



Hands-on Machine Learning with NVIDIA Physics NeMo

Jonathan Dursi, Vanessa Courville, Mohammad Amin Nabian

C3SE 2026 Short Course 01

19 May 2026

Feel free to stay in touch for questions after the session

jdursi@nvidia.com

[Jonathan Dursi | LinkedIn](#)

Approximate Schedule

Time	Content
8 : 30 – 8 : 45 AM	Cluster Setup
8 : 45 – 10 : 00 AM	Intro to PINNs – Slide, First Notebooks
10 : 00 – 10 : 30 AM	Coffee
10 : 30 – 12 : 00 PM	Advanced PINNs, Intro to Operators
12 : 00 – 1 : 00 PM	Lunch
1 : 00 – 2 : 00 PM	Operators Notebooks, Conclusion



Getting Connected To The Cluster

Jupyter notebook

```

Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-31-generic x86_64)
 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro
Expanded Security Maintenance for Applications is not enabled.

668 updates can be applied immediately.
668 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable
Additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Welcome to Base Command Manager 11.6

Based on Ubuntu Noble Numbat 24.04
Cluster Manager ID: #000000

Use the following commands to adjust your environment:
'module avail' - show available modules
'module add <module>' - adds a module to your environment for this session
'module initadd <module>' - configure module to be loaded at every login
                    (Note: initadd is available only for Tcl modules)

Last login: Thu May 14 12:48:25 2026 from 10.104.92.10

3s-bagf@cell-headnode:~$
3s-bagf@cell-headnode:~$
3s-bagf@cell-headnode:~$

```



Web shell

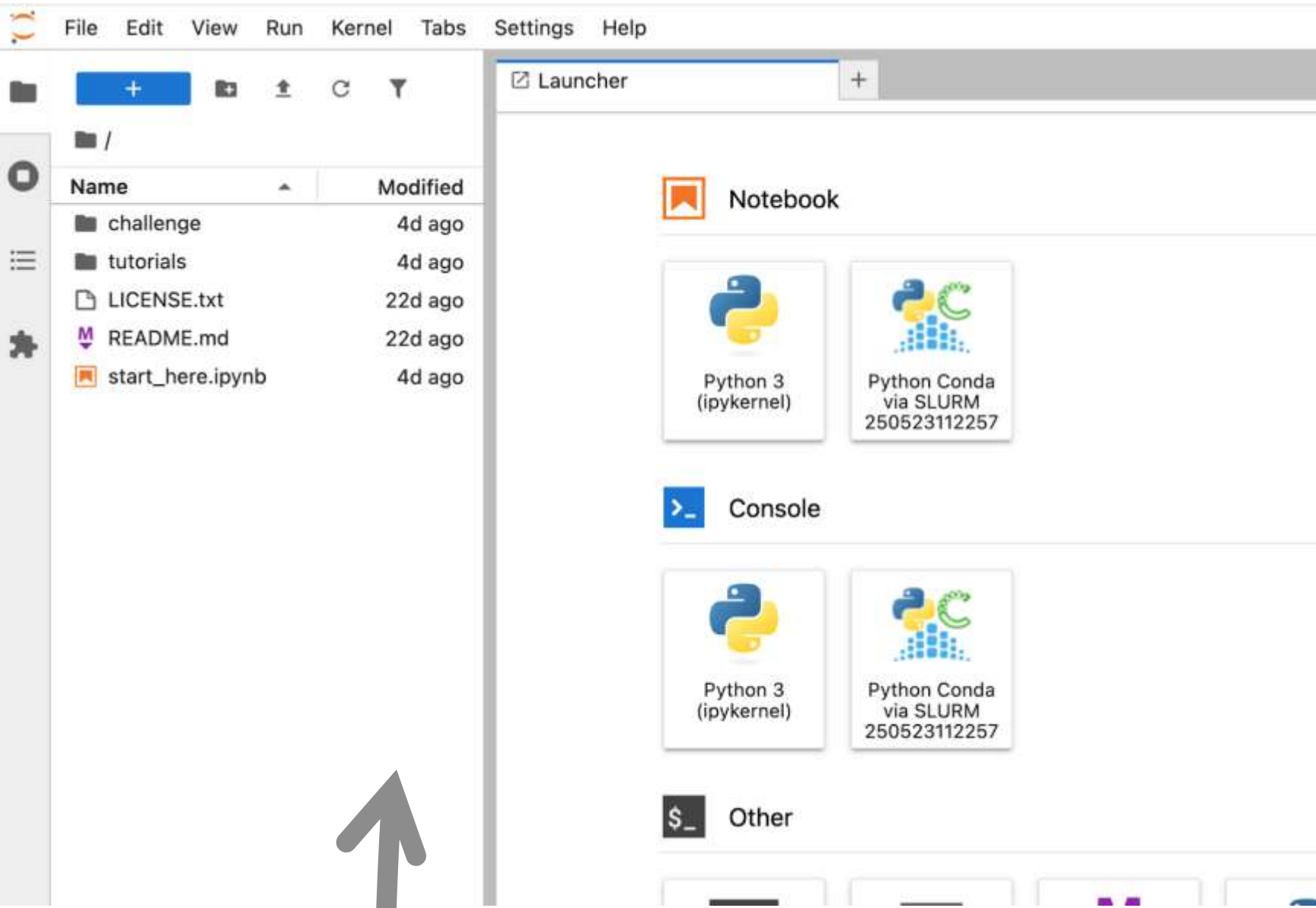
axis

Log in to Axis User Portal

Next

Forgot password?

physicsnemo-script.sh



Axis-raplab

Login code

Ssh command

Localhost via port forwarding



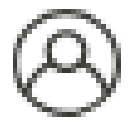
Connecting to the Cluster

Logging In To Axis

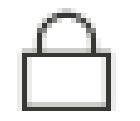
- **Login to Axis with your Axis (C3S-26) credentials**
 - Link : <https://axis-raplabhackathon.axisportal.io/apps>
- Use Chrome browser or make sure your browser does not block pop ups
- **Note! Resources like the link to axis can be found at bit.ly/C3SE-PhysicsNeMo**

Log in to Axis User Portal

C3S-26



.....



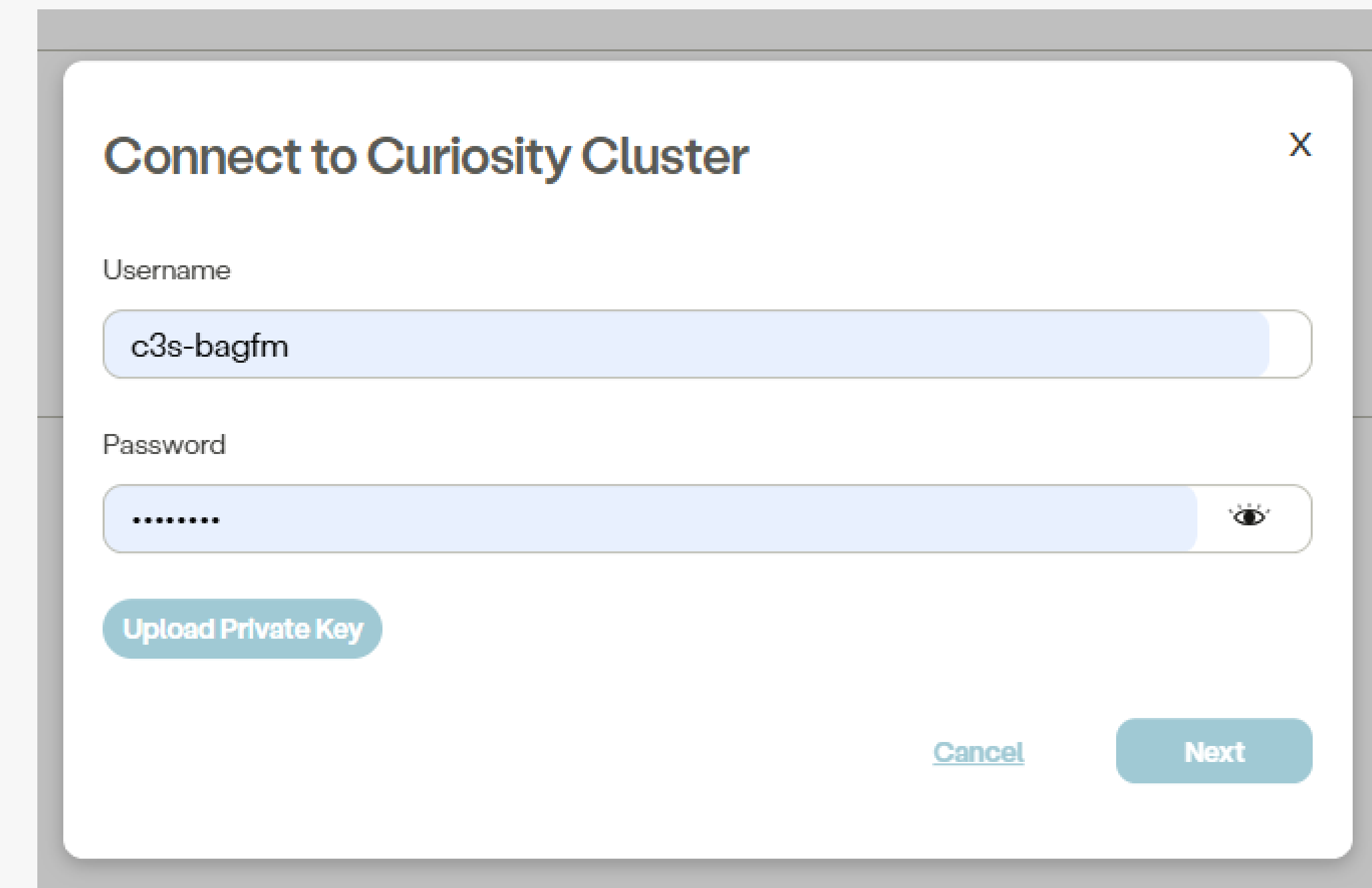
Next

[Forgot password?](#)

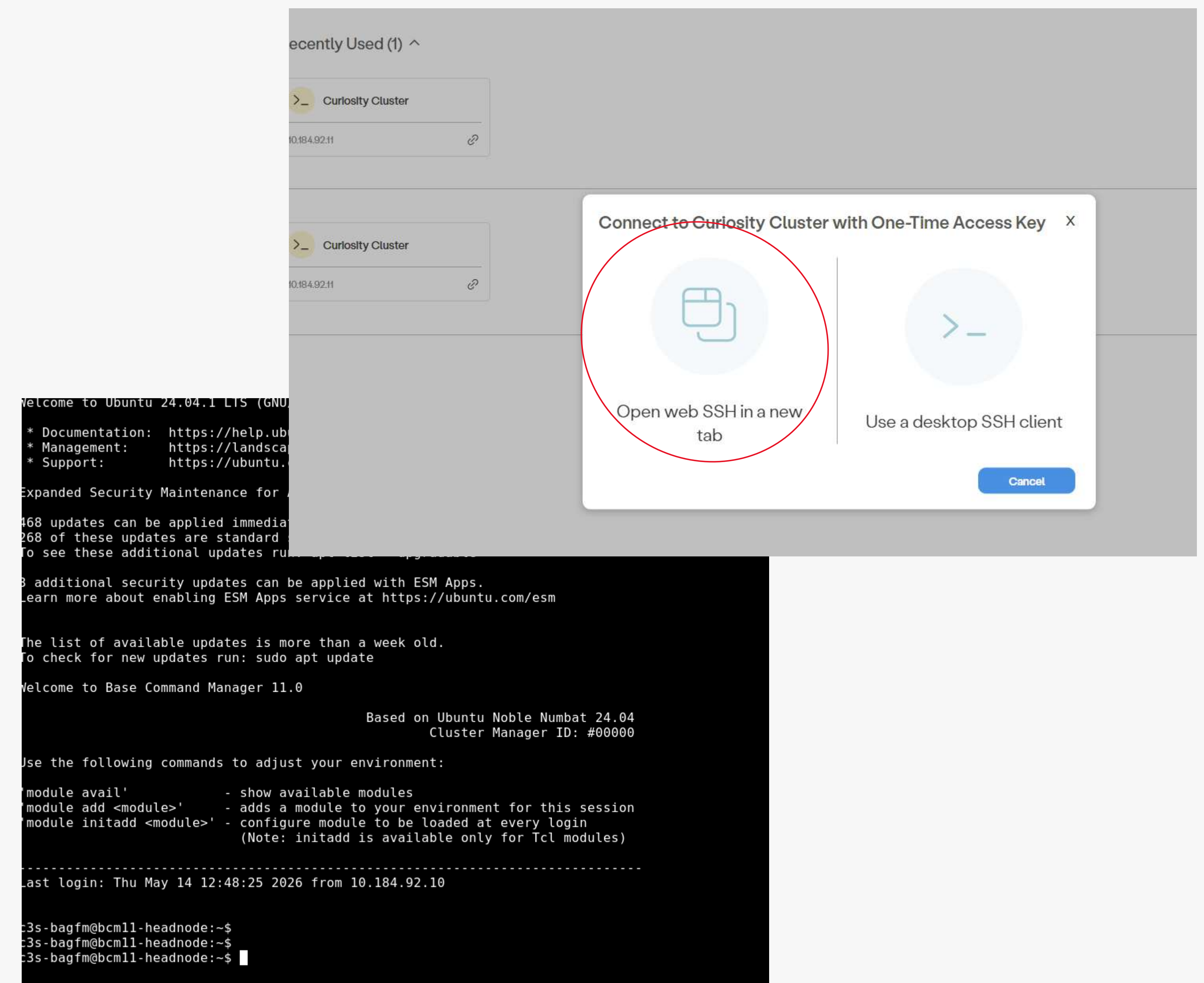
Get Ready to start the Job

Log into Curiosity

- Enter your Curiosity credentials
- Click on Curiosity
- Click on “Open web SSH in a new tab”
- You now have a terminal into curiosity



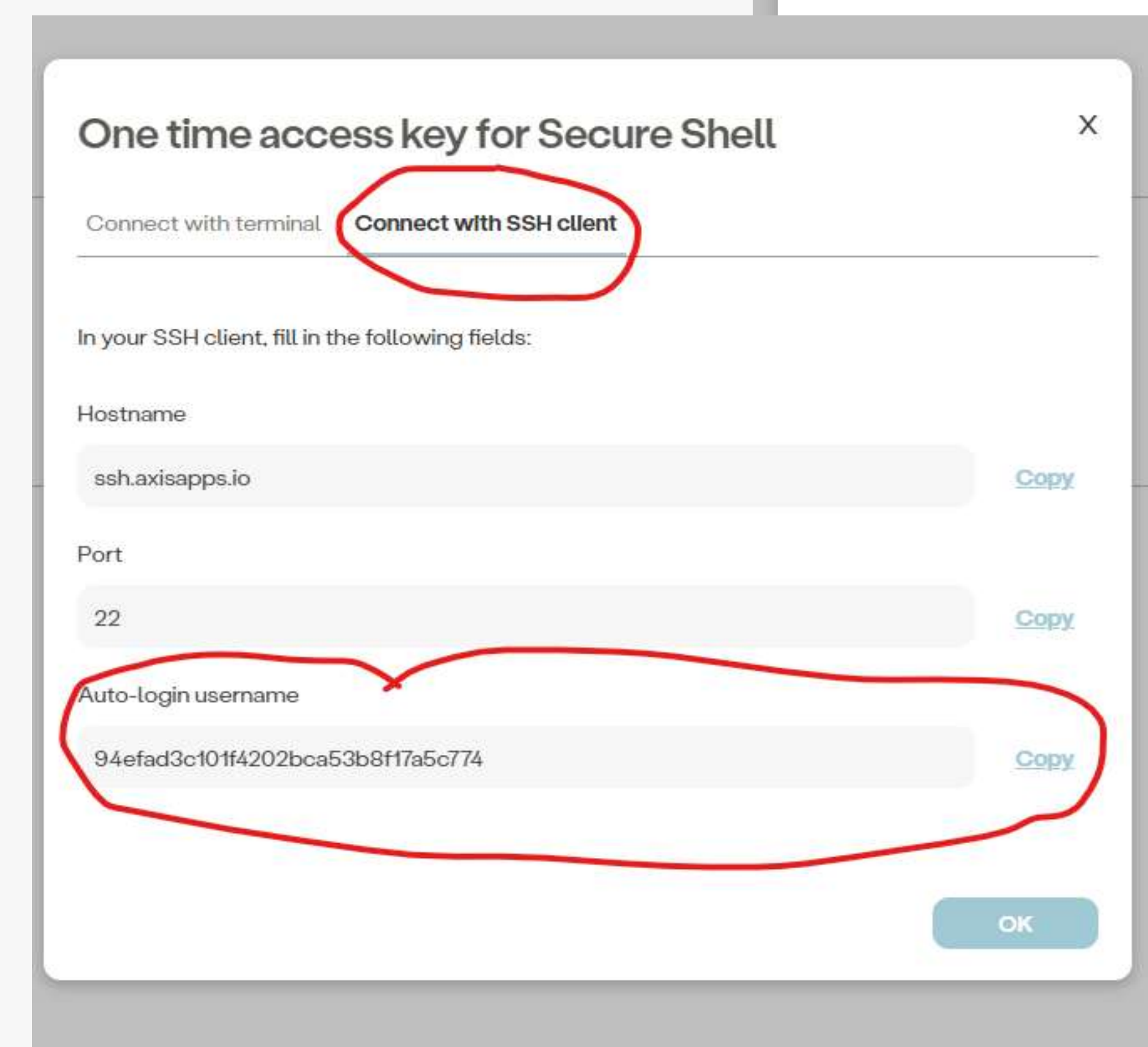
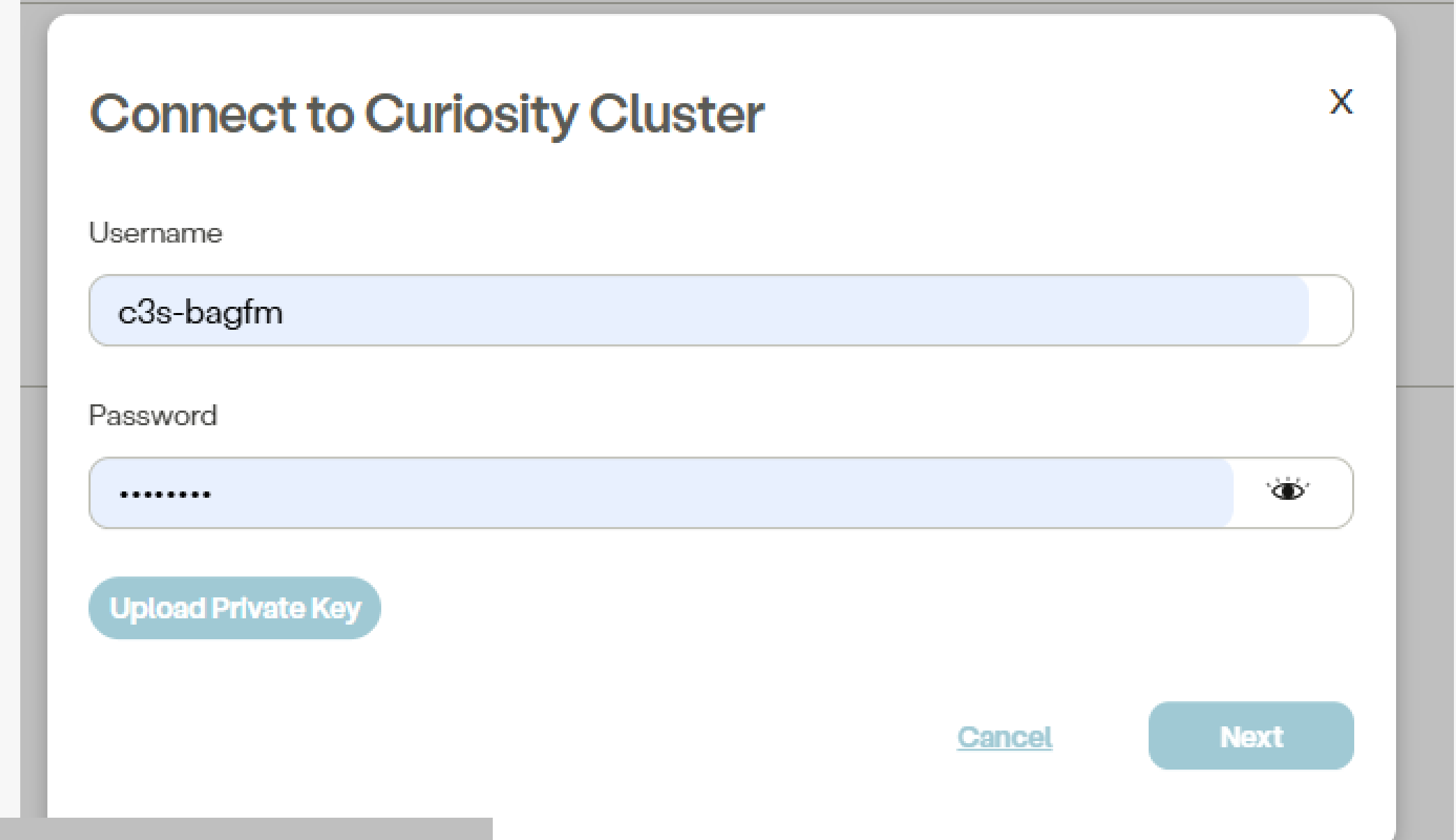
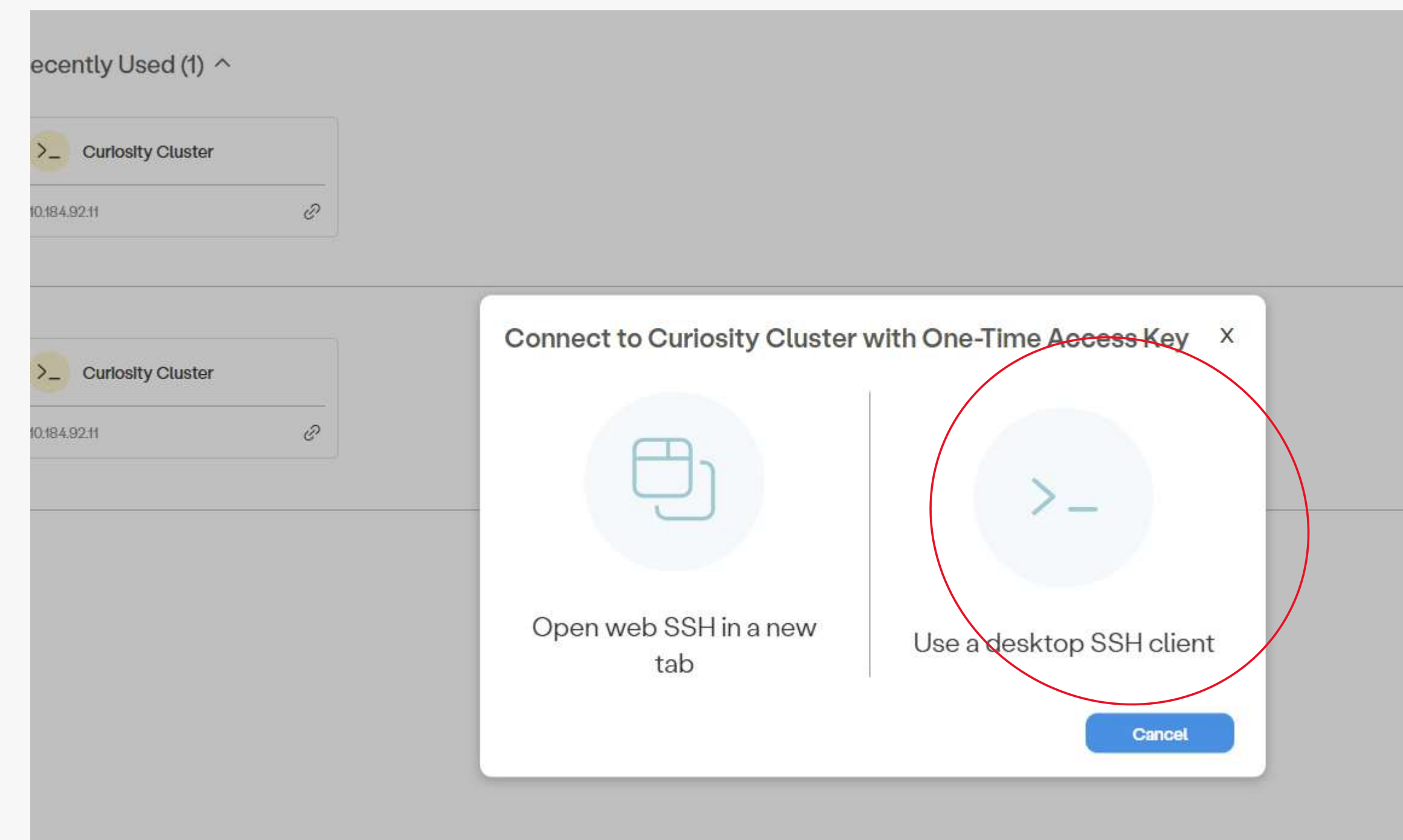
A dialog box titled "Connect to Curiosity Cluster" with a close button (X) in the top right corner. It contains two input fields: "Username" with the value "c3s-bagfm" and "Password" with masked characters ".....". Below the password field is a button labeled "Upload Private Key". At the bottom right are two buttons: "Cancel" and "Next".



Get Ready to start the Job

Get axis key

- Go back to the Axis console
- Click on “Curiosity cluster” and then Click on “Use a desktop SSH client”
- Enter your Curiosity credentials
- Click on “Connect with SSH client”
- Copy that “Auto-login username” – we’re going to use it in the next step. Paste it somewhere safe
- Note – if you used a desktop ssh client last time, you’ll still need to get a new autologin; they’re one use only



Start the Job

Run the script, use the axis key

- Go to the web shell
- Run the command **/lustre/fs01/bootcamps/scripts/End-to-End-AI-for-Science-v4/physicsnemo-script.sh**
 - (Use tab autocomplete, it's way easier)
- Select **1) Run End to End AI for Science Bootcamp Material** by typing 1 and hit on the enter key. Please wait for about 10mins for the loading process to complete and follow the on screen
- While that's running we can start with the introduction
- All these instructions can be found at bit.ly/C3SE-PhysicsNeMo

```
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-51-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

468 updates can be applied immediately.
268 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

8 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Welcome to Base Command Manager 11.0

Based on Ubuntu Noble Numbat 24.04
Cluster Manager ID: #000000

Use the following commands to adjust your environment:

module avail'          - show available modules
module add <module>'    - adds a module to your environment for this session
module initadd <module>' - configure module to be loaded at every login
                        (Note: initadd is available only for Tcl modules)

-----
Last login: Thu May 14 12:48:25 2026 from 10.184.92.10

c3s-bagfm@bcm11-headnode:~$
c3s-bagfm@bcm11-headnode:~$
c3s-bagfm@bcm11-headnode:~$
```

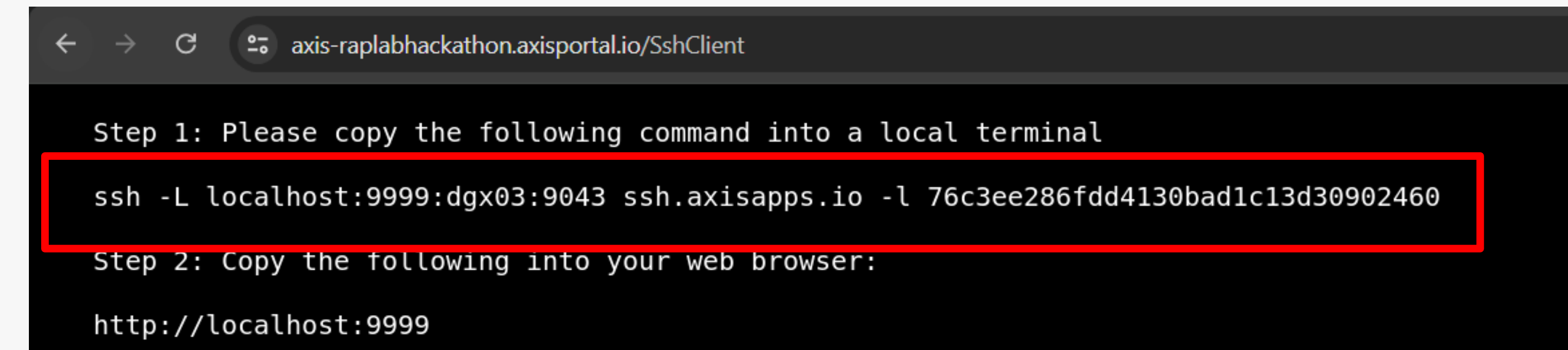
```
Welcome to End to End AI for Science Bootcamp

1) Run End to End AI for Science Bootcamp Material
R) Reset Workspace
T) Reset Docker Container
X) Exit
Choose an option (1, R, T, or X):
```

Connect to the Notebooks

Run a command on a local terminal, connect to localhost on your browser

- In your web terminal when everything is completed, you will see a message along the lines of "Please enter the following command into your terminal" - it's a long command that starts with ssh
- Start a *new terminal on your laptop* (Terminal.app on Mac, PowerShell on windows, or your favourite terminal on Linux, for example), and paste that ssh command in.
- Your terminal is now another shell on the cluster login node, which is fine, but what matters is
- You can now open a new tab on your browser to <https://localhost:9999> to see the Jupyter notebook running on the cluster



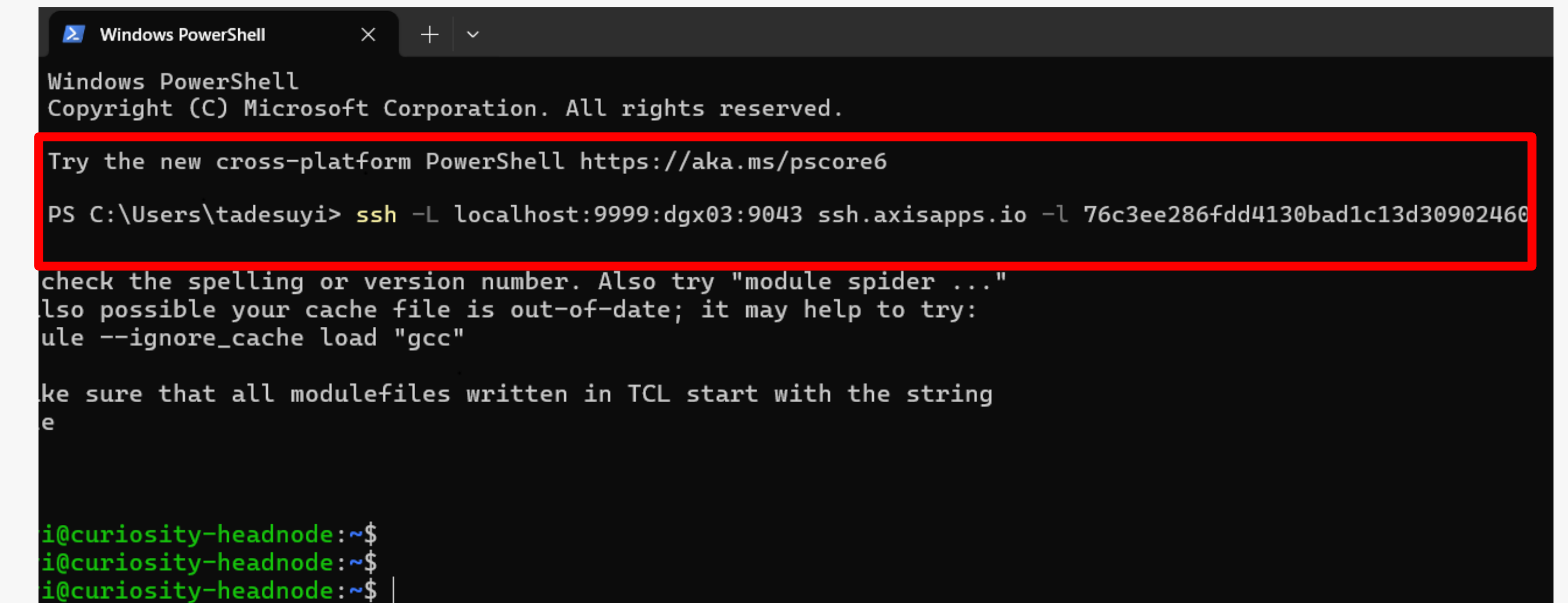
A screenshot of a web browser window with the address bar showing 'axis-raplabhackathon.axisportal.io/SshClient'. The page content includes two steps: 'Step 1: Please copy the following command into a local terminal' followed by a long SSH command, and 'Step 2: Copy the following into your web browser:' followed by the URL 'http://localhost:9999'. The SSH command and the URL are highlighted with red boxes.

```
Step 1: Please copy the following command into a local terminal

ssh -L localhost:9999:dgx03:9043 ssh.axisapps.io -l 76c3ee286fdd4130bad1c13d30902460

Step 2: Copy the following into your web browser:

http://localhost:9999
```



A screenshot of a Windows PowerShell terminal window. The title bar says 'Windows PowerShell'. The text inside shows the prompt 'PS C:\Users\tadesuyi>' followed by the SSH command: 'ssh -L localhost:9999:dgx03:9043 ssh.axisapps.io -l 76c3ee286fdd4130bad1c13d30902460'. Below the command, there is a message about checking spelling or version number, and then the prompt changes to 'i@curiosity-headnode:~\$'.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\tadesuyi> ssh -L localhost:9999:dgx03:9043 ssh.axisapps.io -l 76c3ee286fdd4130bad1c13d30902460

check the spelling or version number. Also try "module spider ..."
also possible your cache file is out-of-date; it may help to try:
ule --ignore_cache load "gcc"

ke sure that all modulefiles written in TCL start with the string
e

i@curiosity-headnode:~$
i@curiosity-headnode:~$
i@curiosity-headnode:~$
```

Cluster troubleshooting

- Everyone can only run 1 job at a time.
- The jobs will run for 4 hours so you do not need to re-submit the sbatch script.
- Each person will get access to 1 GPU only.
- The Hash key is valid for 24 hours.
- You'll have access to the cluster for 24 hours
- Do not use
- The access will be revoked by the end of the day.
- If you get the following error in the final Step :
 - bind [127.0.0.1]:8888: Address already in use
 - channel_setup_fwd_listener_tcpip: cannot listen to port: 8888
 - Could not request local forwarding.
- You will need to change the port 8888 in your ssh command to some other port, examples would be 8890, 8900 etc.
- For example: `ssh -L localhost:8890:dgx05:9538 ssh.axisapps.io -I 7aab57129c95450cb80c4df5c13836bd`



Background

Challenges - Saturating performance in traditional HPC

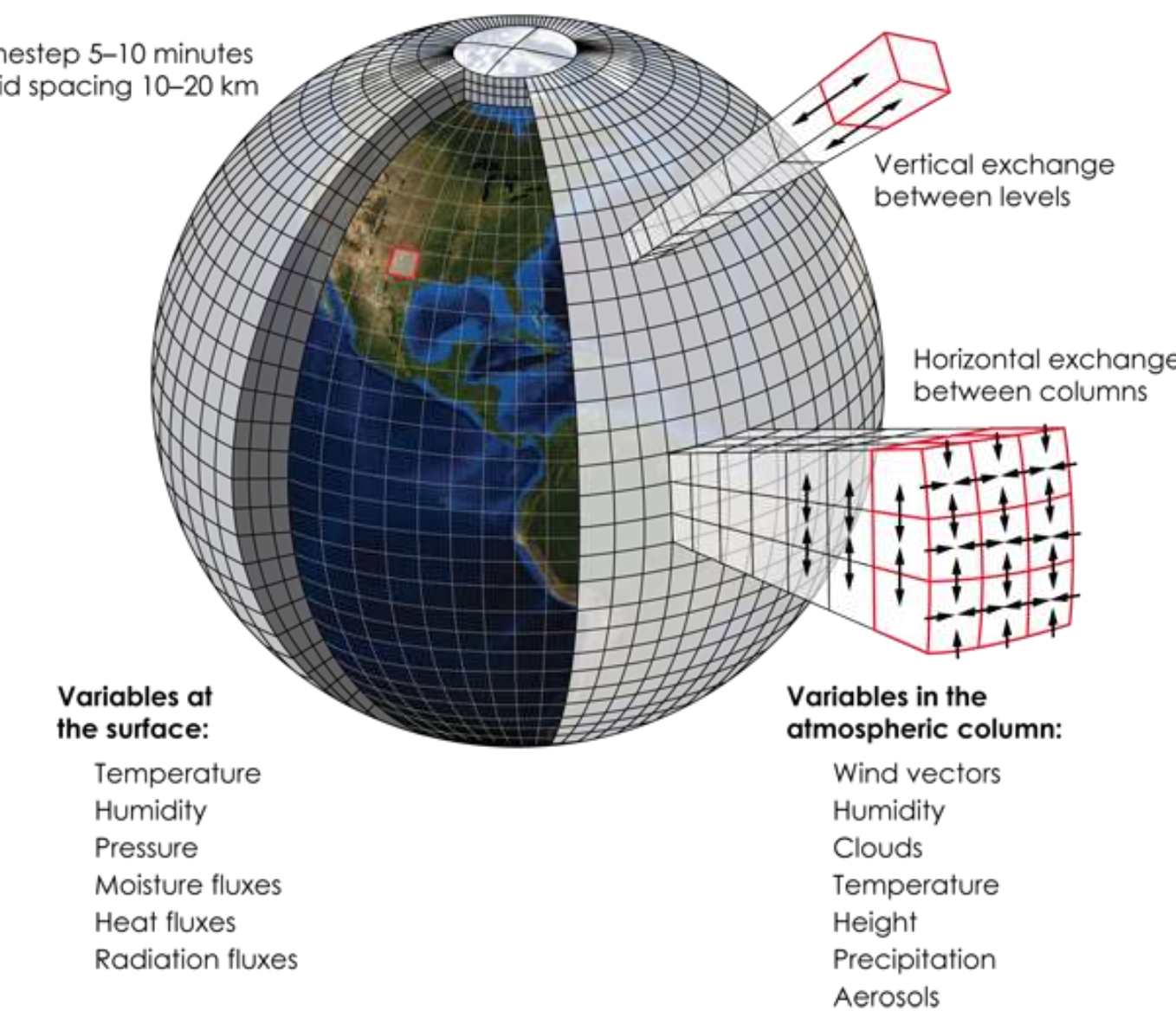
Simulations are getting larger and more complex

Traditional solution methods are:

- Computationally Expensive
- Plagued by Domain Discretization Techniques
- Not suitable for Data-assimilation or Inverse problems

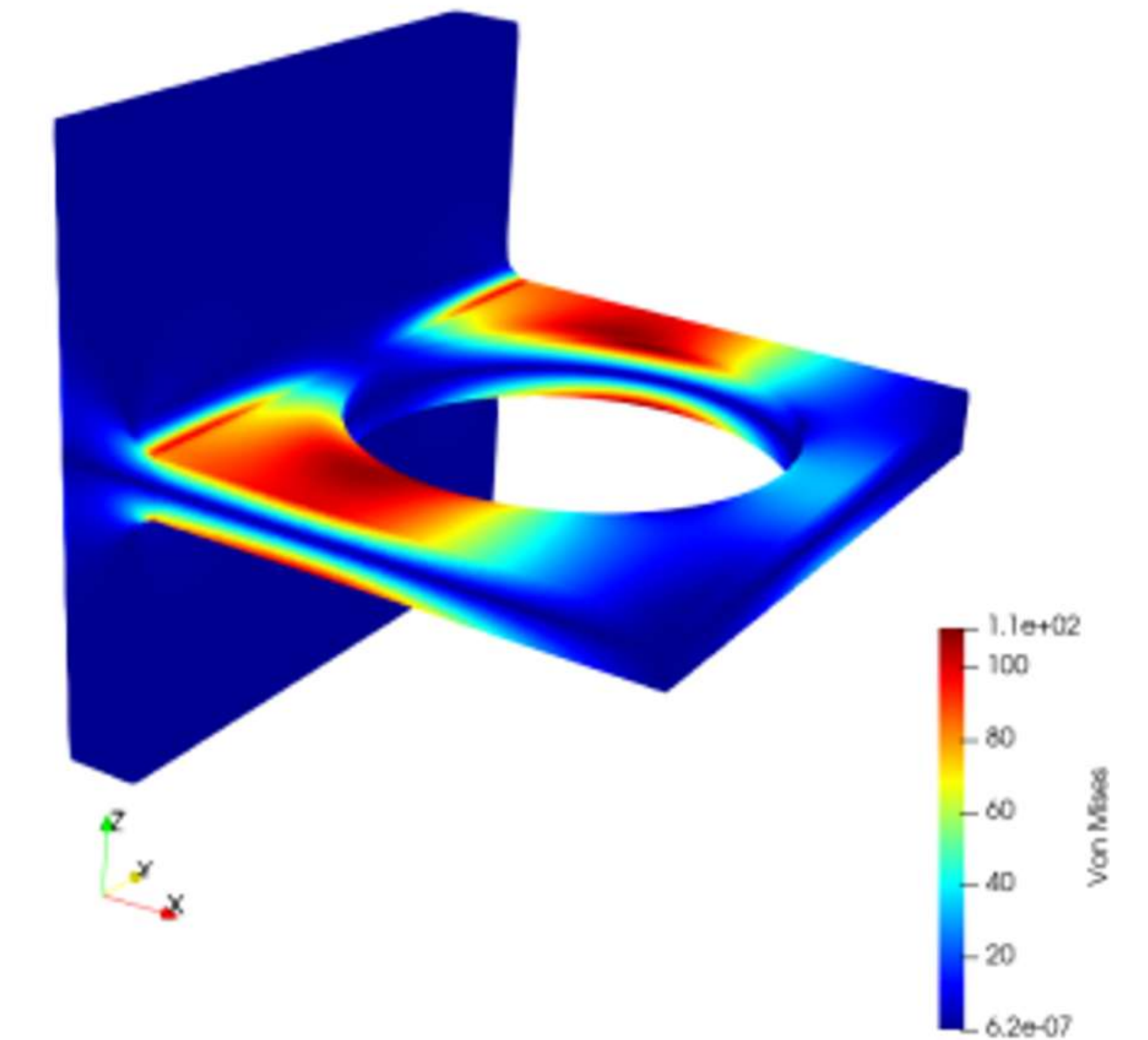
Weather forecast modeling

Timestep 5-10 minutes
Grid spacing 10-20 km



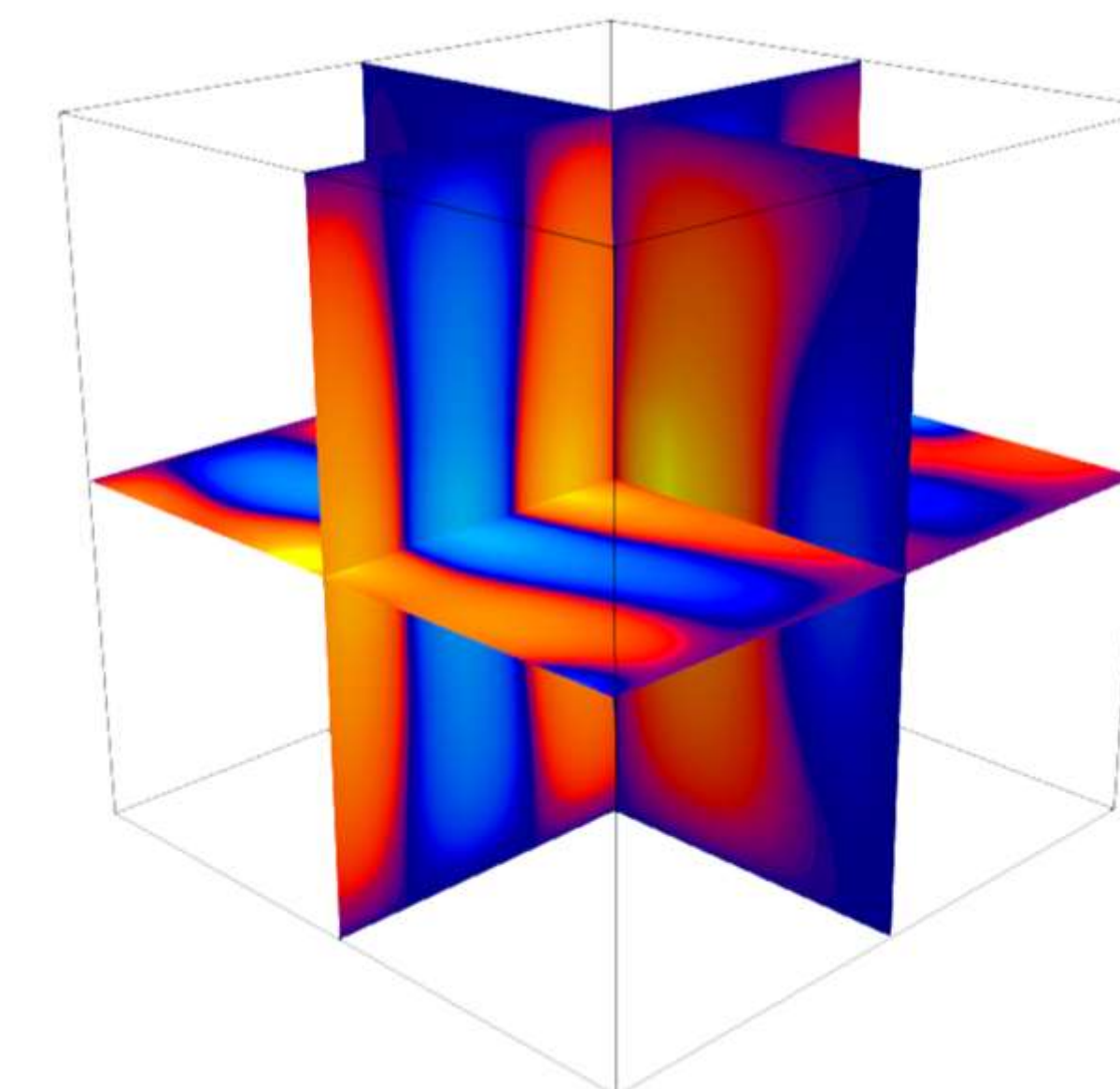
Atmospheric flows

Finite Volume Methods, Sub-grid scale modeling



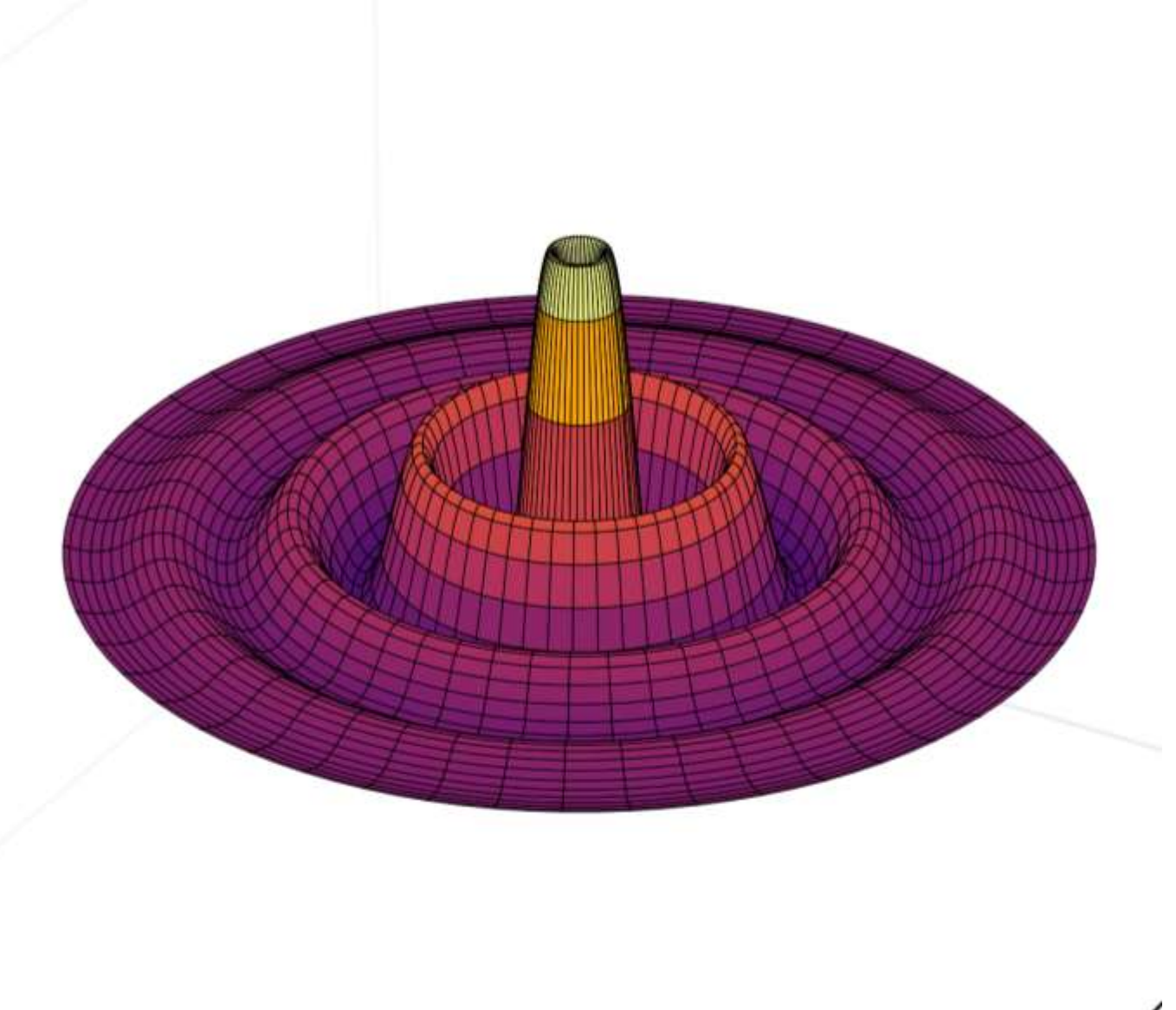
Structural Mechanics

Finite Element Methods



Electromagnetics

Finite Element and Frequency domain methods



Vibrations / Acoustics

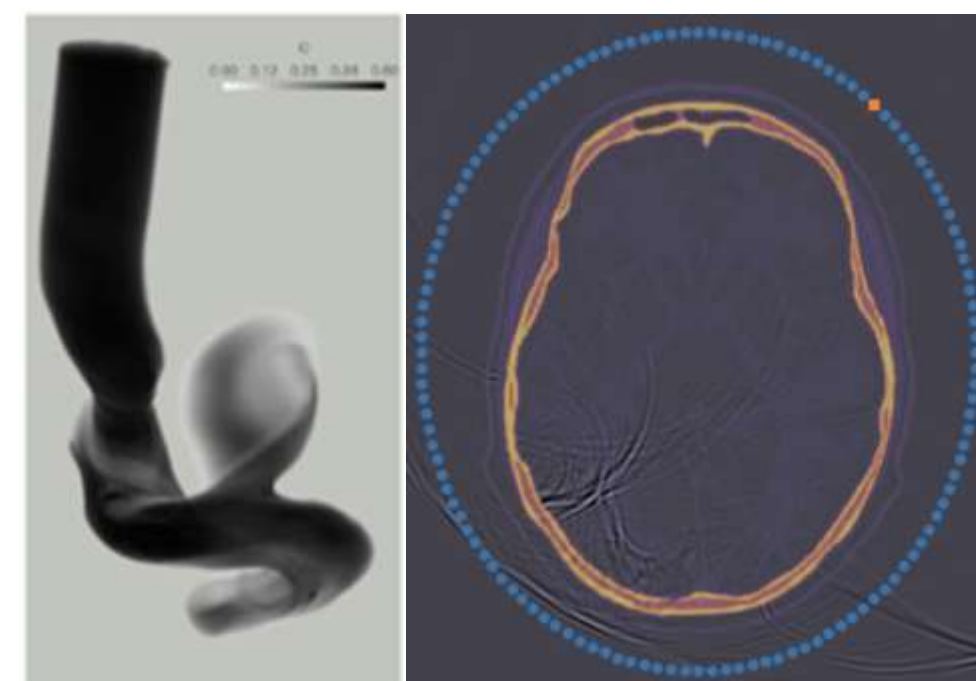
Finite Difference Methods



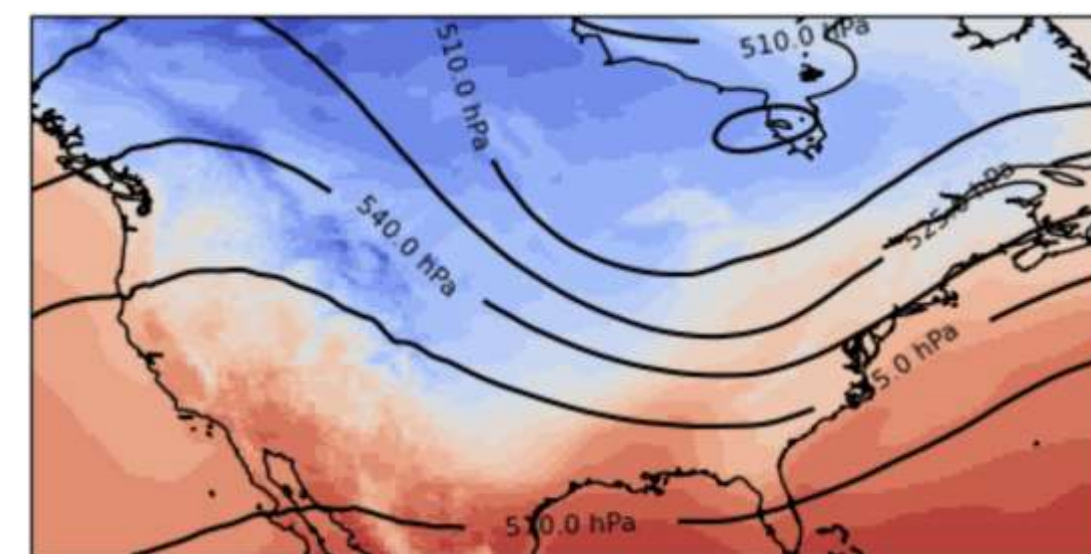
Using AI in Engineering and Science

Use data and governing equations to gather insight

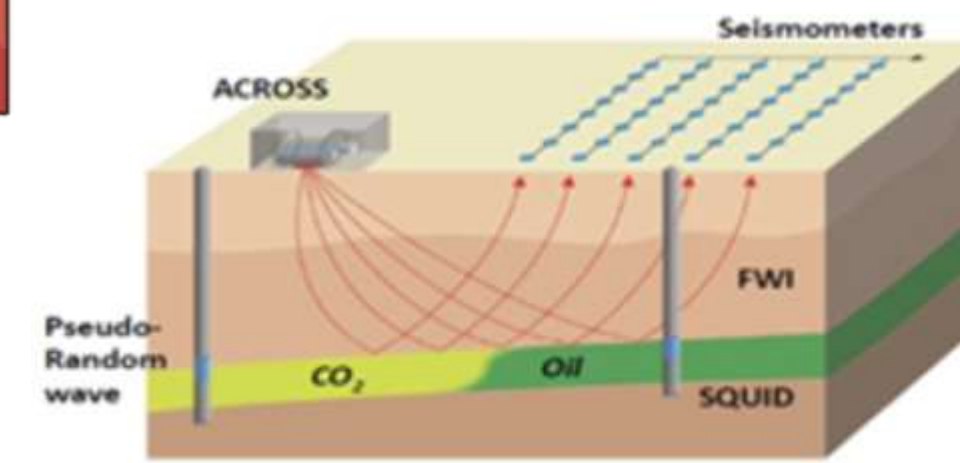
Inverse and Data Assimilation



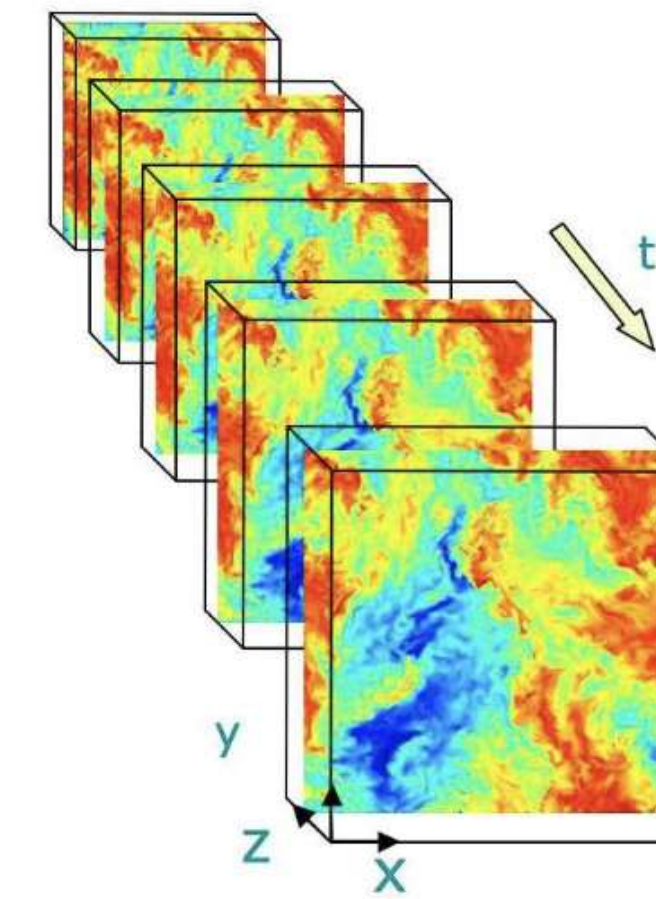
Medical Imaging



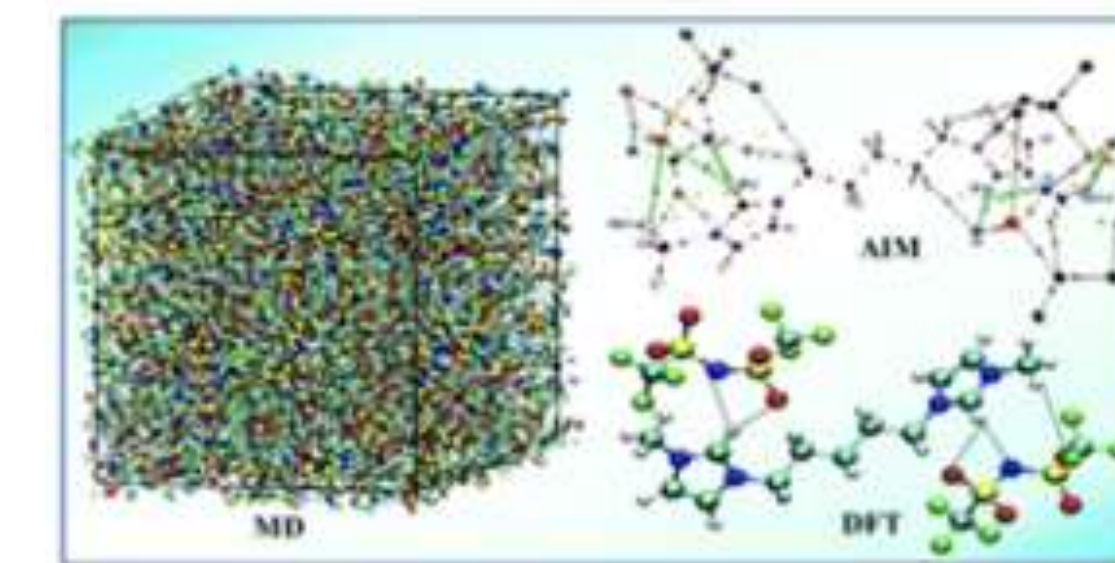
Weather & Climate



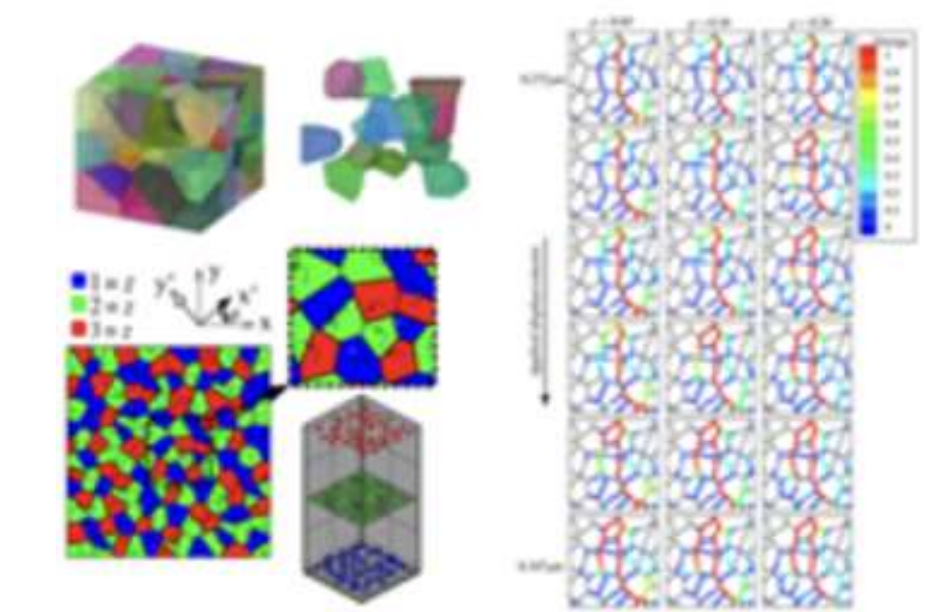
Oil & Gas



Turbulence



Molecular Dynamics

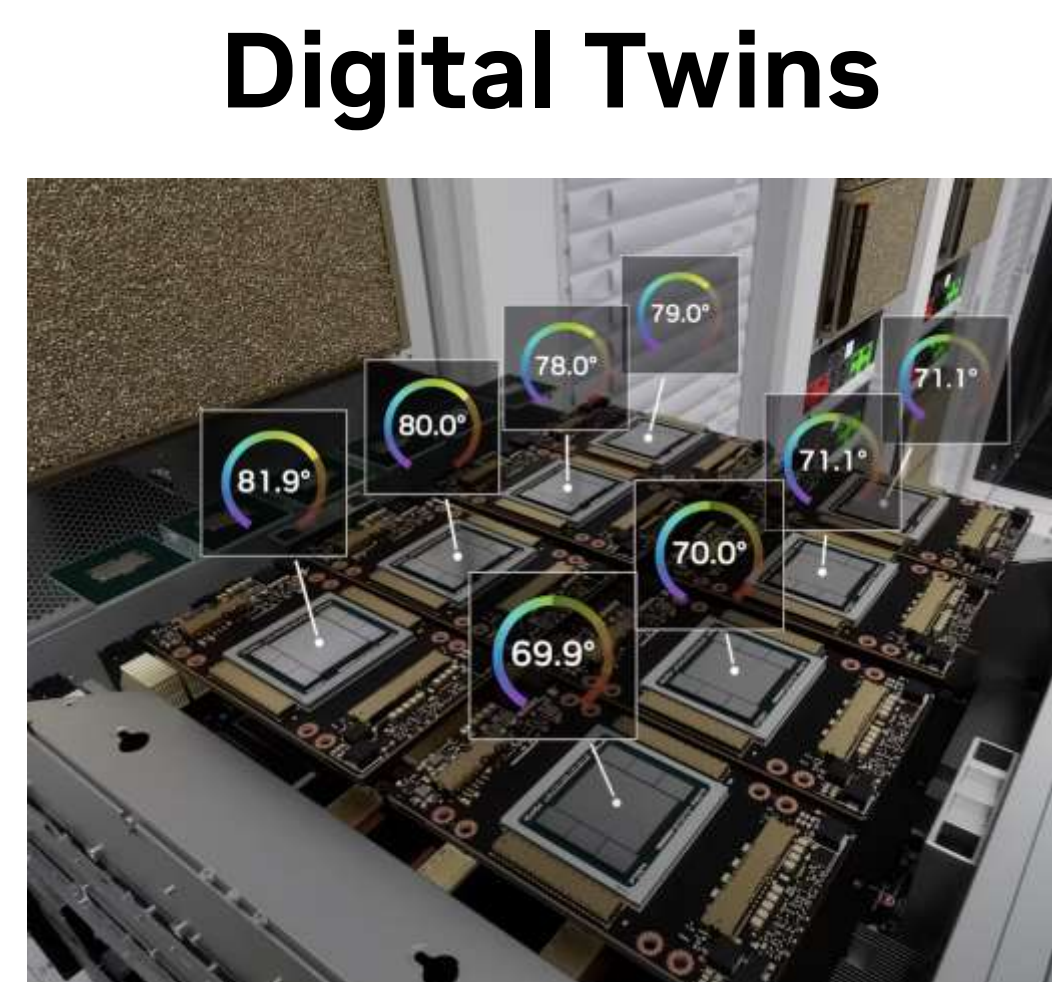


Micro-mechanical
Material Model

Operational Control / Real-time



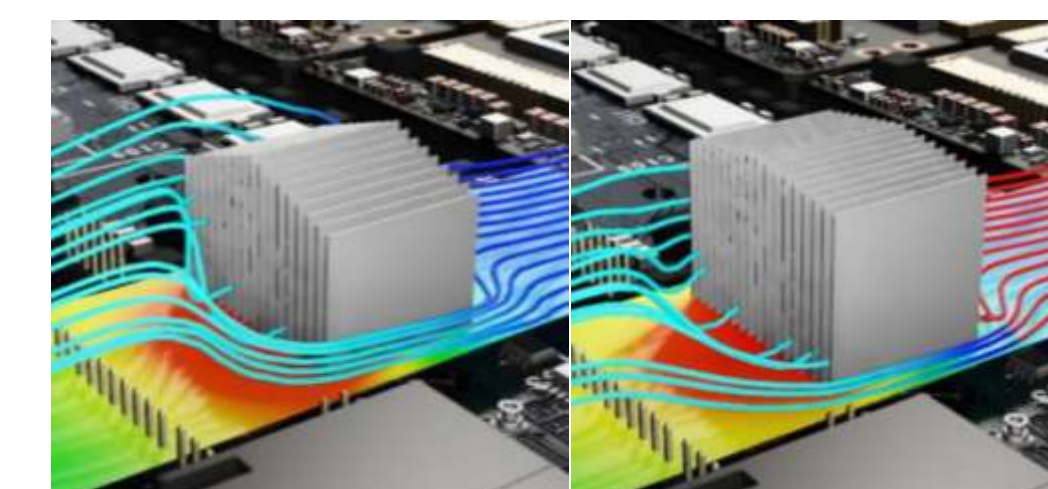
Robotics



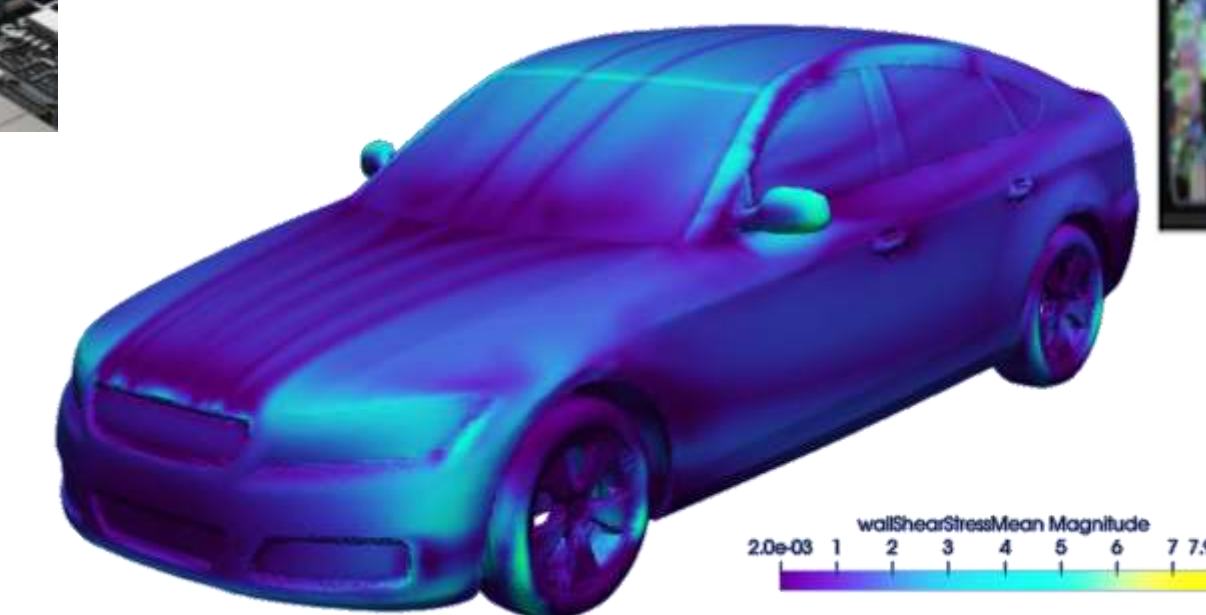
Digital Twins



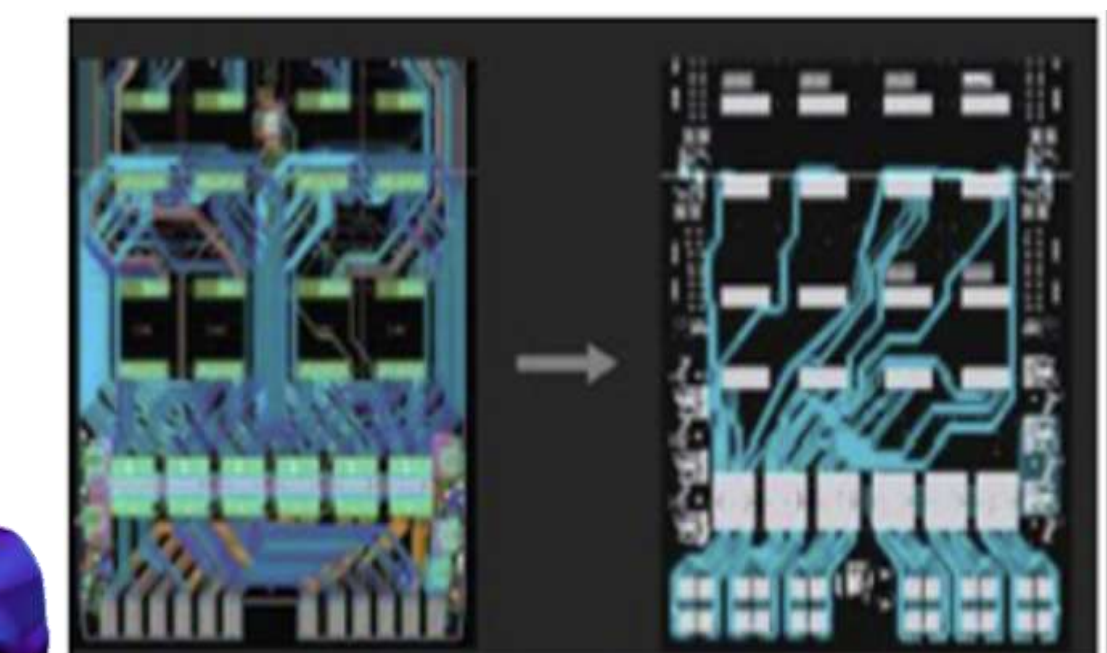
Autonomous
Ride & Handling



Heatsinks



Aerodynamics



Circuit Design

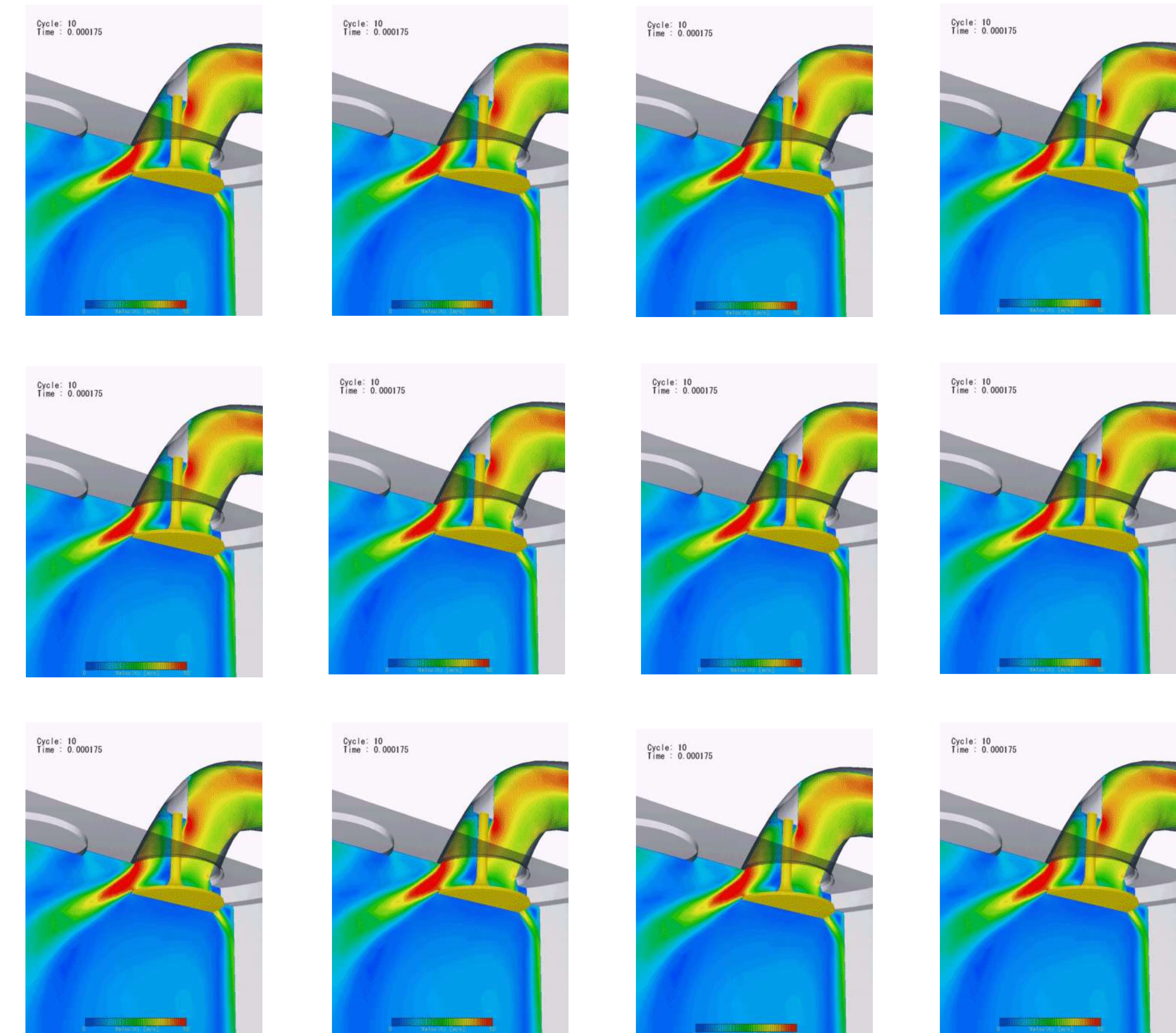
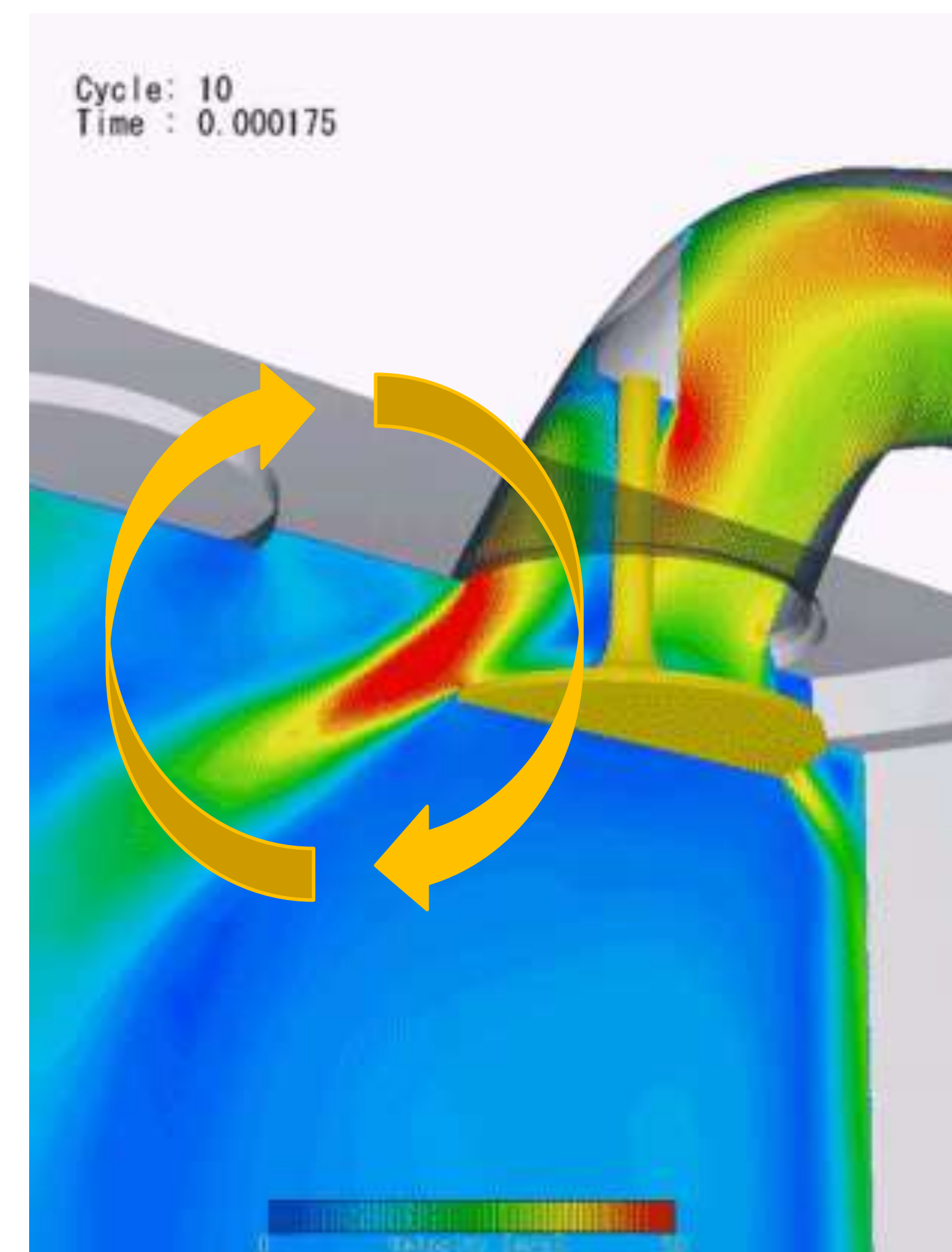
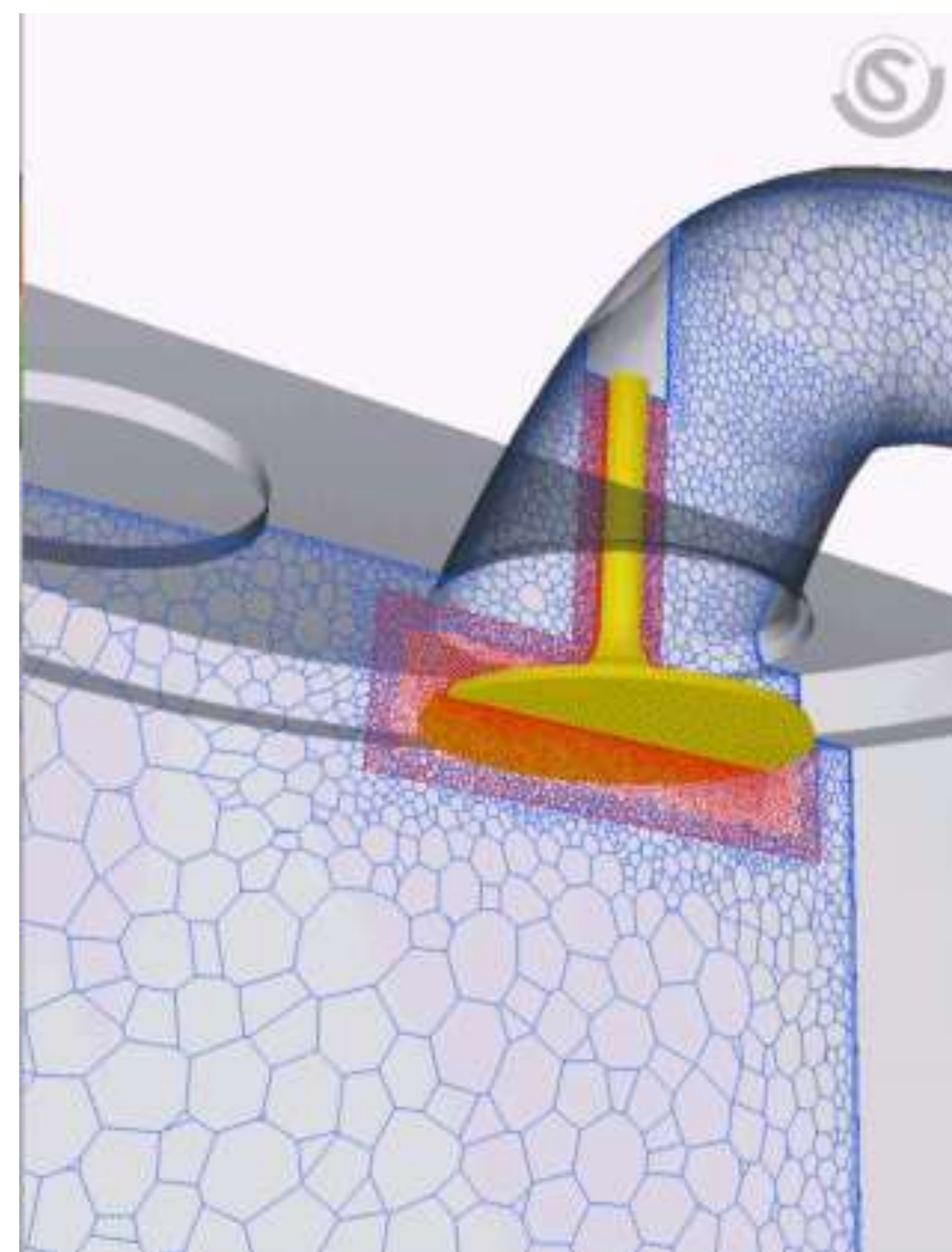
Physics & Data – Little to no gain from Traditional Solver

Physics – Traditional Solver (Speed is a limitation)

Using AI in Engineering and Science

Use data and governing equations to gather insight

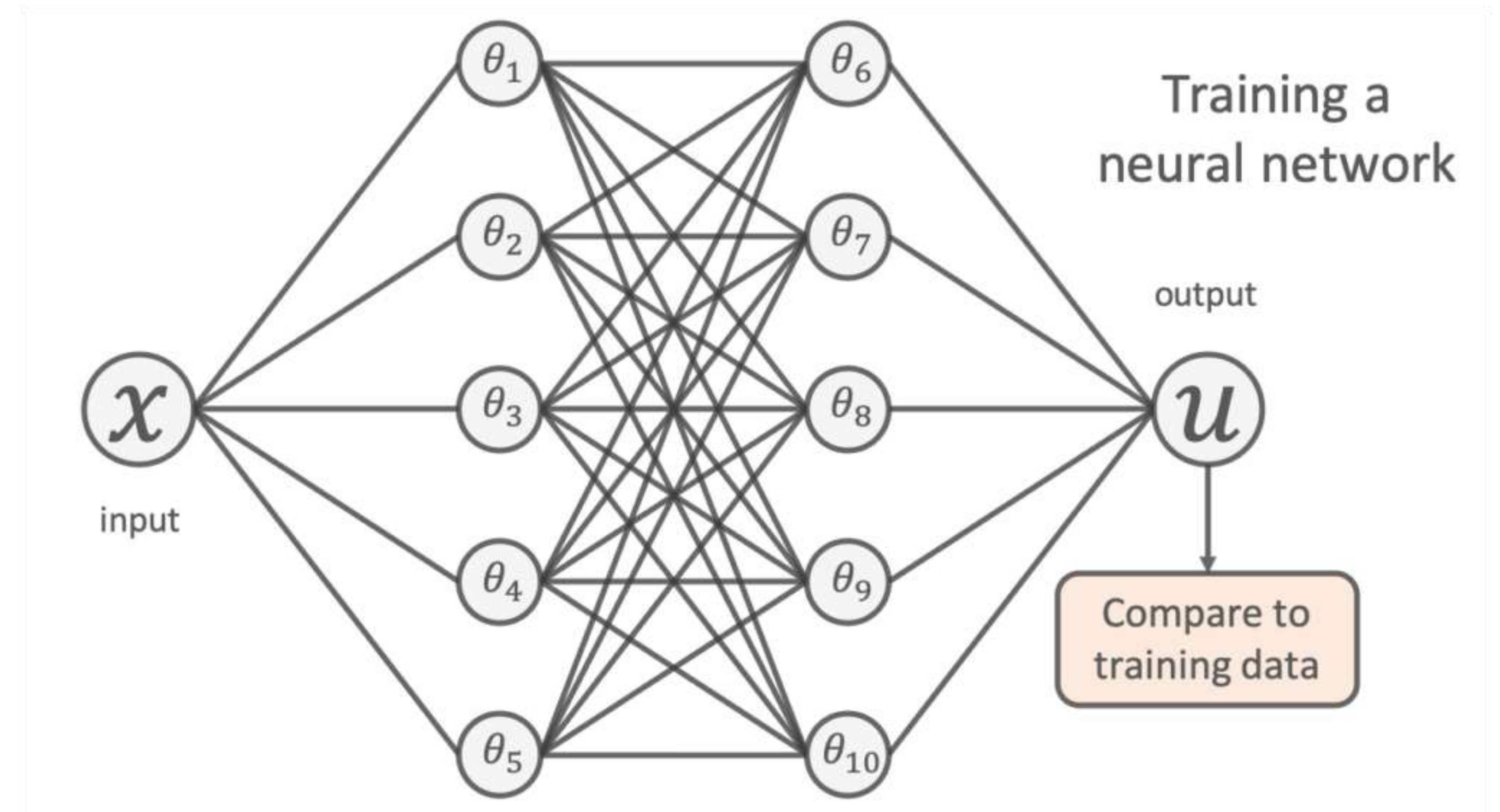
- Like other domains that see the disruption due to AI
- Using AI in simulations unleashes parallelism, real-time outputs, inverse modeling capabilities and generative design



Machine Learning for PDEs

- Rapid advance in ML & with increased amount of scientific data, data driven approaches are becoming more popular.
- An existing theory is not required, instead a ML model can be used to analyze a scientific problem using data only.
- Neural Networks is a popular method to model experimental data.
- Model learns by minimizing the mean-squared error between the predictions and the training points.

$$\min \frac{1}{N} \sum_i^N (u_{\text{NN}}(x_i; \theta) - u_{\text{true}}(x_i))^2$$

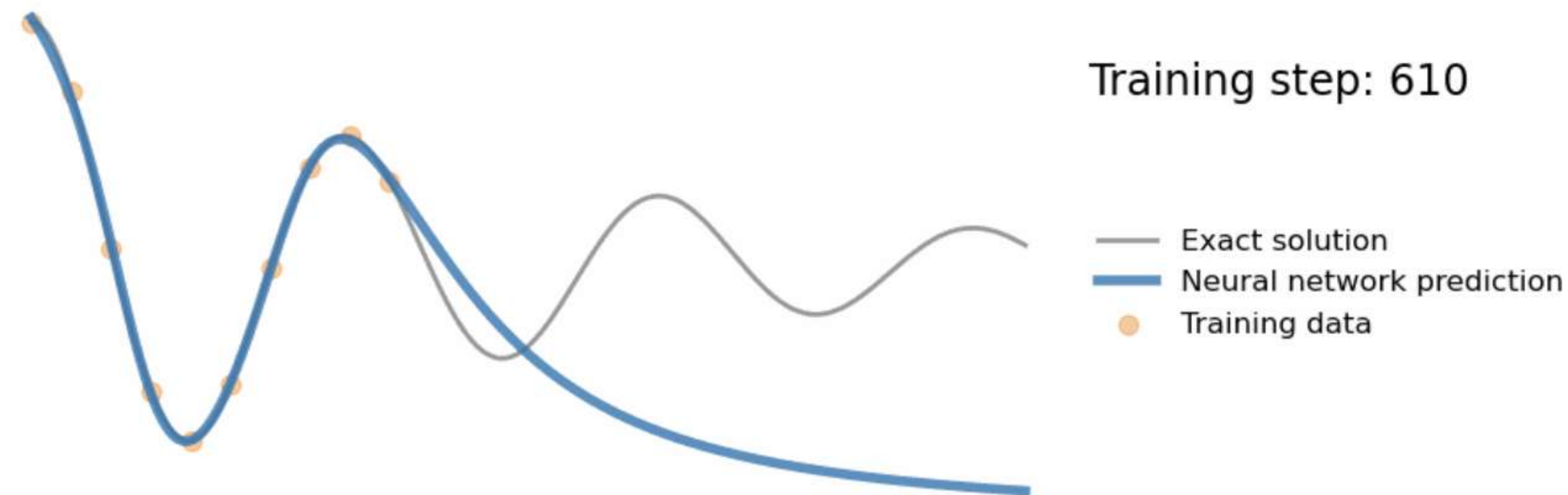


Schematic of a Neural Network

A Machine Learning Revolution in Science

Naively using Deep Learning for scientific problems has its own challenges:

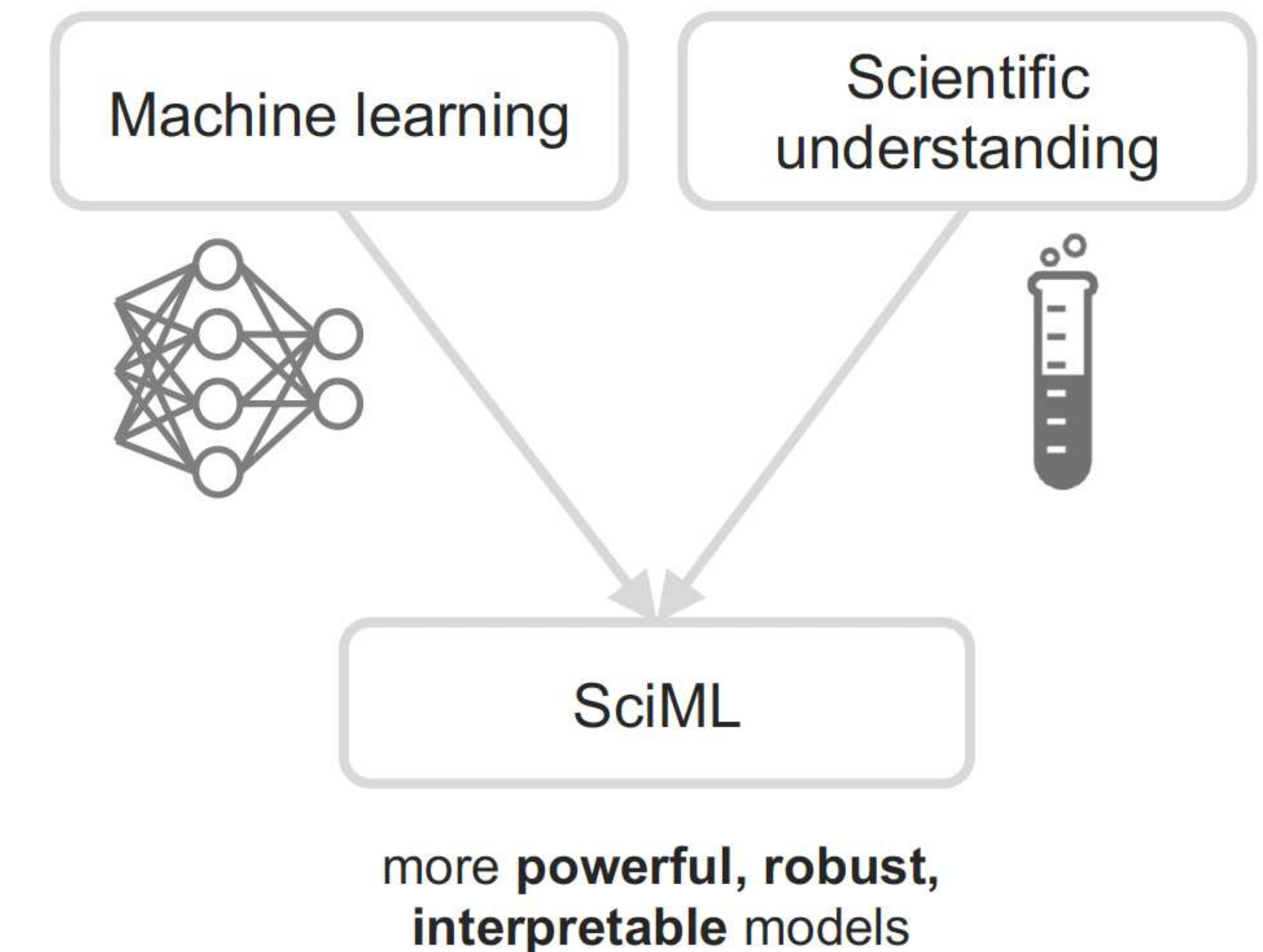
- Lack of interpretability
- Poor generalization – accurately models the process within vicinity of training data, but not away from this data.
- Lack of training data



Scientific Machine Learning

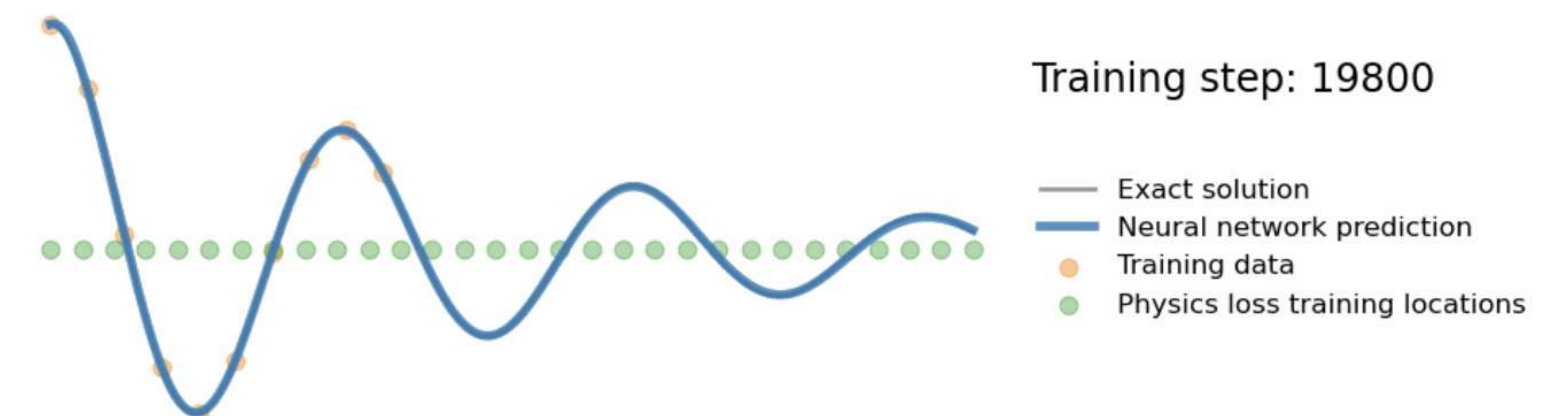
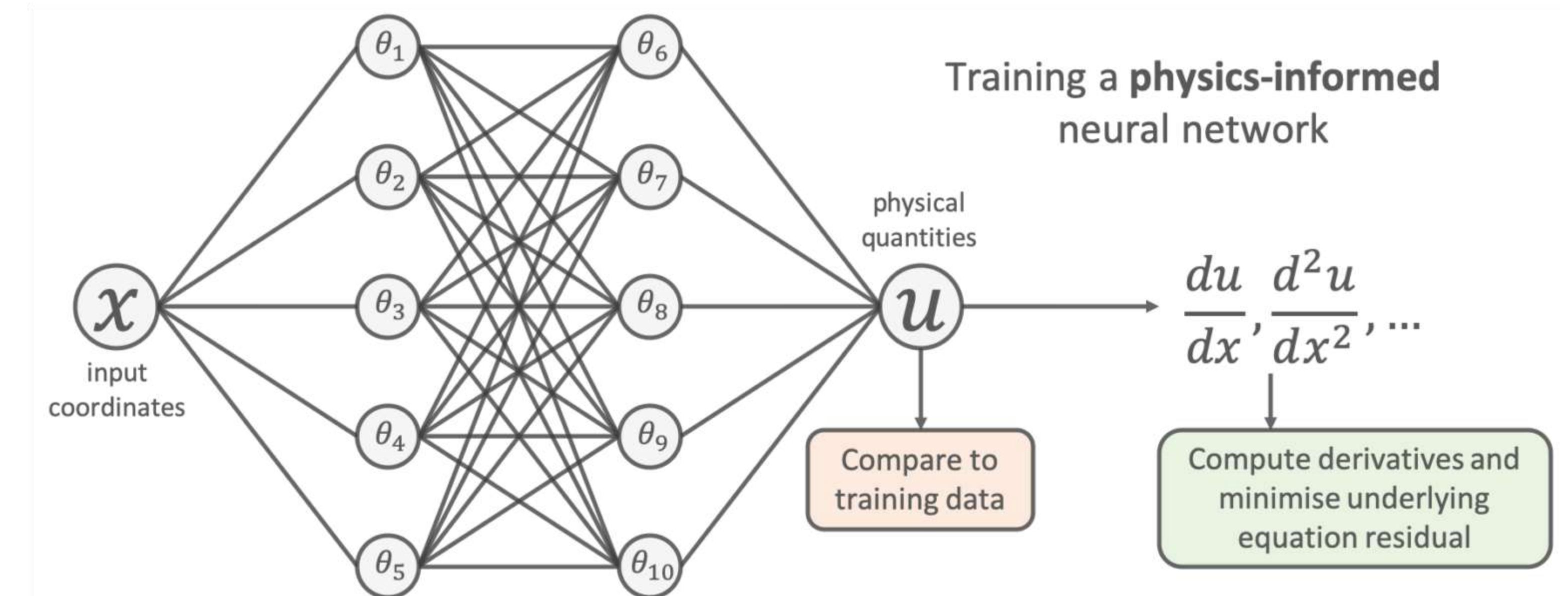
- What if we can somehow add the underlying physics to our ML workflows?
- This way we can take advantage of both the power of Neural Networks + our prior scientific knowledge.
- Advantages:
 - More accurate predictions even if we have sparse or no data, as we can leverage the physical laws.
 - Better generalization as the workflow is not confined to the training data only.
 - Useful in scenarios where data collection is challenging or expensive.

Solution



Physics Informed Neural Networks - PINNs

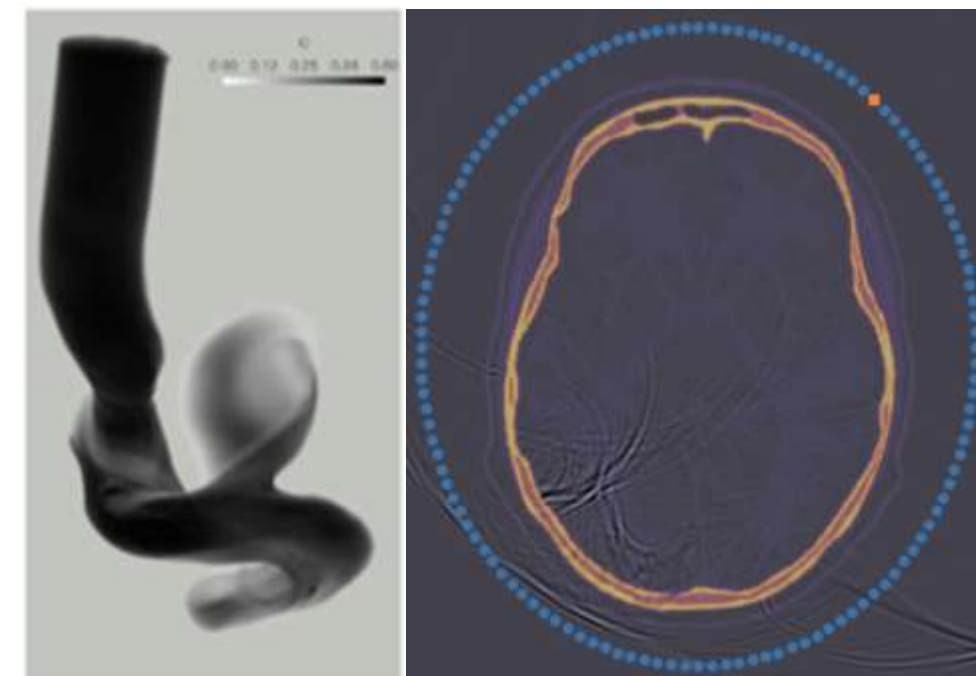
- Idea: Integrate fundamental laws of physics within the network's learning framework.
- How? Add the known differential equations directly into the loss functions of the neural network's training process.
- Now loss function has 2 components:
 - Measure discrepancy between network's predictions and training data points.
 - Quantifies deviation of predictions from the physical model described by differential equations.
 - By minimizing both, the network learns to predict outcomes adhering to both observed data and the physical principles.



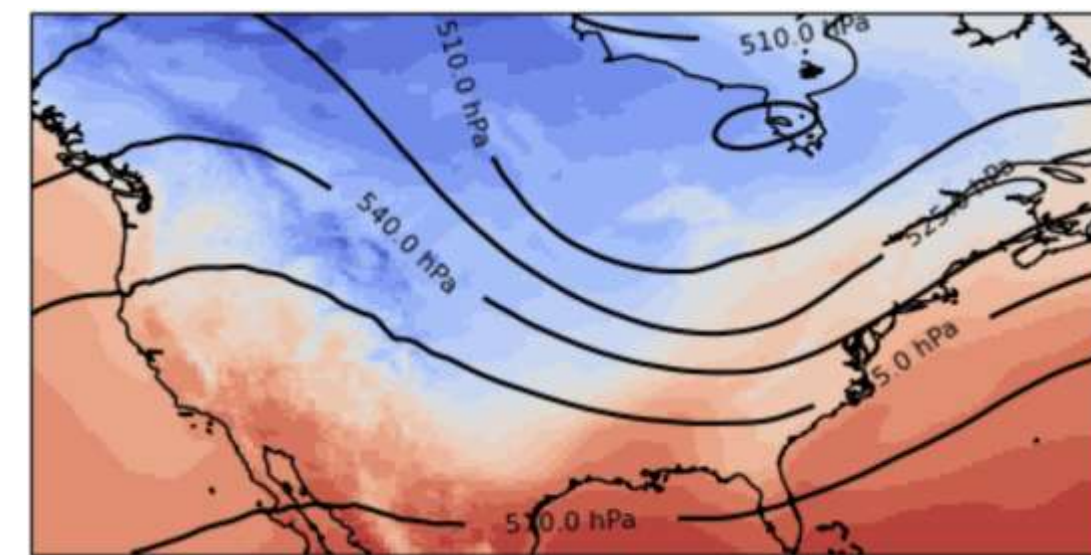
Using AI in Engineering and Science

Use data and governing equations to gather insight

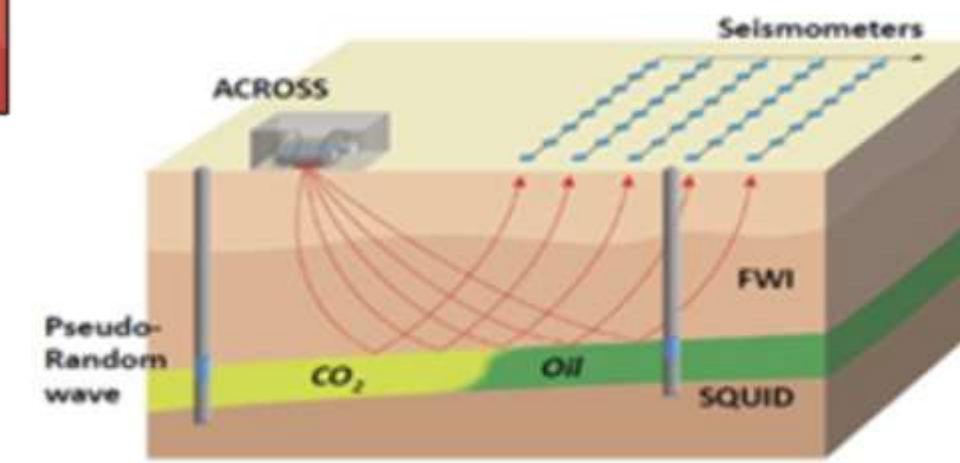
Inverse and Data Assimilation



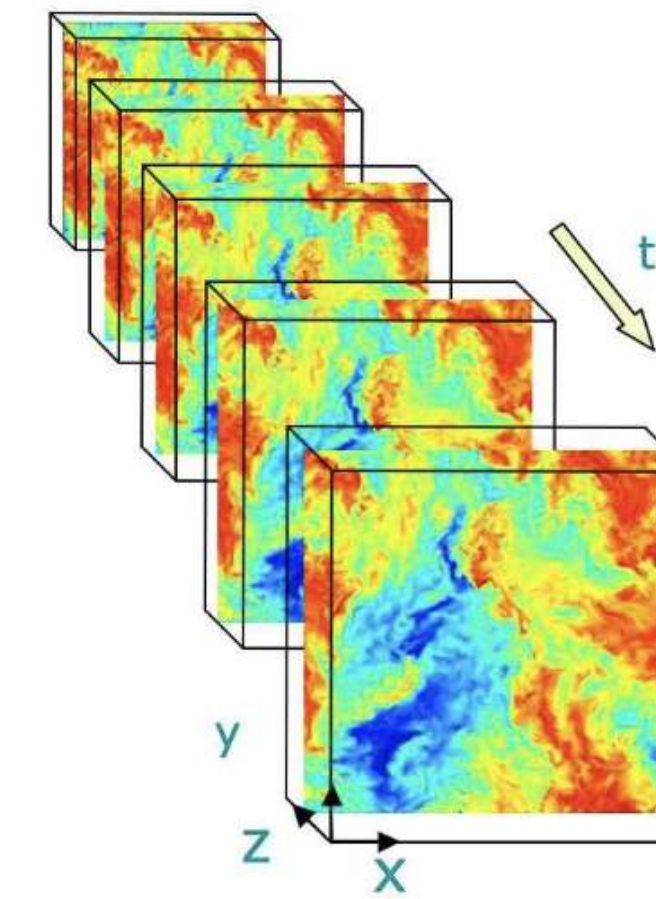
Medical Imaging



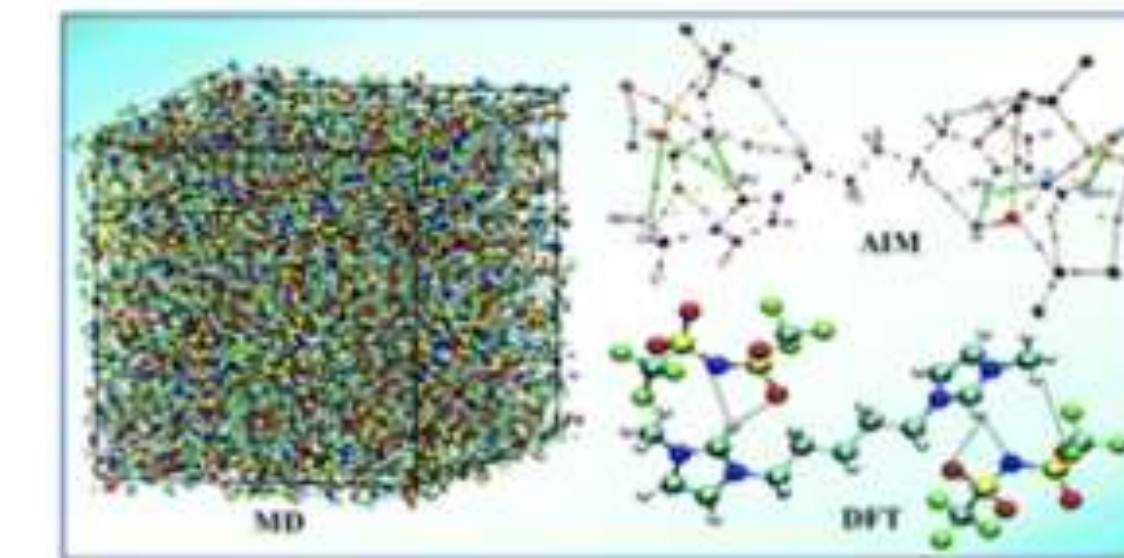
Weather & Climate



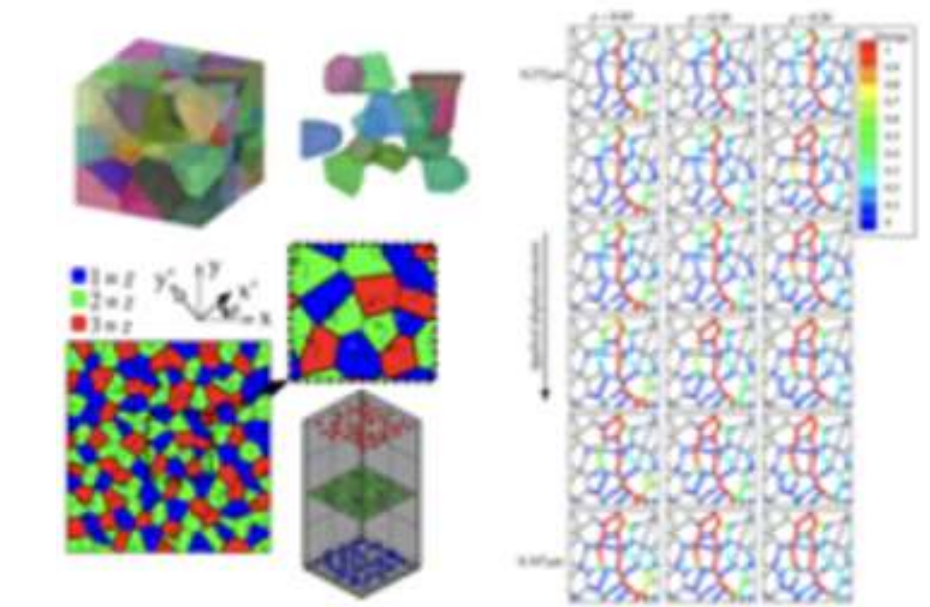
Oil & Gas



Turbulence



Molecular Dynamics



Micro-mechanical
Material Model

Operational Control / Real-time

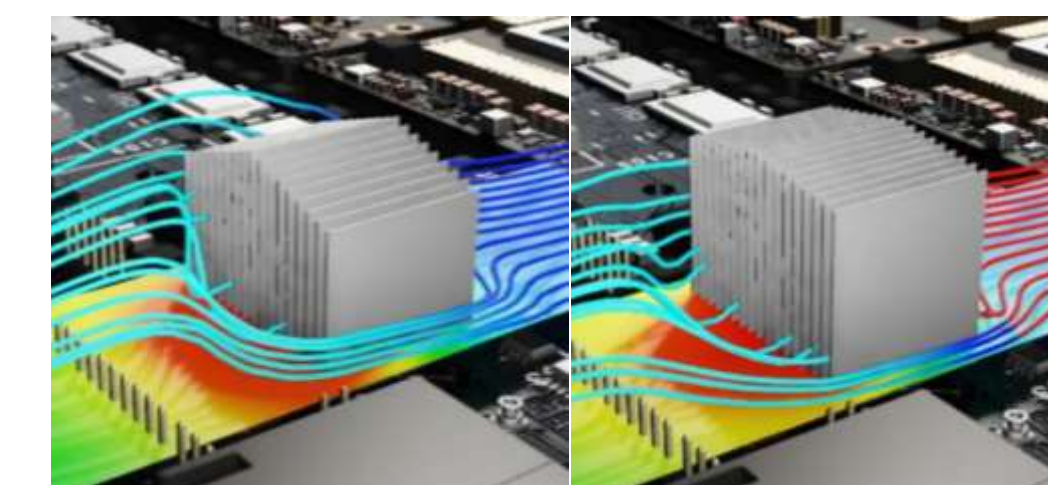


Robotics

Digital Twins

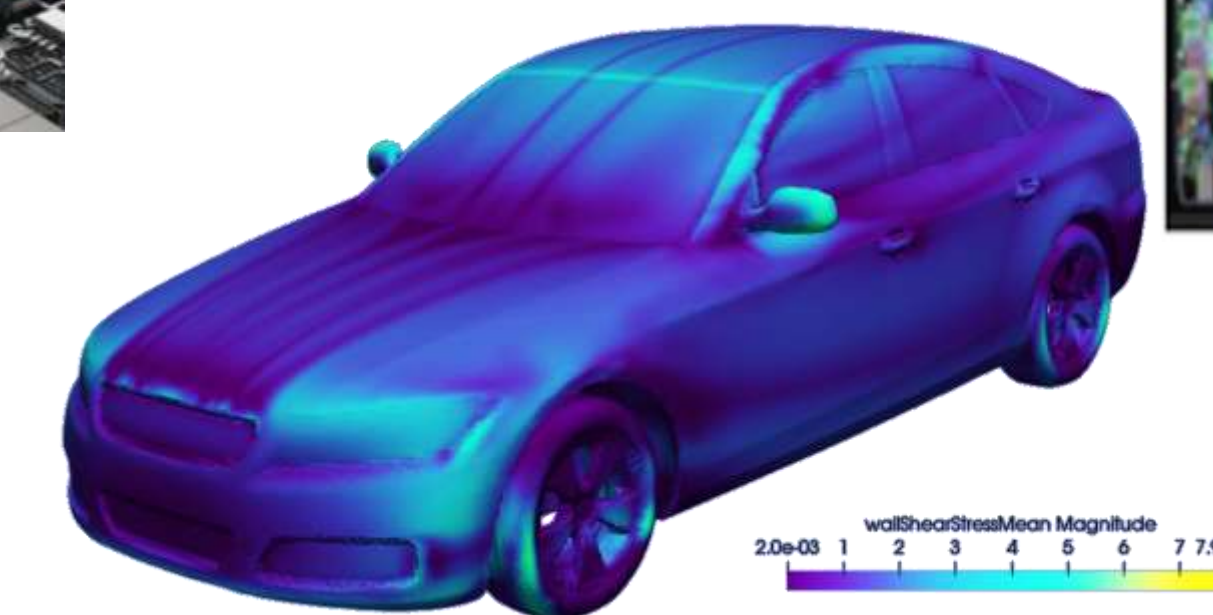


Autonomous
Ride & Handling

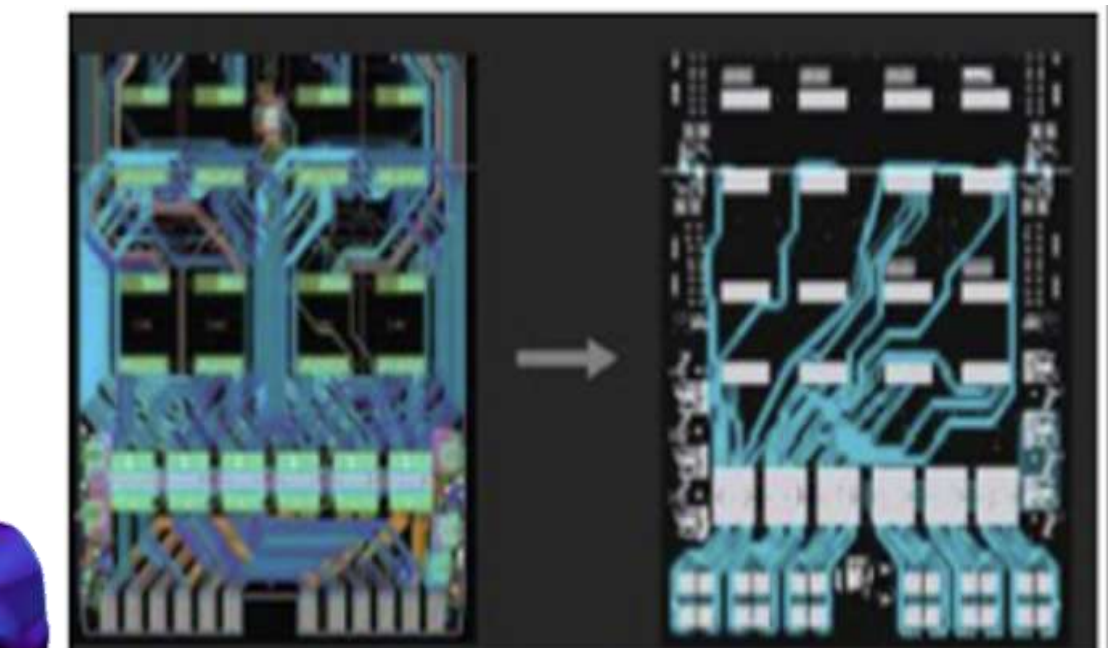


Heatsinks

Aerodynamics



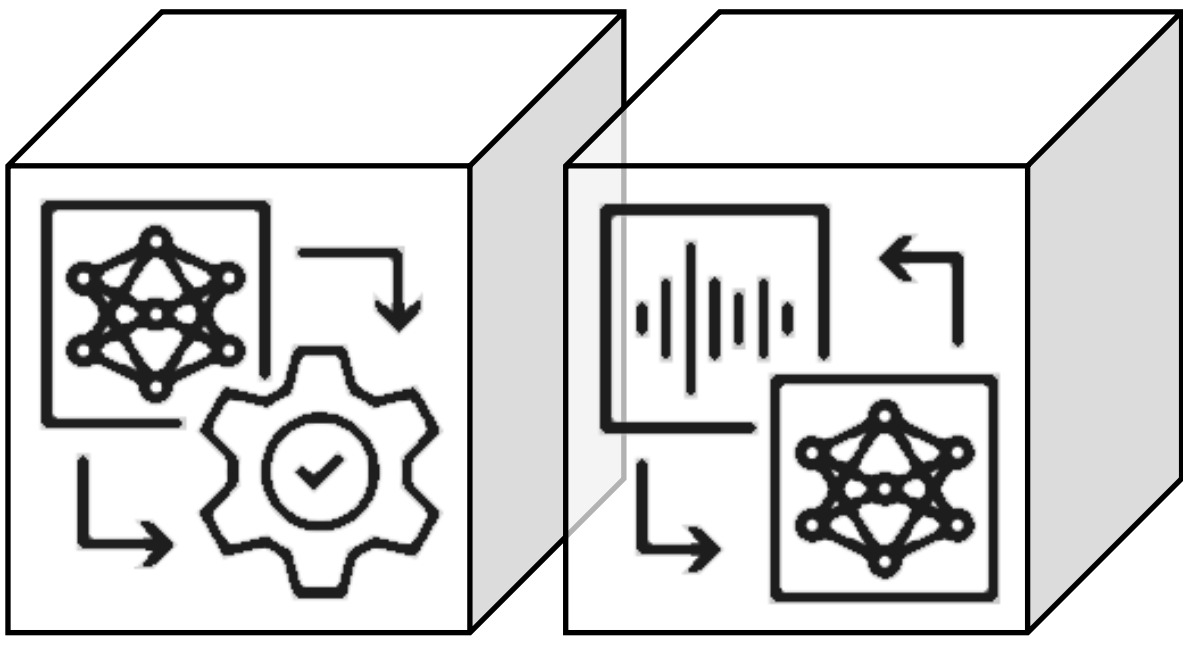
Circuit Design



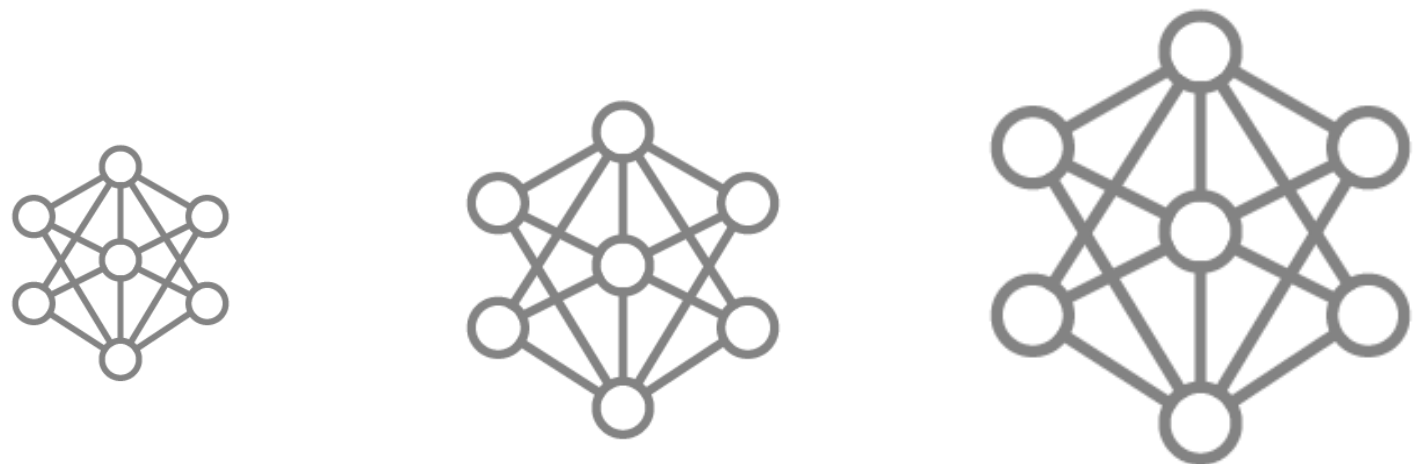
Multiple ways to incorporate AI for Scientific Research and Discovery

NeMo : Generative AI

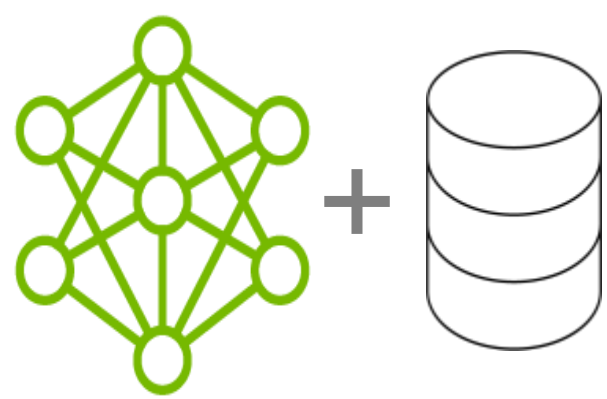
NeMo Framework



Pretrained and Community models
from NGC or HuggingFace



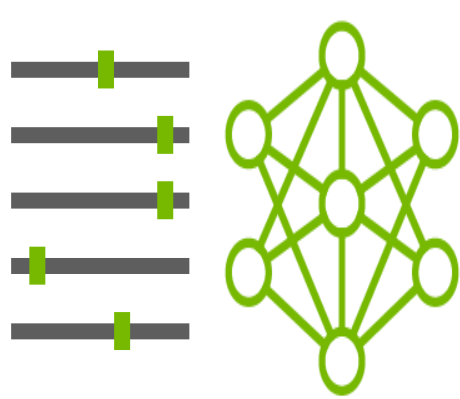
Early development techniques for
researchers/developers



Nemo Inform



Nemo
Guardrails



Nemo
SteerLM

BioNeMo : Drug Discovery

Customer Data



Optimize

Train

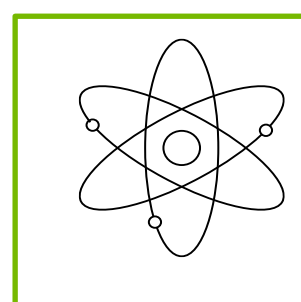
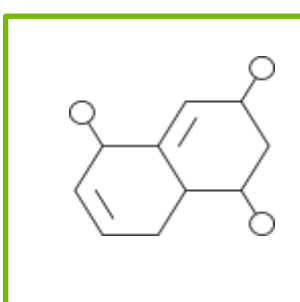
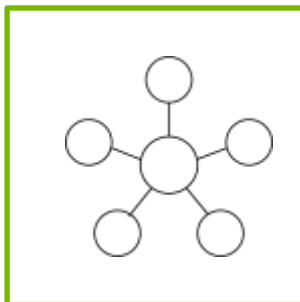
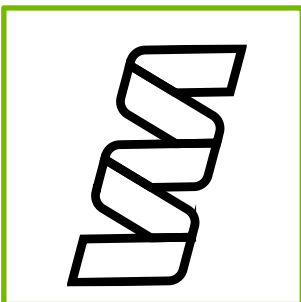
Fine Tune

Inference

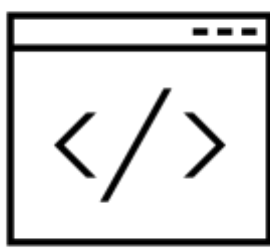
Customer
Model



Available Models: AlphaFold2, OpenFold,
ESMFold, ESM1, ESM2, ProtGPT2,
MEgaMoBART, MoFlow, DiffDock



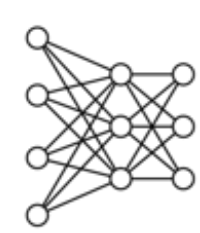
Customer Application



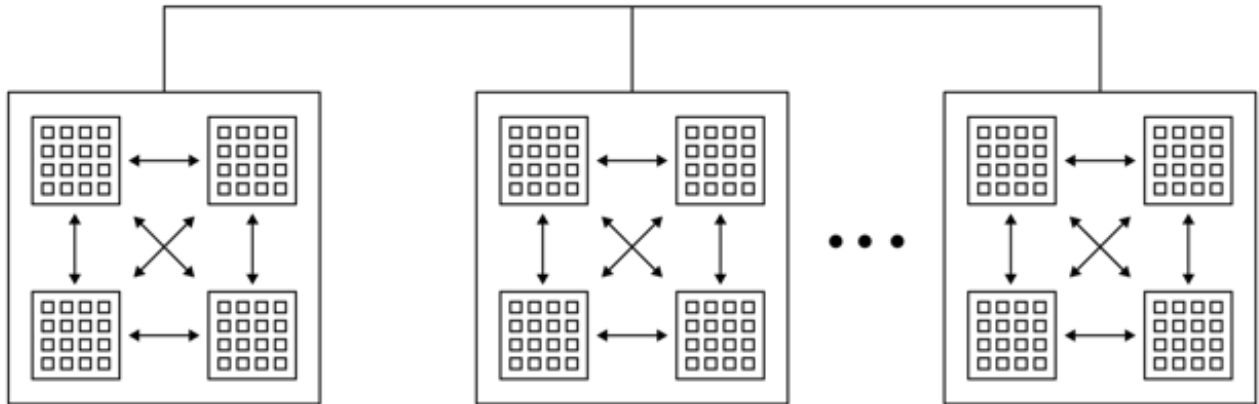
A Cloud Managed Service for
Customize and Run Generative AI for
Computer Aided Drug Discovery

PhysicsNeMo : Physics-Based ML

$$\frac{\partial p}{\partial p} + \nabla \cdot (pu) = 0$$
$$p \frac{Dh}{Dt} = \frac{Dp}{Dt} + \nabla \cdot (k \nabla T) + \Phi$$



$$\begin{matrix} \text{Neural Network} & + & \frac{\partial p}{\partial p} + \nabla \cdot (pu) = 0 \\ \text{Neural Network} & + & p \frac{Dh}{Dt} = \frac{Dp}{Dt} + \nabla \cdot (k \nabla T) + \Phi \end{matrix}$$



Training neural networks using
both data and the governing
equations

NVIDIA PhysicsNeMo

Open-Source platform for developing Physics-ML algorithms

NVIDIA PhysicsNeMo is a neural network framework that blends the power of physics in form of governing partial differential equations (PDEs) with data to build high-fidelity, parameterized surrogate models with near-real-time latency. It offers:

- **AI Toolkit**

- Offers building blocks for developing physics-ML surrogate models

- **Scalable Performance**

- Solves larger problems faster by scaling from single GPU to multi-node implementation

- **Near-Real-Time Inference**

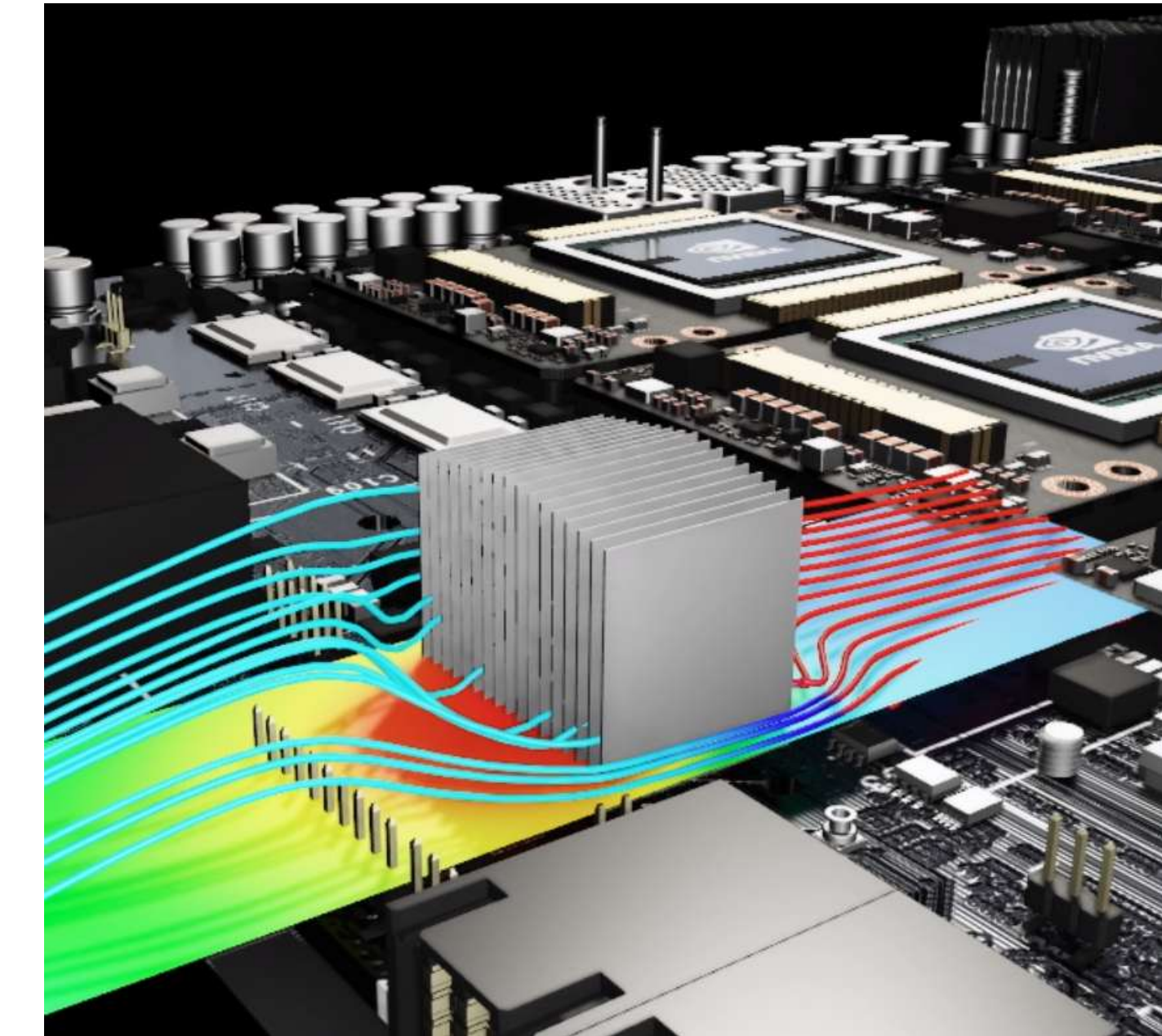
- Provides parameterized system representation that solves for multiple scenarios in near real time, trains once offline to infer in real time repeatedly

- **Easy Adoptability**

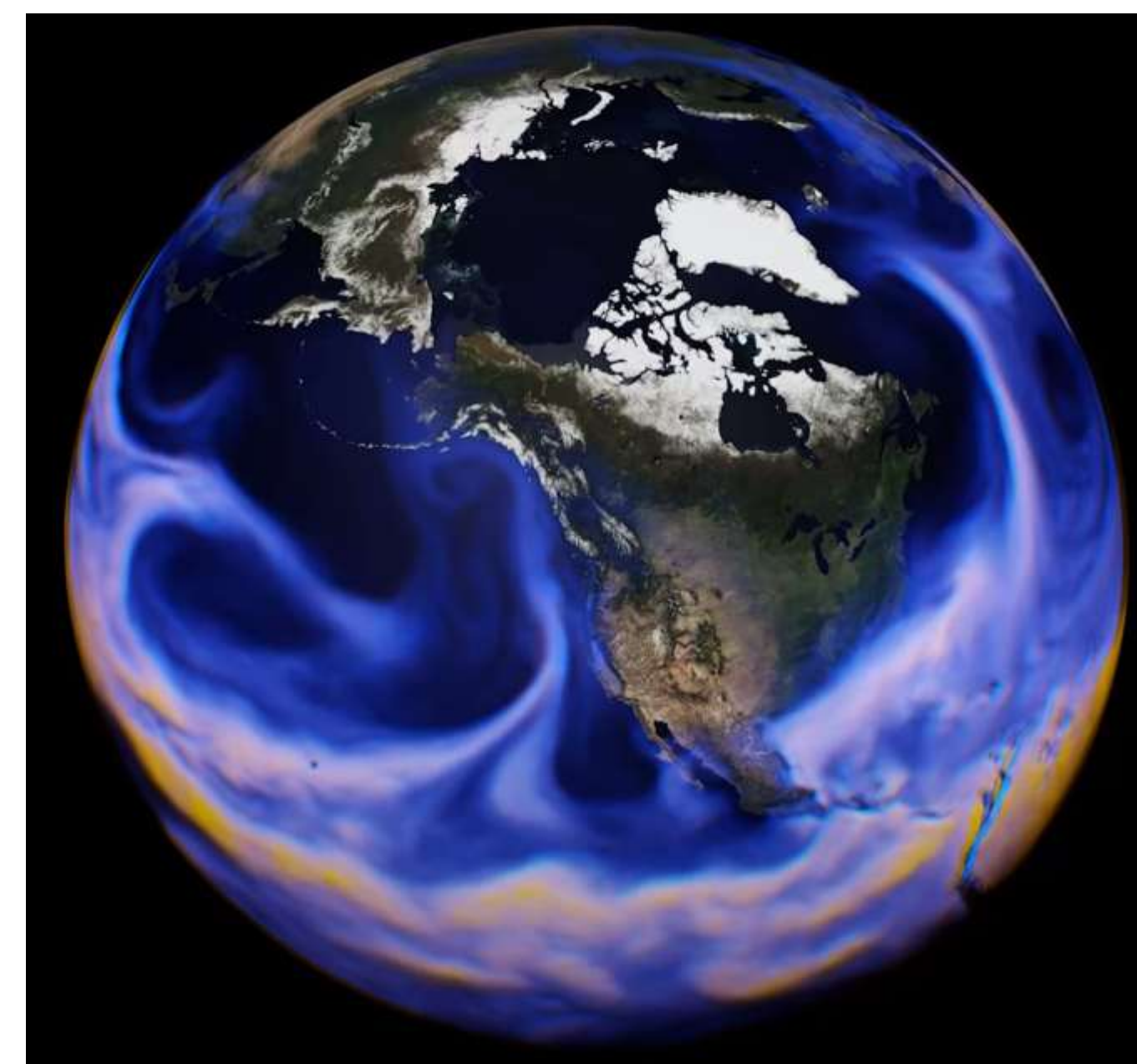
- Includes APIs for domain experts to work at a higher level of abstraction. Extensible to new applications with reference applications serving as starting points



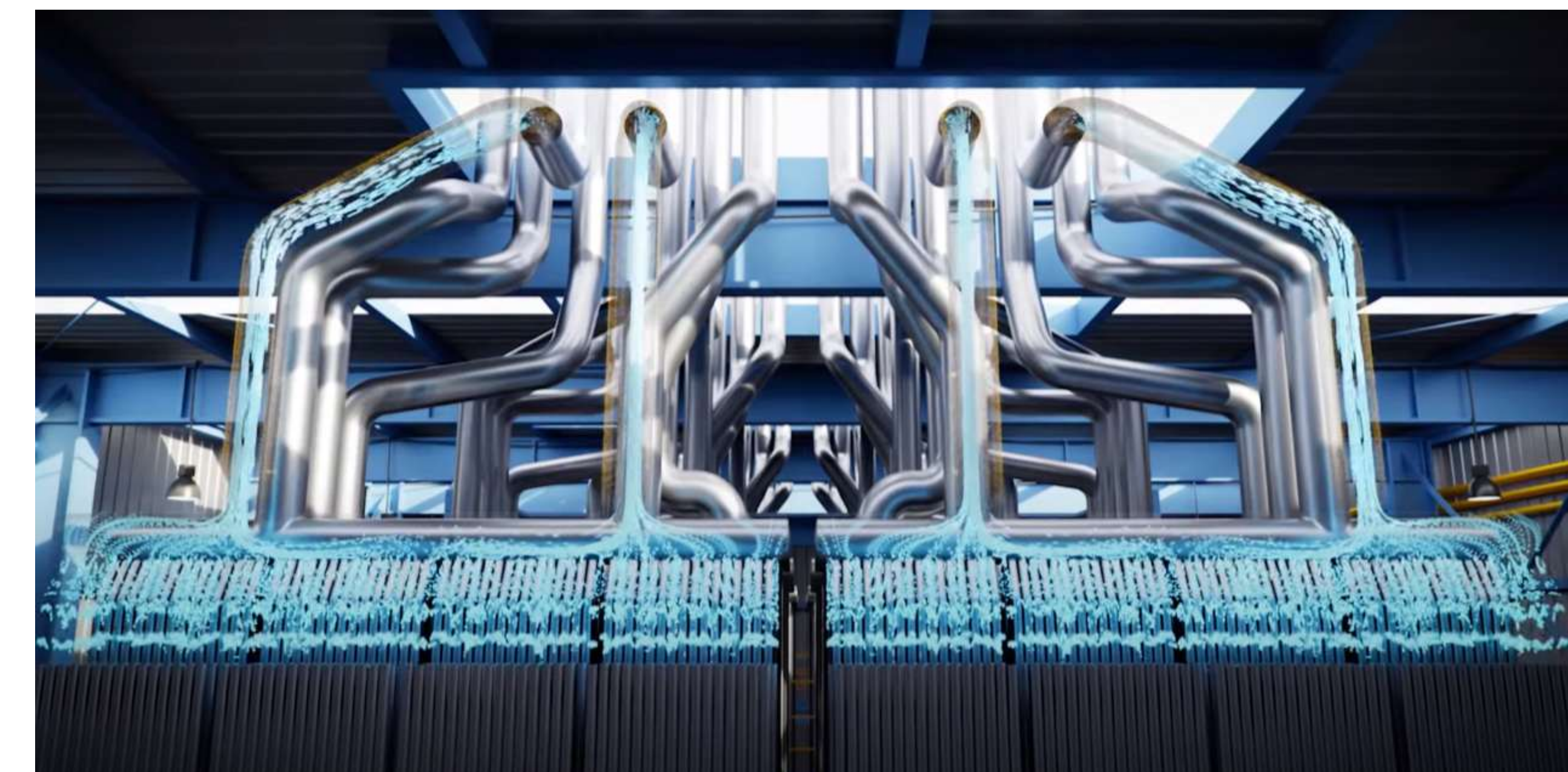
Wind Farm Super Resolution



FPGA Heatsink Design Optimization



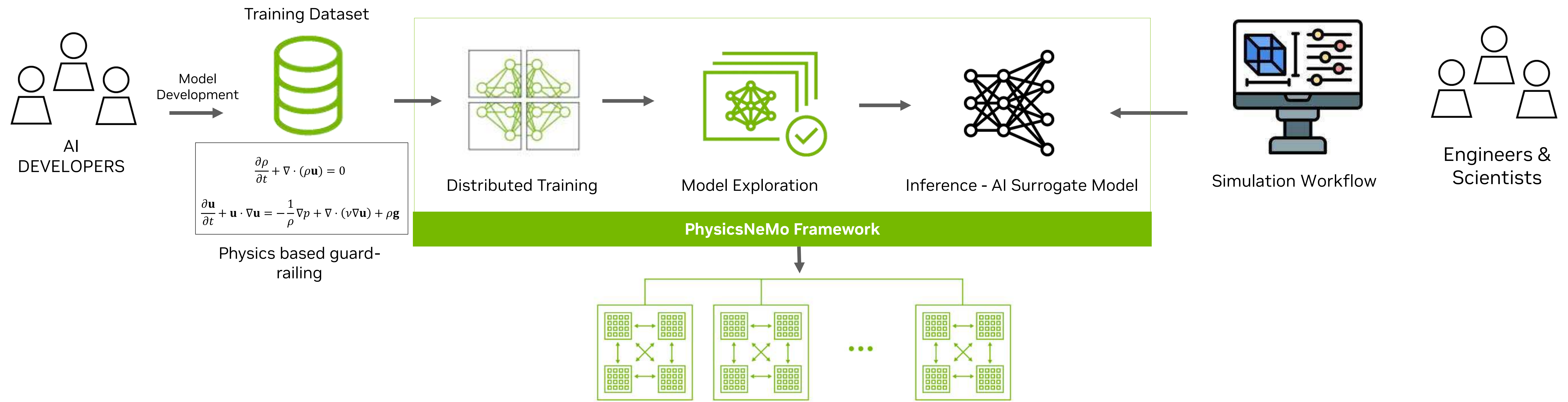
Extreme Weather Prediction



Industrial Digital Twin

PhysicsNeMo framework: Overview

Framework to build and customize Physics-ML models



Multi-domain support

Build physics-ml models for CFD, Heat Transfer, Structural, Electromagnetics, Molecular Dynamics

Optimized Training

Accelerate training and throughput by parallelizing the model and the training data across multi-node.

SOTA Model Architectures

Easily explore physics-ml model architectures – Neural Operators, PINNs, GNNs, Diffusion Models.

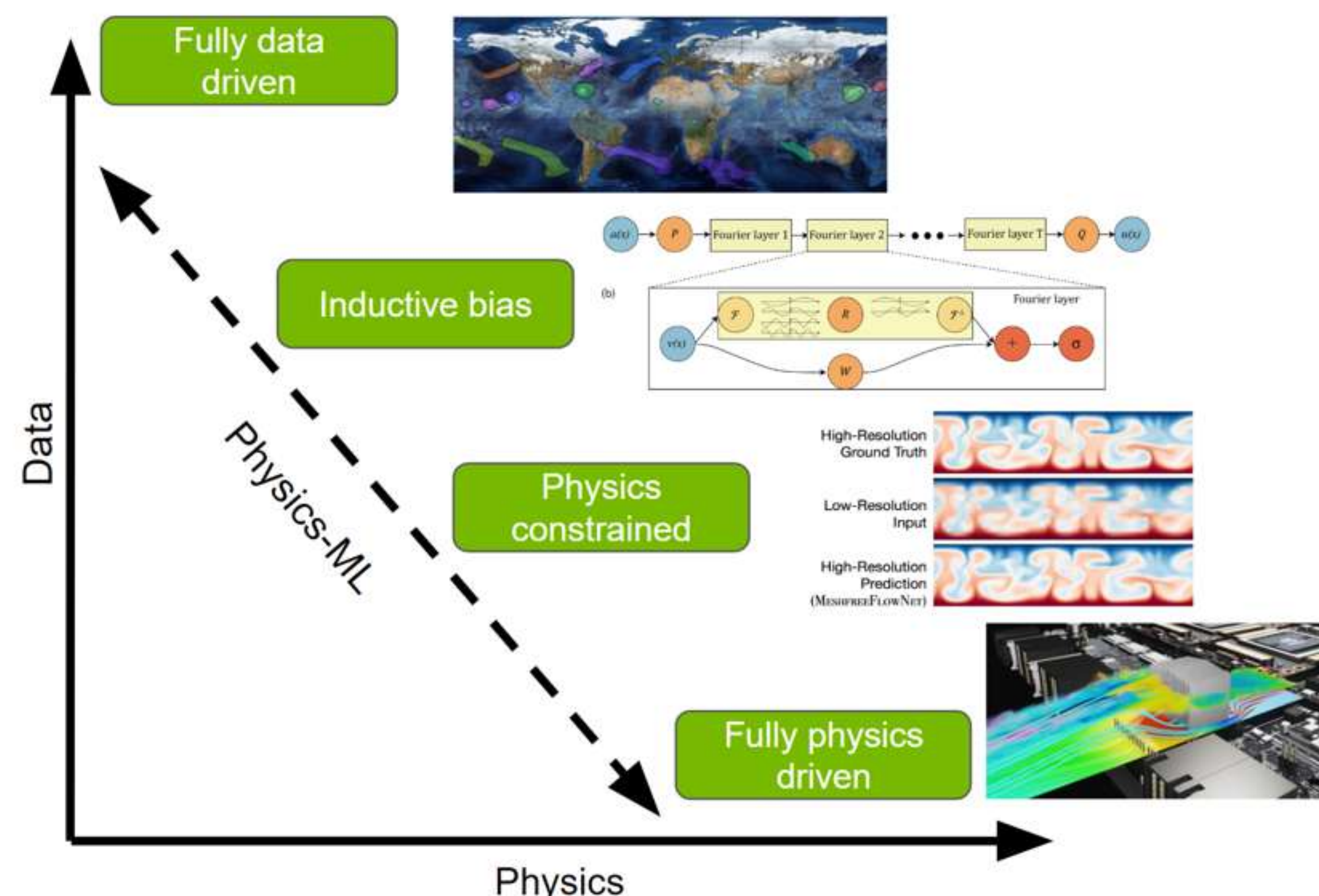
Support

NVIDIA AI Enterprise and experts by your side to keep projects on track

PhysicsNeMo framework: Open-Source AI for Physics-based ML

- PhysicsNeMo Model Zoo - Diverse Physics-ML approaches:
 - Fully Physics driven AI models
 - Fully data driven AI models
 - Hybrid (data + Physics) AI models
- Neural Operators:
 - Fourier Neural Operator family (FNO, AFNO, PINO)
 - DeepONet
- GNNs:
 - GraphCast
 - MeshGraphNet ...
- Diffusion Models:
 - DDPM++
 - NCSN++
 - ADM ...
- PDE informed Neural Networks:
 - Fourier Feature Network
 - Spatial-temporal Fourier Feature Networks
 - SIREN Net ...

- Bringing novel AI architectures that have demonstrated success for engineering and science problems
- Using case studies as reference starting points



How does PhysicsNeMo compliment PyTorch?

Features that can aid data and/or physics driven problems

Data & Physics oriented utils

- Performance Enhancements

CUDA graphs, kernel fusion, JIT compilation, data parallelism, model parallel, etc.

- Pre-built Network Architectures

Diffusion Models, Neural Hash Encoding, Neural Operators, Graph Networks, DCT-RNN, several variants of MLPs etc.

- Hydra Configuration

Hyper-parameter tuning and customization

- Data Pipeline

For very large data-driven problems using Zarr, NVComp, GDS

- Data and Inference Tools

Pre-defined datasets for common data formats (VTK, HDF5, ...). Model export functions to TensorRT and Triton

- Integration with Other Products

Omniverse, PySDF, NVFuser, Triton, Tensor RT, DALI, Warp, etc.

Physics oriented utils

- Geometry Module

Integrated, parameterized geometry module with point cloud/SDF

- Symbolic PDE Loss Construction

Automated PDE loss construction using SymPy API

- Automated Optimized Gradient Calculations

Automatic gradient calculations for physics-informed learning with optimizations such as FuncTorch, AMP16, mesh free derivatives etc.

- Convergence and Stabilization Methods

Mass balance control planes, loss balancing schemes, AdaHessian support, learning rate annealing, etc.

- Exact Boundary Enforcement

Exact enforcement of continuity or boundary conditions

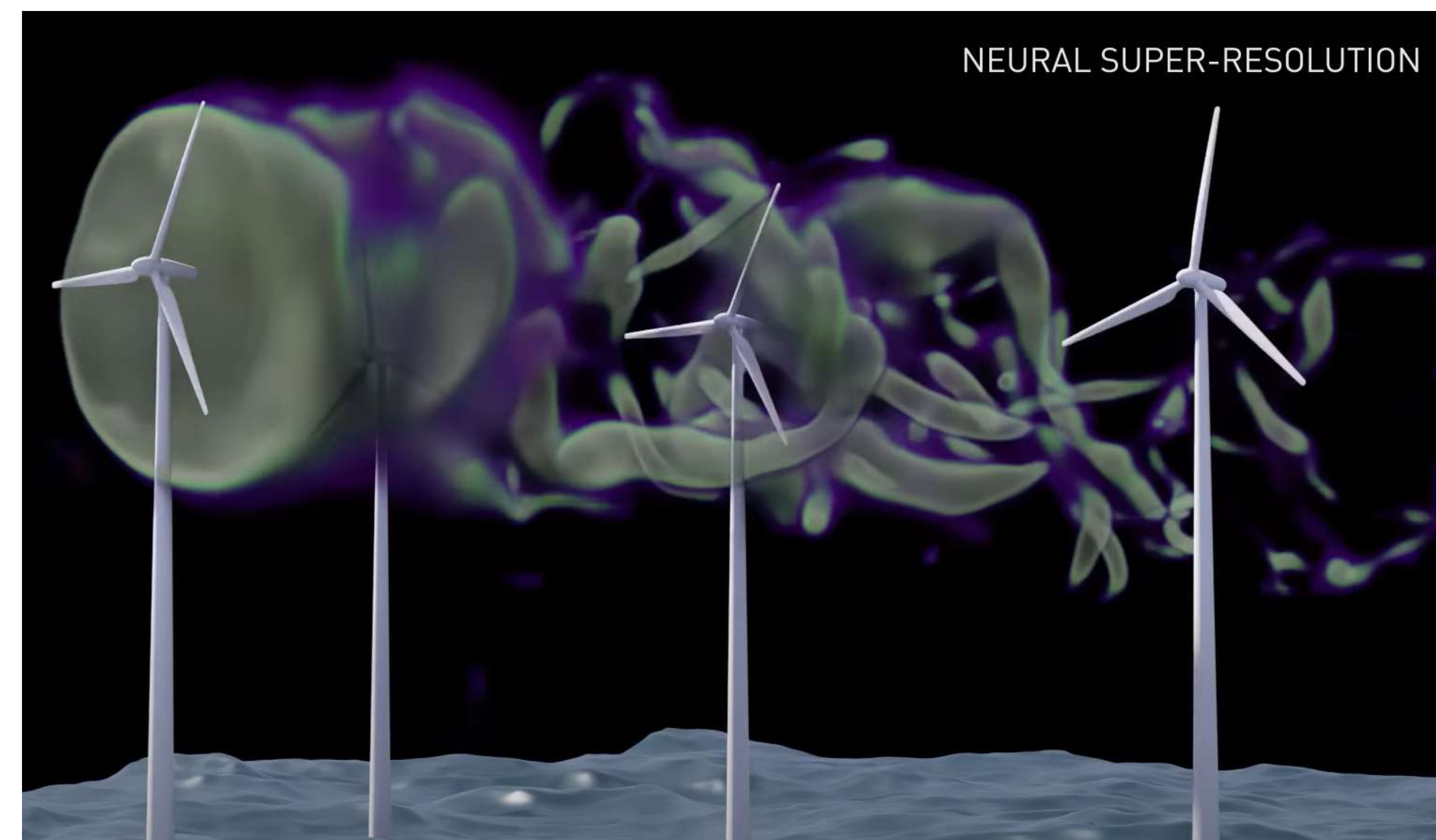
- Variational Learning

Solving PDE systems using variational formulations

PhysicsNeMo

Open-Source on GitHub

- [NVIDIA PhysicsNeMo](https://github.com/NVIDIA/PhysicsNeMo) is an **open-source** framework for building, training and fine-tuning Physics-ML models
- Available open-source on GitHub: <https://github.com/NVIDIA/PhysicsNeMo>



physicsnemo Public

Watch 41 Fork 318 Star 1.4k

main 14 Branches 12 Tags

Go to file Add file Code

ktangsali Update README.md for DOMINO (#846) b5c2377 · yesterday 464 Commits

.github	Post rename updates (#816)	last month
.gitlab	Post rename updates (#816)	last month
docs	Update from_checkpoint docs (#843)	last week
examples	Update README.md for DOMINO (#846)	yesterday
physicsnemo	Cordiff usability and performance enhancements for cust...	last week
test	Cordiff usability and performance enhancements for cust...	last week
.dockerignore	RC fixes 1	last month
.gitattributes	Restore contents that were previously in LFS (#187)	2 years ago
.gitignore	FIGConvUNet model and example (#679)	7 months ago
.gitmodules	23.08 Release preparation (#71)	2 years ago
.markdownlint.yaml	Fix license tests (#324)	last year
.pre-commit-config.yaml	Post rename updates (#816)	last month
CHANGELOG.md	Stormcast Customization (#799)	2 days ago
CITATION.cff	Post rename updates (#816)	last month
CONTRIBUTING.md	Post rename updates (#816)	last month
Dockerfile	Dockerfile Fixes (#835)	2 weeks ago
FAQ.md	Post rename updates (#816)	last month
LICENSE.txt	Github alpha release	2 years ago
Makefile	name change	last month
README.md	Post rename updates (#816)	last month
pyproject.toml	Fixes DeprecationWarning introduced in setuptools>=77 (...)	last week

About

Open-source deep-learning framework for building, training, and fine-tuning deep learning models using state-of-the-art Physics-ML methods

developer.nvidia.com/physicsnemo

machine-learning deep-learning physics pytorch nvidia-gpu

Readme Apache-2.0 license Cite this repository Activity Custom properties 1.4k stars 41 watching 318 forks Report repository

Releases 12

v1.0.1 Latest 3 weeks ago

+ 11 releases

Packages

No packages published

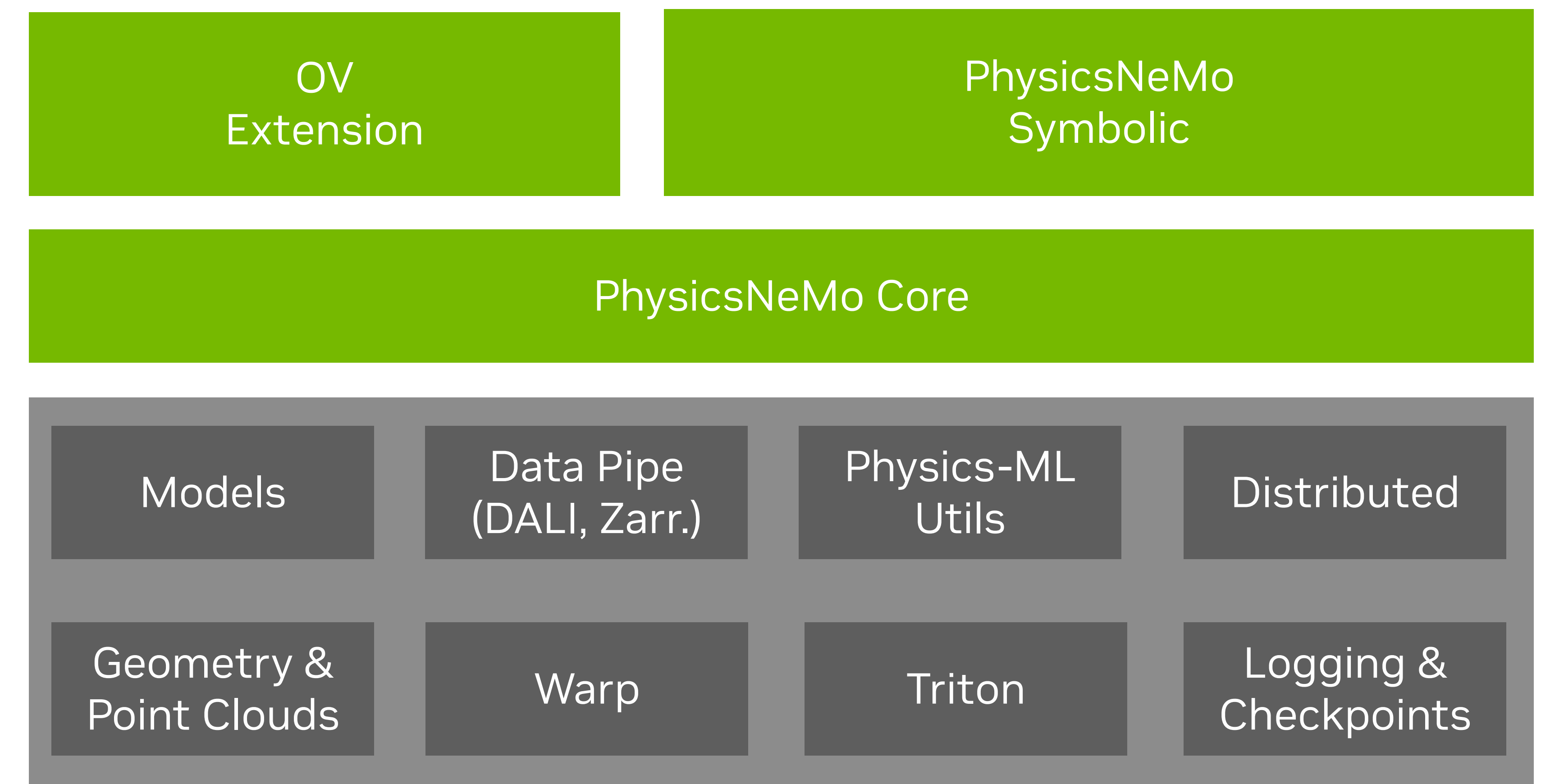
Contributors 62

+ 48 contributors

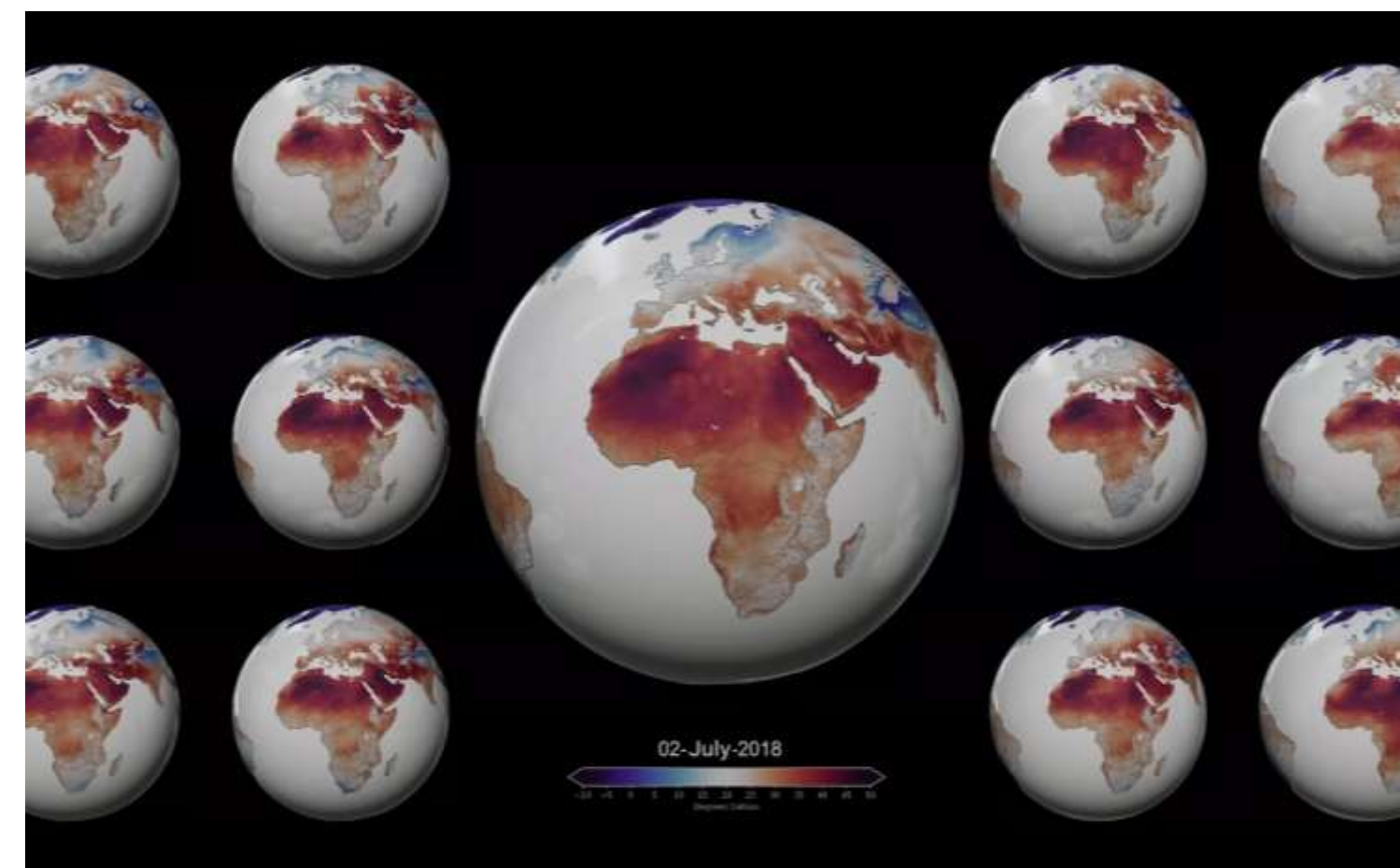
PhysicsNeMo framework: Software stack and accessibility

Modular architecture to support domain experts as well as seasoned developers

- PhysicsNeMo Core provides fundamental and optimized implementations of data pipes, layers, models and utilities to setup distributed training pipelines
- PhysicsNeMo Sym provides abstractions to setup physics ML training with features like geometry module, PDEs, gradient utilities and optimized training loop
- OV extension allows easy export of models to NVIDIA Omniverse for visualization
- General availability via PyPi, NGC Container registry and open source on GitHub.



Physics-ML Success stories – PhysicsNeMo Case Studies



Weather modeling

Demo: [Link 1](#), [Link 2](#), [Link 3](#)



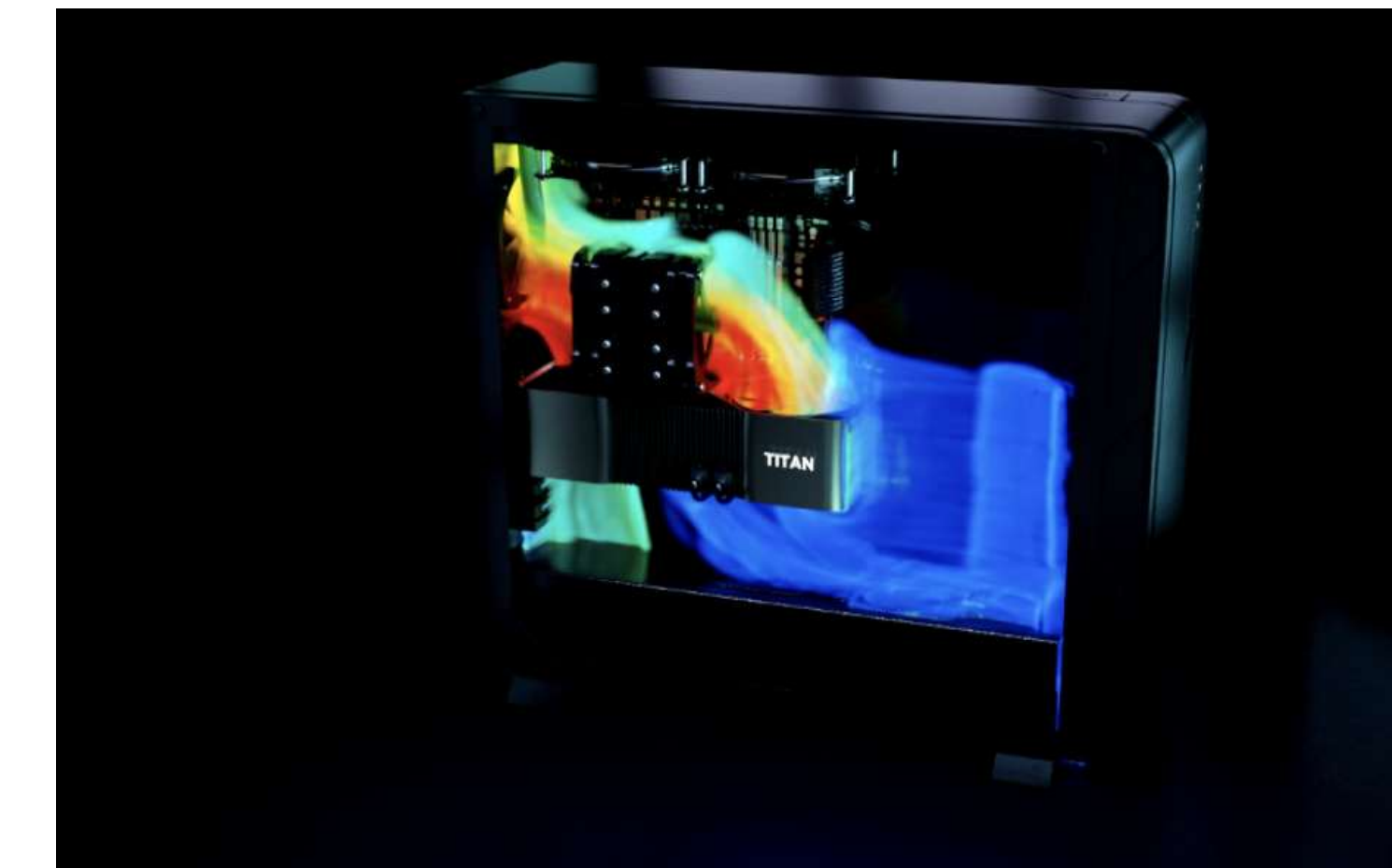
Wind farm Super Resolution

Demo: [Link](#), Blog: [Link](#)



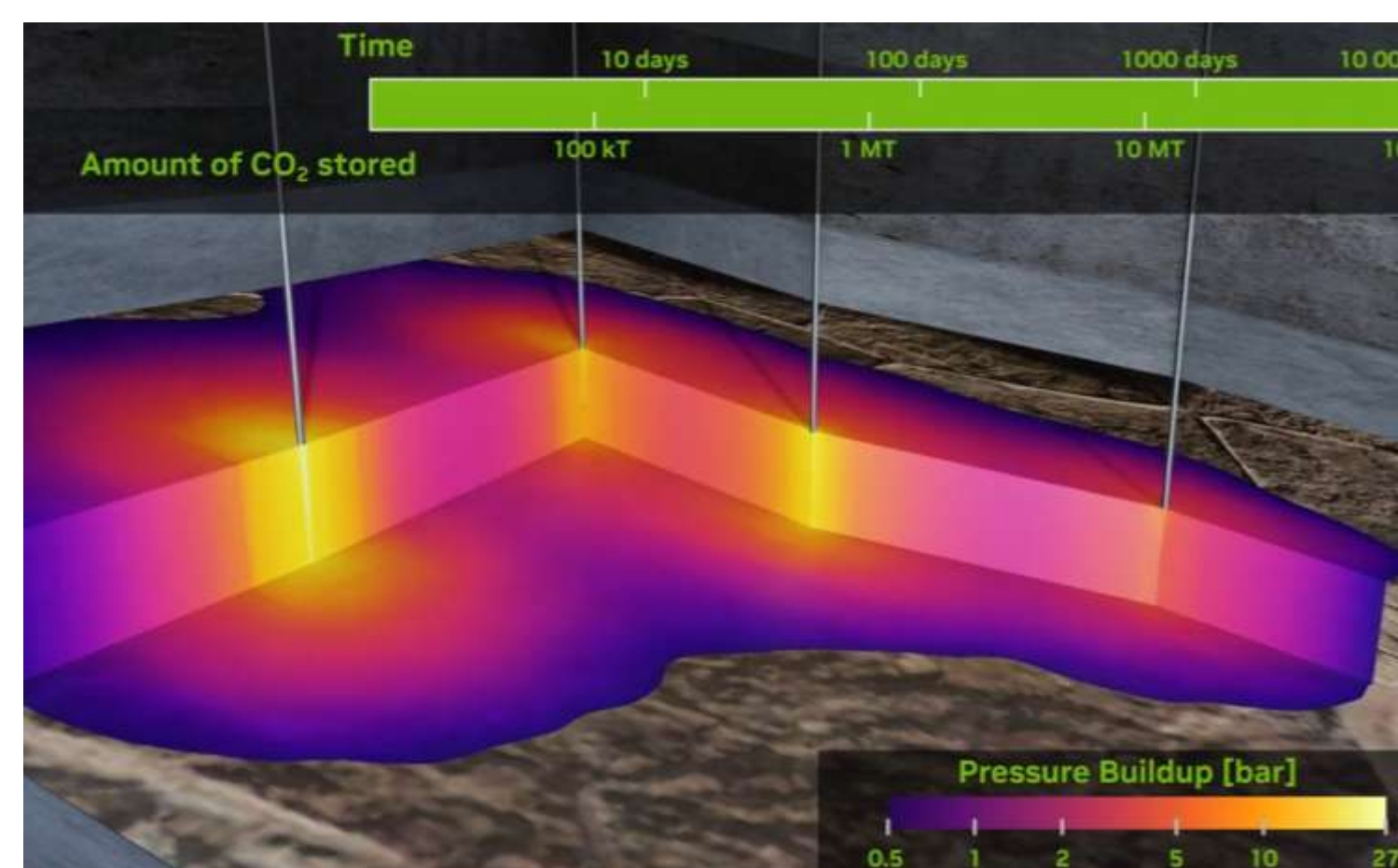
Automotive CFD

[GTC 2024 Keynote](#)



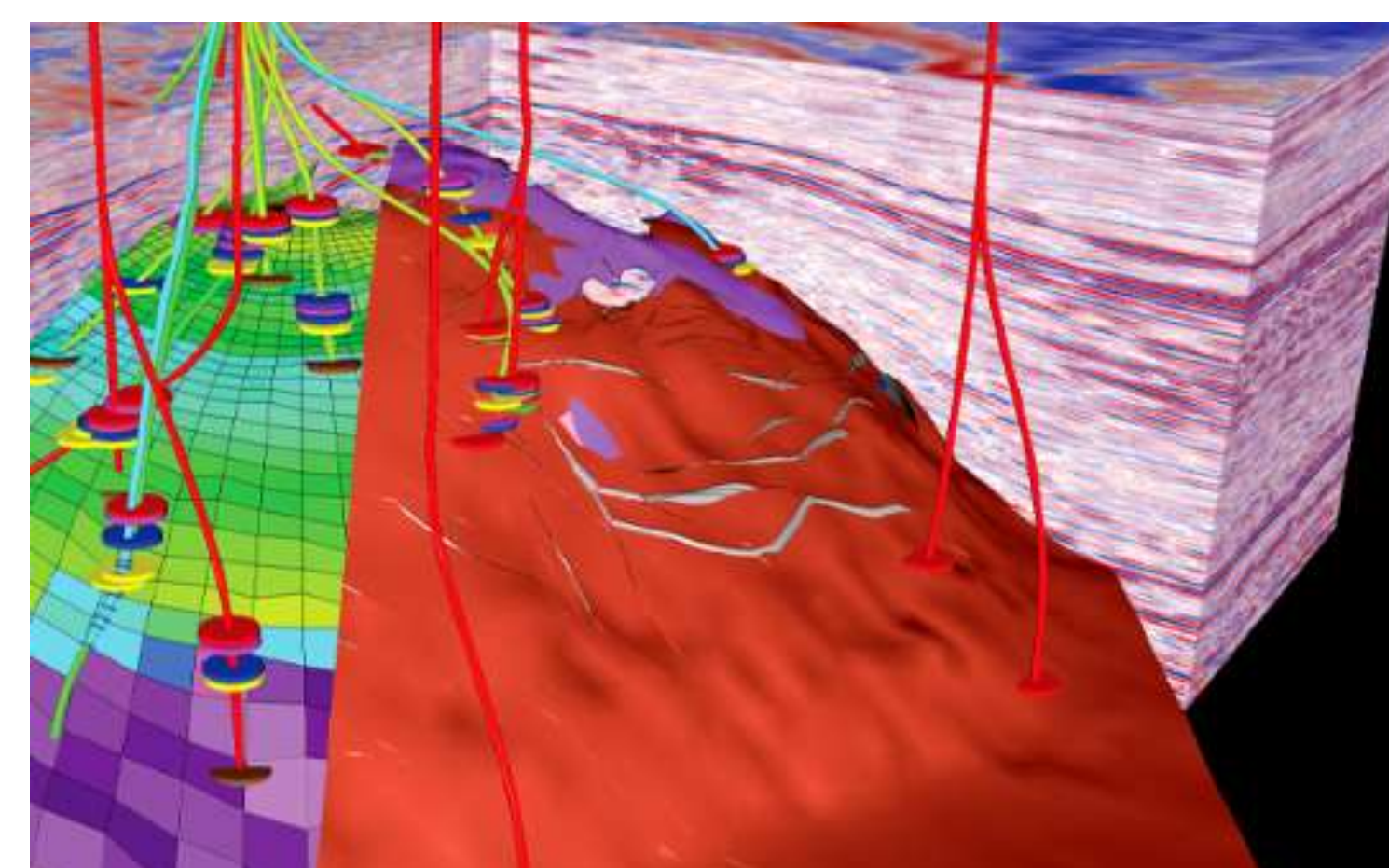
RTX 4090 heat sink design

Demo: [Link](#)



Carbon capture and storage

Demo: [Link](#), Blog: [Link](#)



Sub surface simulations

Resource: [Link](#)



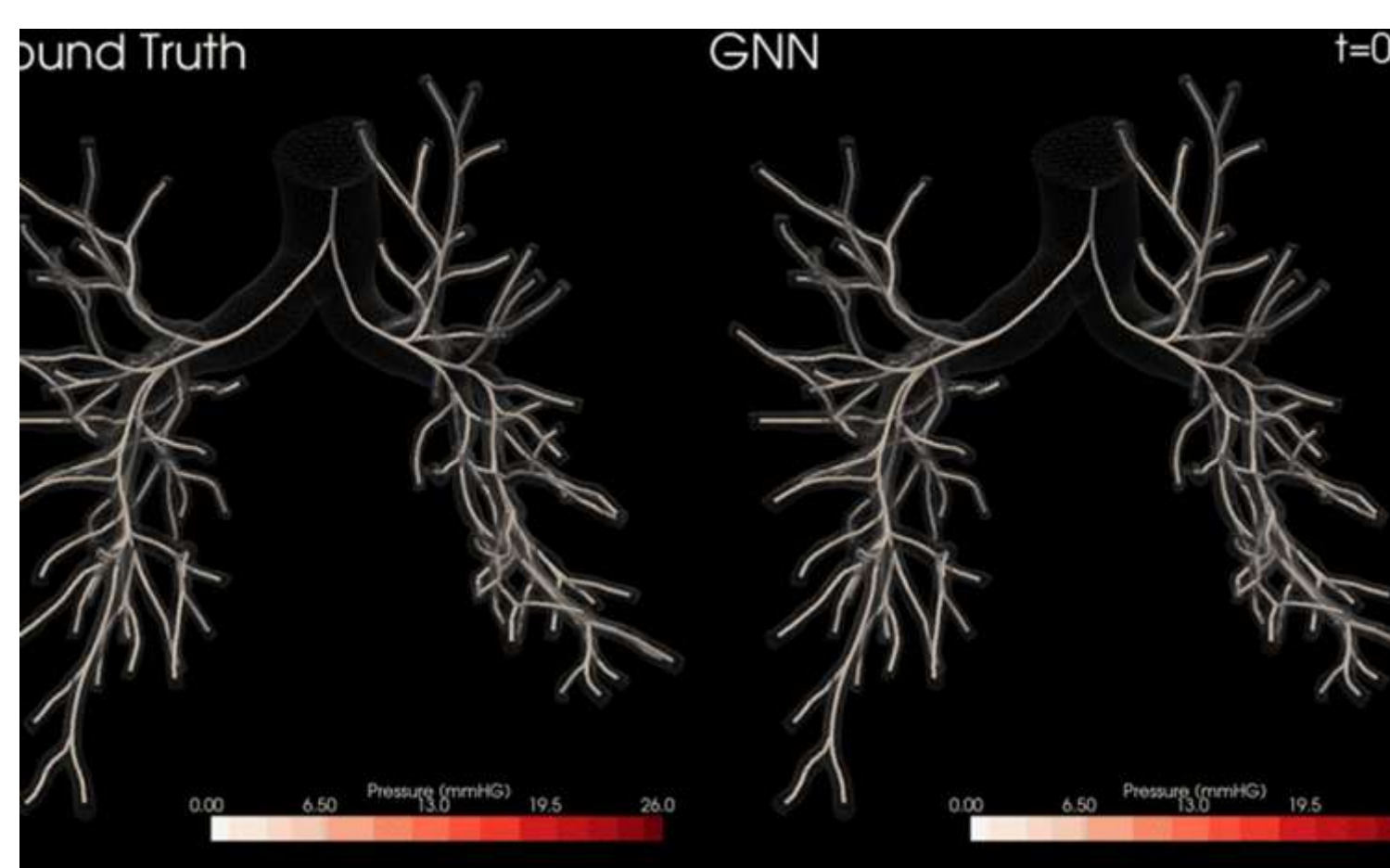
Data Center Digital Twin

Blog: [Link](#), GTC Session: [Link](#)



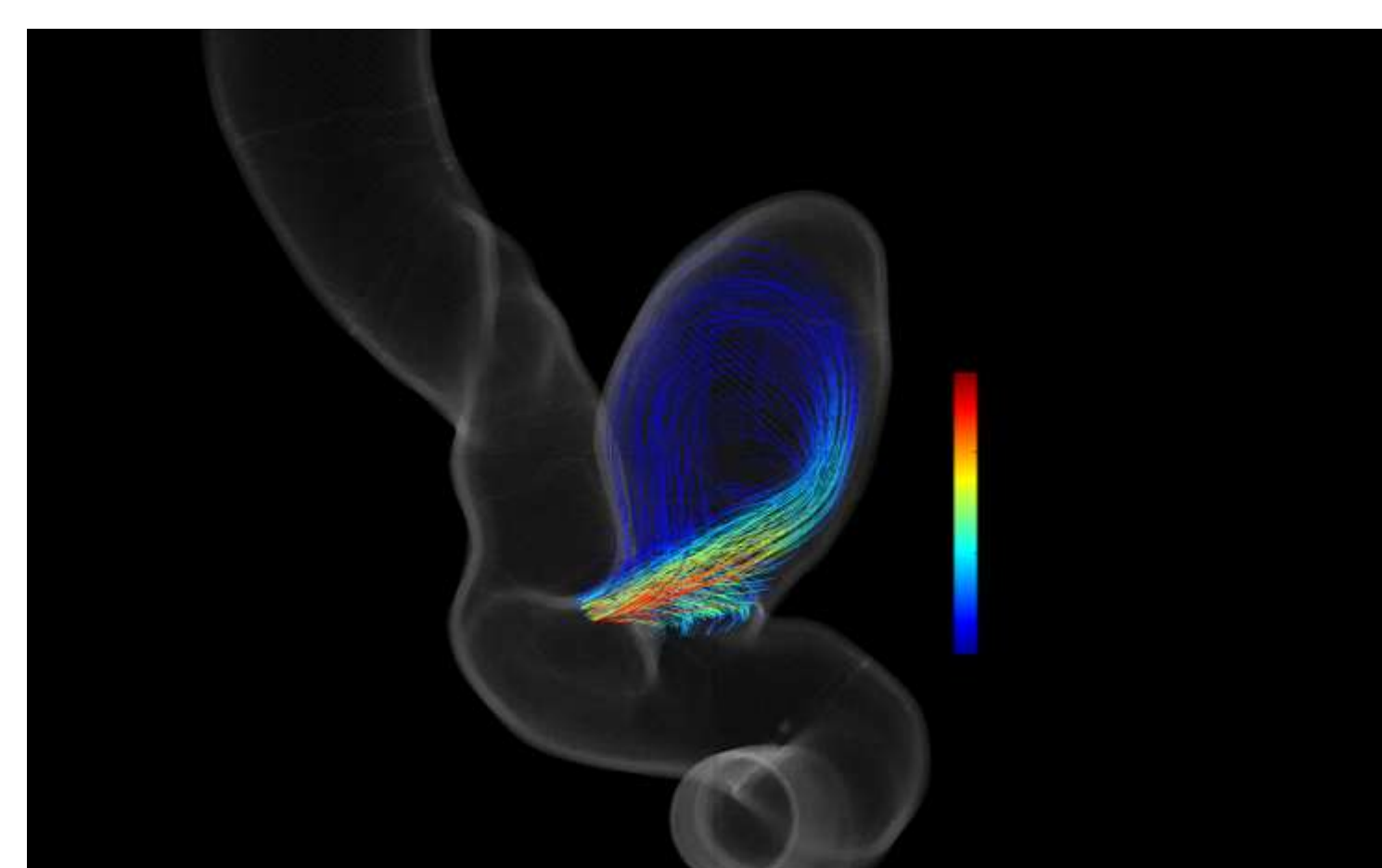
HRSG Digital Twin

Demo: [Link](#), GTC Session: [Link](#)



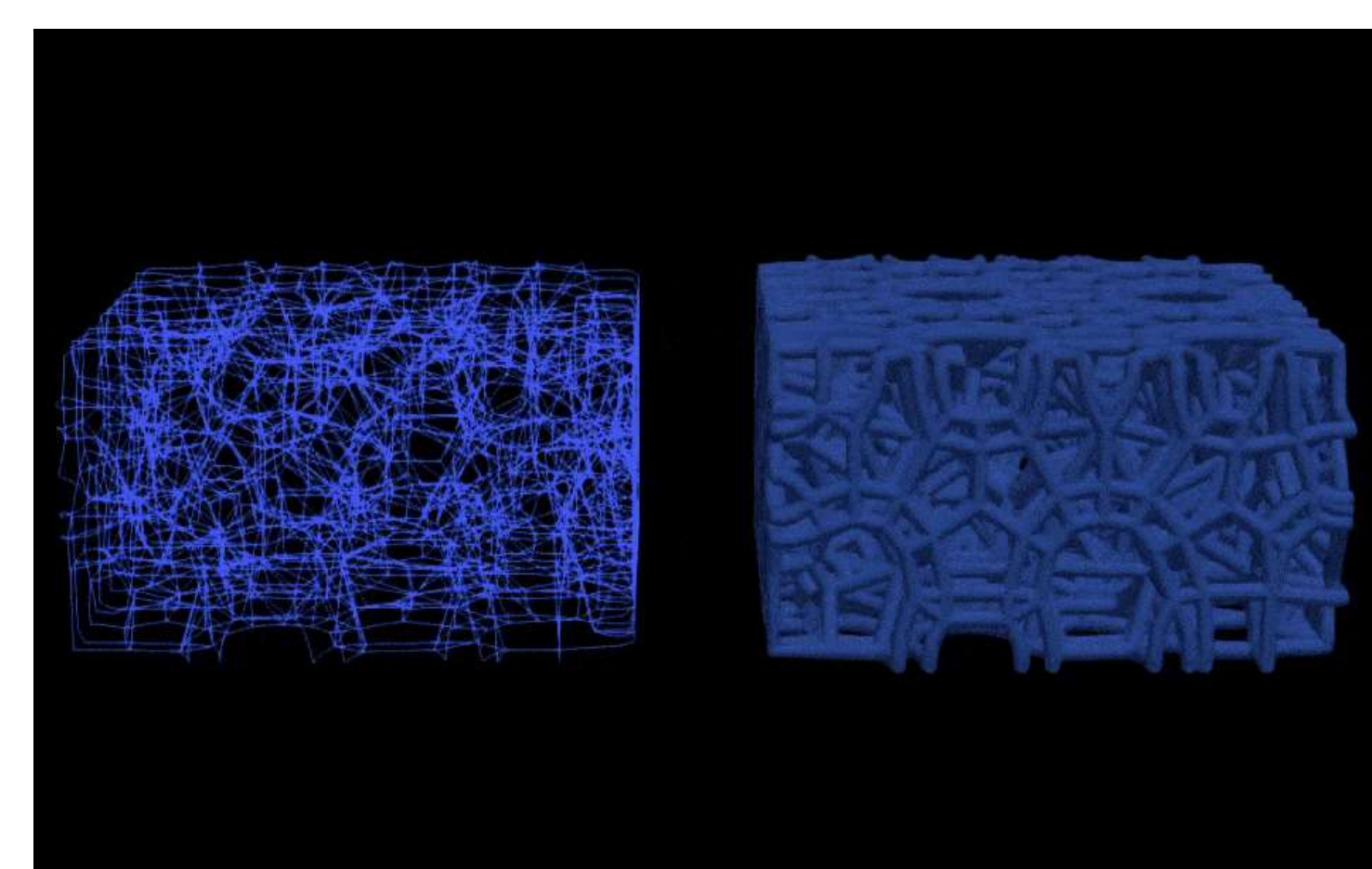
Cardiovascular Simulation

Blog: [Link](#)



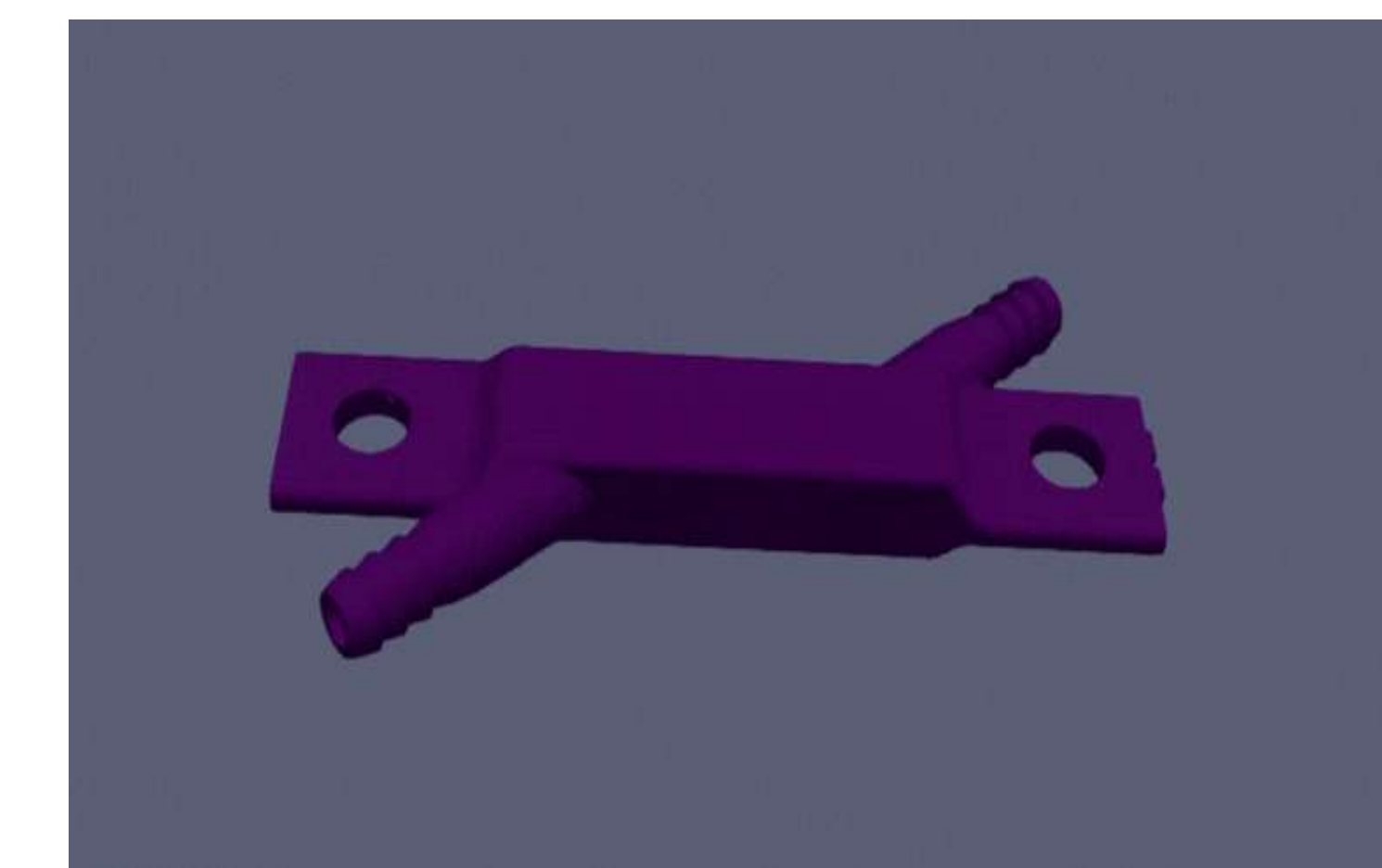
Brain Aneurysm Simulation

Demo: [Link](#)



Additive Manufacturing: Lattice Simulation

Blog: [Link](#)



Additive Manufacturing: 3D Printing

Blog: [Link](#)

PhysicsNeMo access

Availability via PyPi, NGC Container registry and GitHub

- Download [PhysicsNeMo Docker Container](#)

```
docker run --gpus all -it nvidia/physicsnemo/physicsnemo:25.03 bash
```

- Download [PhysicsNeMo via PyPi](#)

```
pip install nvidia-physicsnemo nvidia-physicsnemo.sym
```

- Develop using PhysicsNeMo ([OSS on GitHub](#))

```
git clone https://github.com/NVIDIA/physicsnemo.git
```

PhysicsNeMo NGC resources

Pre-trained checkpoints, datasets and resources available via NGC to enable scientific exploration

Models

The NGC catalog offers 100s of pre-trained models for computer vision, speech, recommendation, and more. Bring AI faster to market by using these models as-is or quickly build proprietary models with a fraction of your custom data.



physicsnemo

Relevance

Use Case

Forecasting3

Simulation and Modeling3

NVIDIA NIM1No results found

NVIDIA Platform

PhysicsNeMo3

PyTorch1

NVIDIA EnterpriseNo results found

Framework

Industry

SolutionNo results found

Publisher

Displaying 3 models



PhysicsNeMo Checkpoints: A...

This deep learning model, based on the Adaptive Fourier Neural Operator framework (AFNO),...



PhysicsNeMo Checkpoints: A...

This deep learning model, based on the Adaptive Fourier Neural Operator framework (AFNO),...



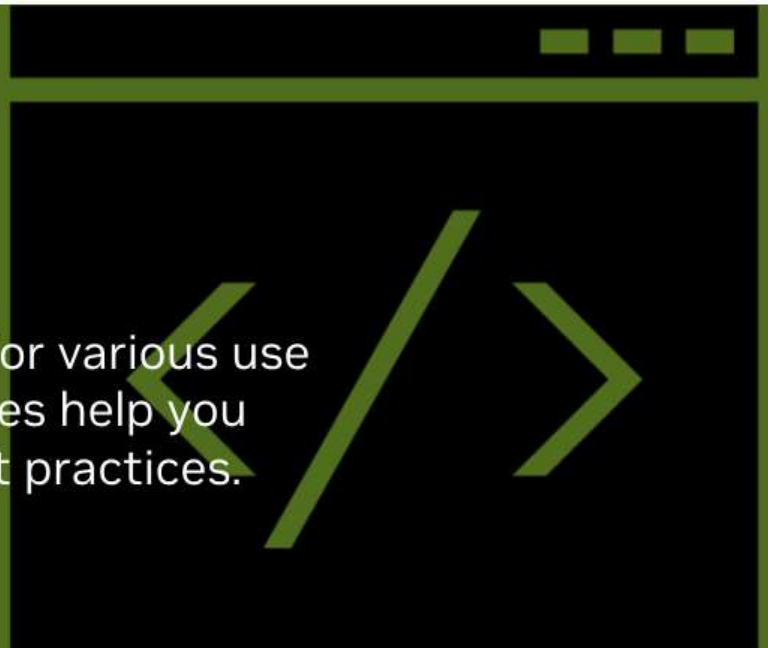
PhysicsNeMo Checkpoints: A...

This deep learning model, based on the Adaptive Fourier Neural Operator framework (AFNO),...

[PhysicsNeMo Models on NGC](#)
Pre-trained model packages

Resources

The NGC catalog offers step-by-step instructions and scripts through Jupyter Notebooks for various use cases, including machine learning, computer vision, and conversational AI. These resources help you examine, understand, customize, test, and build AI faster, while taking advantage of best practices.



physicsnemo

Relevance

Use Case

Simulation and Modeling1

NVIDIA Platform

PhysicsNeMo1

NVIDIA NIM1No results found

NVIDIA Enterprise

NVIDIA AI Enterprise Supported1

NVIDIA AI Enterprise Essentials1

FrameworkNo results found

Industry

SolutionNo results found

Displaying 1 resources



NVIDIA PhysicsNeMo Datacen...

Dataset and reference OpenFOAM setup for Datacenter CFD surrogate model training using PhysicsNeMo.

NVIDIA AI Enterprise Essentials

NVIDIA AI Enterprise Supported

[PhysicsNeMo Resources on NGC](#)
Datasets and supplemental materials

This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

29

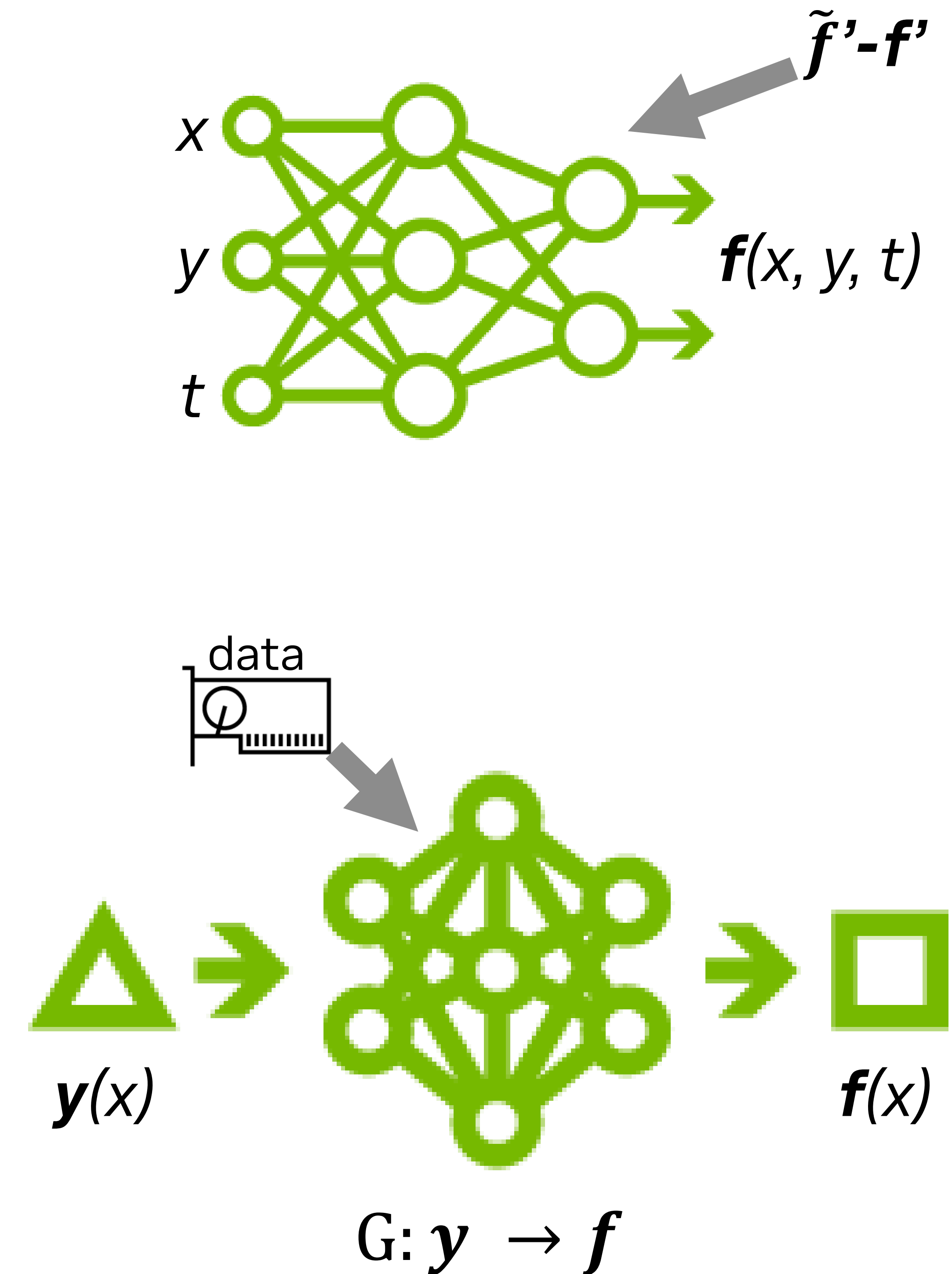


PINNs: Physics-Driven Approach

PINNs and Operators

Two broad frameworks

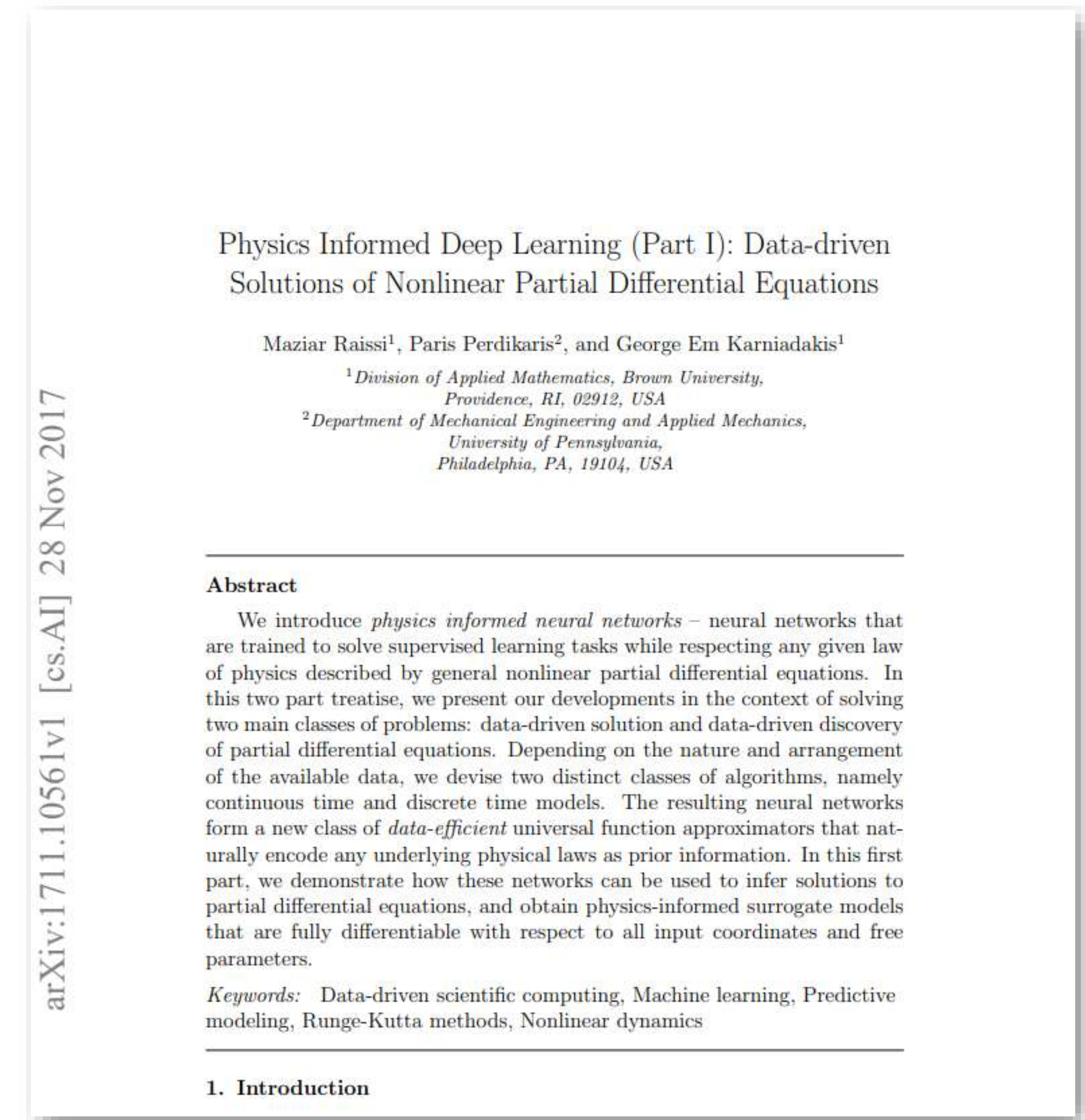
- There's *MANY* really exciting AI for science programmes of research going on today
- Today we're going to touch on two *broad frameworks* of such approaches – PINNs and Neural Operators
- Want to
 - Give you the fundamental ideas behind each approach
 - Give you enough resources that you can start digging into one, both, or the broader effort
 - Let you know that not only are these exciting tools for approaching PDEs, they are interesting objects of *research in and of themselves*
 - Much of the study of PDEs and of numerical analysis can (and is being) applied to these frameworks
- We'll leave you with resources and some places to get started.



Physics Informed Neural Networks - PINNs

Naïve NN + Autodifferentiation + Physics loss

- Physics Informed Neural Networks – PINNs
- Papers that really launched them: Raissi, Perdikaris & Karniadakis
- Application of two basic properties of neural networks (and the toolchains around them)
 - Universal function approximation, and
 - Differentiability/auto-differentiation



References:

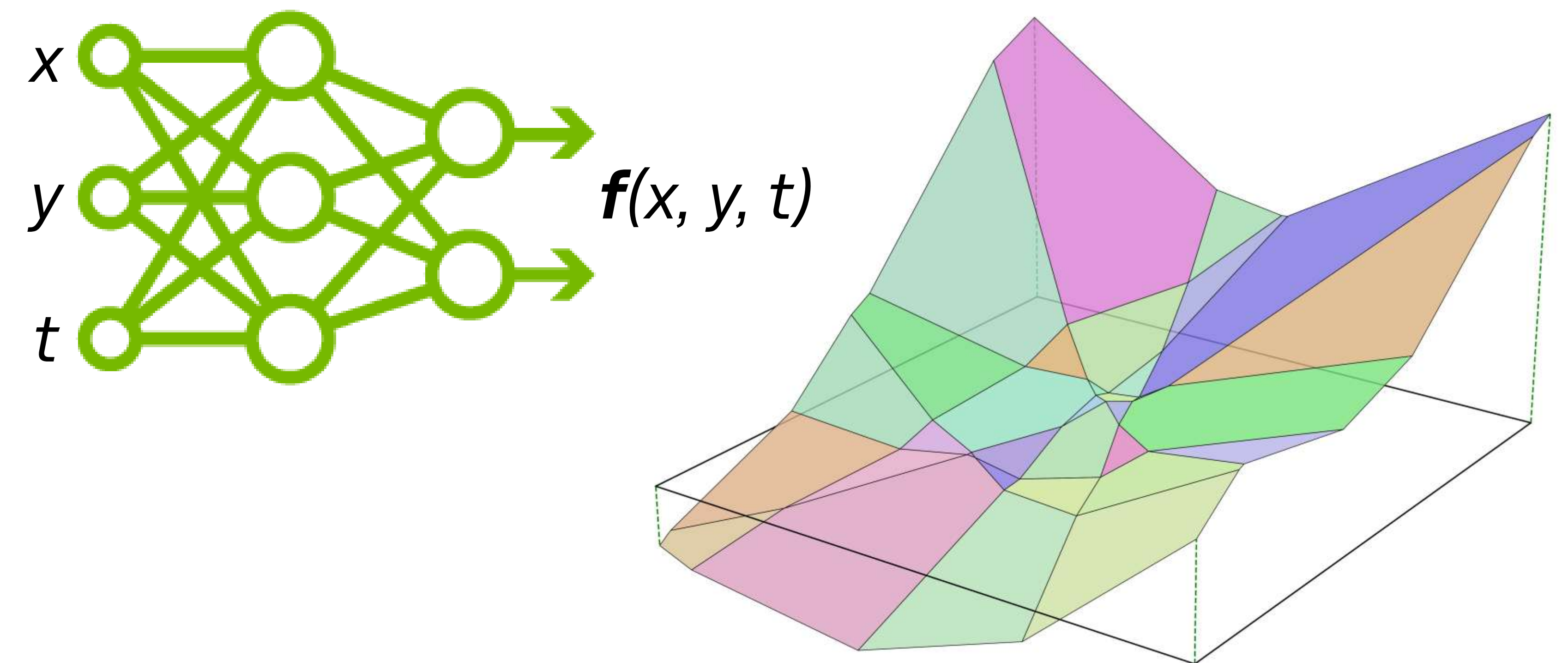
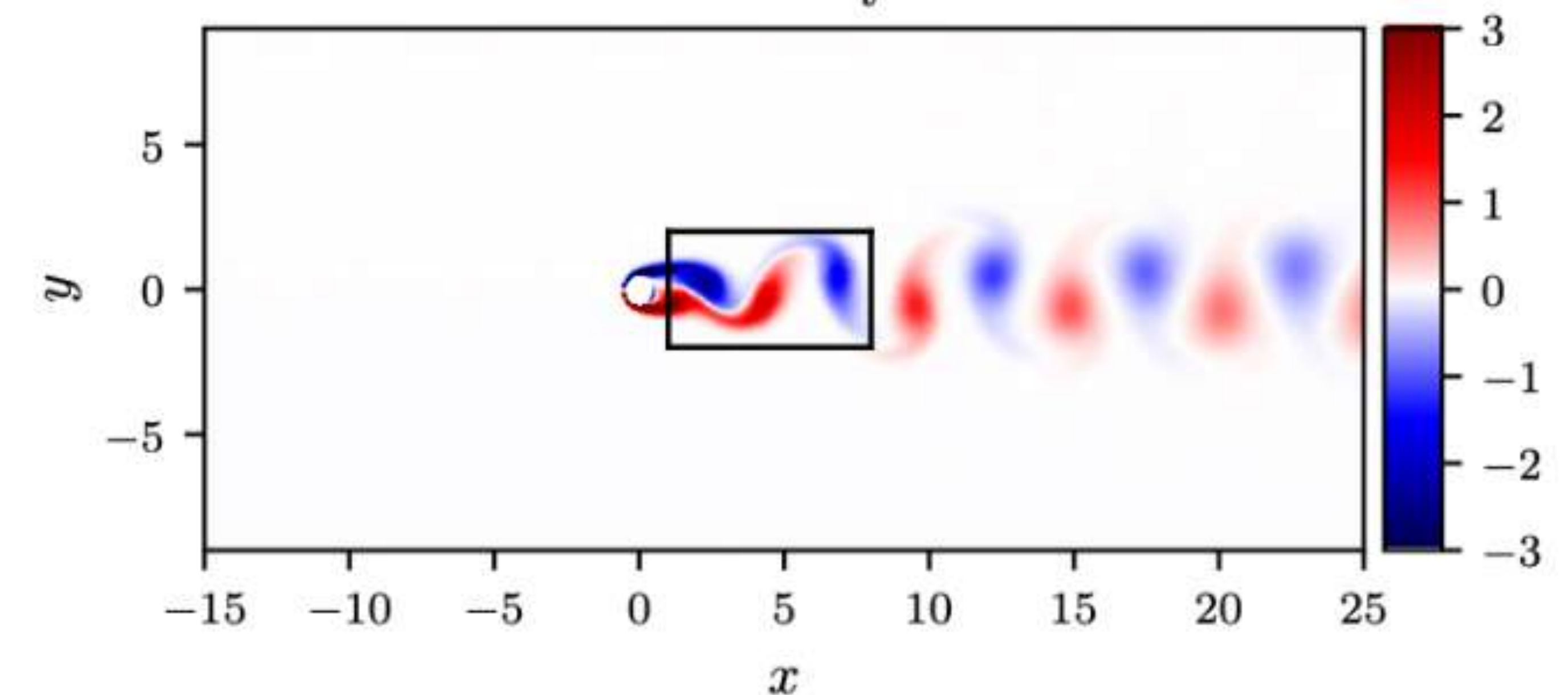
- Raissi, Perdikaris, & Karniadakis (2017a) [arXiv:1711.10561](https://arxiv.org/abs/1711.10561)
- Raissi, Perdikaris, & Karniadakis (2017b) [arXiv:1711.10566](https://arxiv.org/abs/1711.10566)
- Raissi, Perdikaris, & Karniadakis (2019) [JCP 378:686-707](https://doi.org/10.1016/j.jcp.2018.06.024)

Universal Function Approximation

Use a NN as an approximation function

- When we're solving an instance of a PDE, we're looking for a function
- But multi-layer perceptrons can approximate arbitrary functions, given “enough” (surprisingly few) nodes, under various constraints given the details of the function, activations, etc.
- If this is new to you, looking into [Universal Approximation Theorem\(s\)](#) is a good starting point for the mathematics minded, or for the computational physics crowd (like me), I like a [post on Jane Street's blog using ReLU activations to model arbitrary piecewise-linear functions](#)
- Other kinds of networks can do this too! e.g. [Kolmo.gorov-Arnold Networks](#) This is part of the broad research efforts around PINN-type methods

Raissi, Perdikaris, & Karniadakis (2019)

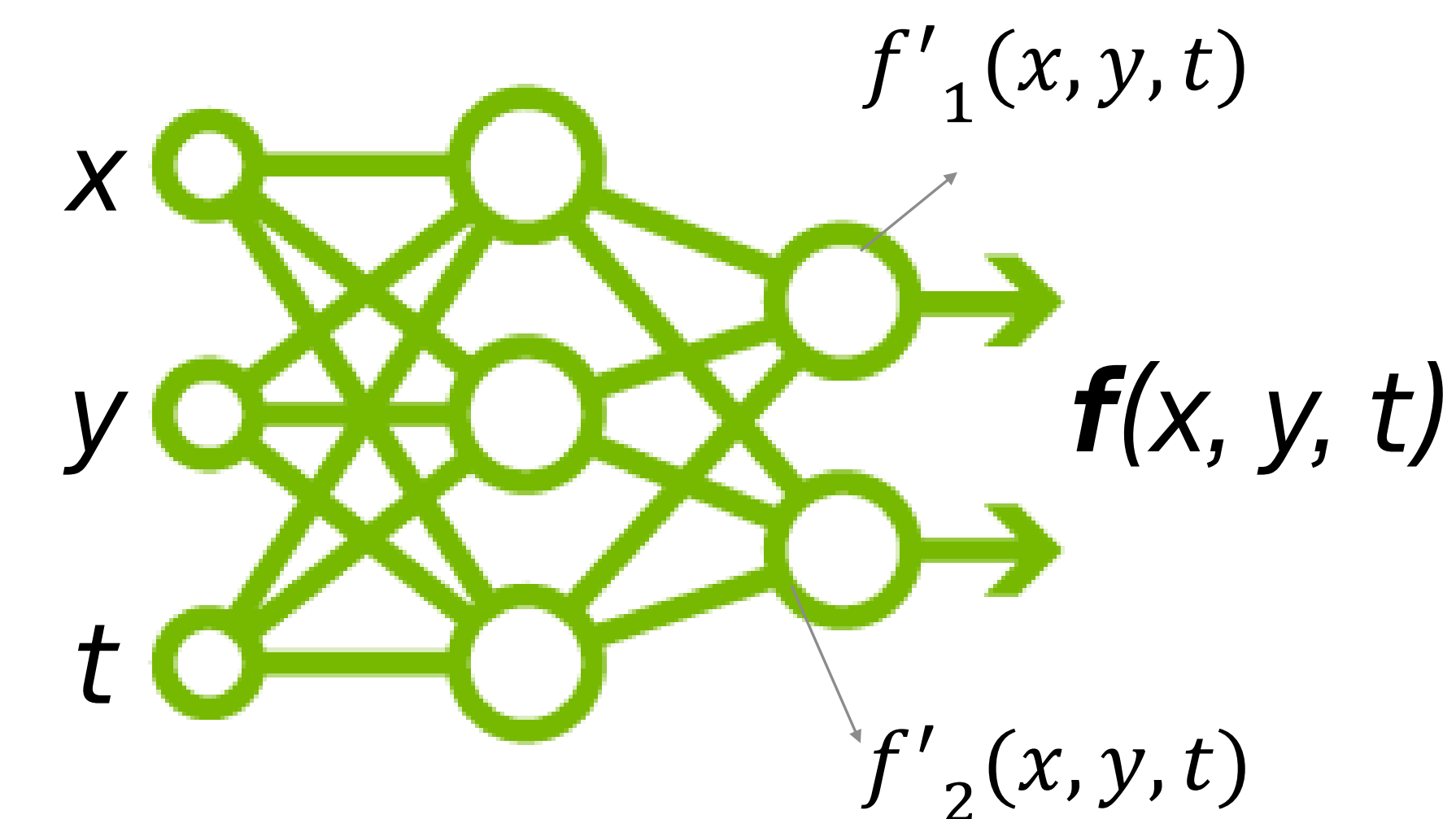


<https://blog.janestreet.com/visualizing-piecewise-linear-neural-networks/>

How To Incorporate The Physics?

Calculate the Differential Operator With Auto-differentiation

- A solution will everywhere satisfy the PDE
- At the heart of our PDE is a differential operator, e.g. $Du = 0$, made up of differential terms
- Our favourite DL framework includes auto-differentiation, mainly for the back-propagation stage – doing gradient descent given the loss
- Here we use the auto-differentiation for the *forward* pass, to calculate the loss in the first place
- (Note – for higher-order derivatives, this can be expensive!)
- The loss is the “PDE residual” – say $||D\hat{u} - Du||$
- And we backpropagate that back to drive the PDE residual to zero



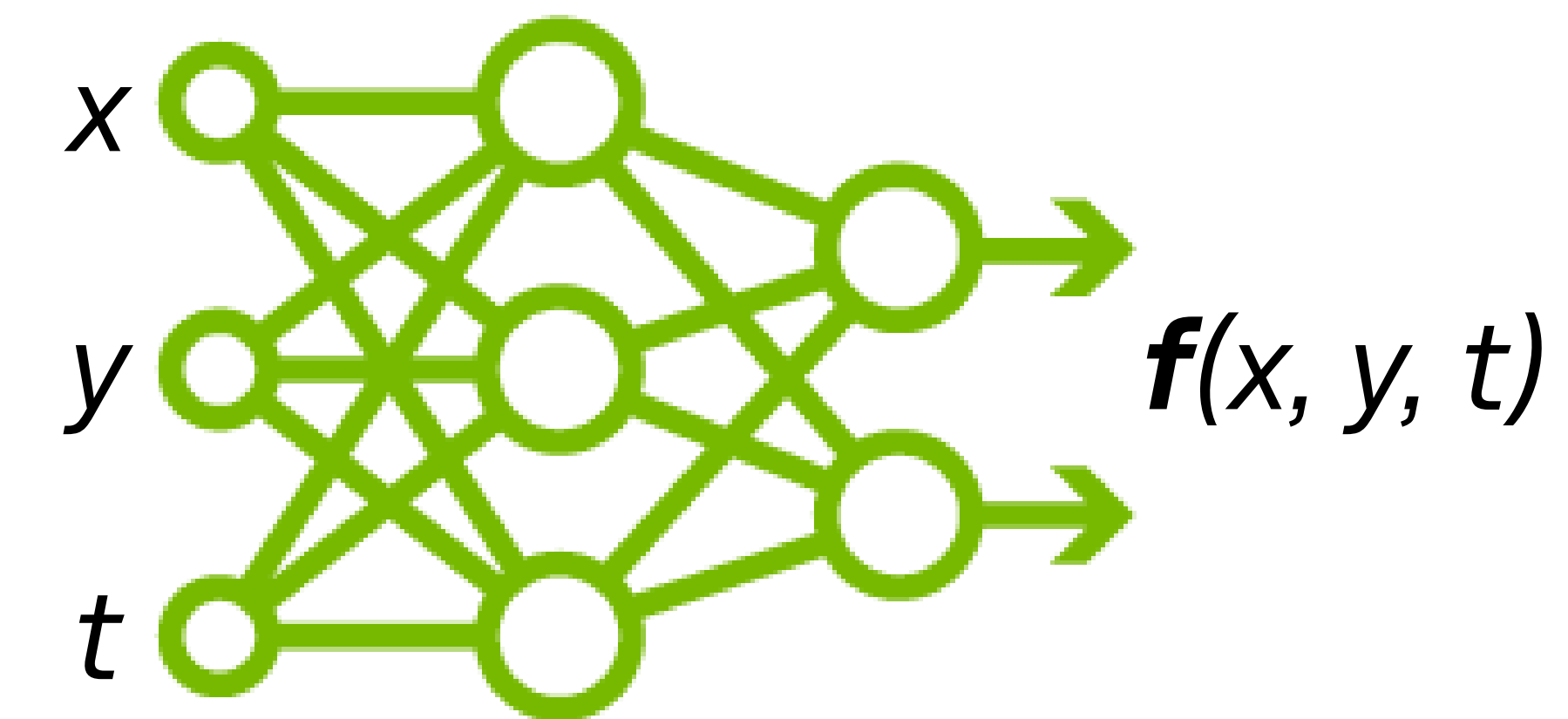
```
def f(t, x):  
    u = u(t, x)  
    u_t = tf.gradients(u, t)[0]  
    u_x = tf.gradients(u, x)[0]  
    u_xx = tf.gradients(u_x, x)[0]  
    f = u_t + u*u_x - (0.01/tf.pi)*u_xx  
    return f
```

<https://maziarraissi.github.io/PINNs/>

But we don't just want *any* solution, we want a *particular* solution

Making the problem well-posed with supervised learning for ICs, BCs

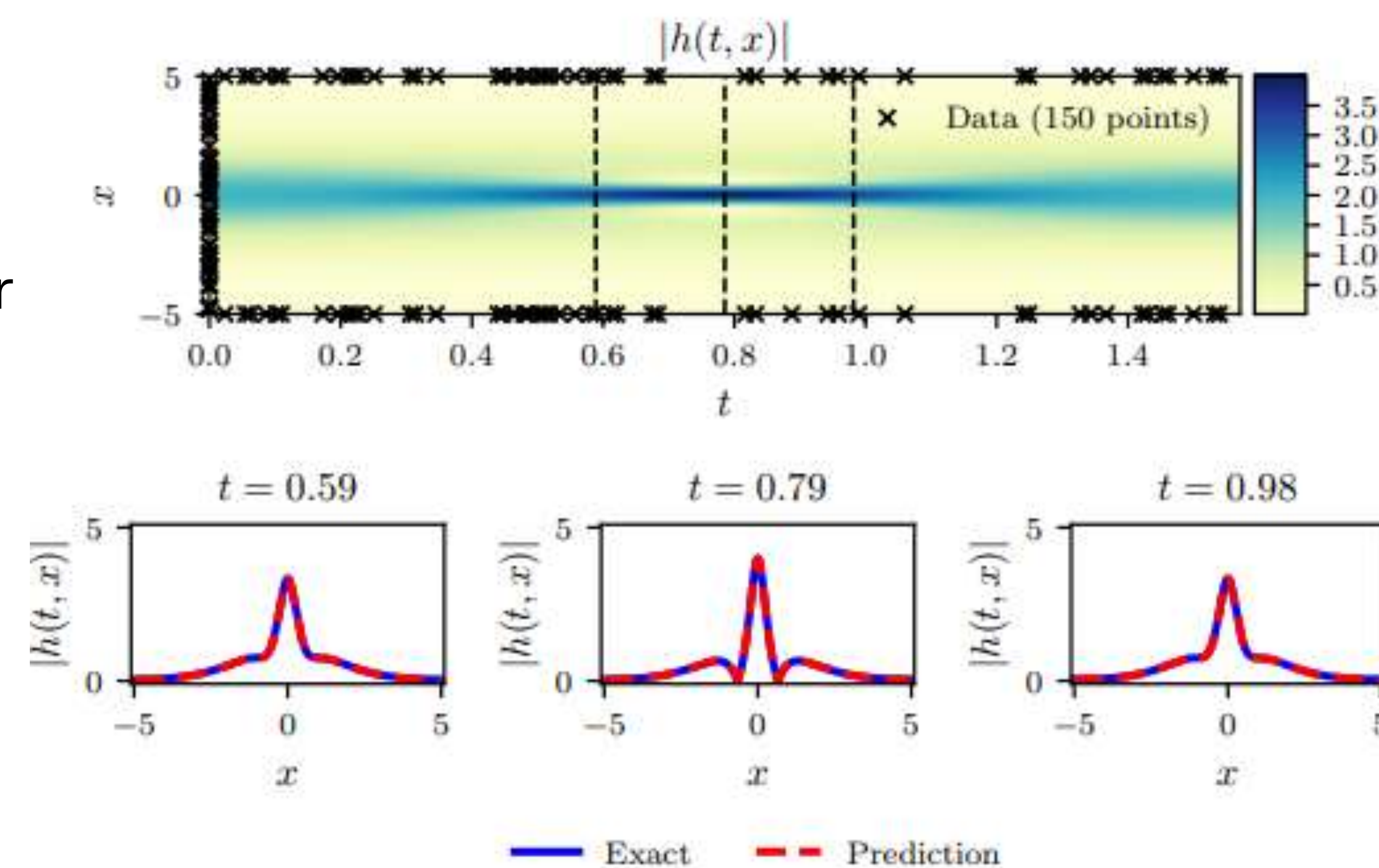
- Lots of functions will satisfy the differential operator – we need our particular solution that meets the initial and boundary conditions
- Here we just use standard supervised learning
 - For ICs, we have given $f(x, y, t)$ we need to see at given x, y, t
 - For Dirichlet BCs, ditto
 - For Neumann BCs, there are $f'(x, y, t)$ we need to see
- So we just have a loss function for the ICs and BCs that looks like standard supervised learning, the loss is MSE (say) of residuals
- Note that there's no real difference between ICs and BCs here, the model doesn't know the difference.



How to actually do the training

Quadrature, not Meshing

- Let's look at Fig 1 from Raissi, Perdikaris, & Karniadakis (2019), a 1-d time-dependent Schrödinger equation
- There's initial conditions at $t = 0$, and boundary conditions at $x = \pm 5$
- We pick “some” points along the IC or BC and evaluate there for calculating those losses (shown), and add points into the interior for the physics loss
- This is not a mesh! This method is meshless.
 - These are better thought of as collocation points
 - We're essentially performing a quadrature of the residual(s) for the purposes of learning our adaptive function
 - We can choose the points them randomly
 - We can choose them to be more heavily weighted in some regions than others
 - We can vary them from training epoch to training epoch!
 - Some of the PINN variants we'll talk about do just this



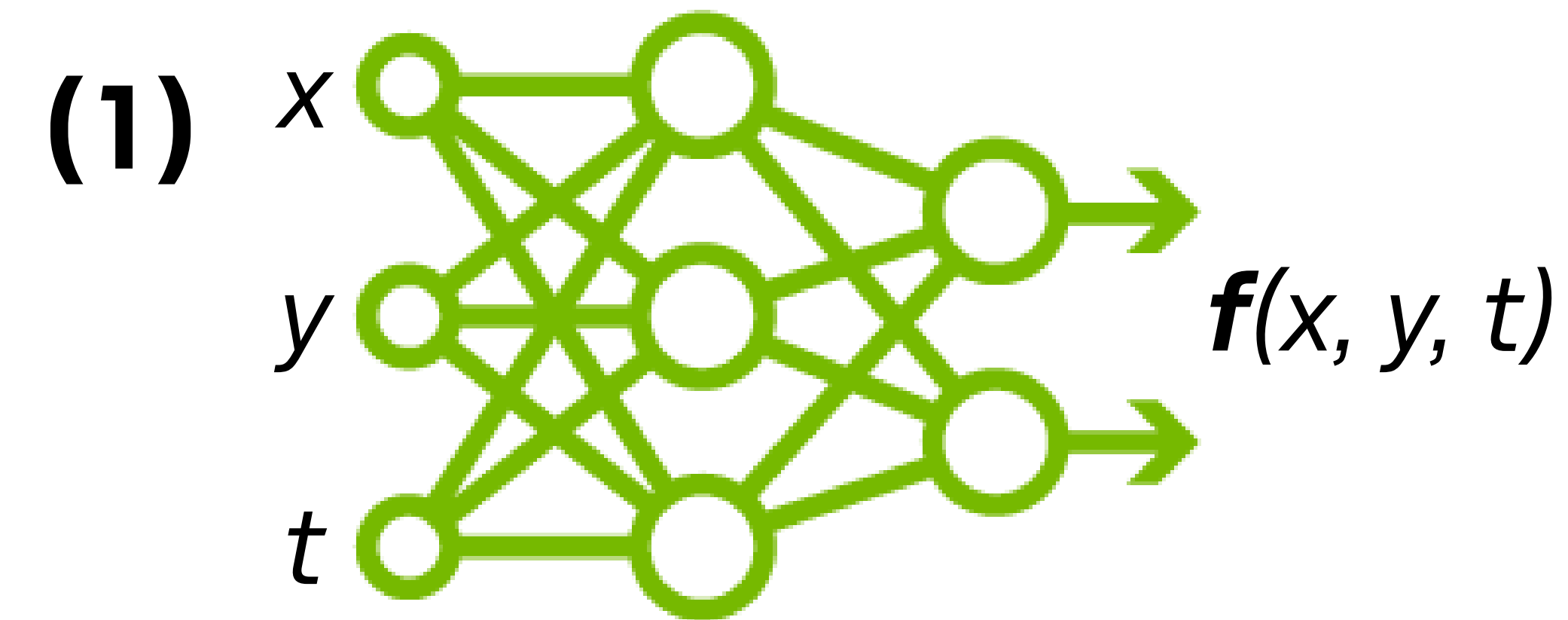
The overall approach

Calculate the Differential Operator With Auto-differentiation

- So the basic approach is:

1. Pick a neural network architecture – it can be a simple, fully-connected network; different networks are used for different problem types. There doesn't have to be *any* physics in the network. Tanh is one common choice for activation function
2. Construct a physics loss (PDE) residual using auto-differentiation
3. Construct the IC/BC loss from difference from desired values
4. Choose collocation points – uniform, random, or otherwise

- And then train on the total loss. (Quasi-newton methods like BFGS or L-BFGS are one set of common choices)



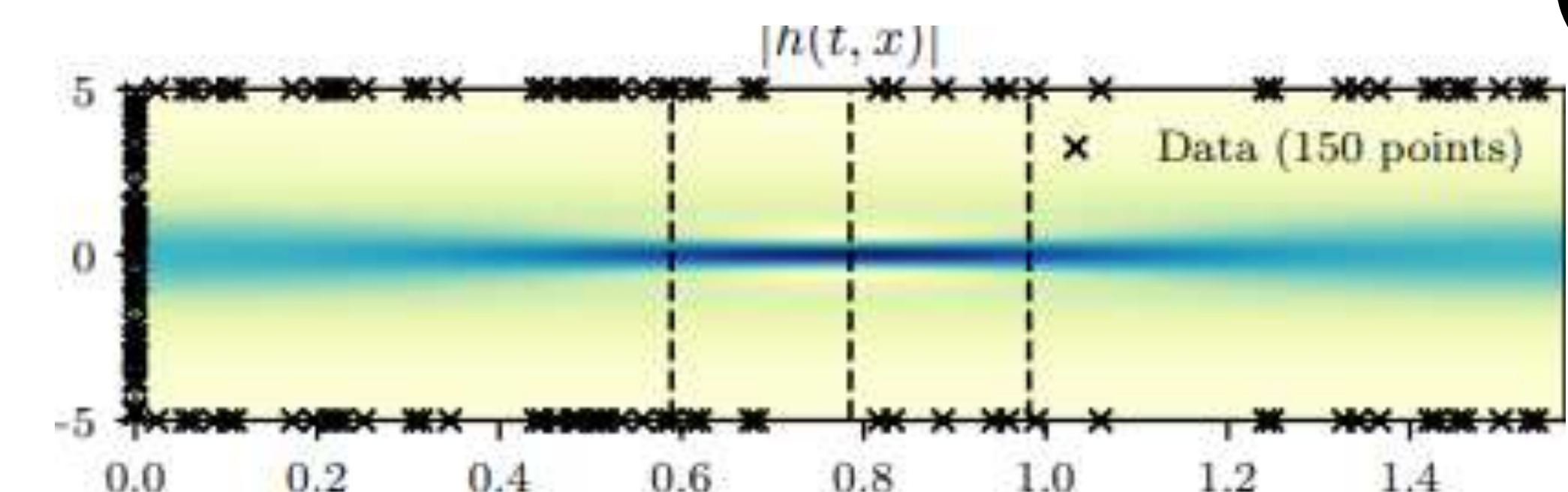
(2)

```
def f(t, x):  
    u = u(t, x)  
    u_t = tf.gradients(u, t)[0]  
    u_x = tf.gradients(u, x)[0]  
    u_xx = tf.gradients(u_x, x)[0]  
    f = u_t + u*u_x - (0.01/tf.pi)*u_xx  
    return f
```

(3)

$$L_{data} = \sum_{x_i \in data} (u_{net}(x_i) - u_{true}(x_i))^2$$

(4)



Let's try an example

Train a neural network using only physical constraints

- Consider an example problem:

$$\mathbf{P:} \begin{cases} \frac{\delta^2 u}{\delta x^2}(x) = f(x) \\ u(0) = u(1) = 0 \end{cases}$$

- To solve the PDE using only the equation and BCs, we construct a neural network $u_{net}(x)$ which has a single value input $x \in \mathbb{R}$ and single value output $u_{net}(x) \in \mathbb{R}$.
- We assume the neural network is infinitely differentiable $u_{net} \in C^\infty$ - Use activation functions that are infinitely differentiable

Computing Physics loss as soft constraints

Train a neural network using only physical constraints: Loss formulation

- Construct the loss function. We can compute the second order derivatives $\left(\frac{\delta^2 u_{net}}{\delta x^2}(x)\right)$ using Automatic differentiation, compute the integrals using Monte-Carlo integration technique

$$L_{BC} = (u_{net}(0) - 0)^2 + (u_{net}(1) - 0)^2$$

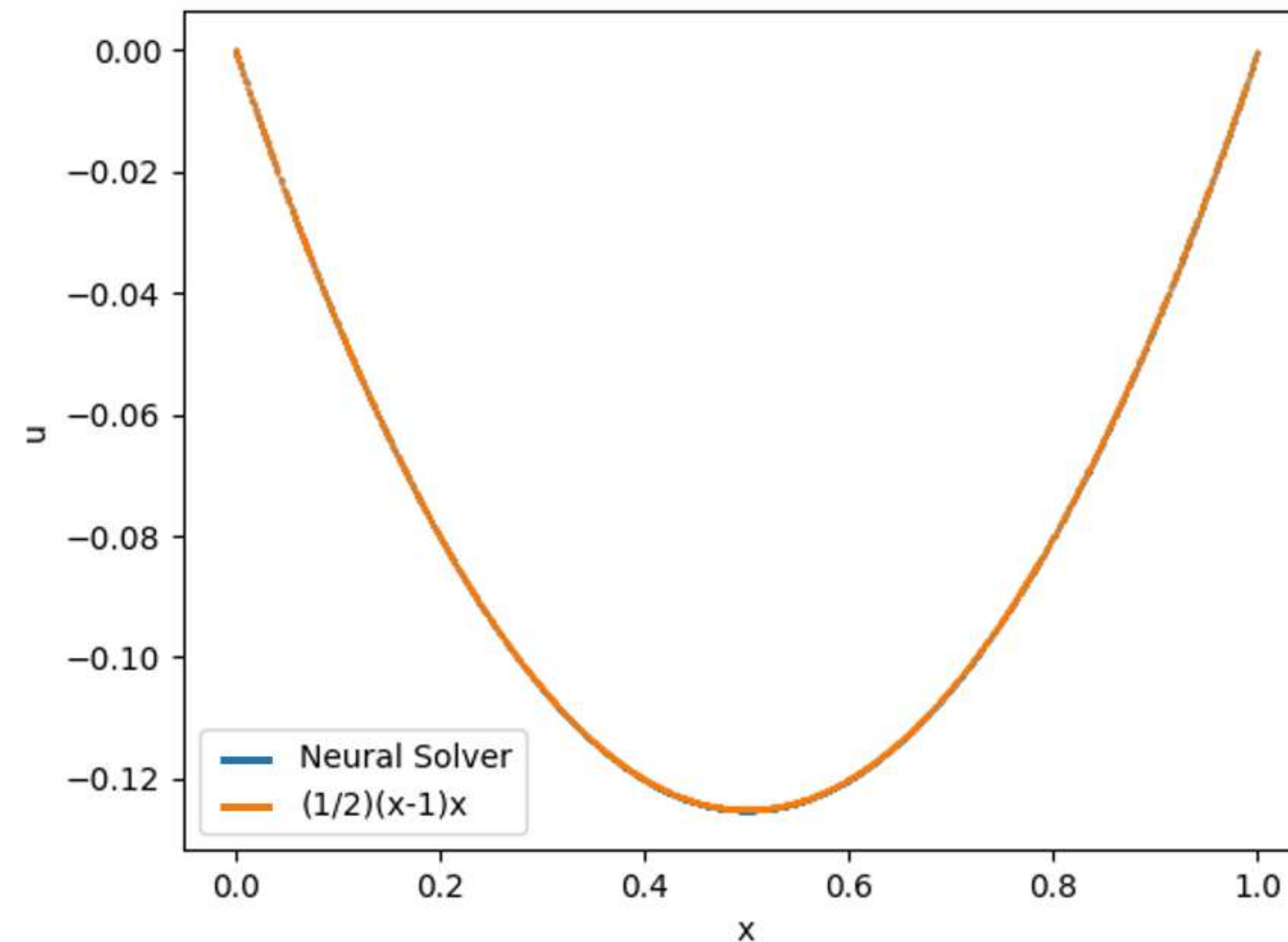
$$L_{Residual} = \int_0^1 \left(\left(\frac{\delta^2 u_{net}}{\delta x^2}(x) - f(x) \right) - 0 \right)^2 dx \approx \left(\int_0^1 dx \right) \frac{1}{N} \sum_{i=0}^N \left(\frac{\delta^2 u_{net}}{\delta x^2}(x_i) - f(x_i) \right)^2$$

- Where x_i are a batch of points in the interior $x_i \in (0, 1)$. Total loss becomes $L = L_{BC} + L_{Residual}$
- Minimize the loss using optimizers like Adam

Computing Physics loss as soft constraints

Train a neural network using only physical constraints: Results

- For $f(x) = 1$, the true solution is $\frac{1}{2}(x-1)x$. After sufficient training we have,



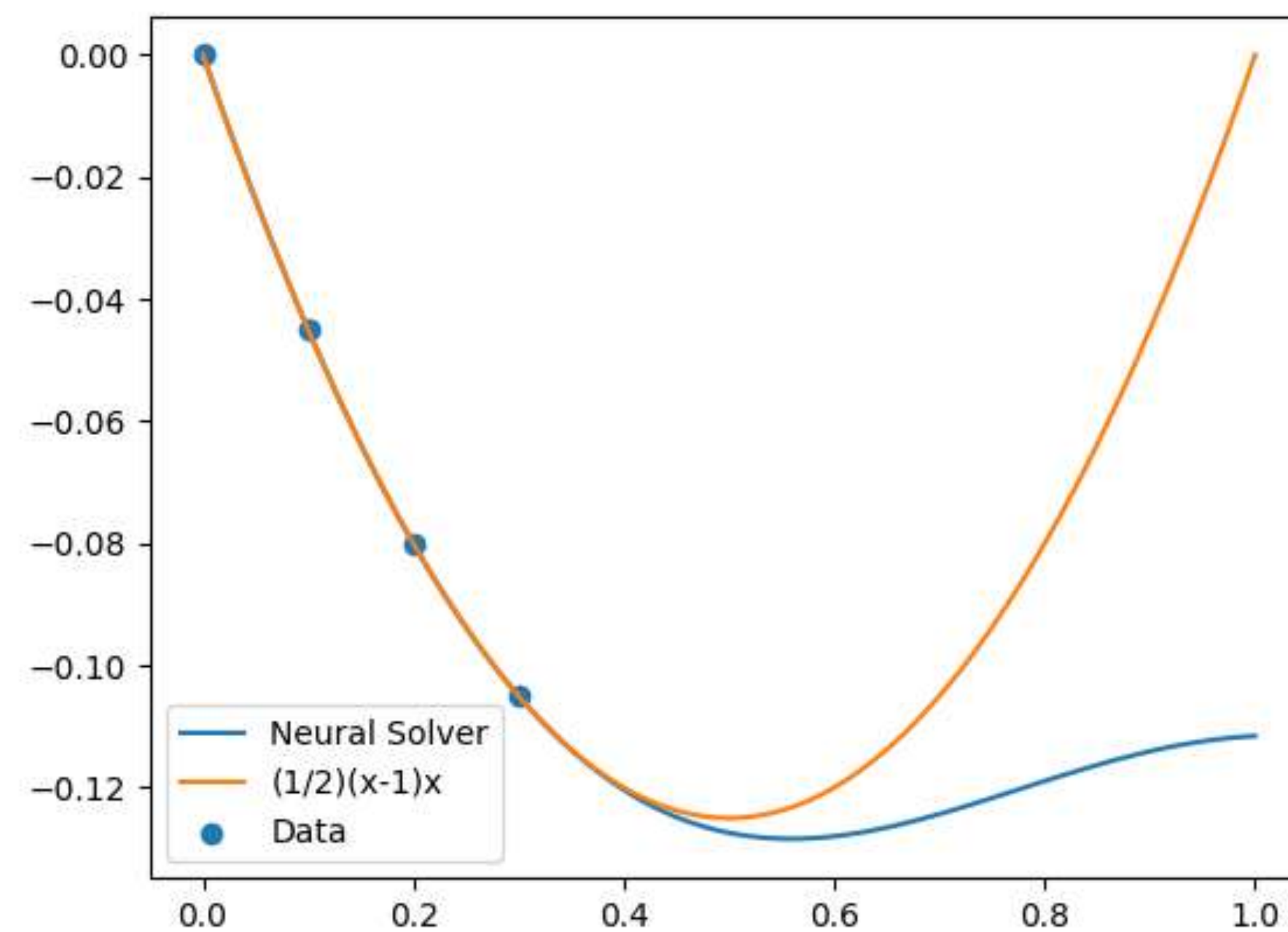
Comparison of the solution predicted by Neural Network with the analytical solution

Adding Physics laws as soft constraints

Data Only

$$L_{data} = \sum_{x_i \in \text{data}} (u_{net}(x_i) - u_{true}(x_i))^2$$

$$L_{total} = L_{data}$$

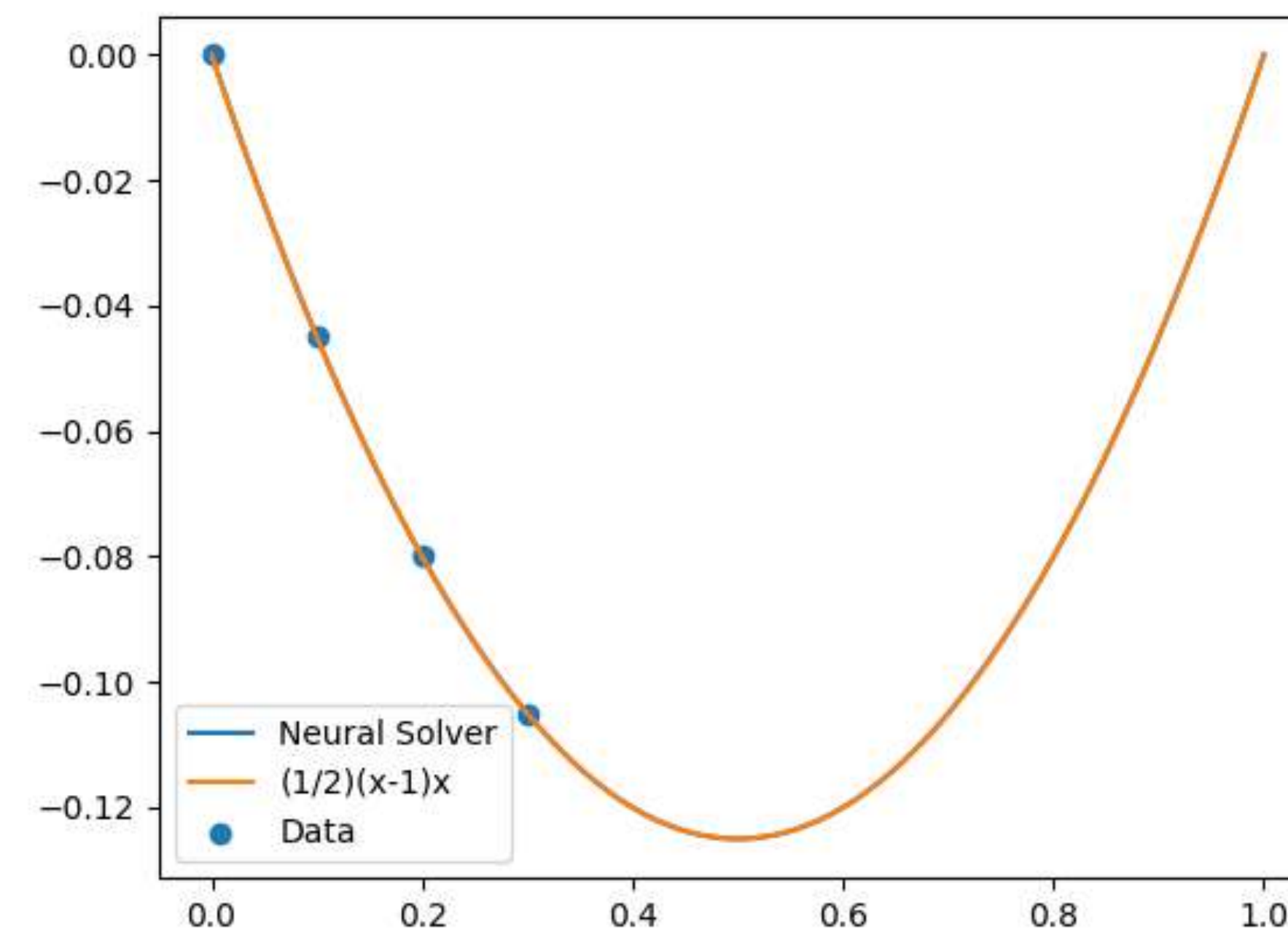


Data + Physics

$$\frac{\delta^2 u}{\delta x^2}(x) = 1$$

$$L_{physics} = \sum_{x_j \in \text{domain}} \left(\frac{\delta^2 u_{net}}{\delta x^2}(x_i) - f(x_i) \right)^2$$

$$L_{total} = L_{data} + L_{physics}$$



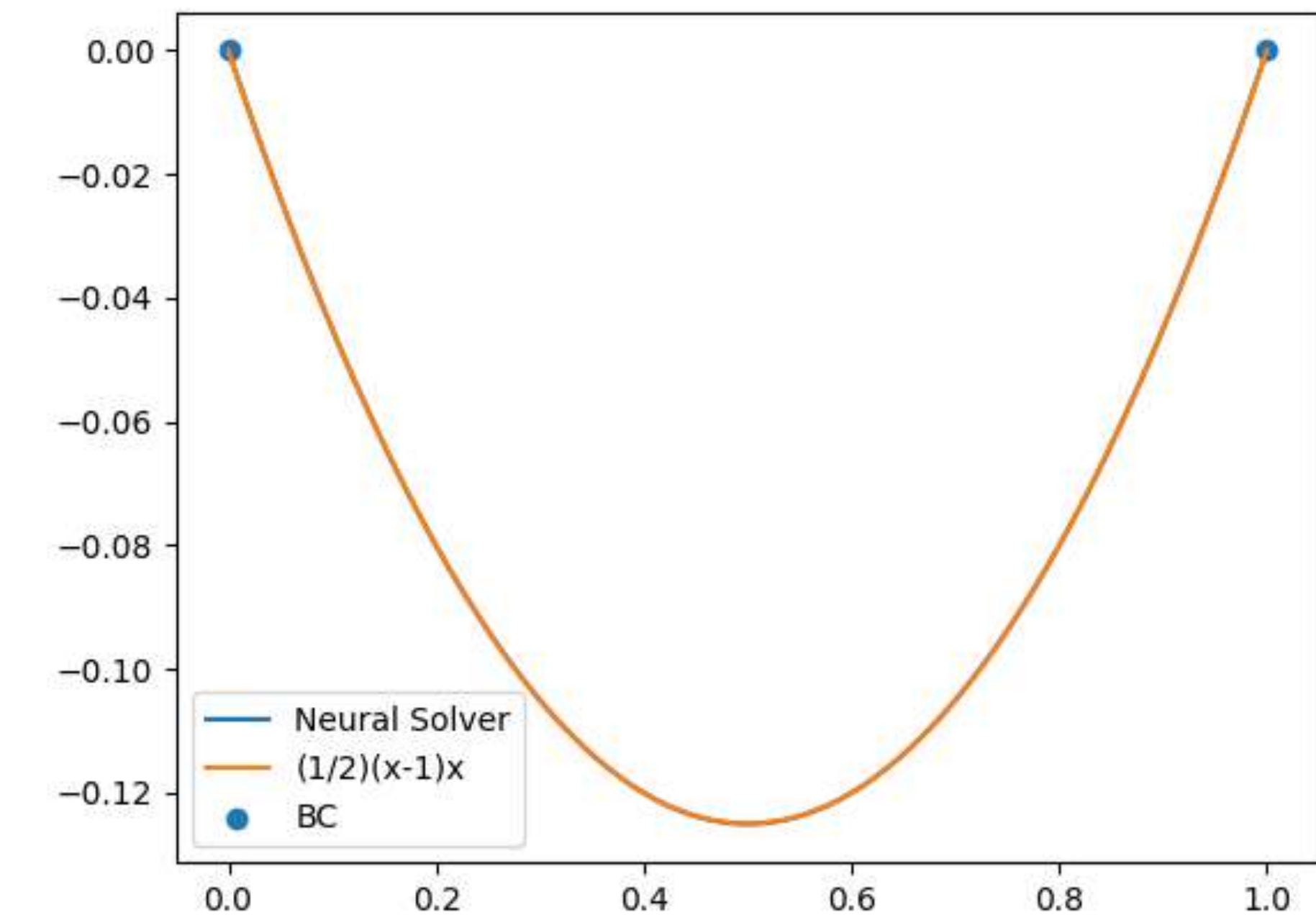
Physics only

$$\frac{\delta^2 u}{\delta x^2}(x) = 1$$

$$u(0) = u(1) = 0$$

$$L_{physics} = L_{residual} + L_{BC}$$

$$L_{total} = L_{physics}$$



PhysicsNeMo Sym

Framework for AI surrogates using Physics-Based symbolic loss functions

- High-level abstract training framework for training physics-constrained models
- Flexible multi-constraint training workflow with many physics-driven training enhancements
- Symbolic paradigm (keys) which enables gradient calculation automation

KEY FEATURES:

Abstraction: High level API for domain experts

Automation: Automated loss and gradient calculations for PINNs training using SymPy

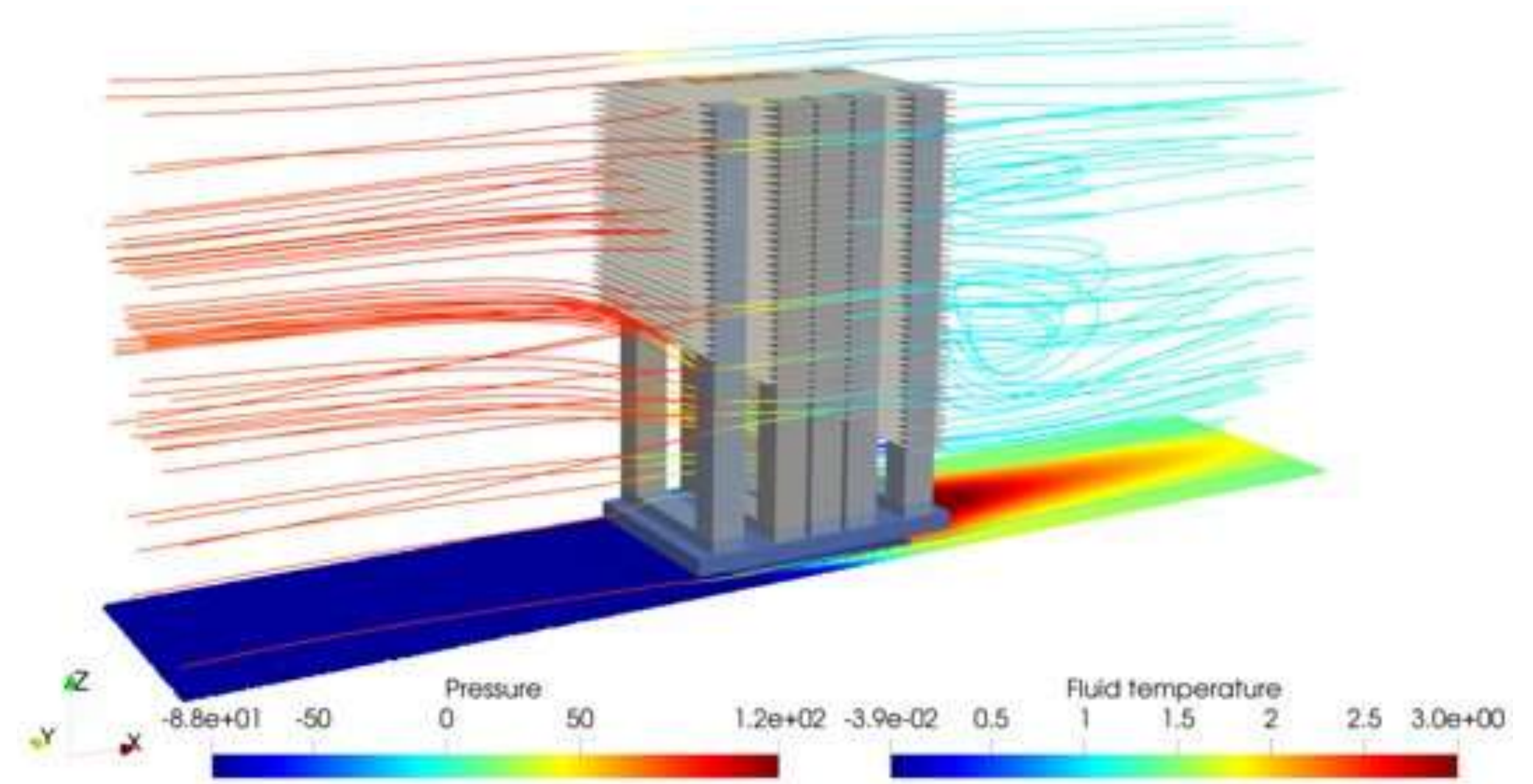
Parallelism: Automated DDP and model parallel setup

Optimization: Similar optimizations as PhysicsNeMo-Launch with some additions (PINNs AMP)

Monitoring / Logging: Tensorboard

Hydra: Configurable through Hydra

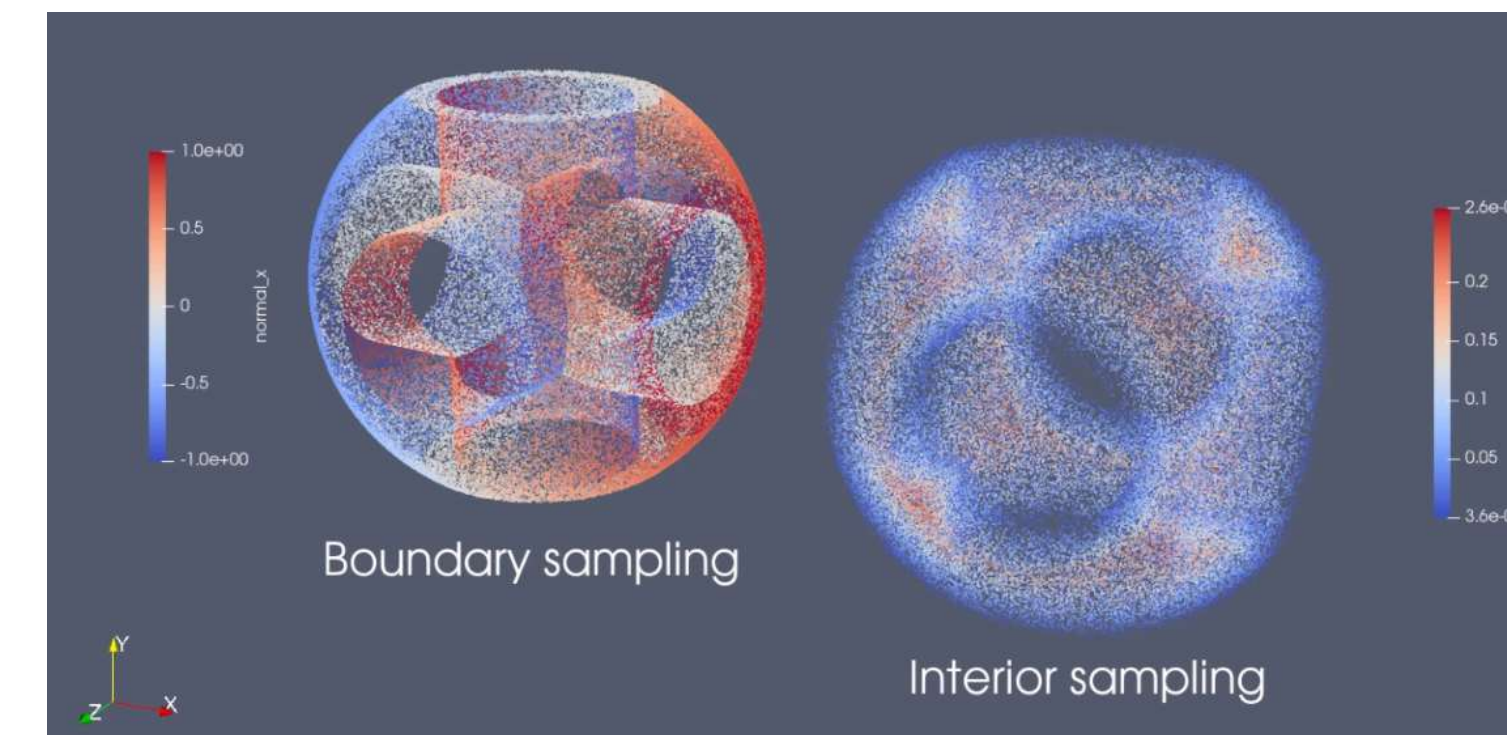
Example Documentation: Comprehensive set of examples for different physical systems



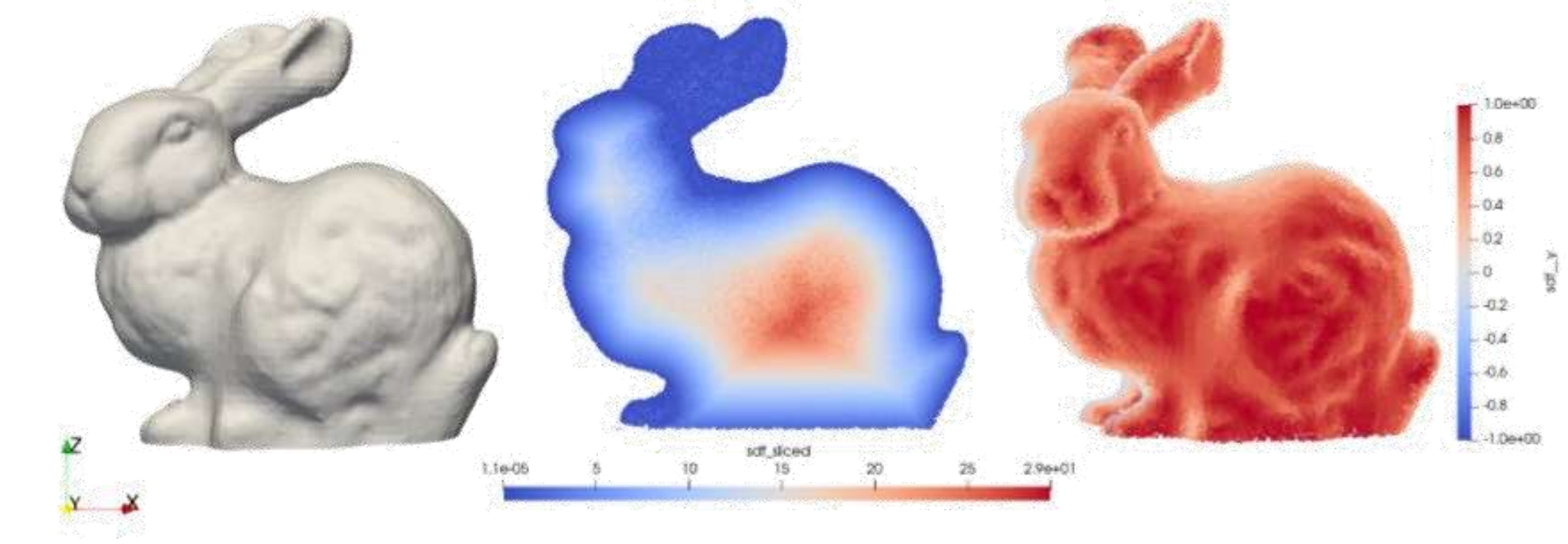
Physics guided training workflows

Leveraging abstracted utils from PhysicsNeMo Sym

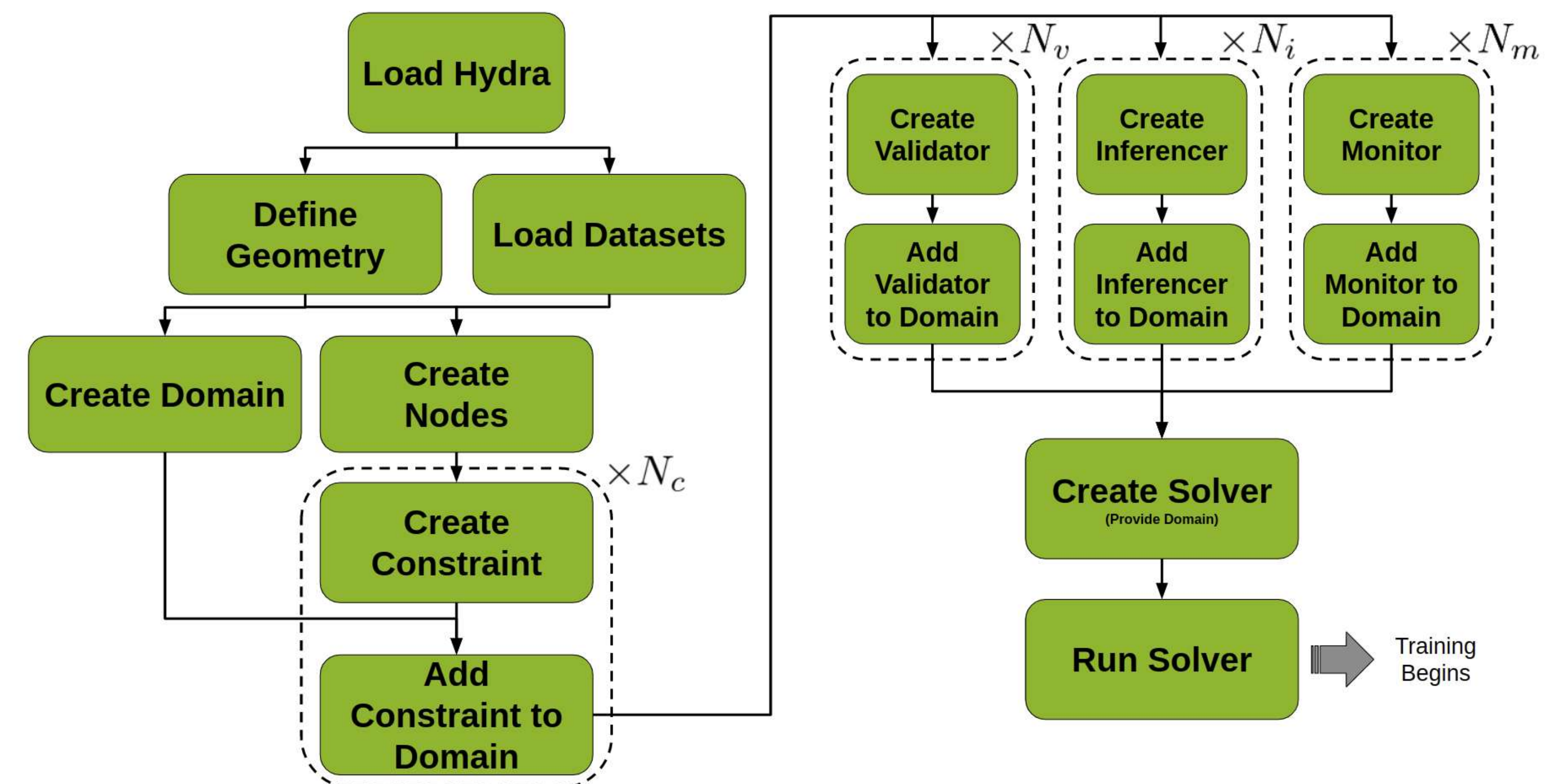
- Key questions:
 - How to determine domain of interest?
 - Full domain
 - Boundaries, ...
 - How to sample the domain?
 - Collocation points
 - Test functions
 - Discretization, ...
 - How to specify the constraints/losses information?
 - Control Volume Formulation
 - DifferentiVariational Formulation, ...
- PhysicsNeMo Sym has utilities to simplify such problem setups. Utils can be used **standalone** or in **abstracted training definition** framework



Constructive Solid Geometry



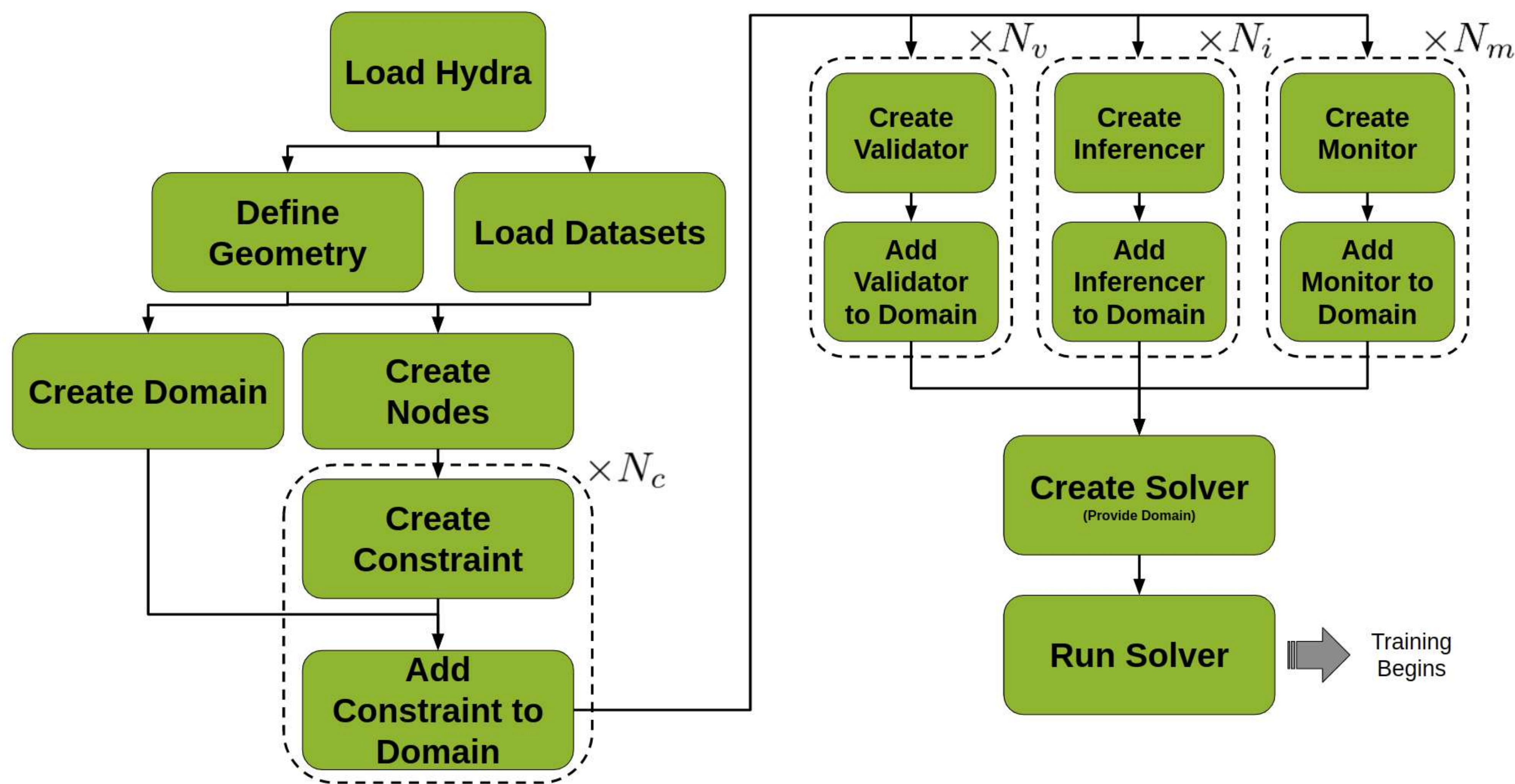
Tessellated Geometry (STL)



PhysicsNeMo Sym's abstracted Training workflow

PhysicsNeMo Sym: Anatomy of a project

Overview



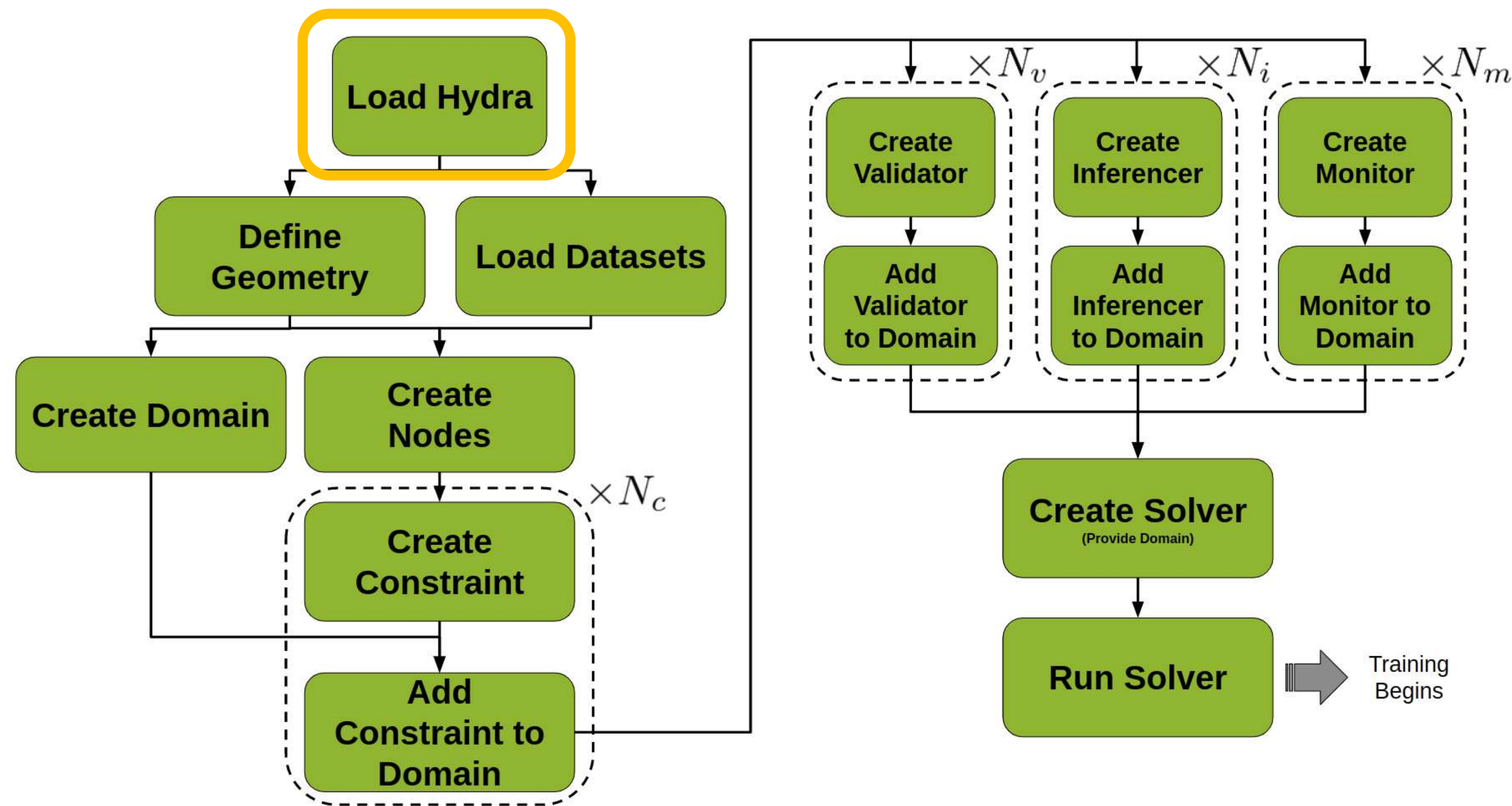
PhysicsNeMo Sym works by:

- Writing models which include at least one adaptable function (a NN)
- Writing objective functions as a combination of these models
- Describing the geometry/dataset where the models should be evaluated
- Minimizing the objective functions by using the provided data, by sampling the geometry, or both
- Running the models to obtain the desired effect

$$\mathbf{P}: \begin{cases} \frac{\delta^2 u}{\delta x^2}(x) = f(x) \\ u(0) = u(1) = 0 \end{cases}$$

Physics guided training workflows

PhysicsNeMo Sym approach: Load Hydra



```

defaults :
- Physicsnemo_default
- scheduler: tf_exponential_lr
- optimizer: adam
- loss: sum
- _self_
scheduler:
decay_rate: 0.95
decay_steps: 200

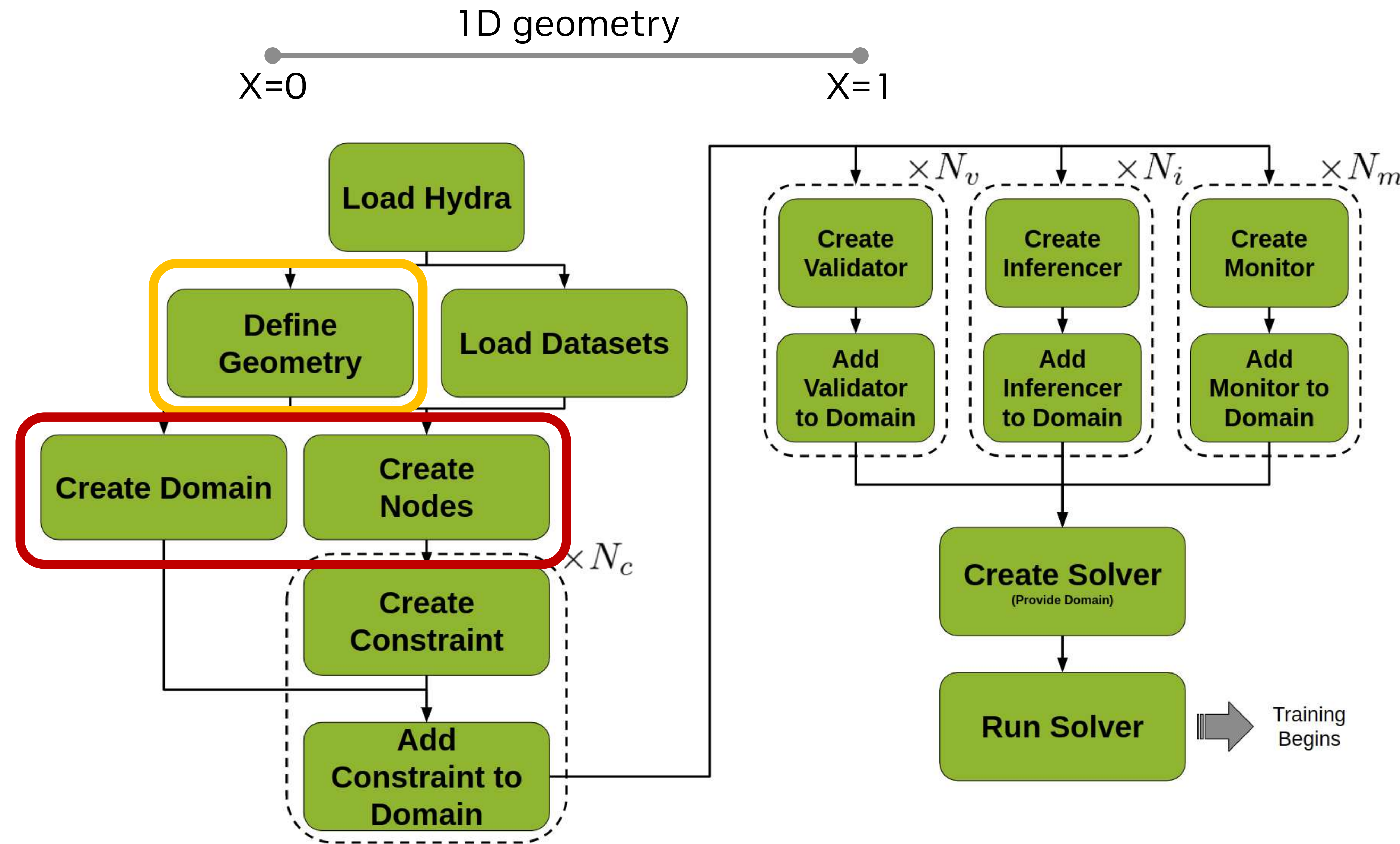
save_filetypes : "vtk,npz"

training:
rec_results_freq : 1000
rec_constraint_freq: 1000
max_steps : 5000
  
```

$$\mathbf{P}: \begin{cases} \frac{\delta^2 u}{\delta x^2}(x) = f(x) \\ u(0) = u(1) = 0 \end{cases}$$

Physics guided training workflows

PhysicsNeMo Sym approach: Create geometry, domain, and nodes



```
@Physicsnemo.main(config_path="conf", config_name="config")
def run(cfg: PhysicsNeMoConfig) -> None:

    # make geometry
    x = Symbol("x")
    geo = Line1D(0, 1)
```

```
# make list of nodes to unroll graph on
eq = CustomPDE(f=1.0)
u_net = FullyConnectedArch(
    input_keys=[Key("x")],
    output_keys=[Key("u")],
    nr_layers=3,
    layer_size=32
)

nodes = eq.make_nodes() + [u_net.make_node(name="u_network")]

# make domain
domain = Domain()
```

Use the model architectures from PhysicsNeMo-Sym that are designed to work with Auto-grad out-of-the-box

PhysicsNeMo:

- physicsnemo.sym.geometry contains implementations of 1D, 2D and 3D primitives that can be assembled to compose complex geometries
- physicsnemo.sym.tessellation enables import of STL geometries
- Sample point cloud inside and on the surface of generated geometries

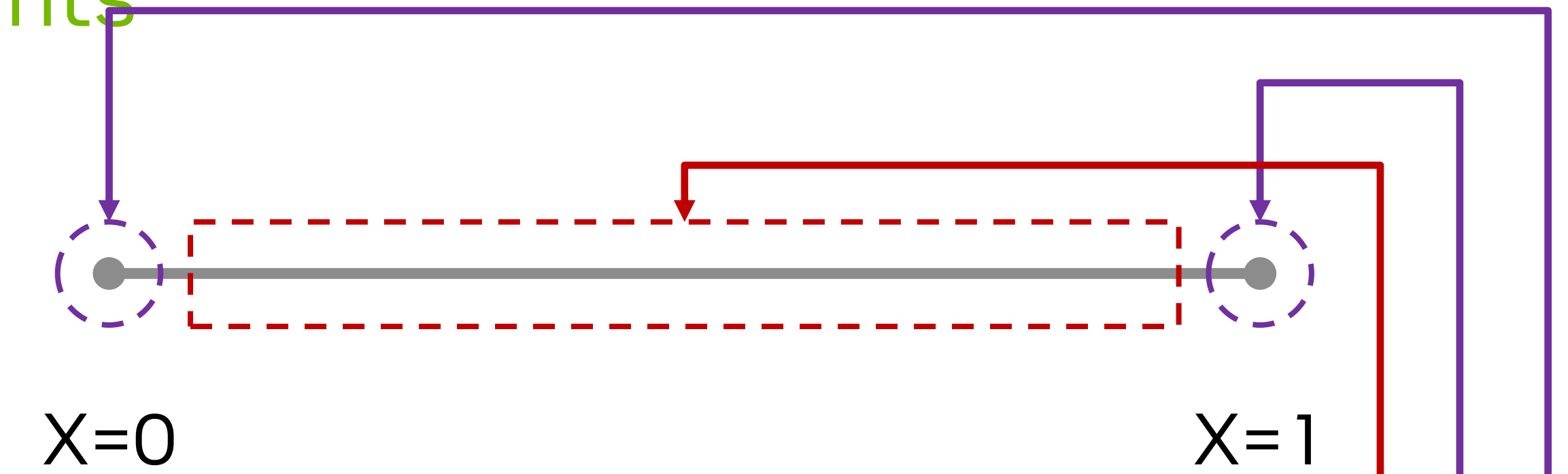
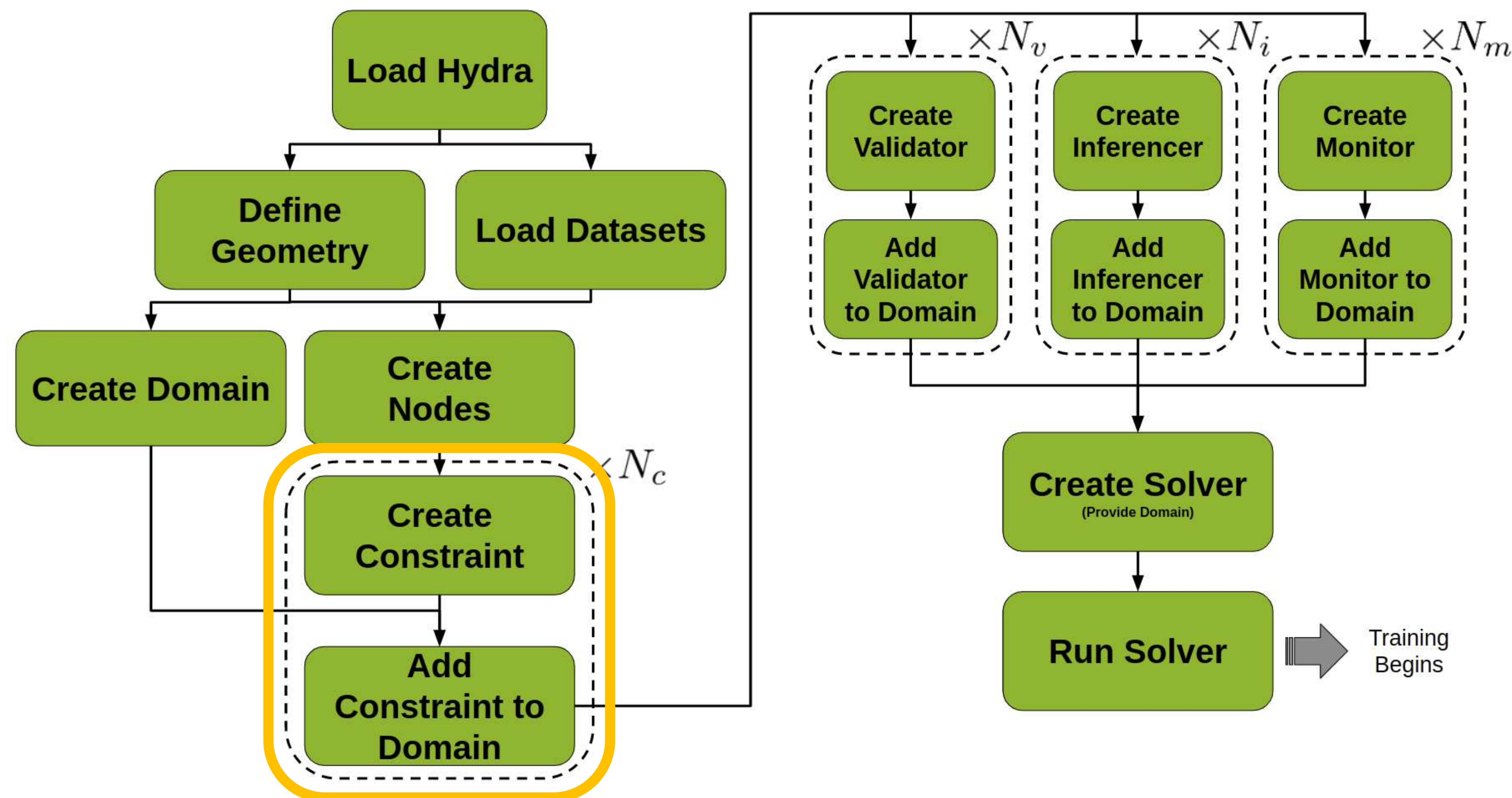
PDE and computational nodes:

- Implementations of fundamental governing equations from domains like: Fluid Mechanics, Linear Elasticity, Electromagnetic, etc.

$$\mathbf{P}: \begin{cases} \frac{\delta^2 u}{\delta x^2}(x) = f(x) \\ u(0) = u(1) = 0 \end{cases}$$

Physics guided training workflows

PhysicsNeMo Sym approach: Add constraints



```
# add constraints

# bcs
bc = PointwiseBoundaryConstraint(
    nodes=nodes,
    geometry=geo,
    outvar={"u": 0},
    batch_size=2,
)
domain.add_constraint(bc, "bc")

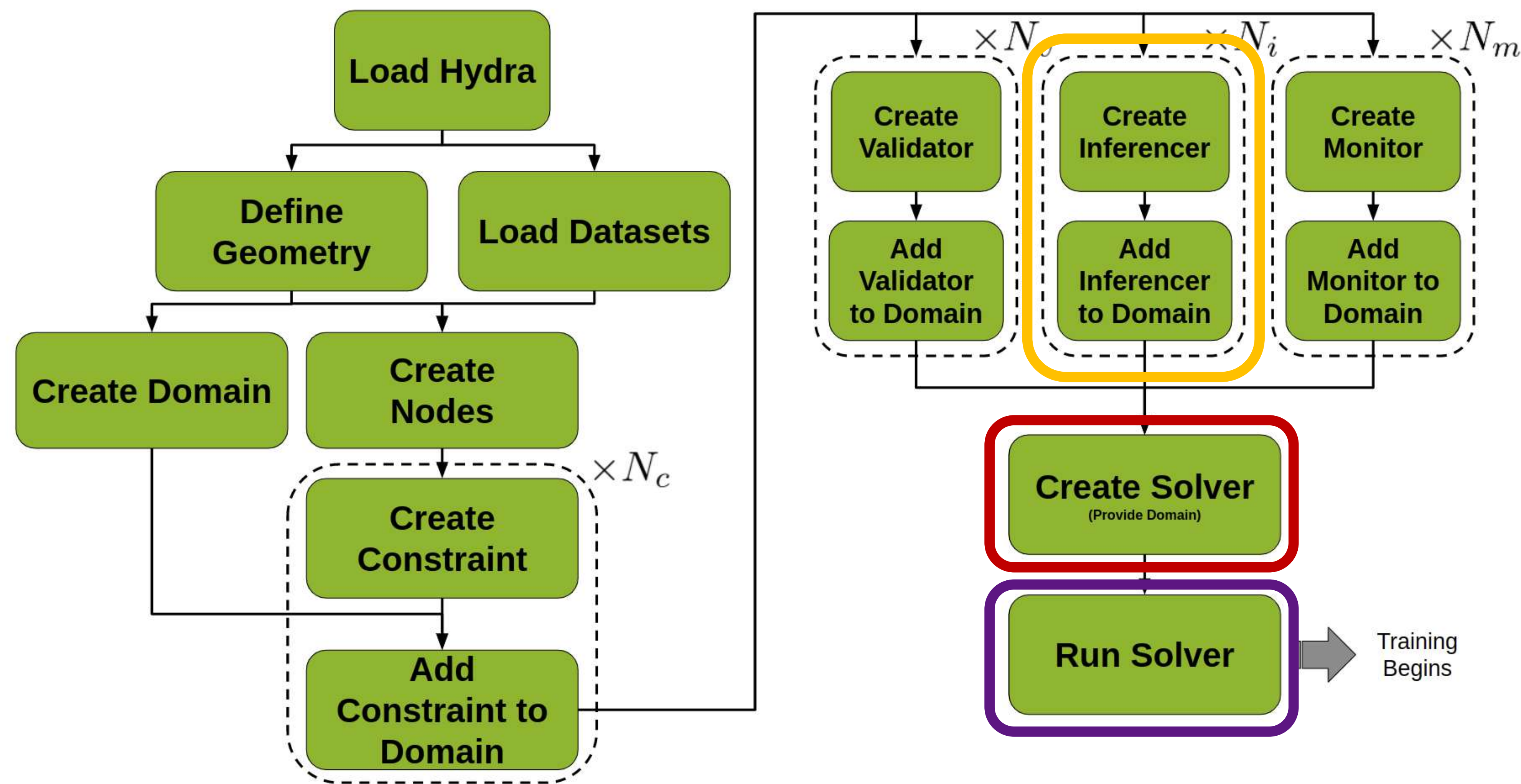
# interior
interior = PointwiseInteriorConstraint(
    nodes=nodes,
    geometry=geo,
    outvar={"custom_pde": 0},
    batch_size=100,
    bounds={x: (0, 1)},
)
domain.add_constraint(interior, "interior")
```

Automatically select the boundary and interior points

$$\mathbf{P}: \begin{cases} \frac{\delta^2 u}{\delta x^2}(x) = f(x) \\ u(0) = u(1) = 0 \end{cases}$$

Physics guided training workflows

PhysicsNeMo Sym approach: Add utils to visualize results, trainer loop



```
# add inferencer
inference = PointwiseInferencer(
    nodes=nodes,
    invar={"x": np.linspace(0, 1.0, 100).reshape(-1,1)},
    output_names=["u"],
)
domain.add_inferencer(inference, "inf_data")
```

```
# make solver
slv = Solver(cfg, domain)

# start solver
slv.solve()

if __name__ == "__main__":
    run()
```

```
python <script_name>.py
mpirun -np <#GPU> <script_name>.py
```

Use the pre-defined optimized training loop

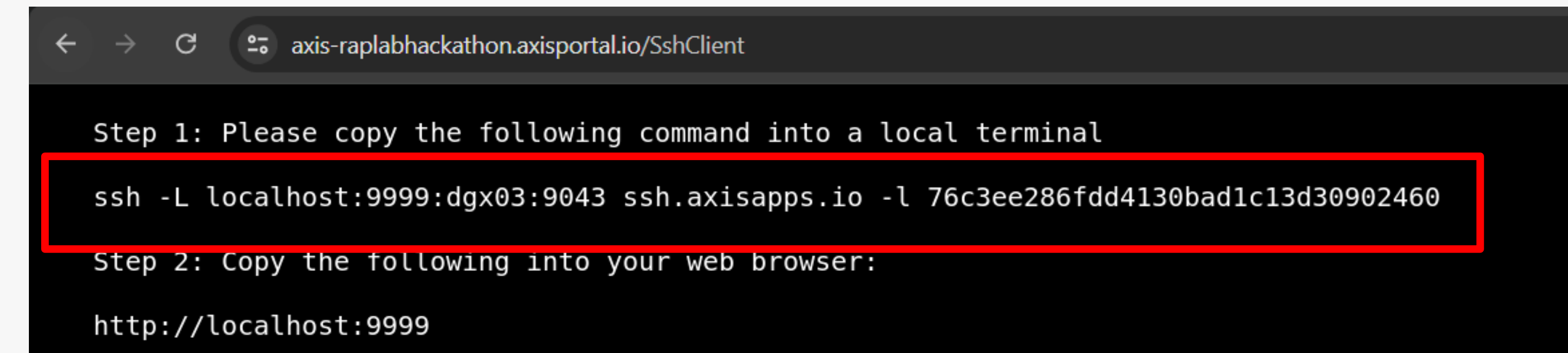
Abstracted and Optimized training loop:

- Solver and Trainer class abstract a lot of complexity of defining the Neural network training allowing users to focus on problem definition

Connect to the Notebooks

Run a command on a local terminal, connect to localhost on your browser

- In your web terminal when everything is completed, you will see a message along the lines of "Please enter the following command into your terminal" - it's a long command that starts with ssh
- Start a *new terminal on your laptop* (Terminal.app on Mac, PowerShell on windows, or your favourite terminal on Linux, for example), and paste that ssh command in.
- Your terminal is now another shell on the cluster login node, which is fine, but what matters is
- You can now open a new tab on your browser to <https://localhost:9999> to see the Jupyter notebook running on the cluster



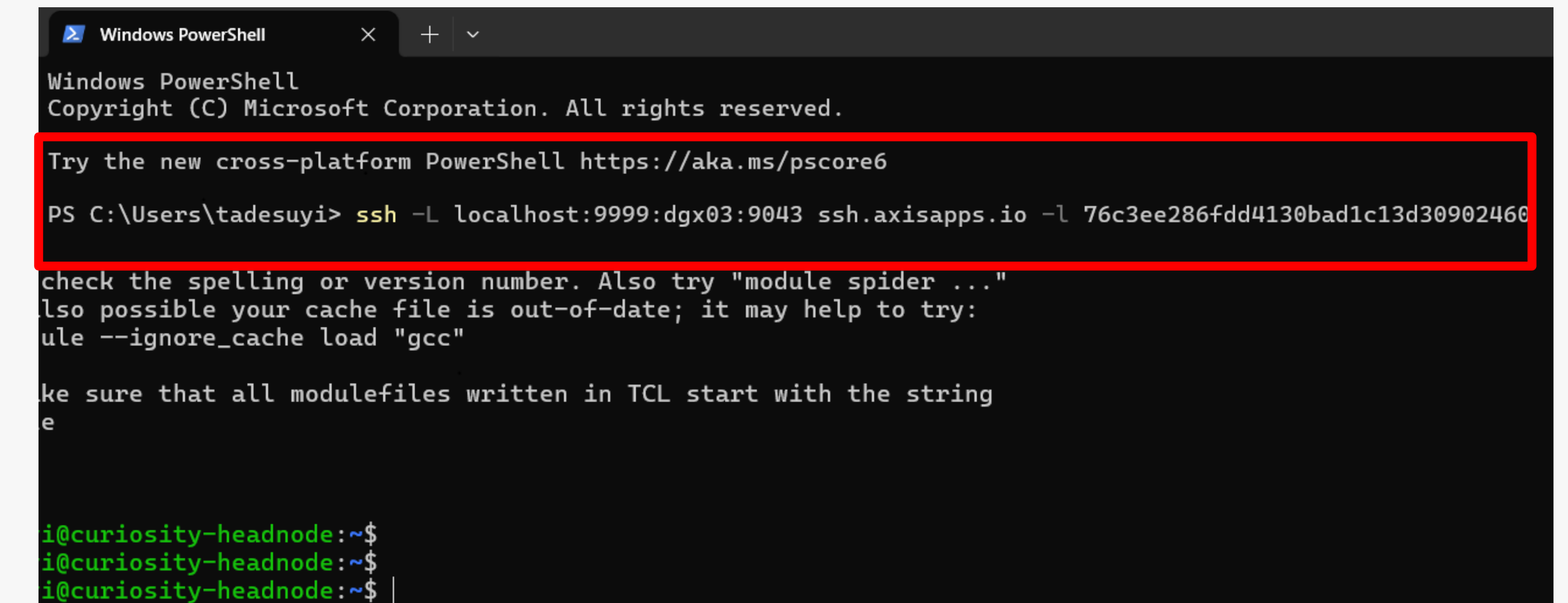
A screenshot of a web browser window with the address bar showing 'axis-raplabhackathon.axisportal.io/SshClient'. The page content includes two steps: 'Step 1: Please copy the following command into a local terminal' followed by a long SSH command, and 'Step 2: Copy the following into your web browser:' followed by the URL 'http://localhost:9999'. The SSH command is highlighted with a red rectangular box.

```
Step 1: Please copy the following command into a local terminal

ssh -L localhost:9999:dgx03:9043 ssh.axisapps.io -l 76c3ee286fdd4130bad1c13d30902460

Step 2: Copy the following into your web browser:

http://localhost:9999
```



A screenshot of a Windows PowerShell terminal window. It shows the execution of the SSH command from the previous screenshot. The command is highlighted with a red rectangular box. Below the command, there are several lines of output, including a warning about the module spider and a prompt to check the spelling or version number. The terminal shows the user is now at the 'i@curiosity-headnode:~\$' prompt.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\tadesuyi> ssh -L localhost:9999:dgx03:9043 ssh.axisapps.io -l 76c3ee286fdd4130bad1c13d30902460

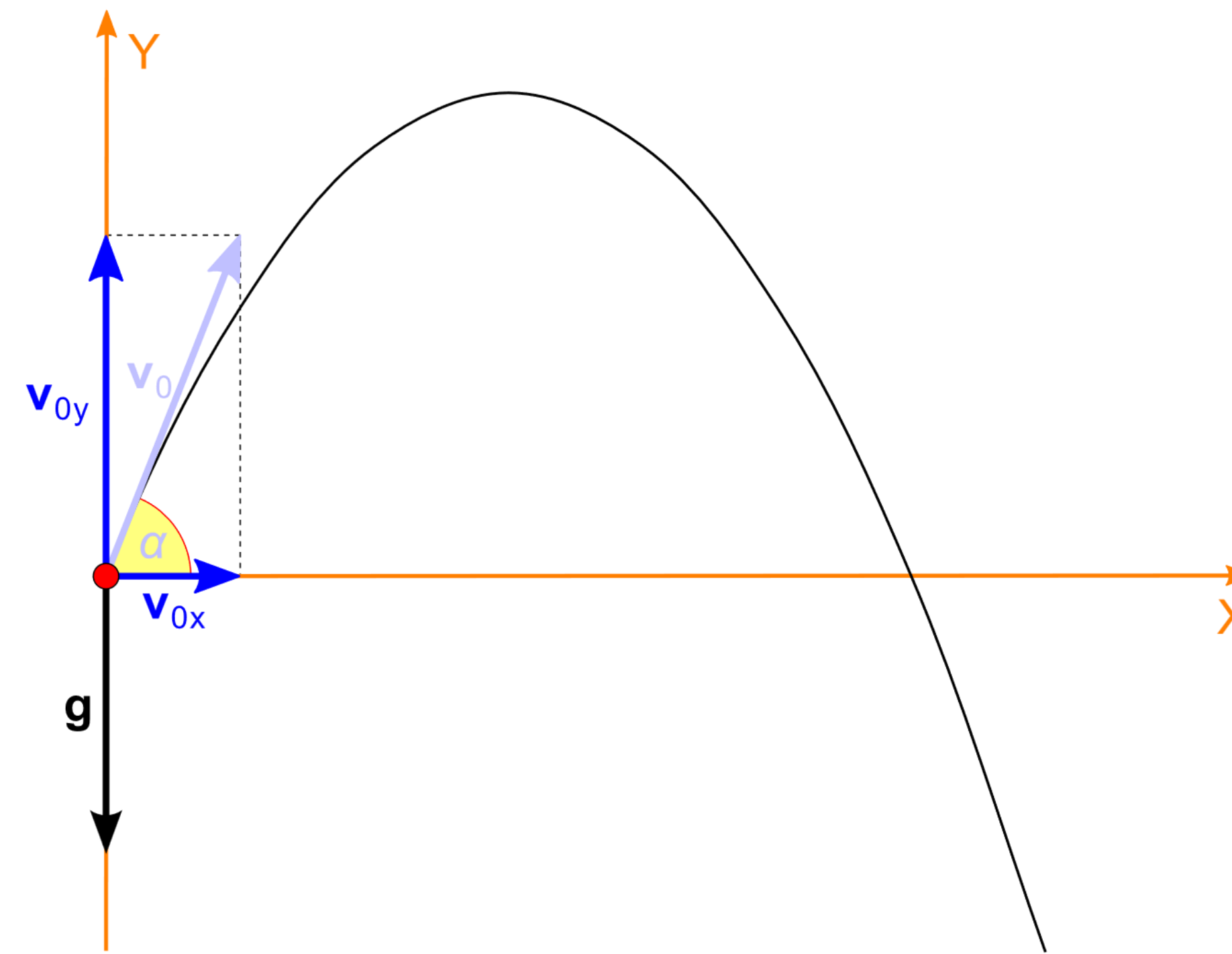
check the spelling or version number. Also try "module spider ..."
also possible your cache file is out-of-date; it may help to try:
ule --ignore_cache load "gcc"

ke sure that all modulefiles written in TCL start with the string
e

i@curiosity-headnode:~$
i@curiosity-headnode:~$
i@curiosity-headnode:~$
```

Projectile motion

Problem description



- Solving projectile motion in the case of absence of air-resistance on the surface on Earth.
- Equations: Equations of motions are as follows.

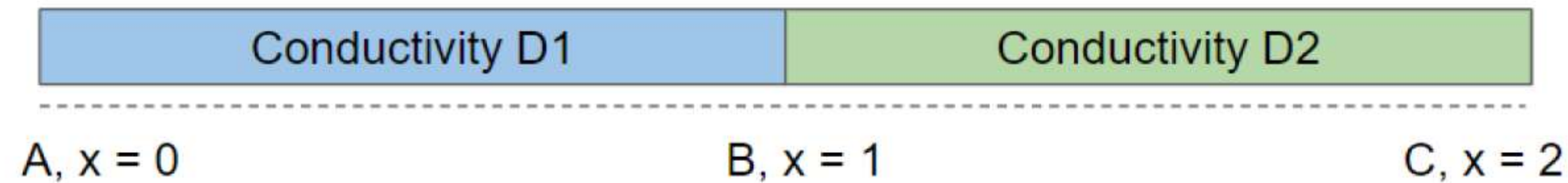
$$\frac{d^2}{dt^2} S_x = 0$$

$$\frac{d^2}{dt^2} S_y = -g$$

- Let us solve these two equations with the ball starting at origin (0,0)

1D diffusion

Problem description



- Composite bar with material of conductivity $D_1 = 10$ for $x \in (0,1)$ and $D_2 = 0.1$ for $x \in (1,2)$. Point A and C are maintained at temperatures of 0 and 100 respectively
- Equations: Diffusion equation in 1D

$$\frac{d}{dx} \left(D_1 \frac{dU_1}{dx} \right) = 0$$

When $0 < x < 1$

$$\frac{d}{dx} \left(D_2 \frac{dU_2}{dx} \right) = 0$$

When $1 < x < 2$

- Flux and field continuity at interface ($x=1$)

$$\left(D_1 \frac{dU_1}{dx} \right) = \left(D_2 \frac{dU_2}{dx} \right)$$
$$U_1 = U_2$$

Visualizing ML experiments

Using tensorboard

- To launch Tensorboard in your environment,
- Open a terminal in the Jupyter window and run the below command. Replace the <PORT NUM> with the appropriate port generated by your port forwarding command (likely 8889)

```
tensorboard --logdir /workspace/python/jupyter_notebook/ --port <PORT NUM>
```

- Once the Tensorboard is up and running, navigate to <http://localhost:6006/> to visualize the results

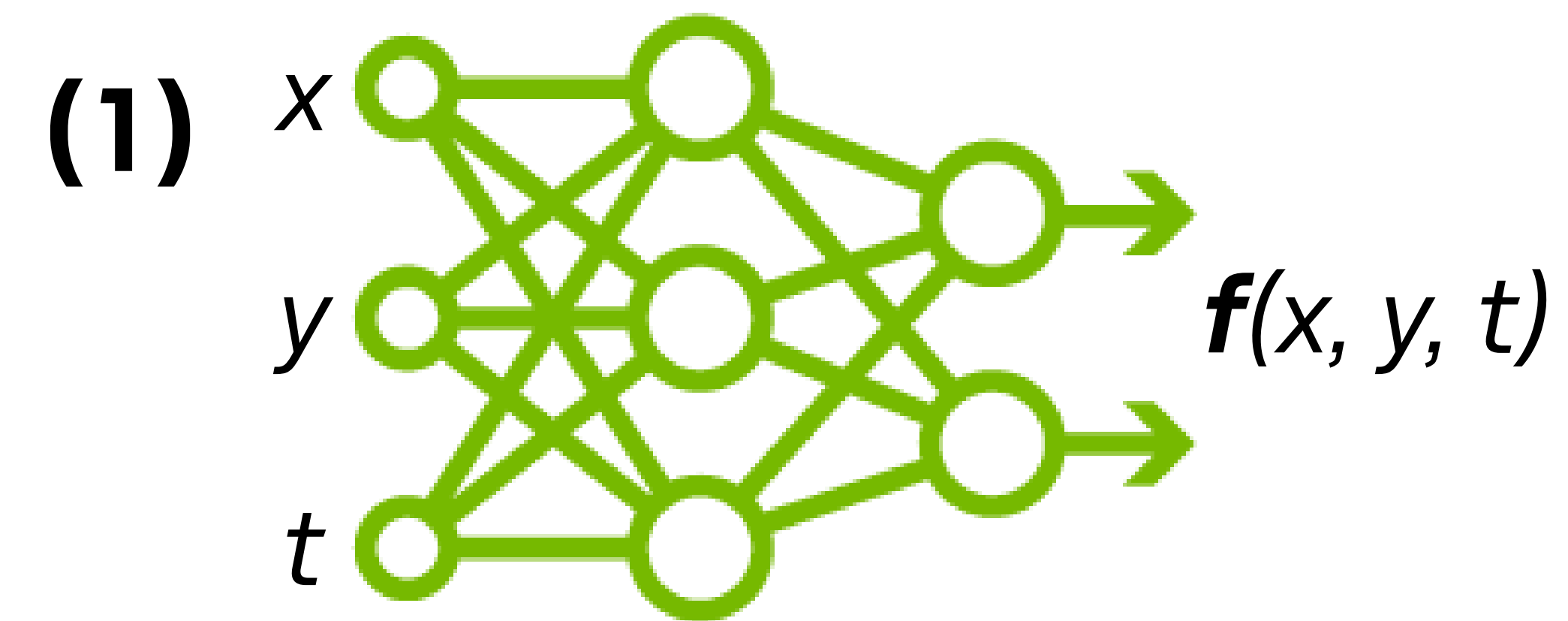


More Advanced PINNs

We can do so much with PINNs!

A single coherent differentiable lab bench for playing with PDEs

- This is a really flexible system.
- Do we want to do data assimilation – we have some (possibly noisy) experimental data we want to include into the solution?
 - Just add it like ICs or BCs! If it's very noisy, weight it downwards in the total loss, and train as usual
- Do we have a family of solutions we want to consider?
 - Parameterize the family and add the parameter as an input dimension, and train as usual
- Do we want to understand how the solution varies with the parameter?
 - We get derivatives “for free” – sensitivity analysis!
- Are we missing some information about the equation? Maybe there's a viscosity we don't know or a subgrid model we're calibrating, and we have some data?
 - Solve the inverse problem – make the parameters learnable parameters, and train as usual
- These are hard to do with traditional PDE solvers!



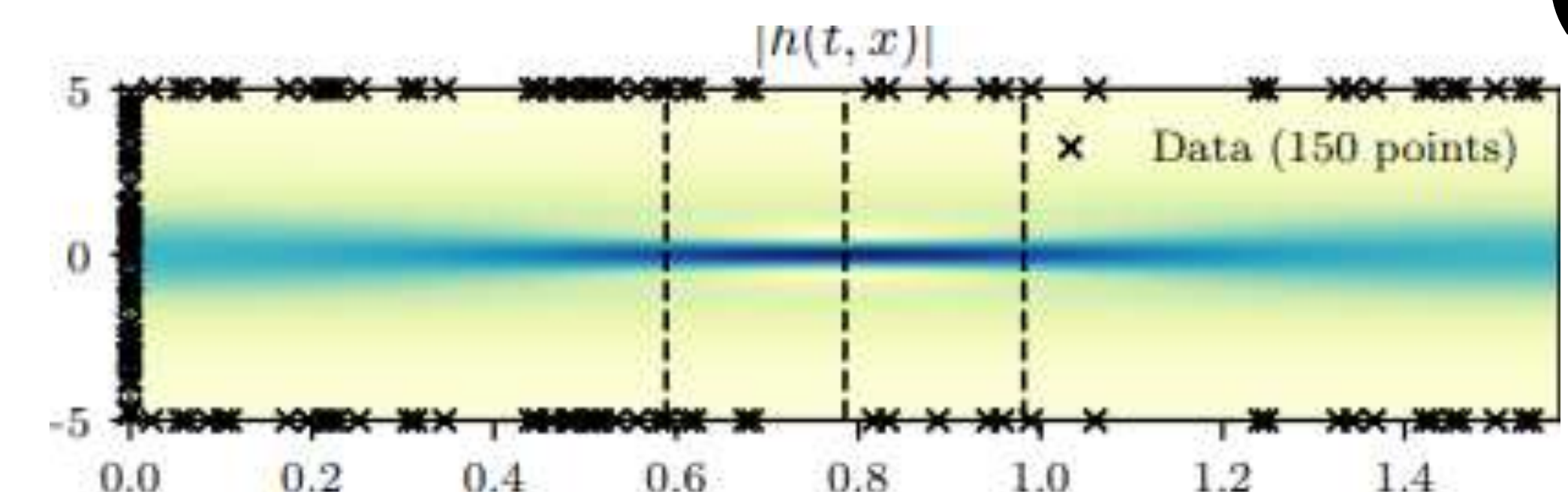
(2)

```
def f(t, x):  
    u = u(t, x)  
    u_t = tf.gradients(u, t)[0]  
    u_x = tf.gradients(u, x)[0]  
    u_xx = tf.gradients(u_x, x)[0]  
    f = u_t + u*u_x - (0.01/tf.pi)*u_xx  
    return f
```

(3)

$$L_{data} = \sum_{x_i \in data} (u_{net}(x_i) - u_{true}(x_i))^2$$

(4)



Parameterized 1D diffusion

Problem description

- Composite bar with material of conductivity D_1 for $x \in (0,1)$ and $D_2 = 0.1$ for $x \in (1,2)$.
- Solve the problem for multiple values of D_1 in the range (5, 25) in a single training
- Same boundary and interface conditions as before
- Equations: Diffusion equation in 1D

$$\frac{d}{dx} \left(D_1 \frac{dU_1}{dx} \right) = 0$$

When $0 < x < 1$

$$\frac{d}{dx} \left(D_2 \frac{dU_2}{dx} \right) = 0$$

When $1 < x < 2$

- Flux and field continuity at interface ($x=1$)

$$\begin{aligned} \left(D_1 \frac{dU_1}{dx} \right) &= \left(D_2 \frac{dU_2}{dx} \right) \\ U_1 &= U_2 \end{aligned}$$

Parameterized Problems

Problem definition

- Consider the parameterized version of the same problem as before. Suppose we want to determine how the solution changes as we move the position on the boundary condition $u(l) = 0$
- Parameterize the position by variable $l \in [1, 2]$ and the problem now becomes:

$$\mathbf{P}: \begin{cases} \frac{\delta^2 u}{\delta x^2}(x) = f(x) \\ u(0) = u(l) = 0 \end{cases}$$

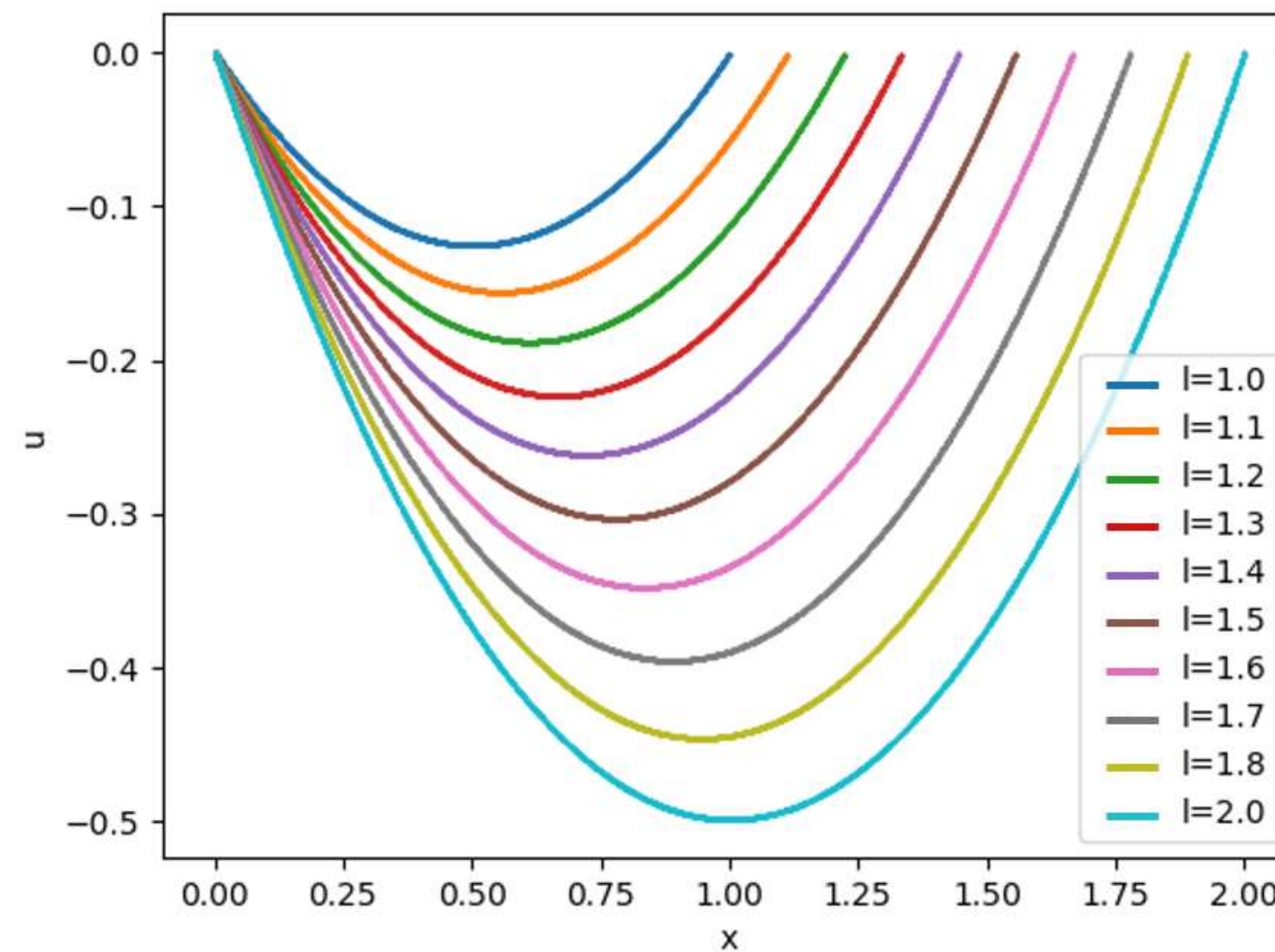
- This time, we construct a neural network $u_{net}(x, l)$ which has x and l as input and single value output $u_{net}(x, l) \in \mathbb{R}$.
- The losses become

$$L_{Residual} = \int_1^2 \int_0^1 \left(\frac{\delta^2 u_{net}}{\delta x^2}(x) - f(x) \right)^2 dx dl \approx \left(\int_1^2 \int_0^1 dx dl \right) \frac{1}{N} \sum_{i=0}^N \left(\frac{\delta^2 u_{net}}{\delta x^2}(x_i, l_i) - f(x_i) \right)^2$$
$$L_{BC} = \int_1^2 \left(u_{net}(0, l) \right)^2 + \left(u_{net}(l, l) \right)^2 dl \approx \left(\int_1^2 dl \right) \frac{1}{N} \sum_{i=0}^N \left(u_{net}(0, l_i) \right)^2 + \left(u_{net}(l_i, l_i) \right)^2$$

Parameterized Problems

Results

- For $f(x) = 1$, for different values of l we have different solutions



Solution to the parametric problem

What if we need hard constraints?

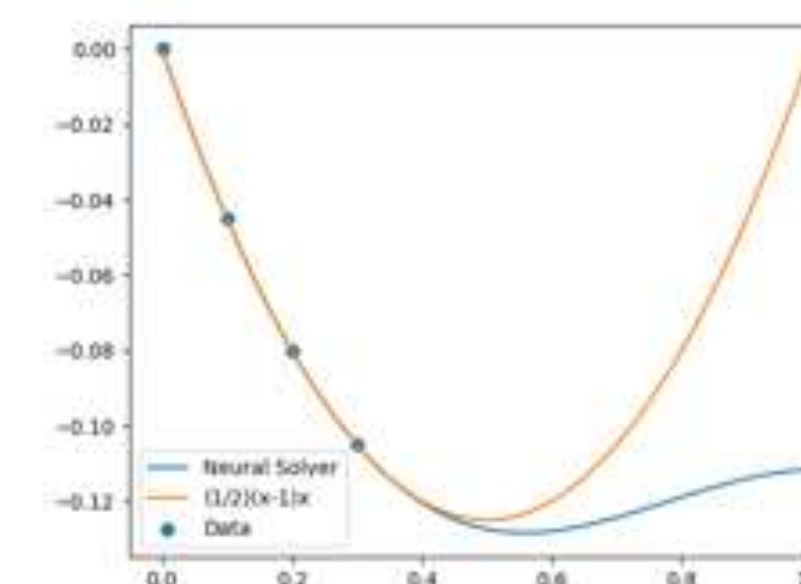
Modify the structure of the problem to go beyond soft losses

- Conservation of energy as a “soft” constraint?!?
 - Can actually work pretty well – we’re not doing a timestepping method (time is meshless, too), so we don’t have to worry about cascading errors
- But sometimes there’s a symmetry or a constraint that if not met will make the solution invalid, or hard to solve
- Generally make an ansatz of some sort
 - Have to have (say) zero at $x = x_l$ and $x = x_r$? If the NN output is f_N , have the solution be $(x - x_l)(x - x_r)f_N$. Basically a weird additional layer on the network
 - Add a second network and combine them (there’s an example of this in the notebook) – this can be used for inverse problems, too.

Adding Physics laws as soft constraints

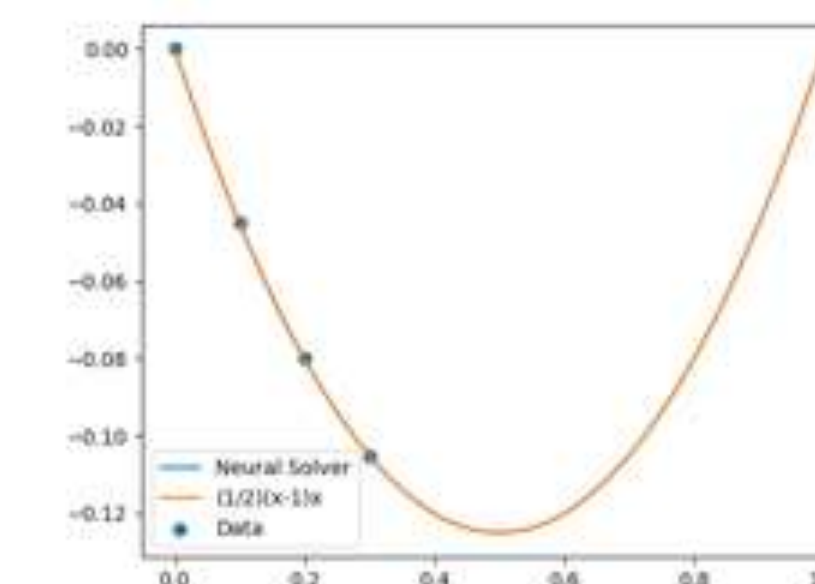
Data Only

$$L_{data} = \sum_{x_i \in \text{data}} (u_{net}(x_i) - u_{true}(x_i))^2$$
$$L_{total} = L_{data}$$



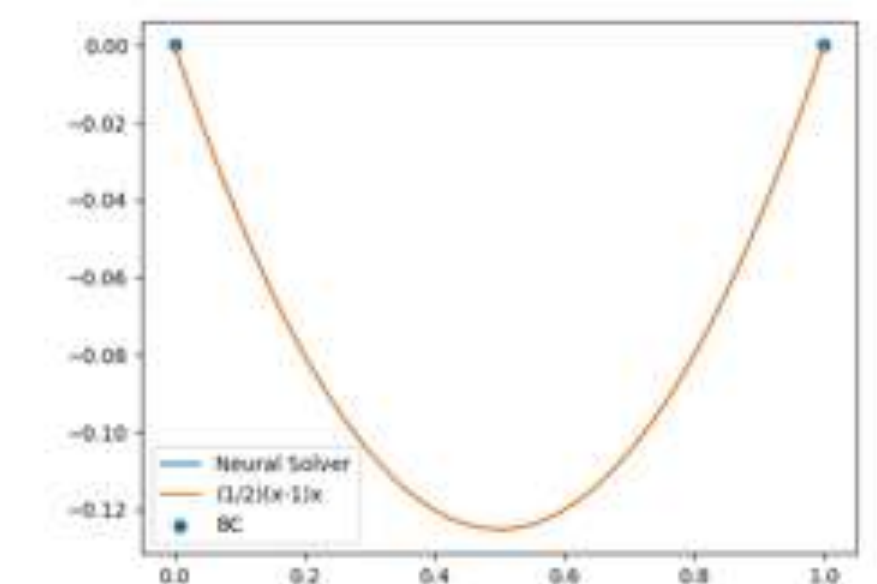
Data + Physics

$$\frac{\delta^2 u}{\delta x^2}(x) = 1$$
$$L_{physics} = \sum_{x_j \in \text{domain}} \left(\frac{\delta^2 u_{net}}{\delta x^2}(x_j) - f(x_j) \right)^2$$
$$L_{total} = L_{data} + L_{physics}$$



Physics only

$$\frac{\delta^2 u}{\delta x^2}(x) = 1 \quad u(0) = u(1) = 0$$
$$L_{physics} = L_{residual} + L_{BC}$$
$$L_{total} = L_{physics}$$



Inverse Problems

Problem definition

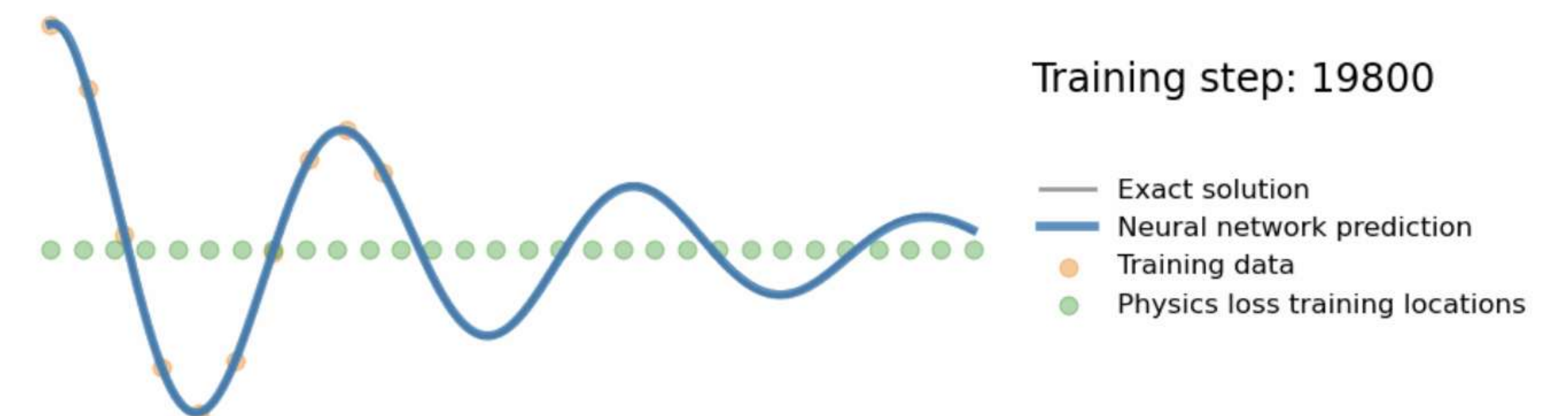
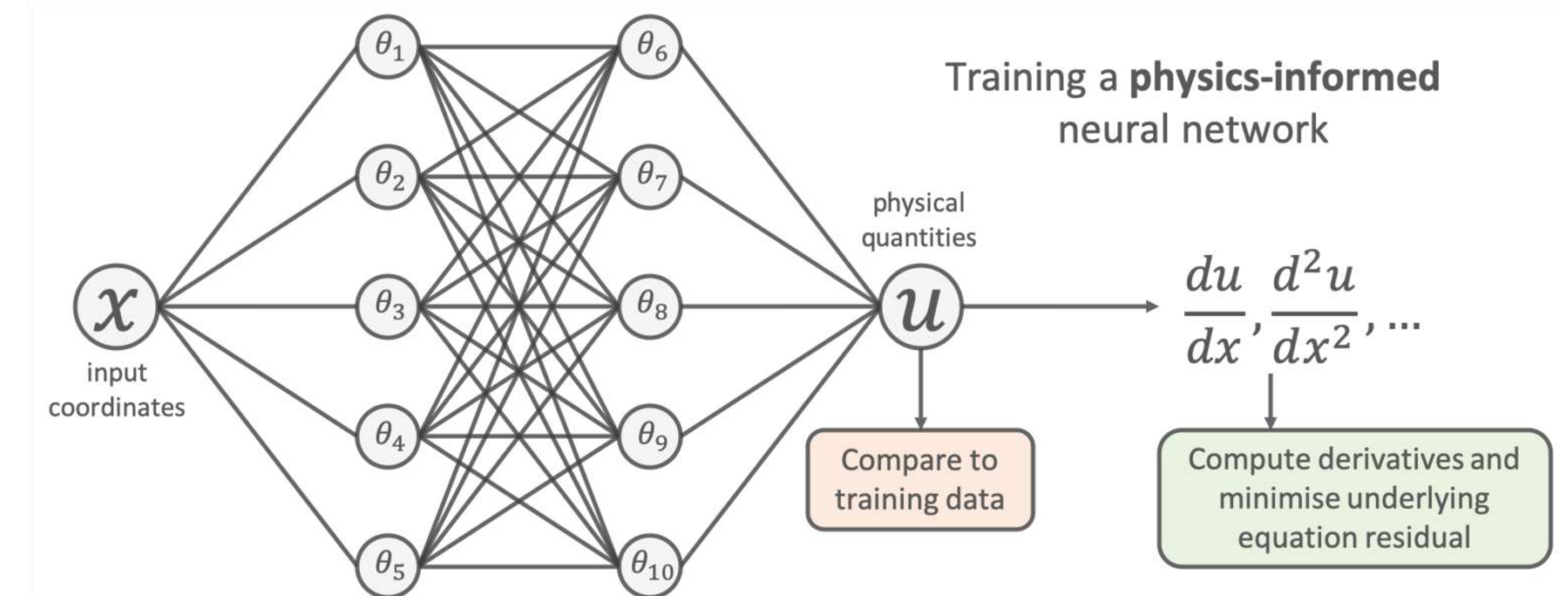
- For inverse problems, we start with a set of observations and then calculate the causal factors that produced them
- For example, suppose we are given the solution $u_{true}(x)$ at 100 random points between 0 and 1 and we want to determine the $f(x)$ that is causing it
- Train two networks $u_{net}(x)$ and $f_{net}(x)$ to approximate $u(x)$ and $f(x)$

$$L_{Residual} \approx \left(\int_0^1 dx \right) \frac{1}{N} \sum_{i=0}^N \left(\frac{\delta^2 u_{net}}{\delta x^2}(x_i) - f(x_i) \right)^2$$

$$L_{Data} = \frac{1}{100} \sum_{i=0}^{100} (u_{net}(x_i) - u_{true}(x_i))^2$$

Physics Informed Neural Networks - PINNs

- Idea: Integrate fundamental laws of physics within the network's learning framework.
- How? Add the known differential equations directly into the loss functions of the neural network's training process.
- Now loss function has 2 components:
 - Measure discrepancy between network's predictions and training data points.
 - Quantifies deviation of predictions from the physical model described by differential equations.
 - By minimizing both, the network learns to predict outcomes adhering to both observed data and the physical principles.



PINN Variants Address Different Challenges

Fourier features, domain decomposition...

- PINNs as we've described them have many strengths:
 - Easily connect to parameterized problems, inverse problems, data assimilation...
 - Work extremely well on problems with smooth, well-behaved solutions with low helicity
 - Work exceedingly well on high-dimensional problems (7d radiation transfer, high-dimensional option pricing...) that are smooth
- But there are challenges that many variants have been created to address
 - Training can be challenging
 - Well known spectral bias issue with neural networks

PINN Variants Address Different Challenges

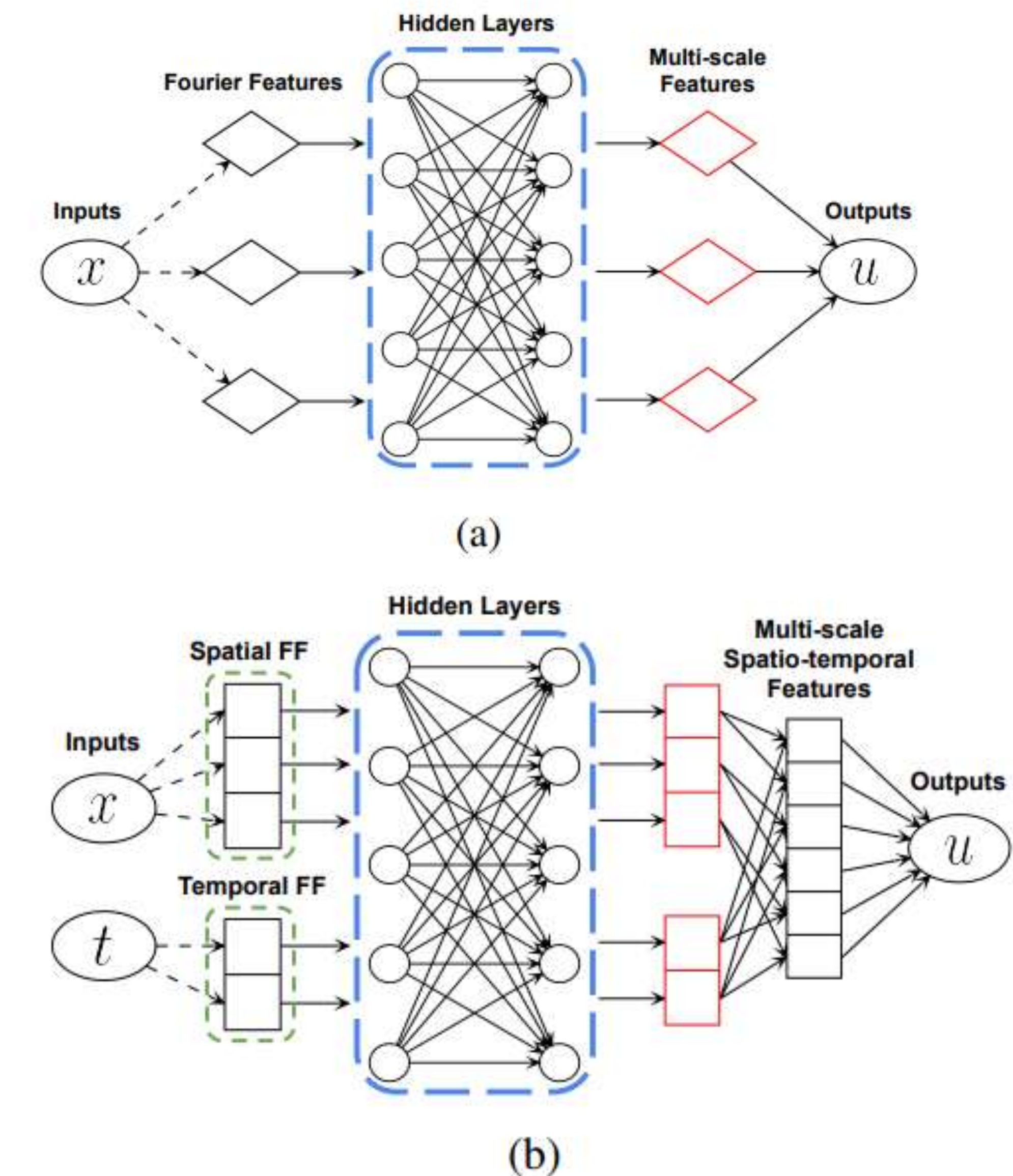
Fourier features, domain decomposition...

- PINNs as we've described them have many strengths:
 - Easily connect to parameterized problems, inverse problems, data assimilation...
 - Work extremely well on problems with smooth, well-behaved solutions with low helicity
 - Work exceedingly well on high-dimensional problems (7d radiation transfer, high-dimensional option pricing...) that are smooth
- But there are challenges that many variants have been created to address
 - Training can be challenging
 - How to handle multiple losses?
 - Might need large number of collocation points for convergence -
 - Well known spectral bias issue with neural networks

Addressing Spectral Bias w/ Fourier Features

Many approaches to this – spectral methods as adapted to PINNs

- Normally in numerical methods we're worried about the explosive growth of very high-frequency modes
- But neural nets **strongly** damp even moderate frequencies
- Want to maintain accuracy across spectrum of modes – spectral methods
- An approach that's proved useful is using Fourier features – $\sin(\mathbf{A} x)$, $\cos(\mathbf{A} x)$ with learnable $\mathbf{A}$.
- Other approaches being taken as well
 - Work specifically in Fourier basis, use in place of auto-differentiation
 - Similar approach but use wavelet features, handles local structures like discontinuities well



[Wang, Wang, Perdikaris \(2021\), J. CMAME 384](#)

Addressing some Computational Challenges w/ Adaptive Methods

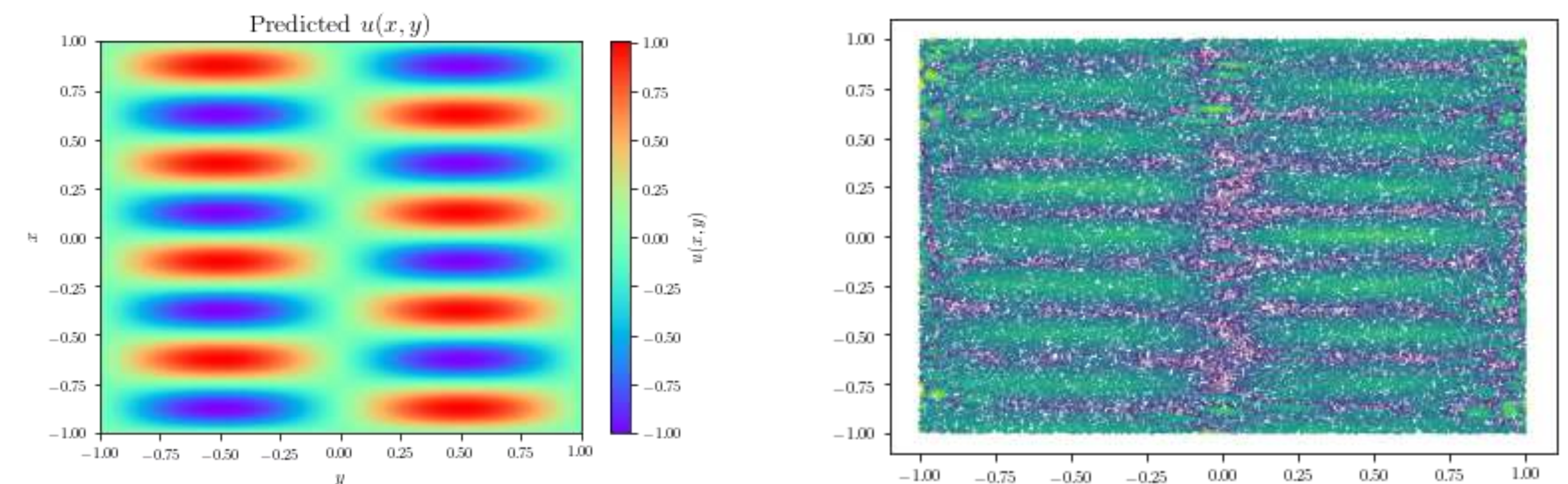
Learn loss weights, collocation points...

- We saw above that we combine the losses... but how do we know how to weigh them?
- Should the weighing even be the same across the domain?
- Adaptive losses – lambdas are learned values or learned functions
- “Soft Attention Mechanism”

$$L_{data} = \sum_{x_i \in data} (u_{net}(x_i) - u_{true}(x_i))^2$$

$$L_{physics} = \sum_{x_j \in domain} \left(\frac{\delta^2 u_{net}}{\delta x^2}(x_j) - f(x_j) \right)^2$$

$$L_{total} = \lambda_d L_{data} + \lambda_p L_{physics}$$

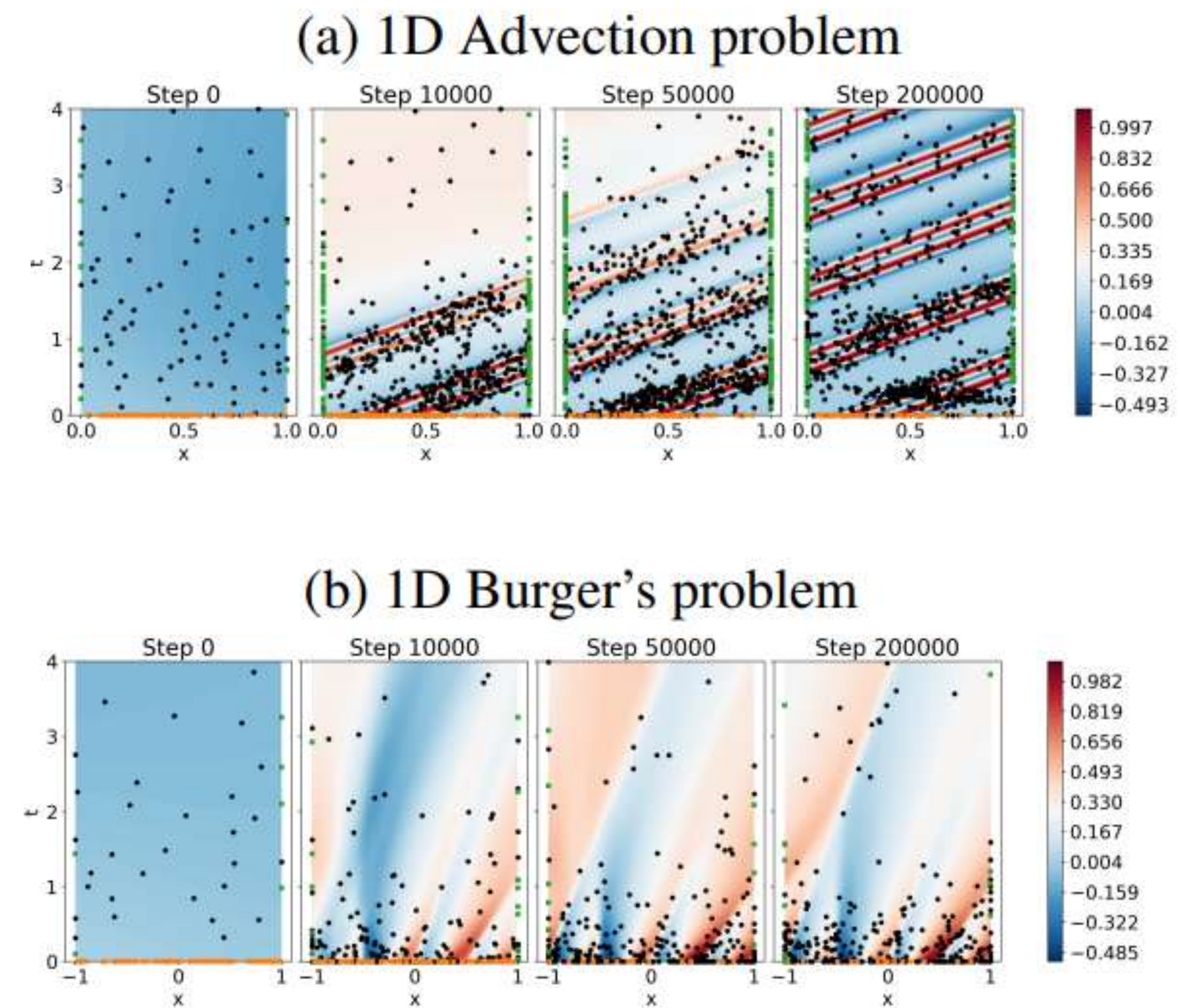


[McClenny & Braga-Neto \(2023\) JCP 474](#)

Addressing some Computational Challenges w/ Adaptive Methods

Learn loss weights, collocation points...

- Similarly, collocation points for training can be moved to where residual is high or rapidly changing, and moved away from or removed from areas with small residuals



[Ruey Lau, Hemachandra, Ng, Low \(2024\) ICLR 2024](#)

Domain Decomposition and PINNs

Effective parallelization, handling complex domains...

- There's a 1-d example of this in the notebooks associated with this talk – used to enforce a hard BC
- Classic domain decomposition, where domain is broken up into disjoint or overlapping domains, and required continuity conditions are enforced on boundary/overlap

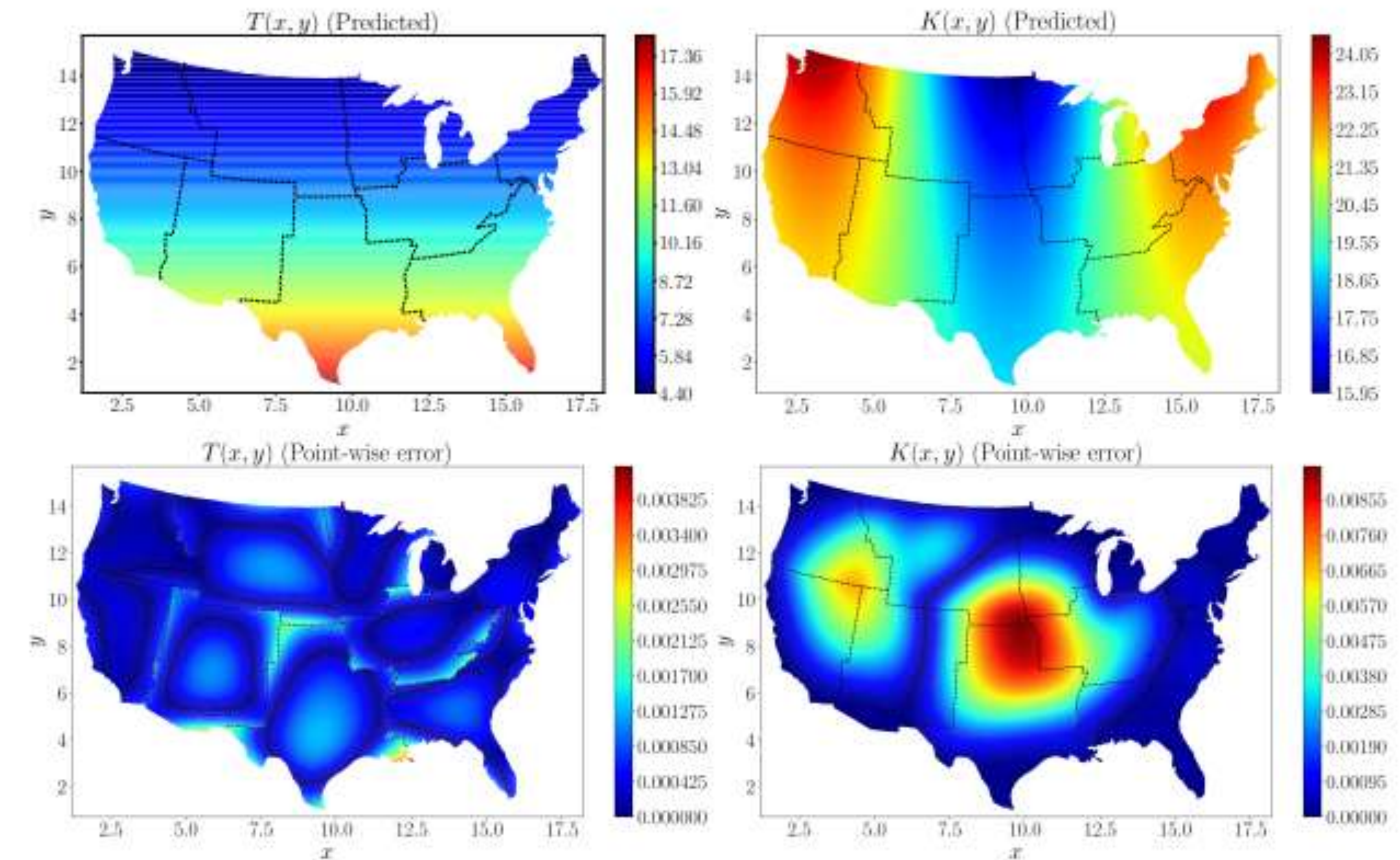


Figure 12: Steady-state heat conduction with variable conductivity: The first row shows the contour plots for the predicted temperature $T(x, y)$ and thermal conductivity $K(x, y)$ while the second row shows the corresponding absolute point-wise errors.

[Shukla, Jagtap, Karniadakis \(2021\) arXiv 2104.10013](#)

Some Good References To Get Started

PINN papers, reviews, and videos

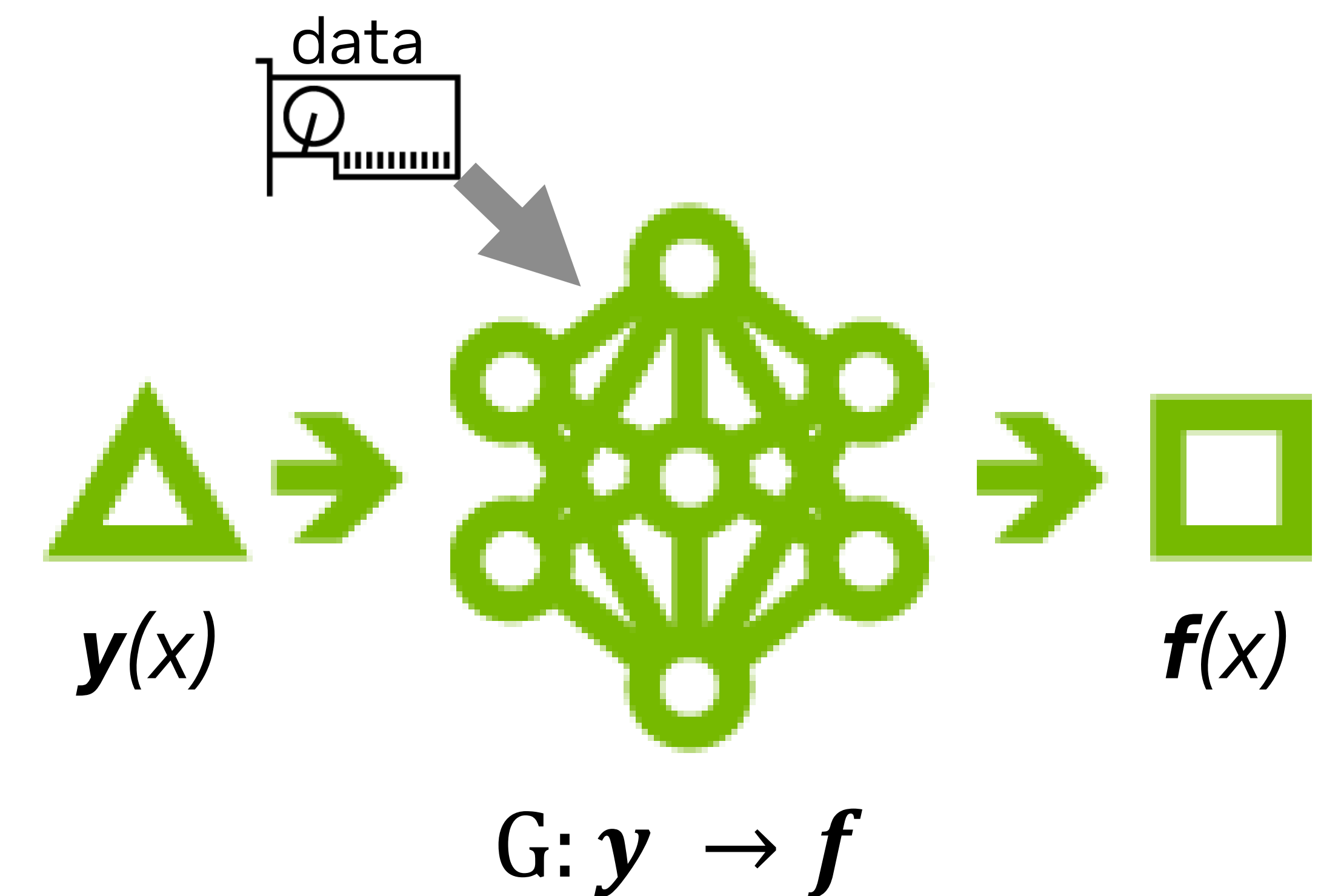
- [Raissi, Perdikaris, & Karniadakis \(2019\) JCP 378:686-707](#) - the first two papers combined and extended. Code at <https://github.com/maziarraissi/PINNs>
- [Cuomo, Di Cola, Giampaolo, Rozza, Raissi, Piccialli \(2022\) J Sci Comp 92:88](#), “Scientific machine learning through physics-informed neural networks: Where we are and what's next” Nice accessible overview of PINNs and early variants
- [Cai, Mao, Wang, Yin, Karniadakis \(2021\) arXiv:2105.09506](#), “Physics-informed neural networks (PINNs) for fluid mechanics: A review” – There’s lots of these reviews for PINNs for [X], which are a great way to ramp up if there’s a particular area you’ll be working in
- [Karniadakis, Kevrekidis, Lu, Perdikaris, Wang, Yang, \(2021\) Nat Rev Phys 3:422](#) – Bigger picture overview, including application areas
- [Mishra, Moseley \(2024\), “AI in the Sciences and Engineering \(2024\)”](#) – Lovely set of lectures (videos, slides) and tutorial exercises on PINNs, Operators, and more
- <https://github.com/openhackathons-org/End-to-End-AI-for-Science> - Set of notebooks for playing with PINNs and Operators in PhysicsNeMo

Operators: Data driven approach

Why not just use PINNs?

The case for learning an operator instead of a function

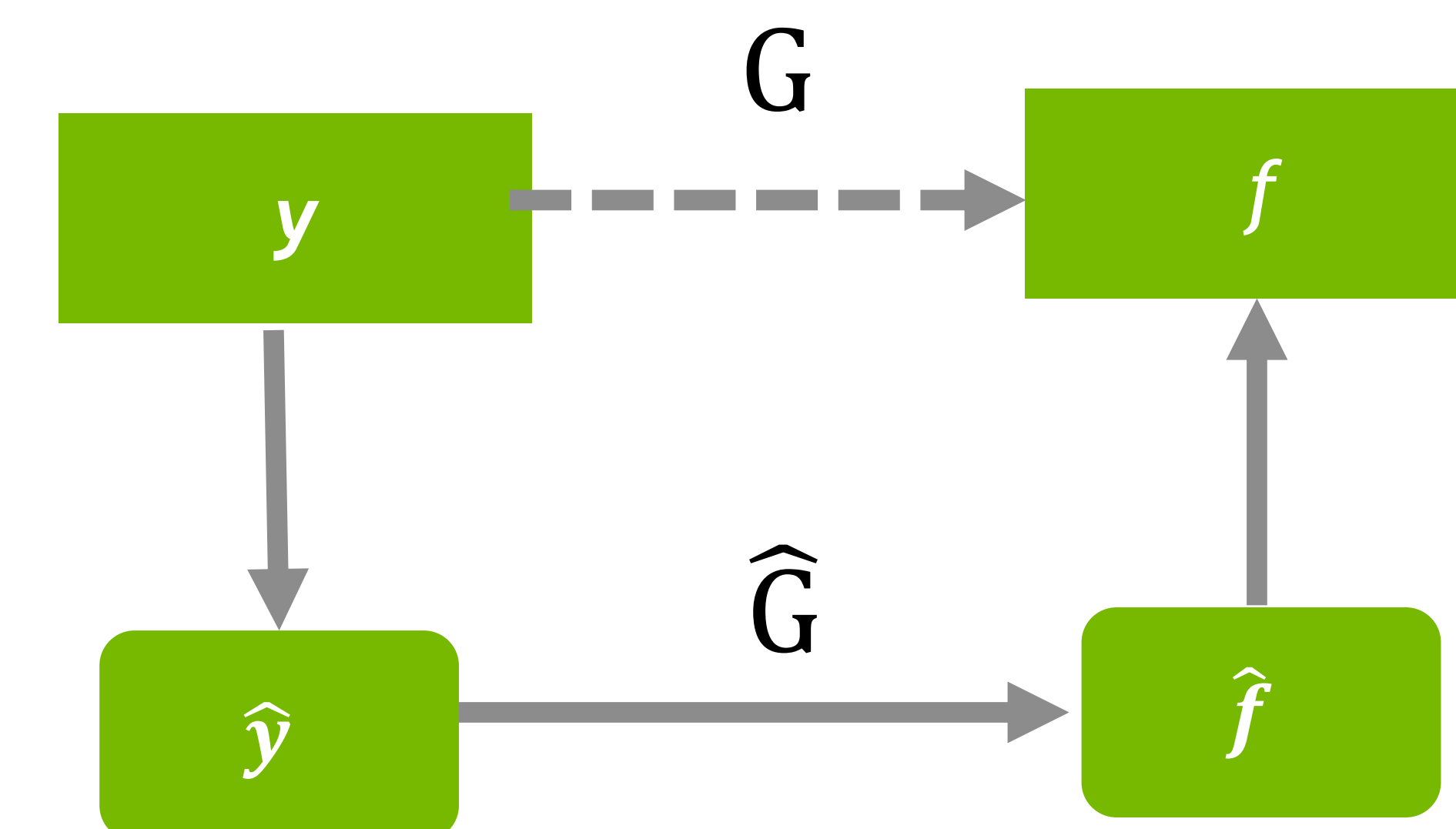
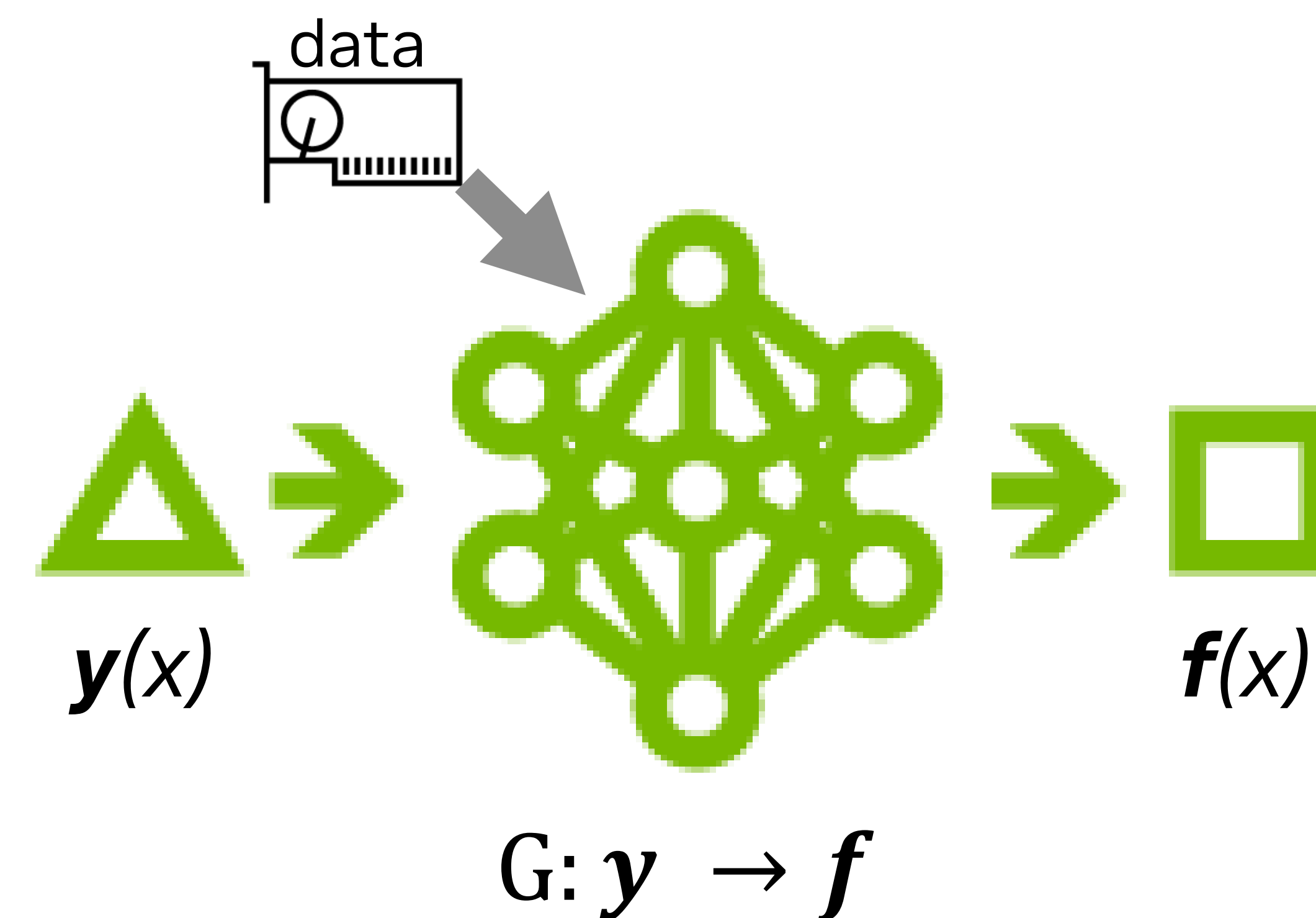
- PINNs work really well for a lot of problems
- But they can be computationally expensive to use:
 - The derivative calculations grow the computational graph, quite a bit for higher order equations
 - Training is complicated, and..
 - You have to retrain for every different set of ICs/BCs! (parameterization helps with this)
- PINNs learn a solution, a function. A traditional PDE solver is more like an operator – mapping from an input function to an output function



How Would Learning An Operator Even Work?

Handling mappings between infinite-dimensional objects

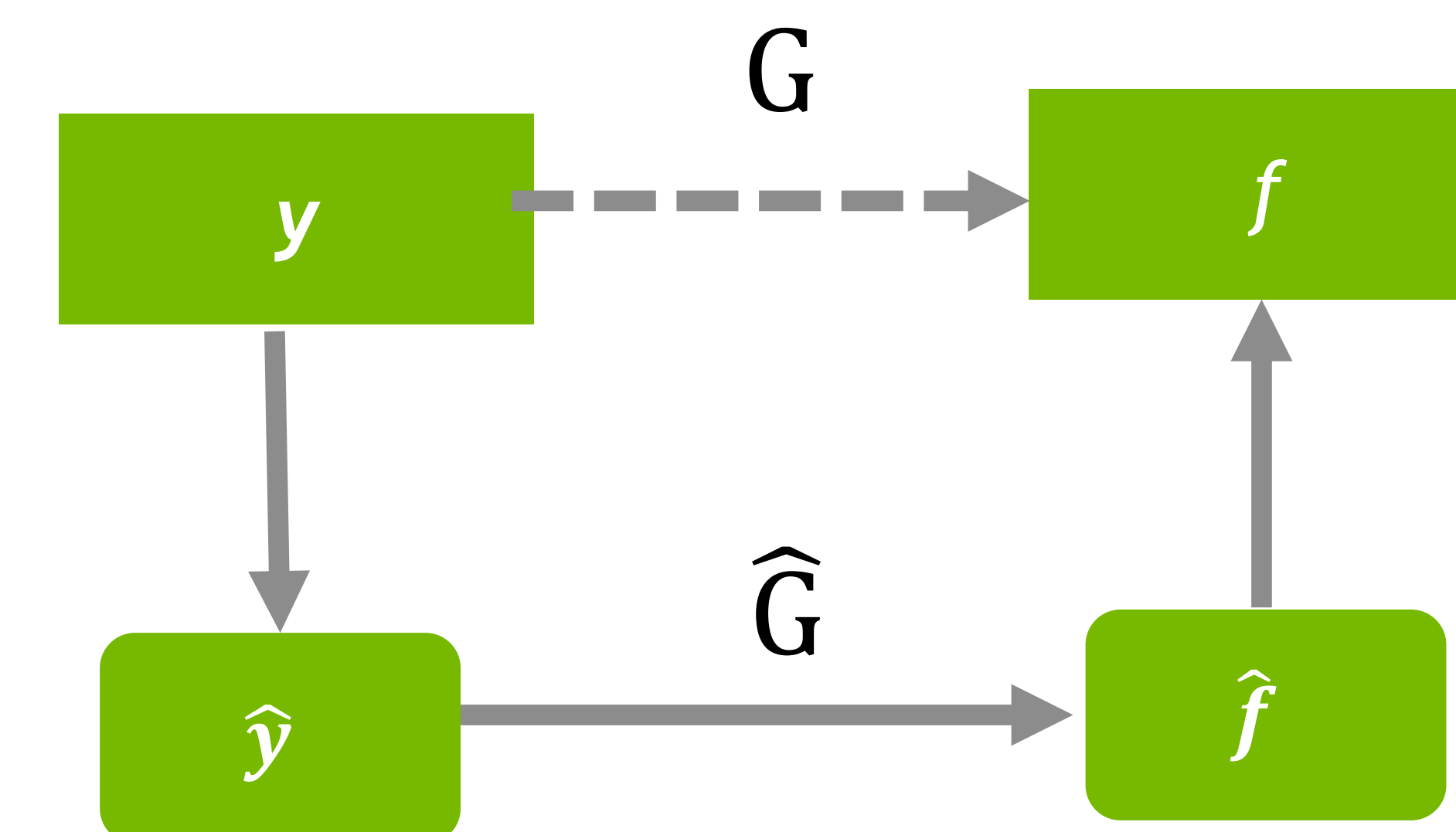
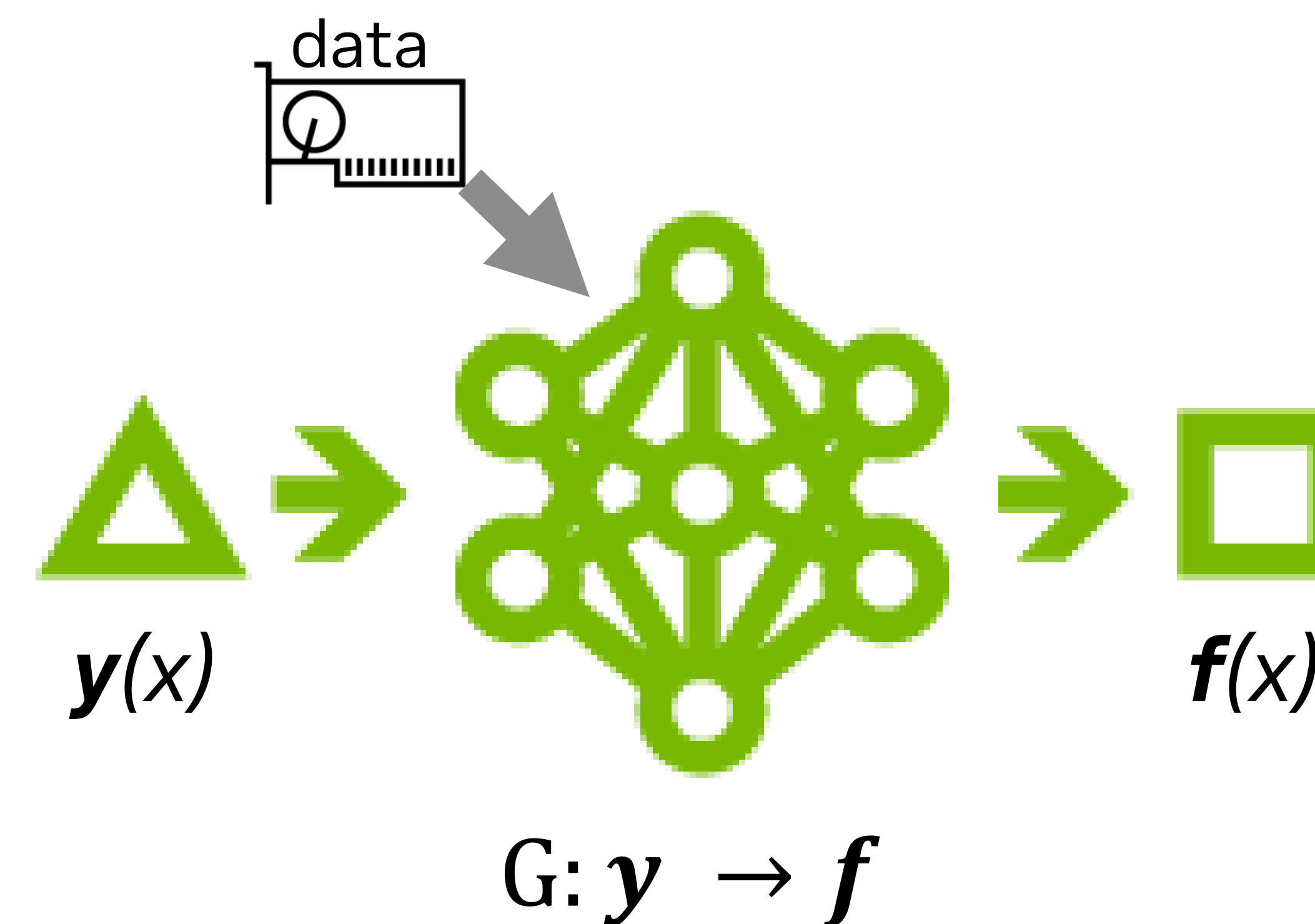
- It turns out there are universal approximation to nonlinear operator theorems, too (e.g. [Chen & Chen \(1995\) IEEE Tran. Neur. Net.](#))
- How do you handle an infinite dimensional input?
- Very traditional numerical approach – we don't
 - Project into some reduced-dimensionality space in some way
 - Work in that reduced space
 - Extrapolate back into the full space
- In contrast to PINNs, which needed no real training data, operator learning requires lots of training samples of input and output functions
 - Experimental data
 - Surrogate data such as simulations
- Training is very expensive, but can then inference very quickly with different ICs (and maybe BCs)



How Would Learning An Operator Even Work?

Handling mappings between infinite-dimensional objects

- It turns out there are universal approximation to nonlinear operator theorems, too (e.g. [Chen & Chen \(1995\) IEEE Tran. Neur. Net.](#))
- How do you handle an infinite dimensional input?
- Very traditional numerical approach – we don't
 - Project into some reduced-dimensionality space in some way
 - Work in that reduced space
 - Extrapolate back into the full space
- In contrast to PINNs, which needed no real training data, operator learning requires lots of training samples of input and output functions
 - Experimental data
 - Surrogate data such as simulations
- Training is very expensive, but can then inference very quickly with different ICs (and maybe BCs)



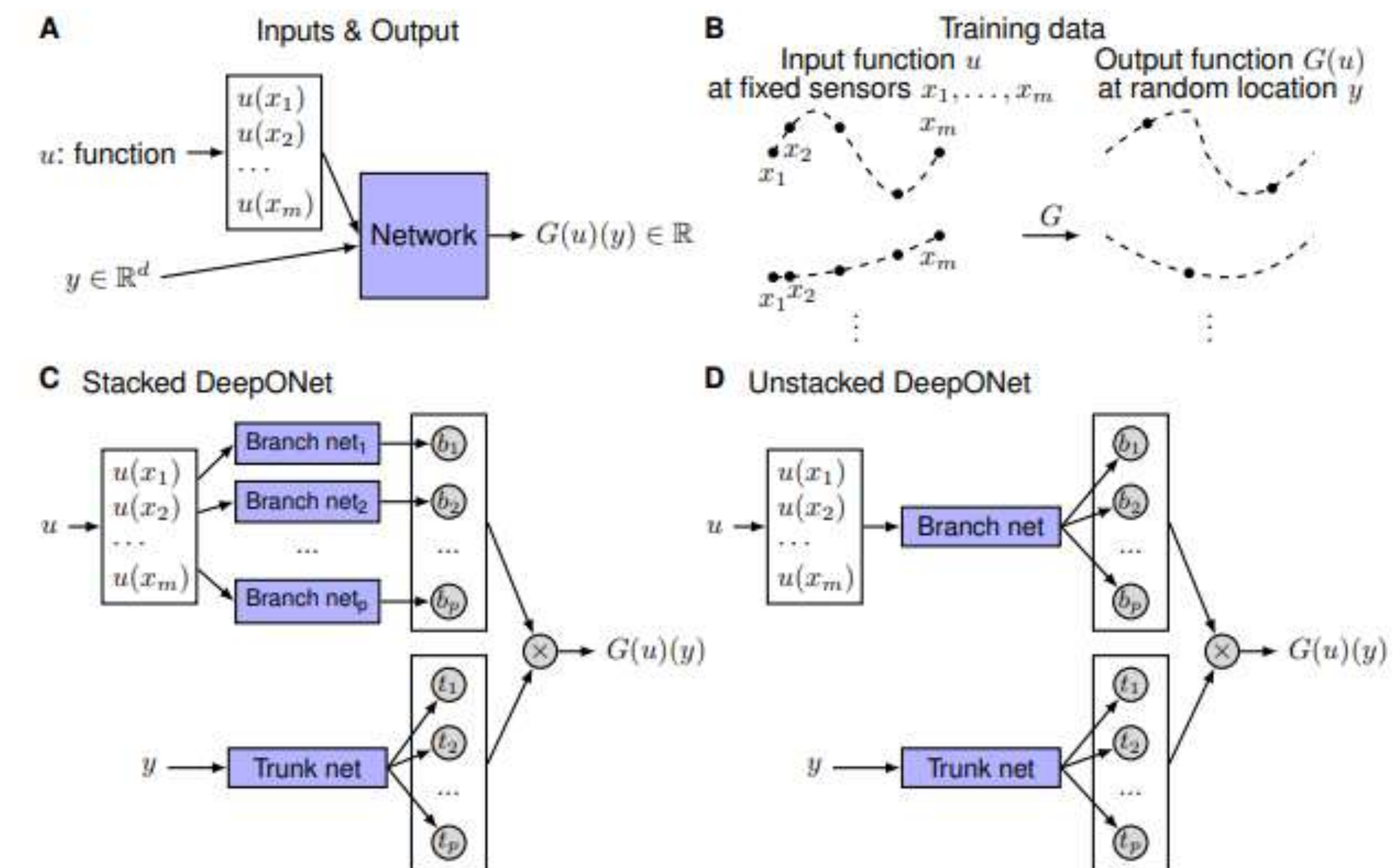
DeepONets

Example of Encoder-Decoder approach

- An early operator approach is DeepONets
- Two different networks – one on the inputs, one on query points within the domain of the output
- Those are combined (inner product) to produce the output
- ICs/BCs are discretized and passed as inputs to the “Branch net”
 - Learns latent representation of input function
 - From fixed mesh representation – higher or lower input resolution would require different network
- As with PINNs, single query points are provided to the trunk network
 - Tradeoff: more flexible, but more expensive if want the whole domain/much cheaper if only want a single point

DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators

Lu Lu¹, Pengzhan Jin², and George Em Karniadakis¹



[Lu, Jin, Karniadakis \(2021\) Nat Mach Intel 3:218](#)

DeepONets

Example of Encoder-Decoder approach

- This is a bit like an encoder-decoder
- Branch net is a learned encoder of the input space
- Trunk + combiner is a decoder

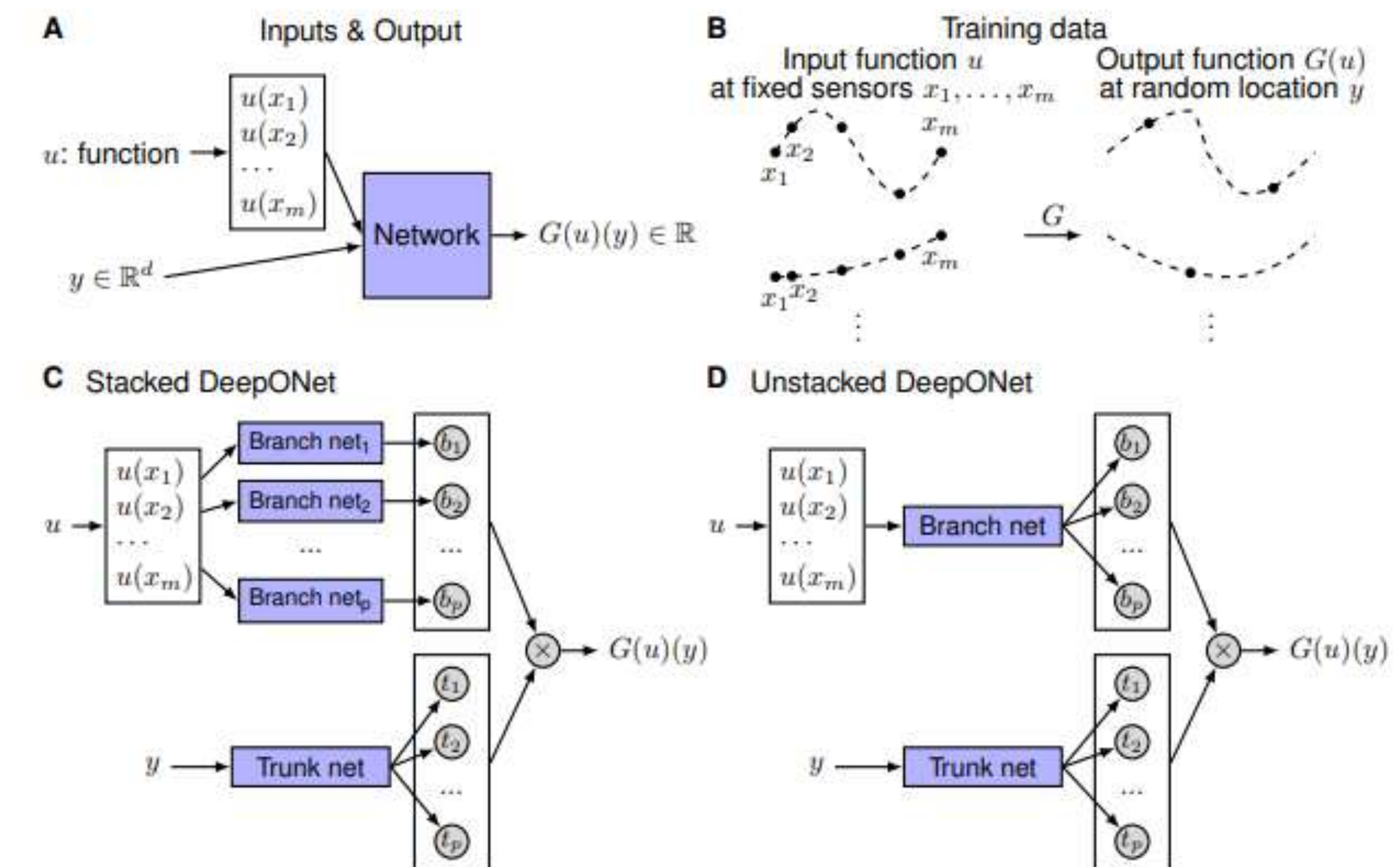
DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators

Lu Lu¹, Pengzhan Jin², and George Em Karniadakis¹

Method	Encoding	Finite-dim. Mapping	Reconstruction
FEM	Galerkin projection	Numerical scheme	Finite element basis
FVM	Cell averages	Numerical scheme	Piecewise polynomial
FD	Point values	Numerical scheme	Interpolation
PCA-Net	PCA projection	Neural network	PCA basis
DeepONet	Linear encoder	Neural network	Neural network basis

TABLE 1. Numerical schemes vs. Encoder-Decoder-Net.

[Kovachki, Lanthaler, Stuart \(2024\) arXiv:2402.15715](https://arxiv.org/abs/2402.15715)



[Lu, Jin, Karniadakis \(2021\) Nat Mach Intel 3:218](https://doi.org/10.1038/s42256-021-00362-1)

Fourier Neural Operators

Learning a more general operator approach

- A common approach to approximating a function – use a Fourier basis!
- The entire input is provided at once
- A “lifting” operator lifts the data into a higher space, adding room for additional features
- Then, a number of layers, each one of which is an operator:
 - Fourier transform
 - Filtering operation
 - Inverse Fourier transform
 - And then combine that with a feed-forward weight and activation
- Note! Have to be a little careful with activation functions here. The basic idea is that we’re operating in a reduced Fourier basis, the activation functions can add modes we’re not modelling...
- Note – we’re operating in Fourier space, with a projection down to a reduced Fourier basis – we’re not learning a representation-specific (e.g. resolution specific) operator!
- Although as practical matter assuming structured mesh (for FFTs)

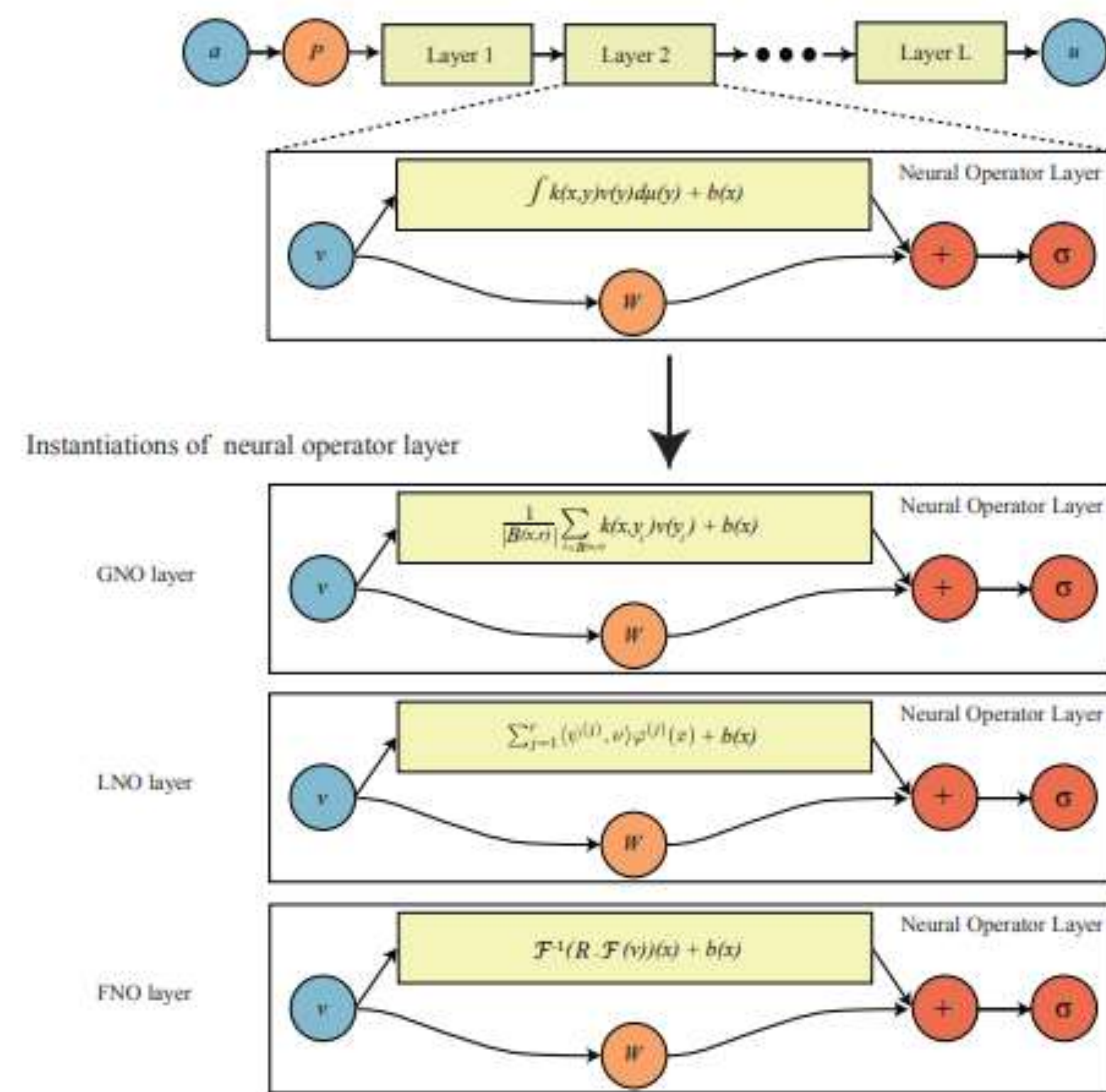


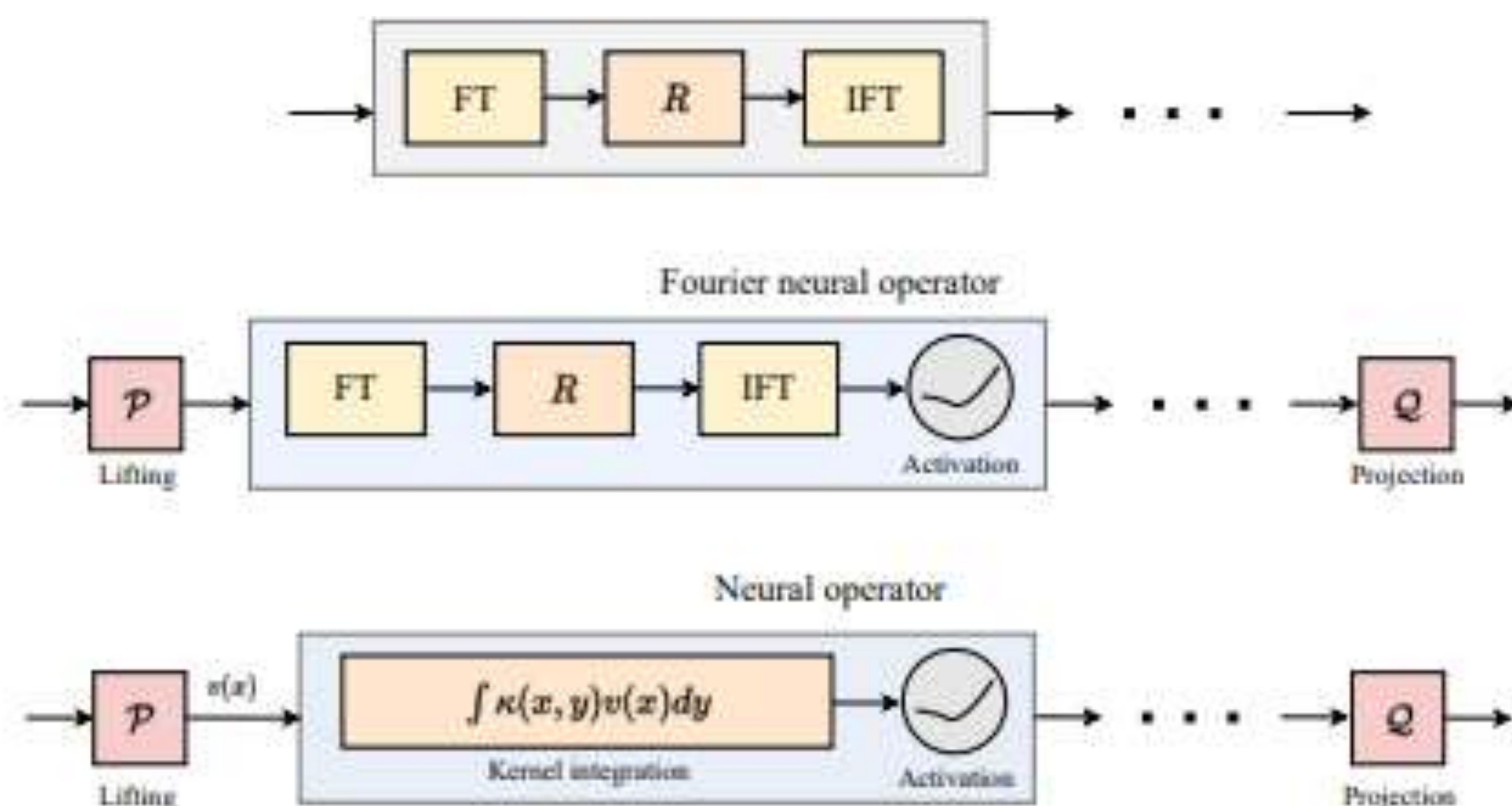
Figure 2: Neural operator architecture schematic

[Kovachki, Li, Liu, Azizzadenesheli, Bhattacharya, Stuart, Anandkumar \(2023\) JMLR 24:1](#)

Fourier Neural Operators

Learning a more general operator approach

- Here the model is more like generalizing neural nets to function spaces
- Looks like spectral operator
- Working on a sphere (e.g. climate?) generalize to spherical harmonics
- Can use other methods. Graph neural operator – the projection is not to a reduced Fourier basis but a spatial coarsening to a particular mesh



[Azizzadenesheli, Kovachki, Li, Liu-Schiaffini, Kossai, Anandkumar \(2023\) arXiv:2309.15325](#)

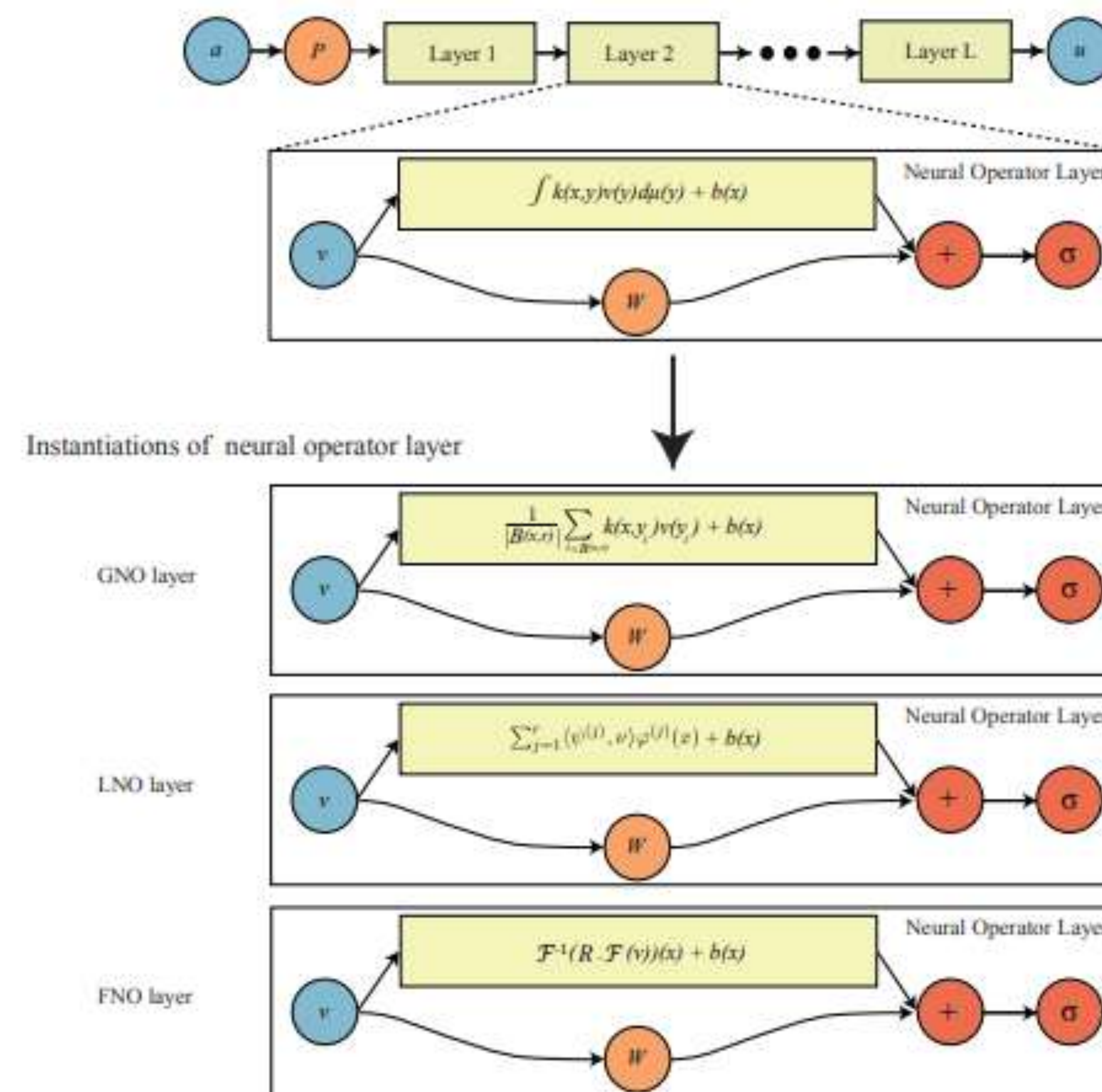


Figure 2: Neural operator architecture schematic

[Kovachki, Li, Liu, Azizzadenesheli, Bhattacharya, Stuart, Anandkumar \(2023\) JMLR 24:1](#)

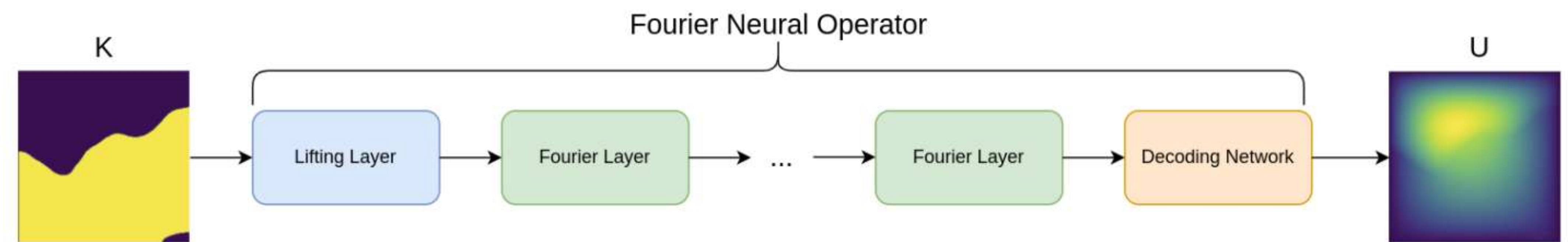
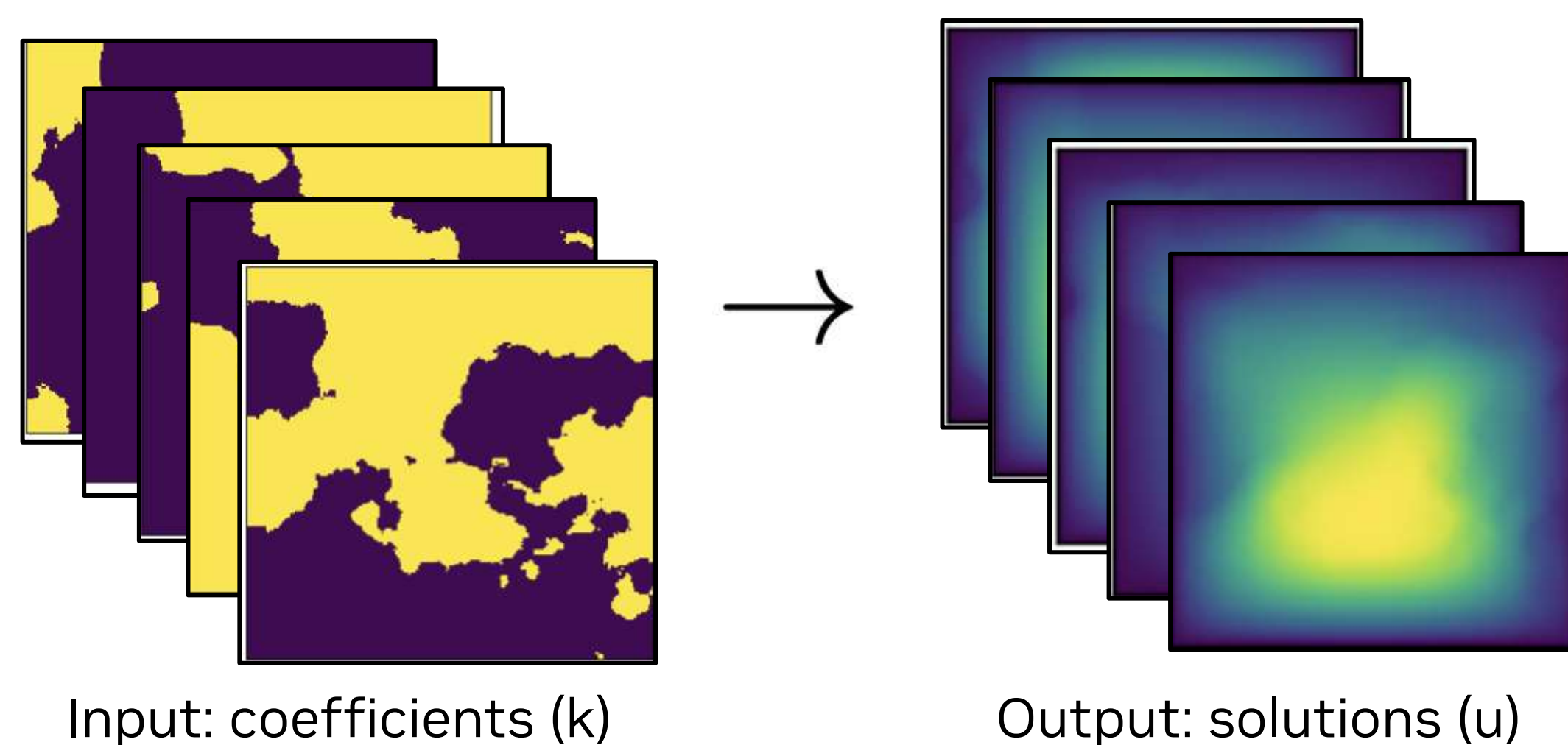
Developing Physics-ML models using data in PhysicsNeMo

Using FNOs to create a surrogate model

- Develop a surrogate model that learns a mapping between a permeability field and pressure field, $\mathbf{K} \rightarrow \mathbf{U}$, for a distribution of permeability fields $\mathbf{K} \sim p(\mathbf{K})$.
- The Darcy PDE (Diffusion equation) is a second order, elliptical PDE with the following form:

$$-\nabla \cdot (k(\mathbf{x}) \nabla u(\mathbf{x})) = f(\mathbf{x}), \quad \mathbf{x} \in D,$$

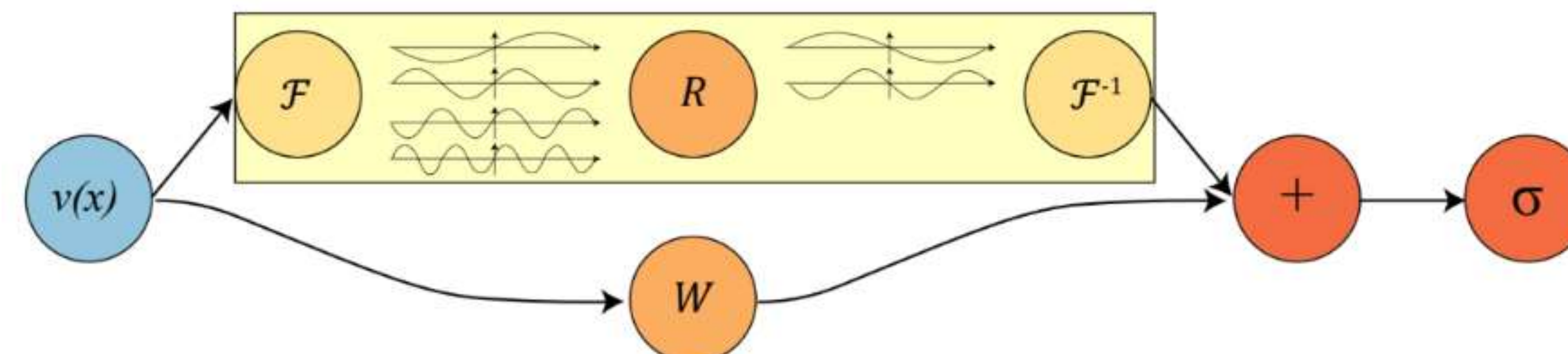
- $u(\mathbf{x})$: Flow pressure
- $k(\mathbf{x})$: Permeability field
- $f(\cdot)$: Forcing function
- $D = \{x, y \in (0,1)\}$ with the boundary condition $u(\mathbf{x}) = 0, \mathbf{x} \in \partial D$.
- Both the permeability and flow fields are discretized into a 2D matrix $\mathbf{K}, \mathbf{u} \in \mathbb{R}^{N \times N}$.



Developing Physics-ML models using data in PhysicsNeMo

Using FNOs to create a surrogate model

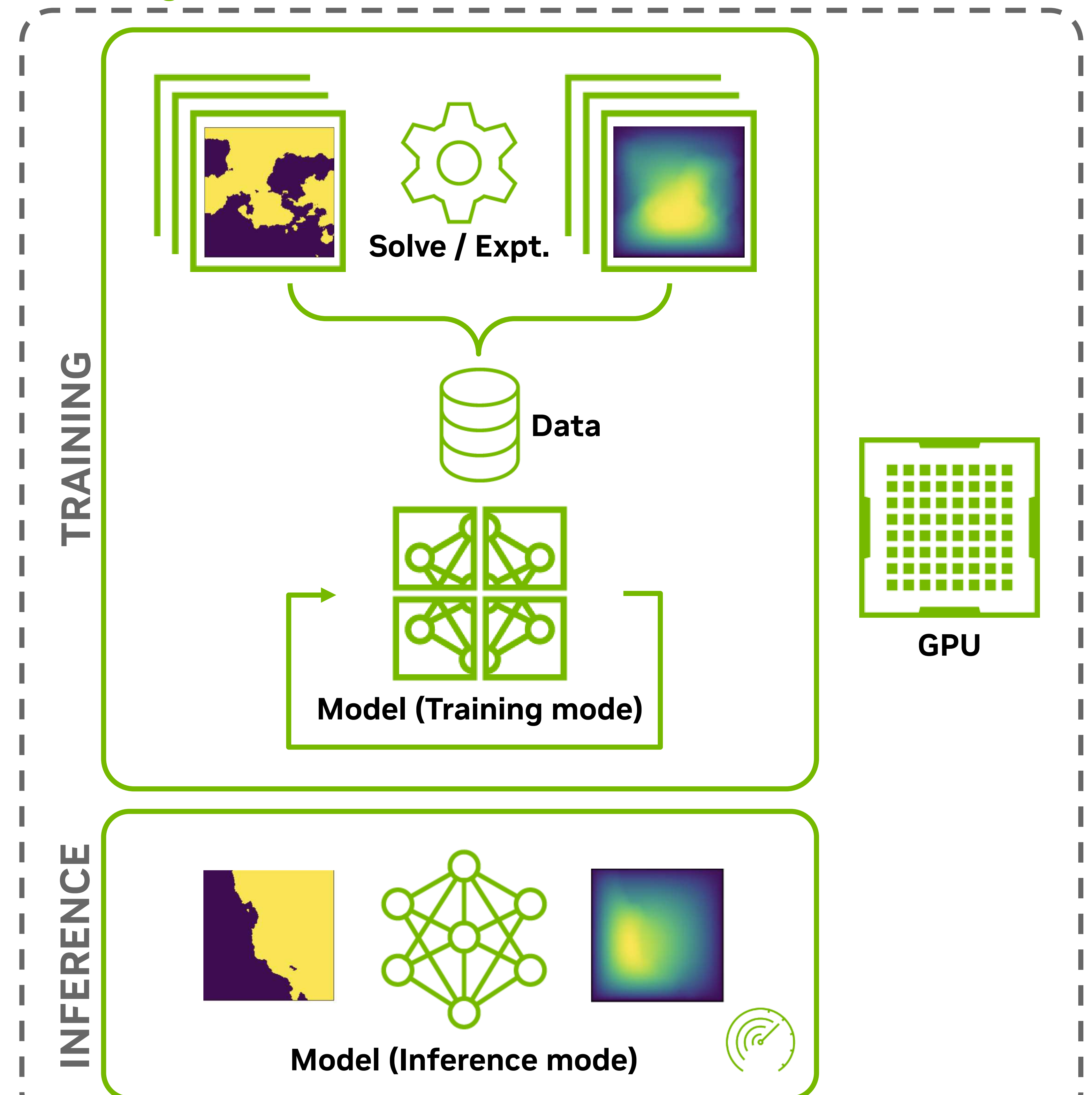
- The key feature of FNO is the spectral convolutions: operations that place the integral kernel in Fourier space:
$$(K(\mathbf{w})\phi)(x) = F^{-1}(R_{\mathbf{w}} \cdot (F)\phi)(x), \quad \forall x \in D$$
 - $R_{\mathbf{w}}$: Transformation containing the learnable parameters $\mathbf{w}$, calculated over the entire *structured Euclidean* domain D .
- Fourier coefficients are truncated to only the lower modes - explicit control over the dimensionality of the spectral space and linear operator.
- The FNO model is a composition of a fully-connected “lifting” layer, L spectral convolutions with point-wise linear skip connections and a decoding point-wise fully-connected neural network at the end:
$$u_{net}(\Phi; \theta) = Q \circ \sigma(\mathbf{W}_L + \mathbf{K}_L) \circ \dots \circ \sigma(\mathbf{W}_1 + \mathbf{K}_1) \circ P(\Phi), \quad \Phi = \{\phi(x); \forall x \in D\}$$
 - $\sigma(\mathbf{W}_i + \mathbf{K}_i)$: Spectral convolution layer i with the point-wise linear transform $\mathbf{W}_i$ and activation function $\sigma(\cdot)$.
 - P : Point-wise lifting network that projects the input into a higher dimensional latent space.
 - Q : Point-wise fully-connected decoding network.
- Since all fully-connected components of FNO are point-wise operations, the model is invariant to the dimensionality of the input.



Developing Physics-ML models using data in PhysicsNeMo

Using FNOs to create a surrogate model

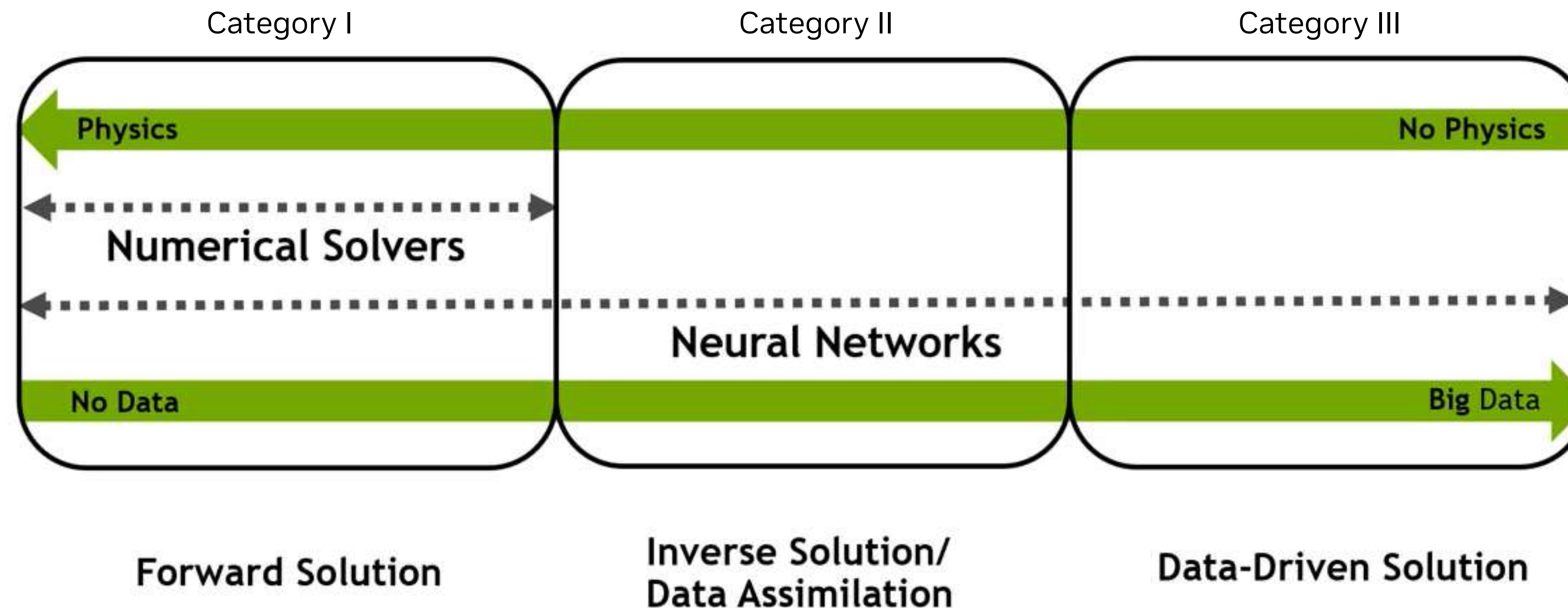
- Train a data-driven model:
 - Given input-output pairs, find the map (operator)
- Define the training loop using interoperable PhysicsNeMo-PyTorch utilities
 - Optimized data loaders and data pipes for domain specific use cases
 - Optimized Model Zoo and layers targeted for Physics-ML applications
 - Utilities for Distributed training and scaling
 - Easily apply performance optimizations like CUDA Graphs, AMP
 - Utilities for easier logging
 - Checkpointing utilities to save and load checkpoints
 - Load model checkpoints without model arguments
- Utilities for deployment and inference



They Can be Combined With Physics Loss - PINOs!

Physics Losses + Data-driven approaches

- We've talked now about the extremes on this diagram below – operators with no explicit physics but lots of data, and PINNs, with little/no data necessary but driven by the PDE
- These approaches combine – you can use more data with PINNs, you can add physics losses to operators (PINOs)



Some Good References To Get Started

Operator papers, reviews, and videos

- [Kovachki, Li, Liu, Azizzadenesheli, Bhattacharya, Stuart, Anandkumar \(2023\) JMLR 24:1](#), “Neural Operator: Learning Maps Between Function Spaces With Applications to PDEs” – describes FNOs and a framework for thinking about operators
- [Guibas et al. \(2022\) ICLR](#) “Adaptive Fourier Neural Operators: Efficient Token Mixers for Transformers” — ViT with Fourier token mixing, $O(N \log N)$ instead of $O(N^2)$ attention.
- [Azizzadenesheli et al. \(2024\) Nat Rev Phys 6:320](#) “Neural Operators for Accelerating Scientific Simulations and Design” Nature Reviews Physics survey, accessible and big-picture
- [Kovachki, Lanthaler, Stuart \(2024\) arXiv:2402.15715](#), “Operator Learning: Algorithms and Analysis” – comparison of a few key methods from an approximation theory point of view
- [Mishra, Moseley \(2024\), “AI in the Sciences and Engineering \(2024\)”](#) – Lovely set of lectures (videos, slides) and tutorial exercises on PINNs, Operators, and more
- <https://github.com/openhackathons-org/End-to-End-AI-for-Science> - Set of notebooks for playing with PINNs and Operators in PhysicsNeMo

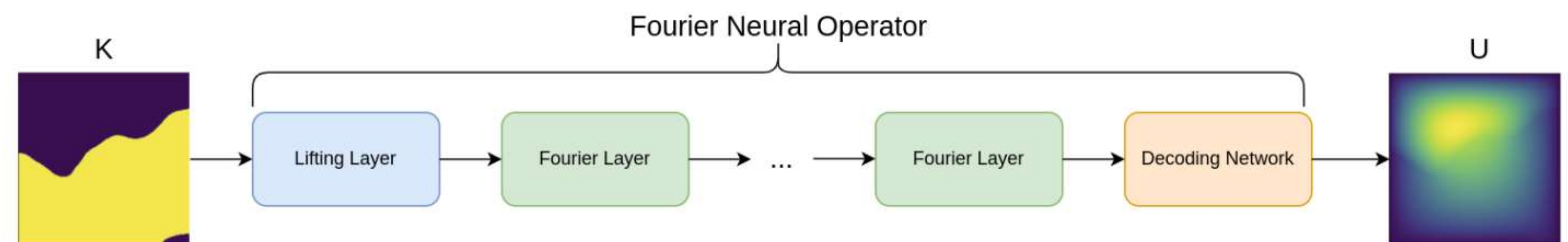
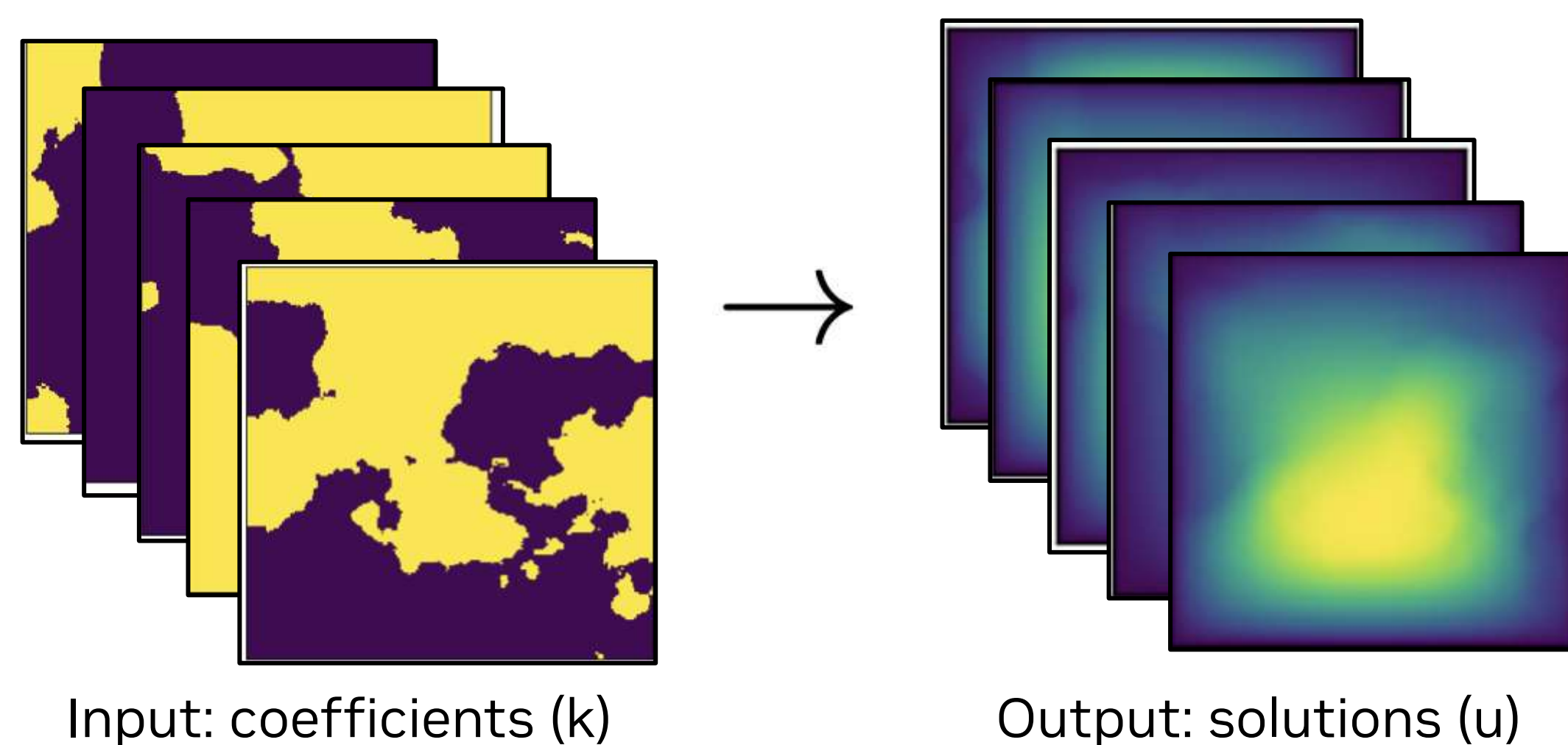
Developing Physics-ML models using data in PhysicsNeMo

Using FNOs to create a surrogate model

- Develop a surrogate model that learns a mapping between a permeability field and pressure field, $\mathbf{K} \rightarrow \mathbf{U}$, for a distribution of permeability fields $\mathbf{K} \sim p(\mathbf{K})$.
- The Darcy PDE (Diffusion equation) is a second order, elliptical PDE with the following form:

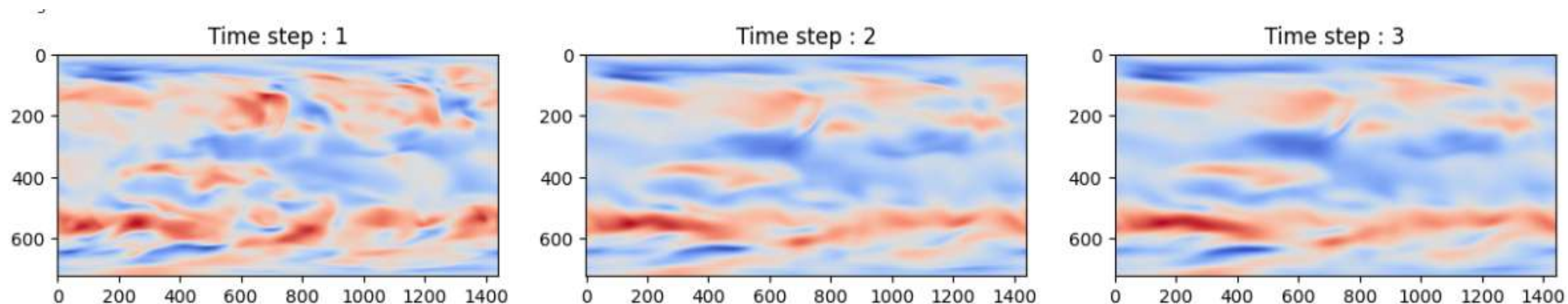
$$-\nabla \cdot (k(\mathbf{x}) \nabla u(\mathbf{x})) = f(\mathbf{x}), \quad \mathbf{x} \in D,$$

- $u(\mathbf{x})$: Flow pressure
- $k(\mathbf{x})$: Permeability field
- $f(\cdot)$: Forcing function
- $D = \{x, y \in (0,1)\}$ with the boundary condition $u(\mathbf{x}) = 0, \mathbf{x} \in \partial D$.
- Both the permeability and flow fields are discretized into a 2D matrix $\mathbf{K}, \mathbf{u} \in \mathbb{R}^{N \times N}$.

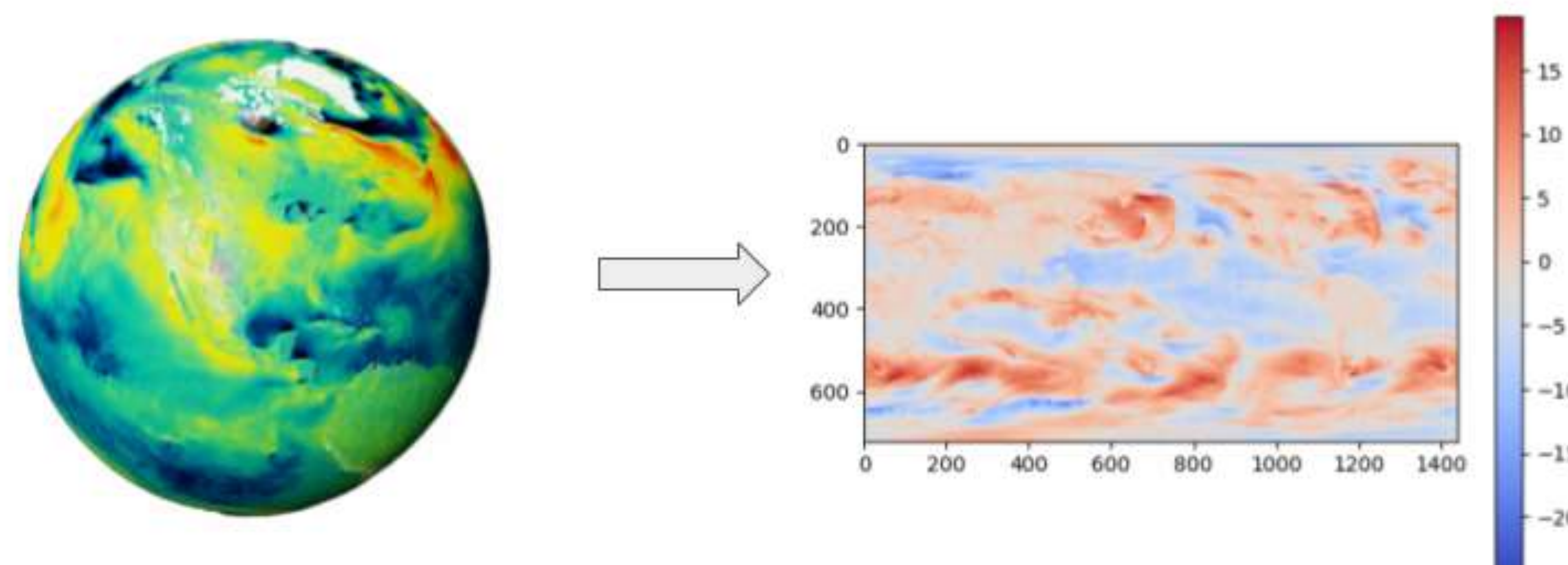


Weather prediction using Navier-Stokes

Problem description



- We aim to predict the velocities for 6-hour timesteps using the Navier-Stokes equation.
- The Navier-Stokes equations mathematically express the conservation of momentum and conservation of mass for Newtonian fluids.
- We will take a 2d projected input from the ERA5 Reanalysis dataset to be used as initial conditions for our input. The process of taking the 3D sphere and projecting it onto a 2D mesh is shown in the diagram below, the 2D mesh is of the size (1440,720)



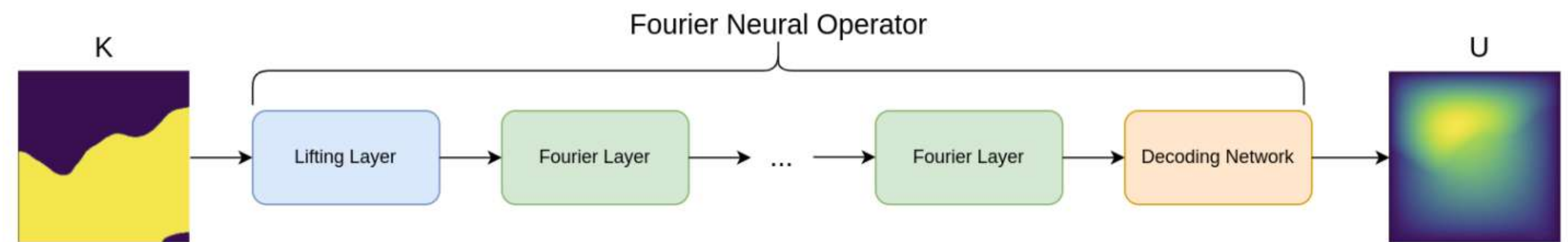
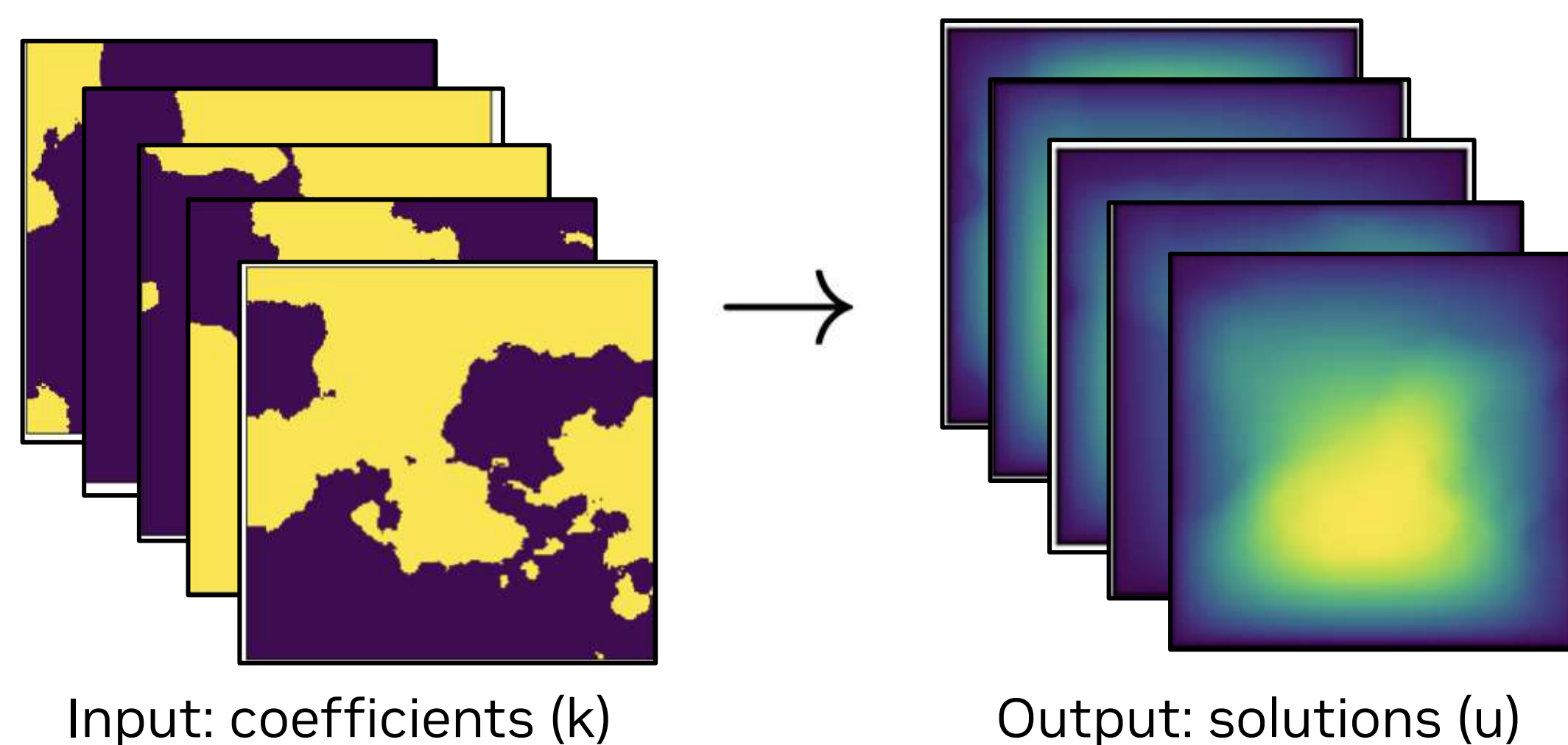
Developing Physics-ML models using data in PhysicsNeMo

Using FNOs to create a surrogate model

- Develop a surrogate model that learns a mapping between a permeability field and pressure field, $\mathbf{K} \rightarrow \mathbf{U}$, for a distribution of permeability fields $\mathbf{K} \sim p(\mathbf{K})$.
- The Darcy PDE (Diffusion equation) is a second order, elliptical PDE with the following form:

$$-\nabla \cdot (k(\mathbf{x}) \nabla u(\mathbf{x})) = f(\mathbf{x}), \quad \mathbf{x} \in D,$$

- $u(\mathbf{x})$: Flow pressure
- $k(\mathbf{x})$: Permeability field
- $f(\cdot)$: Forcing function
- $D = \{x, y \in (0,1)\}$ with the boundary condition $u(\mathbf{x}) = 0, \mathbf{x} \in \partial D$.
- Both the permeability and flow fields are discretized into a 2D matrix $\mathbf{K}, \mathbf{u} \in \mathbb{R}^{N \times N}$.



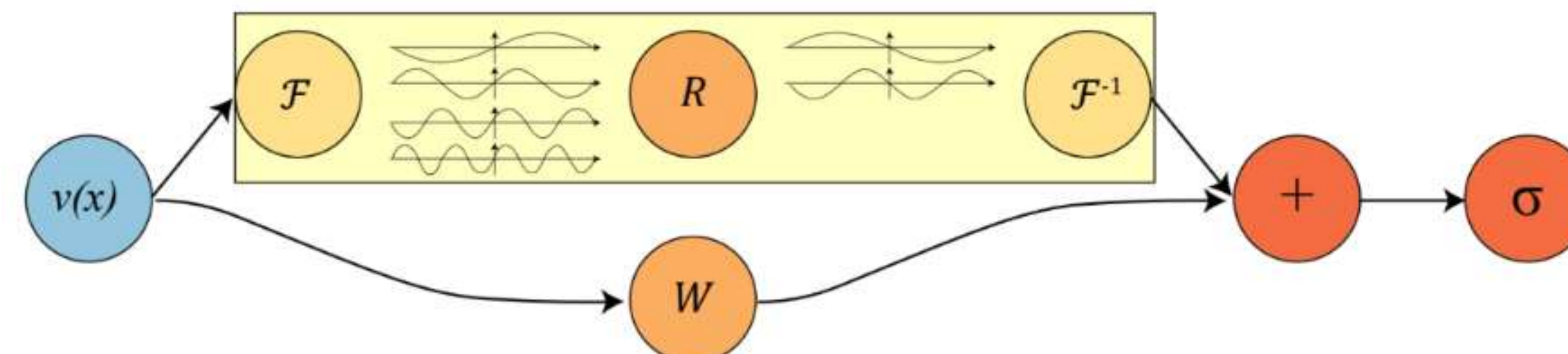
Developing Physics-ML models using data in PhysicsNeMo

Using FNOs to create a surrogate model

- The key feature of FNO is the spectral convolutions: operations that place the integral kernel in Fourier space:

$$(K(\mathbf{w})\phi)(x) = F^{-1}(R_{\mathbf{w}} \cdot (F)\phi)(x), \quad \forall x \in D$$

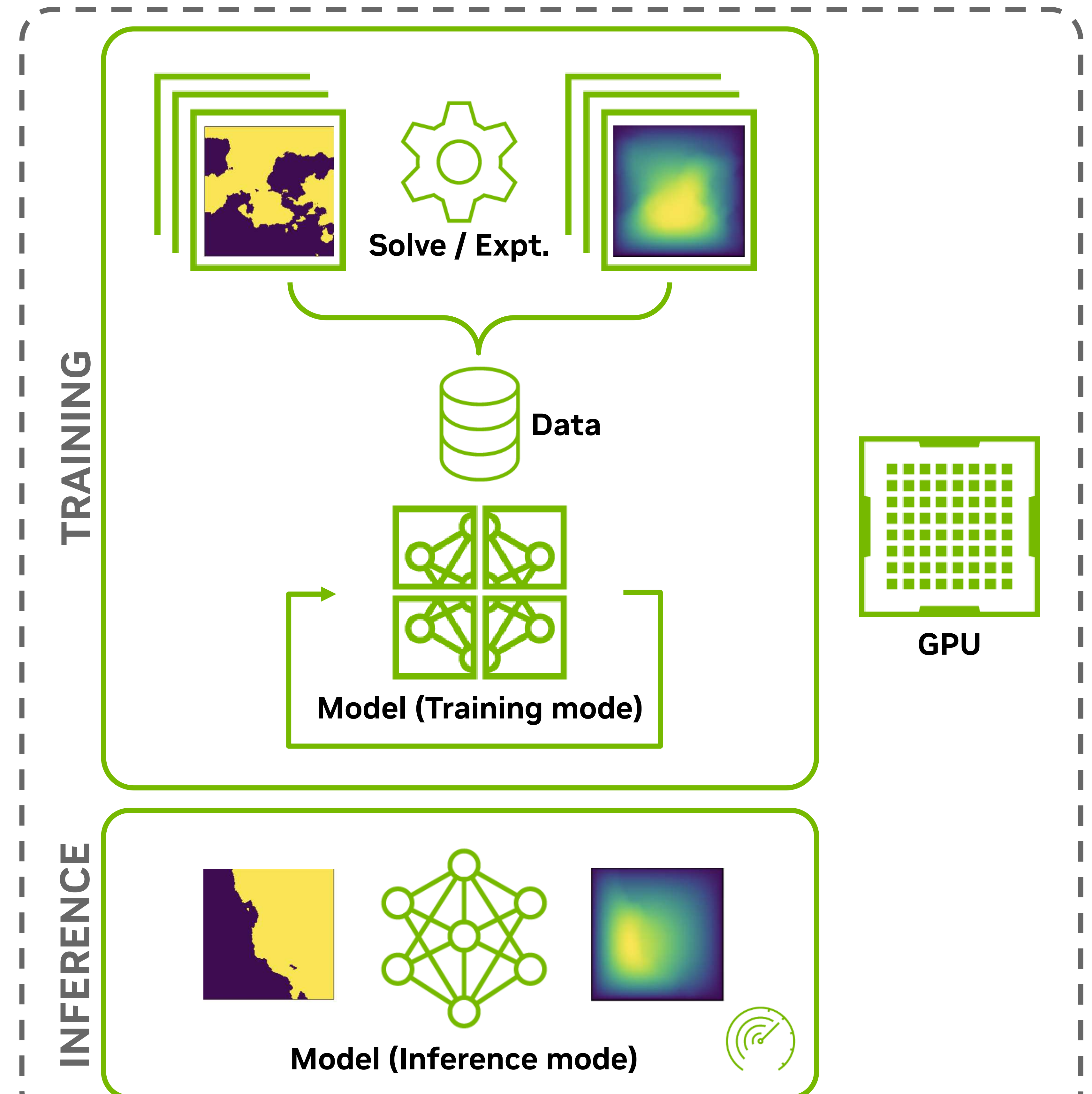
- $R_{\mathbf{w}}$: Transformation containing the learnable parameters $\mathbf{w}$, calculated over the entire *structured Euclidean* domain D .
- Fourier coefficients are truncated to only the lower modes - explicit control over the dimensionality of the spectral space and linear operator.
- The FNO model is a composition of a fully-connected “lifting” layer, L spectral convolutions with point-wise linear skip connections and a decoding point-wise fully-connected neural network at the end:
$$u_{net}(\Phi; \theta) = Q \circ \sigma(\mathbf{W}_L + \mathbf{K}_L) \circ \dots \circ \sigma(\mathbf{W}_1 + \mathbf{K}_1) \circ P(\Phi), \quad \Phi = \{\phi(x); \forall x \in D\}$$
 - $\sigma(\mathbf{W}_i + \mathbf{K}_i)$: Spectral convolution layer i with the point-wise linear transform $\mathbf{W}_i$ and activation function $\sigma(\cdot)$.
 - P : Point-wise lifting network that projects the input into a higher dimensional latent space.
 - Q : Point-wise fully-connected decoding network.
- Since all fully-connected components of FNO are point-wise operations, the model is invariant to the dimensionality of the input.



Developing Physics-ML models using data in PhysicsNeMo

Using FNOs to create a surrogate model

- Train a data-driven model:
 - Given input-output pairs, find the map (operator)
- Define the training loop using interoperable PhysicsNeMo-PyTorch utilities
 - Optimized data loaders and data pipes for domain specific use cases
 - Optimized Model Zoo and layers targeted for Physics-ML applications
 - Utilities for Distributed training and scaling
 - Easily apply performance optimizations like CUDA Graphs, AMP
 - Utilities for easier logging
 - Checkpointing utilities to save and load checkpoints
 - Load model checkpoints without model arguments
 - Utilities for deployment and inference



Developing Physics-ML models using data in PhysicsNeMo

Typical training workflow

Python Imports

```
import hydra
from omegaconf import DictConfig
import torch
import numpy as np
import matplotlib.pyplot as plt
from hydra.utils import to_absolute_path
import torch.nn.functional as F
from torch.utils.data import DataLoader
from itertools import chain

from physicsnemo.models.fno import FNO
from physicsnemo.launch.logging import LaunchLogger
from physicsnemo.launch.utils.checkpoint import save_checkpoint

from utils import HDF5MapStyleDataset

@hydra.main(version_base="1.3", config_path="conf", config_name="config_pure_data.yaml")
def main(cfg: DictConfig):

    LaunchLogger.initialize()

    dataset = HDF5MapStyleDataset(
        to_absolute_path("./datasets/Darcy_241/train.hdf5"), device="cuda"
    )
    validation_dataset = HDF5MapStyleDataset(
        to_absolute_path("./datasets/Darcy_241/validation.hdf5"), device="cuda"
    )

    dataloader = DataLoader(dataset, batch_size=2, shuffle=True)
    validation_dataloader = DataLoader(validation_dataset, batch_size=1, shuffle=False)

    model_branch = FNO(
        in_channels=2,
        out_channels=1,
    ).to("cuda")

    optimizer = torch.optim.Adam(
        chain(model_branch.parameters()),
        betas=(0.9, 0.999),
        lr=cfg.start_lr,
        weight_decay=0.0,
    )

    scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=cfg.gamma)
```

Data loading

Model

Optimizer

Training loop with loss computation

Saving model

```
for epoch in range(cfg.max_epochs):
    # wrap epoch in launch logger for console logs
    with LaunchLogger(
        "train",
        epoch=epoch,
        num_mini_batch=len(dataloader),
        epoch_alert_freq=10,
    ) as log:
        for data in dataloader:
            optimizer.zero_grad()
            truevar = data[1]

            # compute forward pass
            outvar = model_branch(data[0][:, 0].unsqueeze(dim=1))

            # Compute data loss
            loss_data = F.mse_loss(outvar, truevar)

            # Compute total loss
            loss = loss_data

            # Backward pass and optimizer and learning rate update
            loss.backward()
            optimizer.step()
            scheduler.step()

            log.log_epoch({"Learning Rate": optimizer.param_groups[0]["lr"]})

        # test model on validation dataset
        with LaunchLogger("valid", epoch=epoch) as log:
            error = validation_step(model_branch, validation_dataloader, epoch)
            log.log_epoch({"Validation error": error})

        # save the checkpoint
        save_checkpoint(
            "./checkpoints",
            models=[model_branch],
            optimizer=optimizer,
            scheduler=scheduler,
            epoch=epoch,
        )

if __name__ == "__main__":
    main()
```

Darcy Flow – Adaptive Fourier Neural Operators (AFNO)

Results and discussion

AFNO accurately learns the solution of this system.

For more information, please refer to the official [PhysicsNeMo user guide example](#).

AFNO is the key backboard for NVIDIA's groundbreaking [FourCastNet](#):

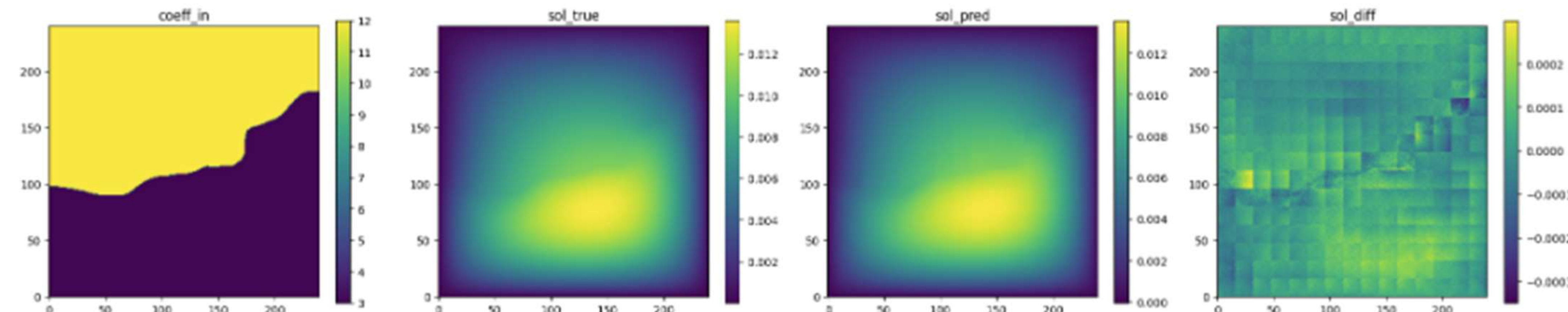
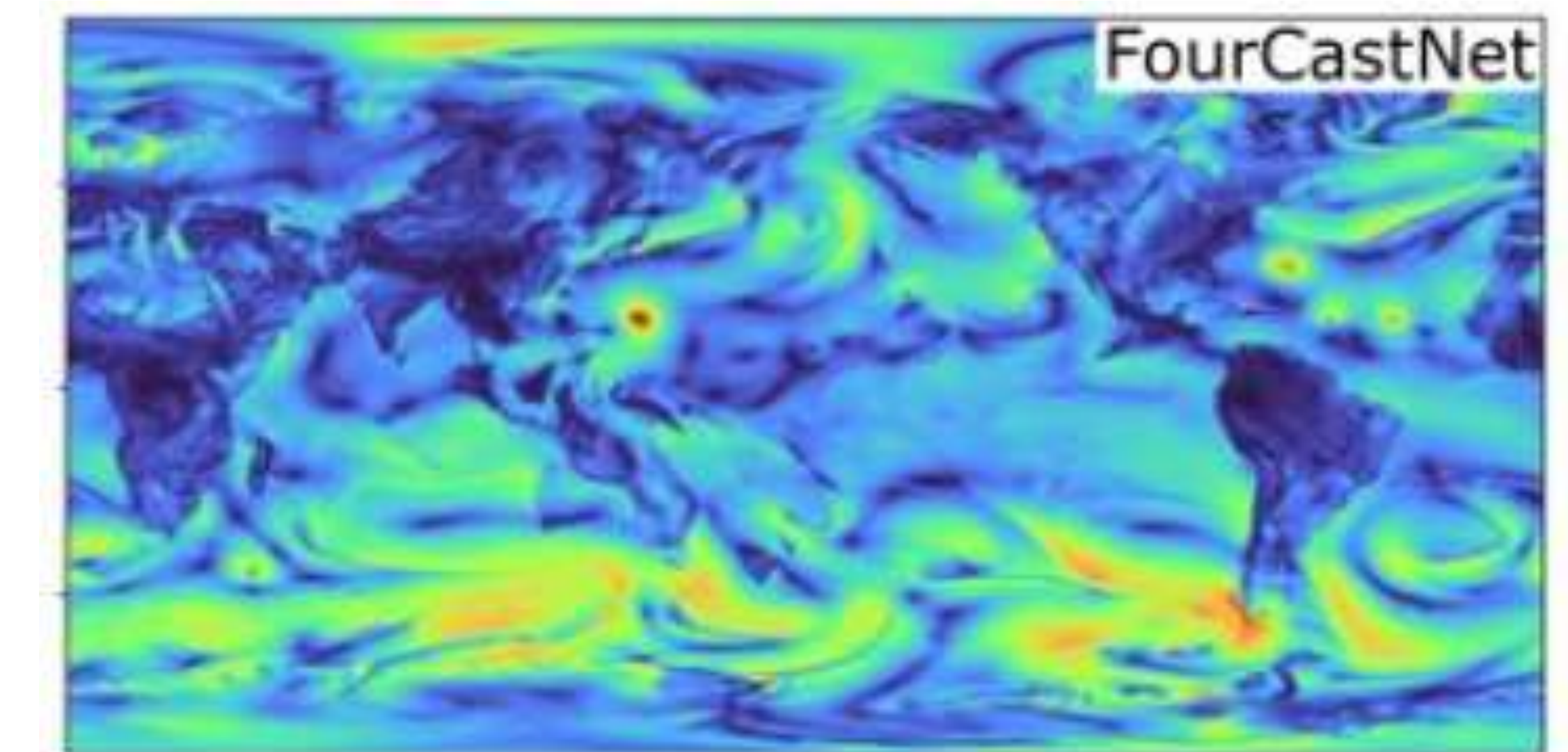


Fig. 65 AFNO validation prediction 1. (Left to right) Input permeability, true pressure, predicted pressure, error.

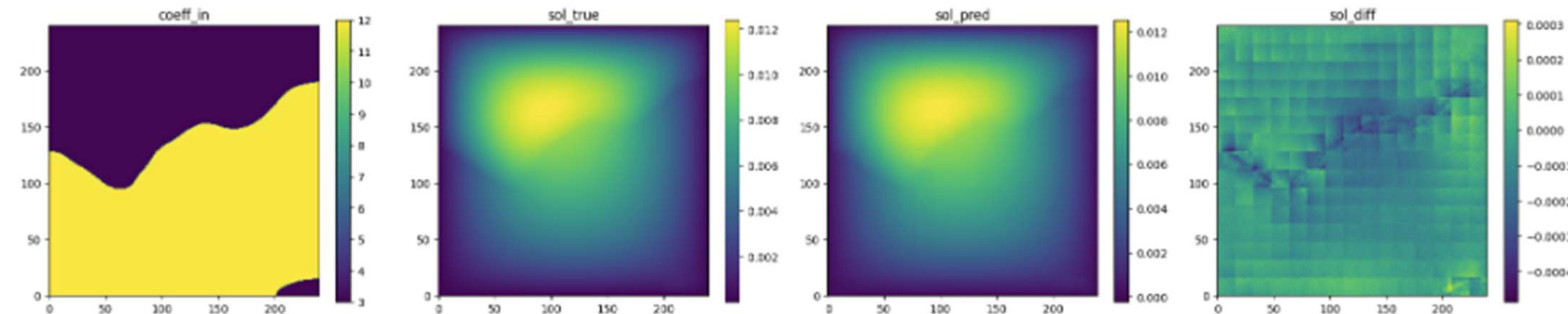
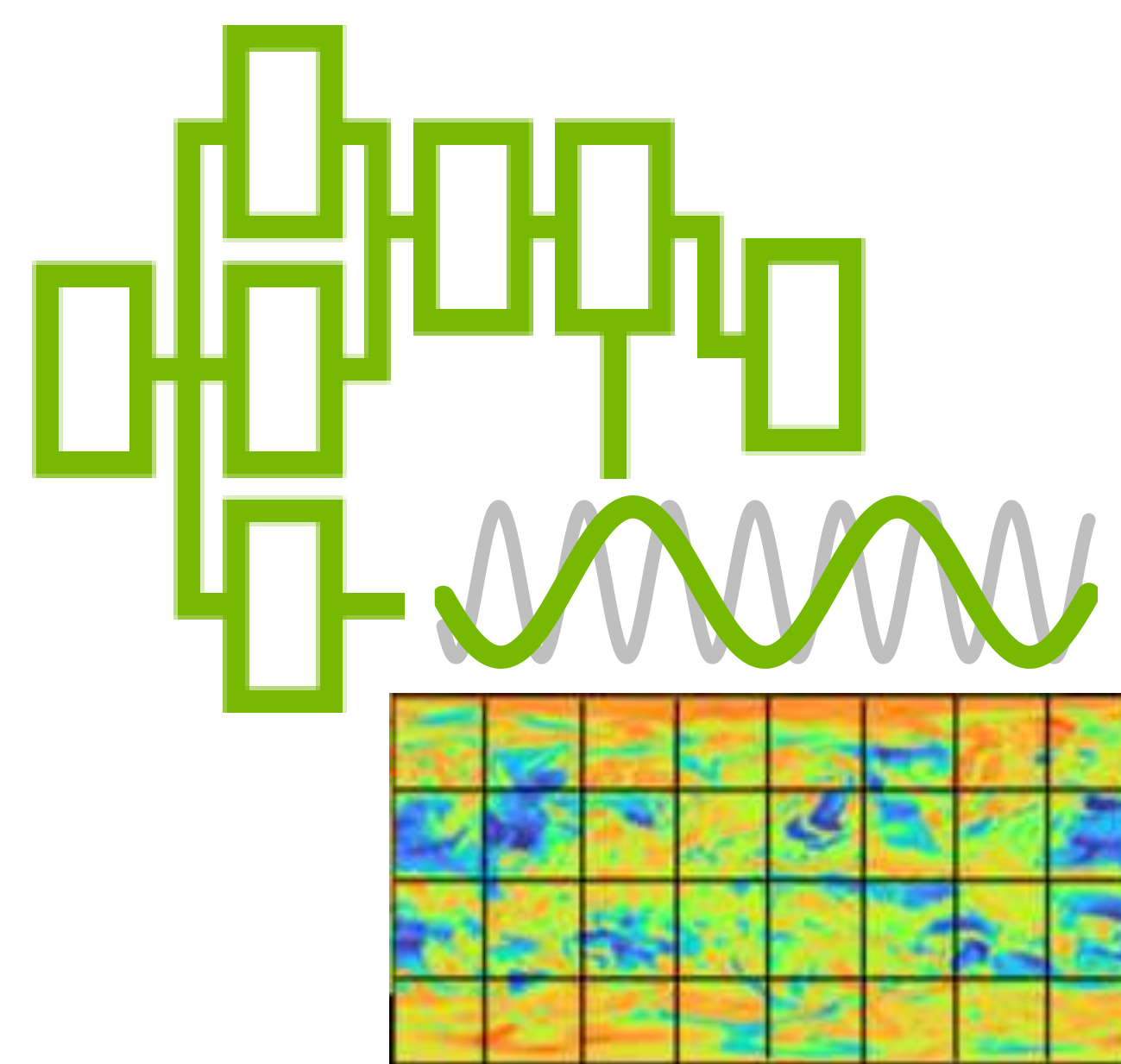


Fig. 66 AFNO validation prediction 2. (Left to right) Input permeability, true pressure, predicted pressure, error.

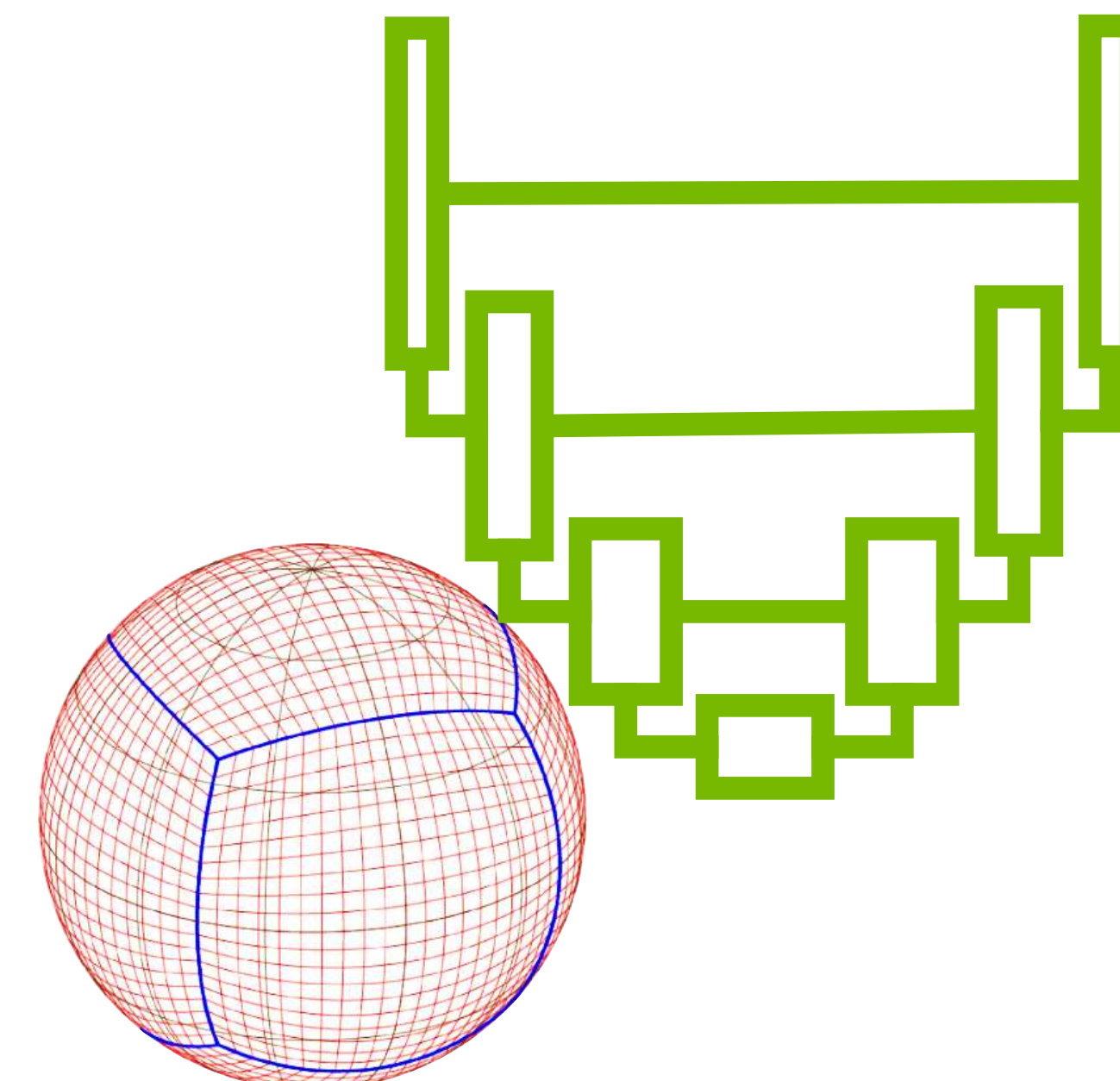
Model architectures – Weather and Climate

Developing models at scale using unified recipes in PhysicsNeMo



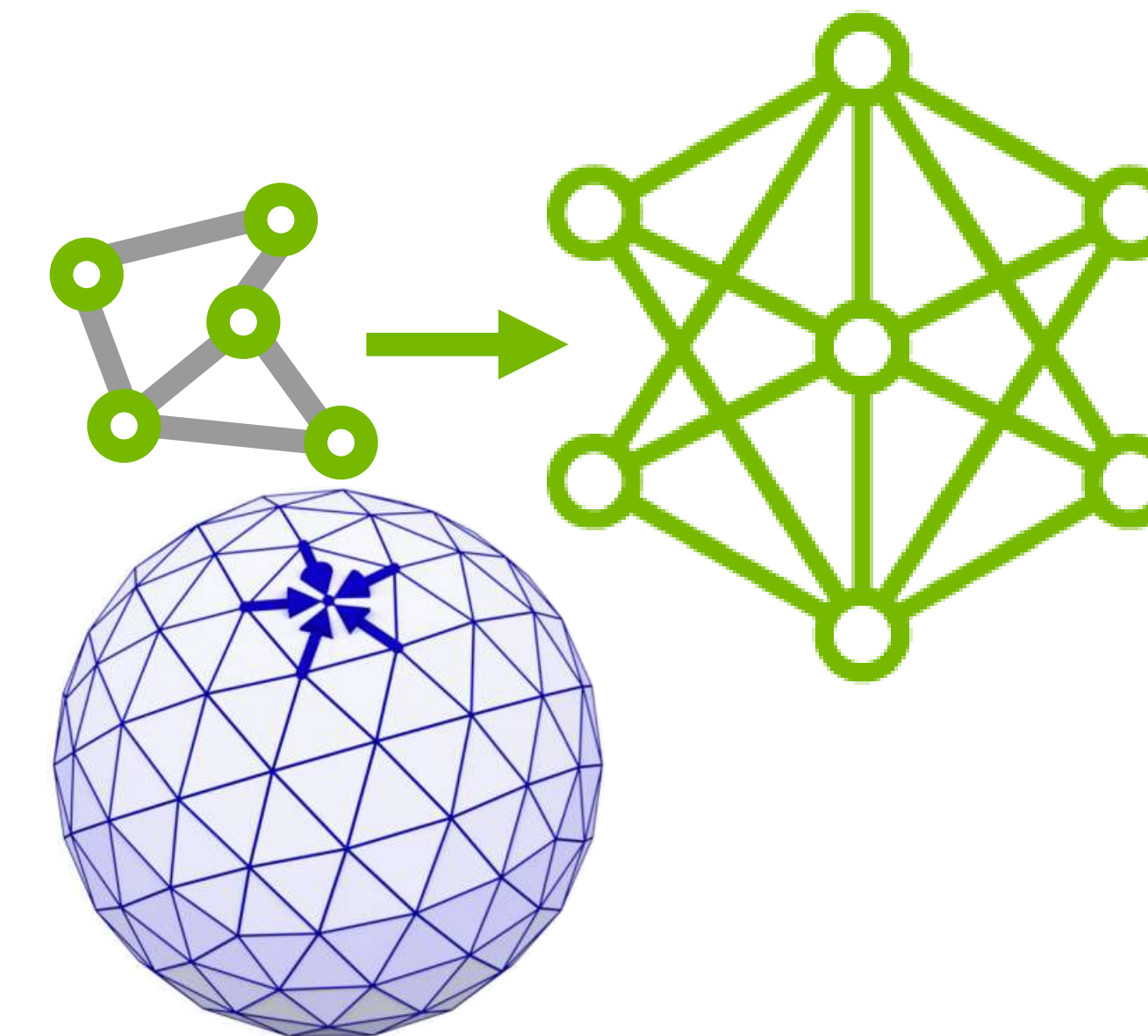
FourCastNet

A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators. FourCastNet matches the forecasting accuracy of the ECMWF Integrated Forecasting System (IFS).



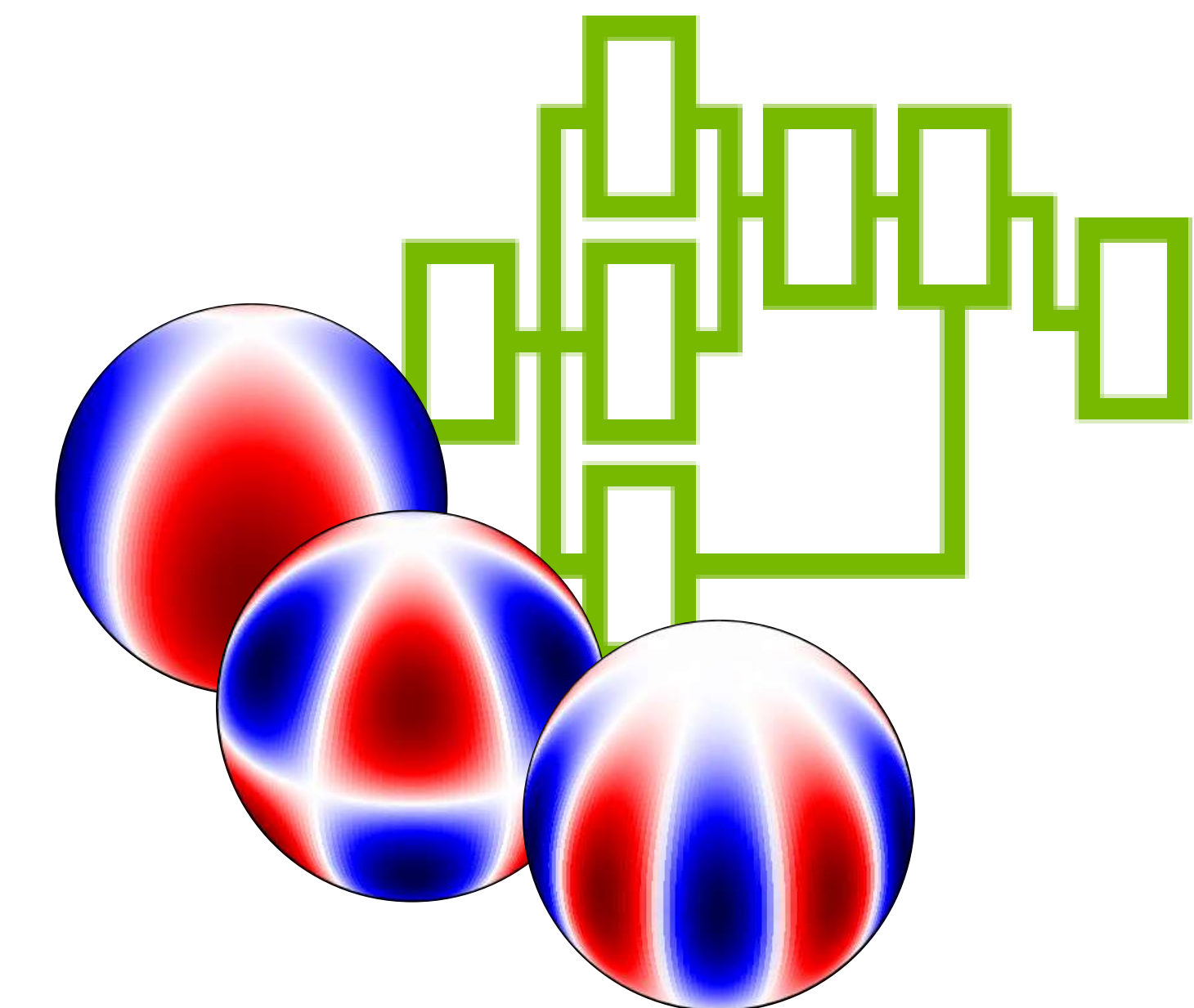
DLWP - CubeSphere

A light-weight deep learning weather prediction model that uses Cube-sphere grid and can produce forecasts up to 6 weeks using a 320-member ensemble within a couple of minutes.



GraphCast

Based on GNNs in an “encode-process-decode” configuration. GraphCast’s forecast skill and efficiency compared to HRES shows DL methods are now competitive with traditional weather forecasting methods.

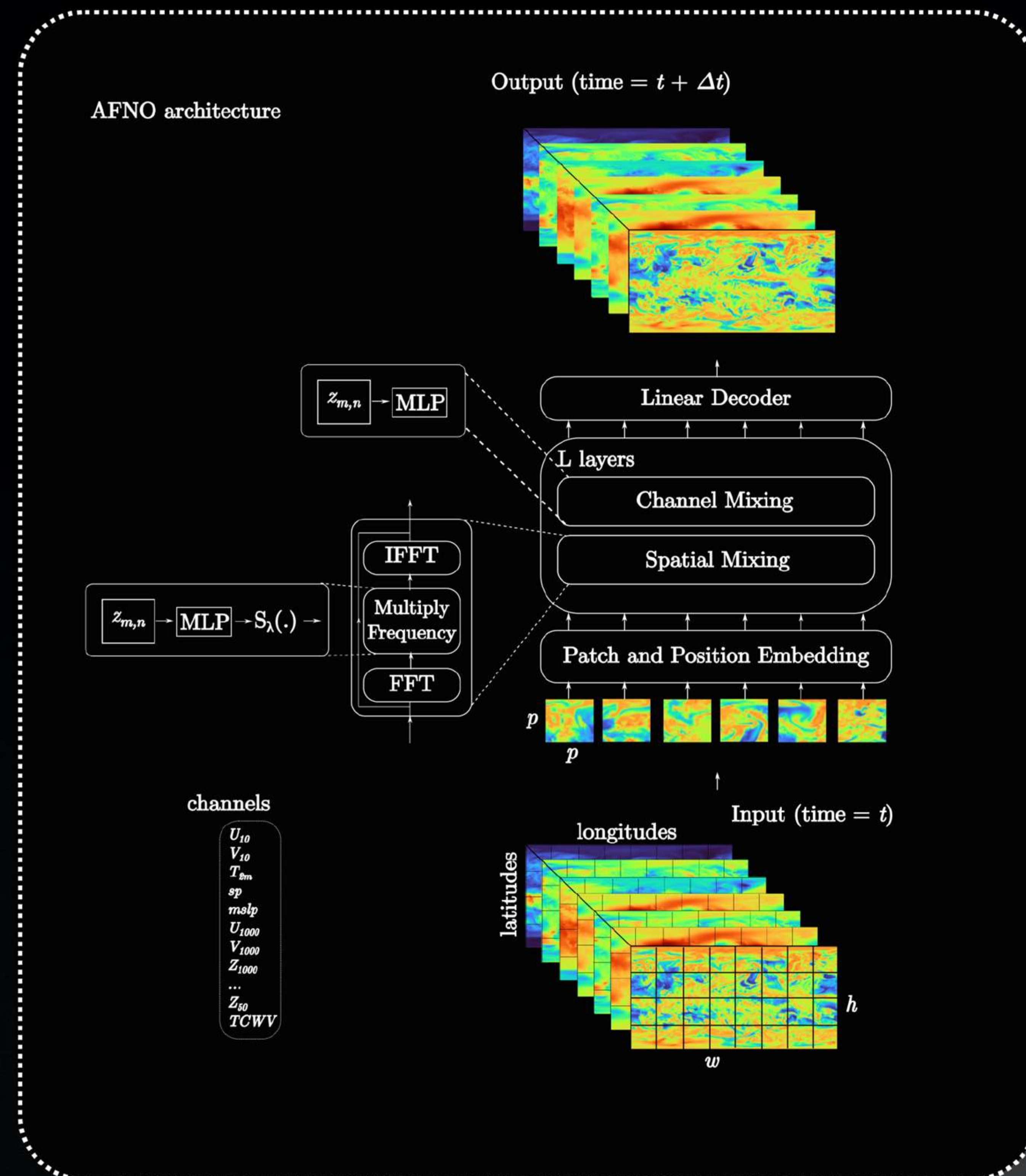


FourCastNet v2

A novel SFNO equivariant architecture for modeling nonlinear chaotic dynamical systems on the sphere. The high accuracy and long-term stability promises for the application of Spherical Fourier Neural Operators for long term forecasting.

Unified training recipe to leverage the same optimized training pipeline across all the model architectures at scale

FOURCASTNET (FOURIER FORECASTING NETWORK)



Purely data-driven machine learning surrogate weather model

Trained on ERA5 reanalysis data at the native resolution of 0.25 degrees.

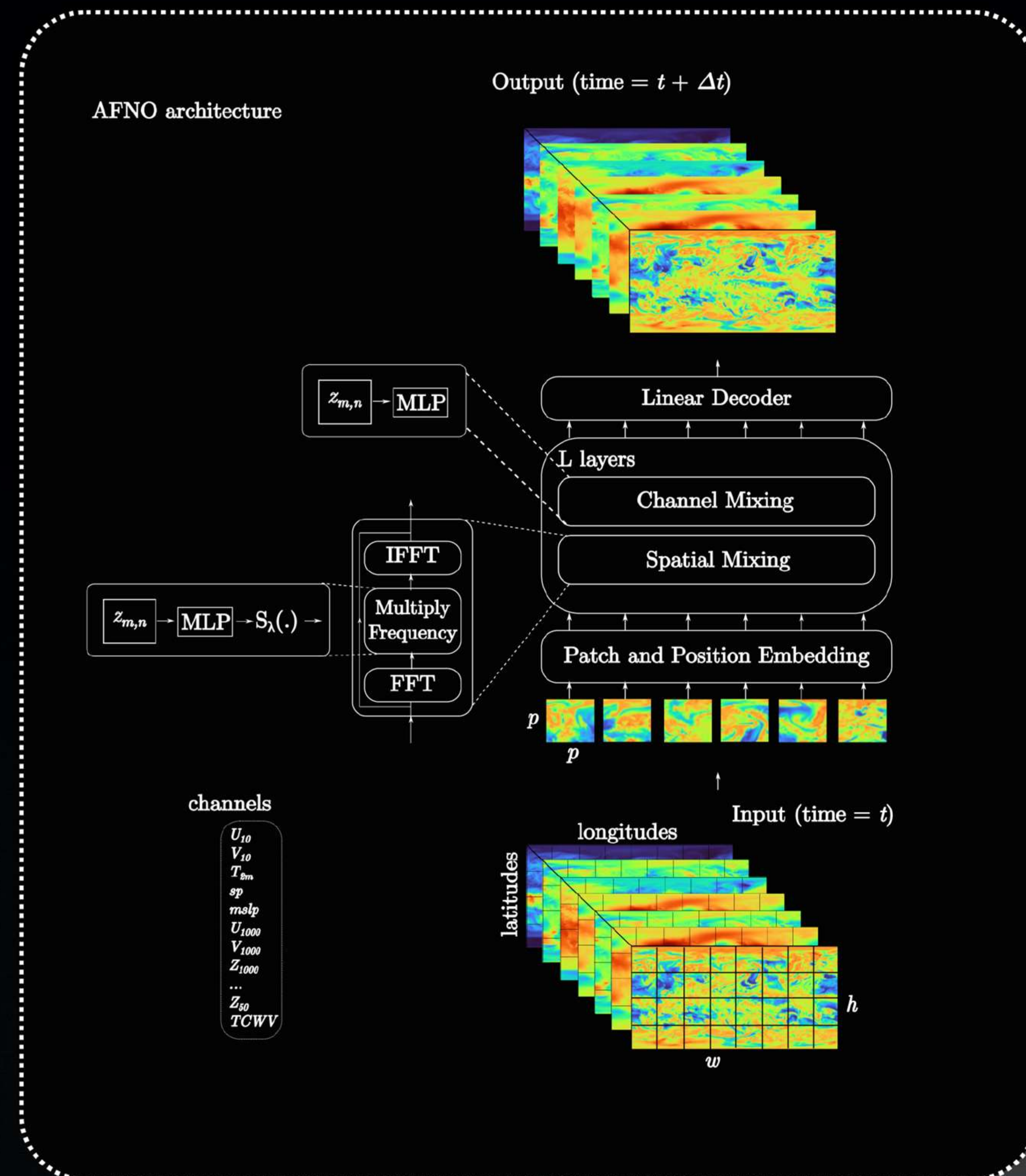
State-of-the-art for Deep Learning based weather surrogate models.

Highest resolution data driven model ever trained.

Guibas et al. (2022), Adaptive Fourier Neural Operators: Efficient Token Mixers for Transformers, ICLR 2022.

Pathak et al. (2022), FourCastNet: A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators, arXiv:2202.11214

FOURCASTNET (FOURIER FORECASTING NETWORK)



Vertical Level	Variables
Surface	$U_{10}, V_{10}, T_{2m}, sp, mslp$
1000hPa	U, V, Z
850hPa	T, U, V, Z, RH
500hPa	T, U, V, Z, RH
50hPa	Z
Integrated	$TCWV$

Currently models 21 atmospheric variables.

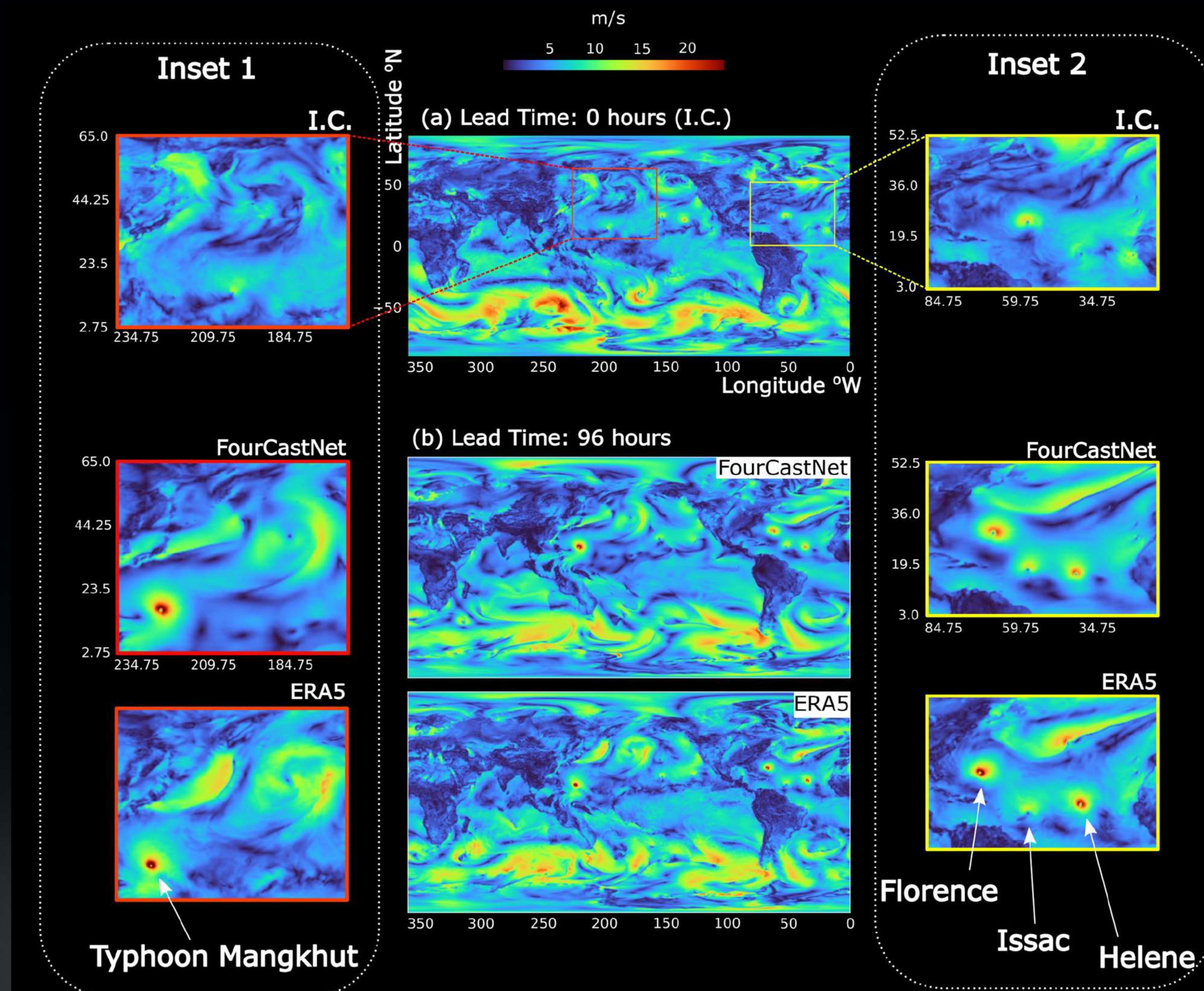
Soon to be extended to a larger set of variables to include radiation processes, vapor transport, more moisture variables, clouds.

Training set: 1979 to 2015

Validation set: 2016, 2017

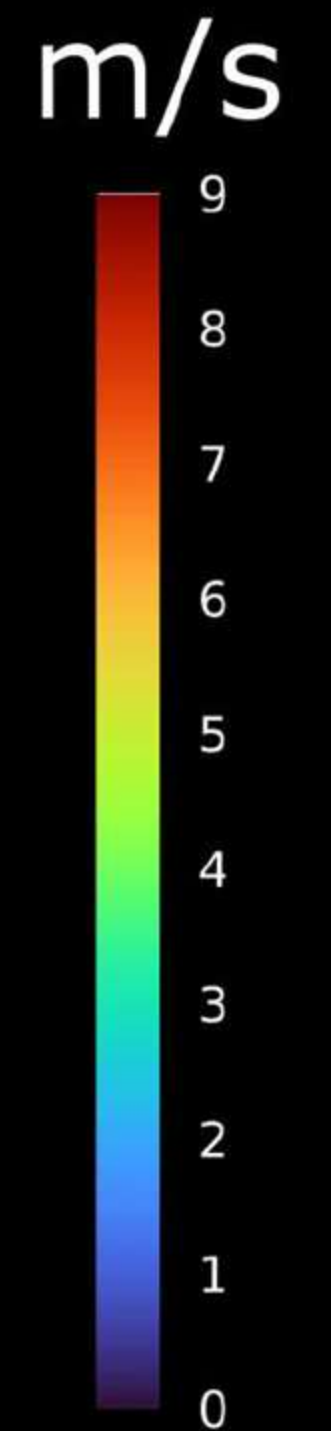
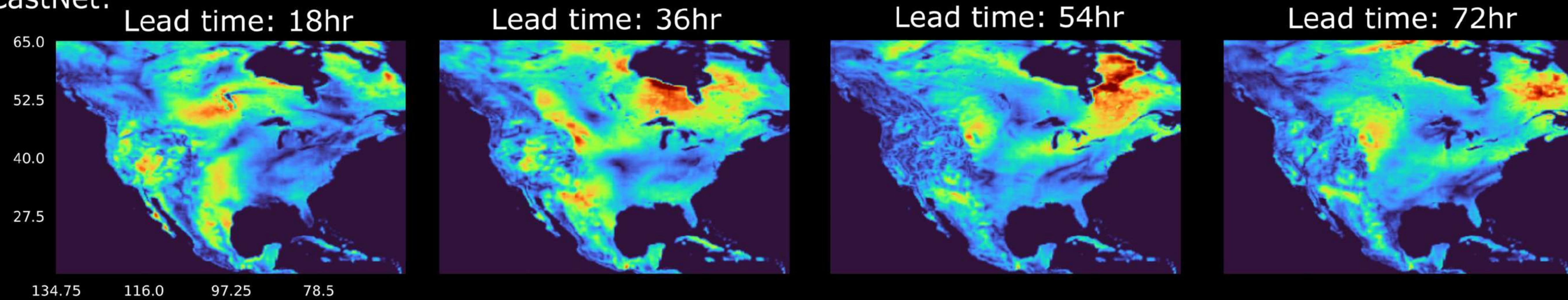
Held out: 2018 onwards

EXCELLENT SKILL ON FORECASTING SURFACE WINDS

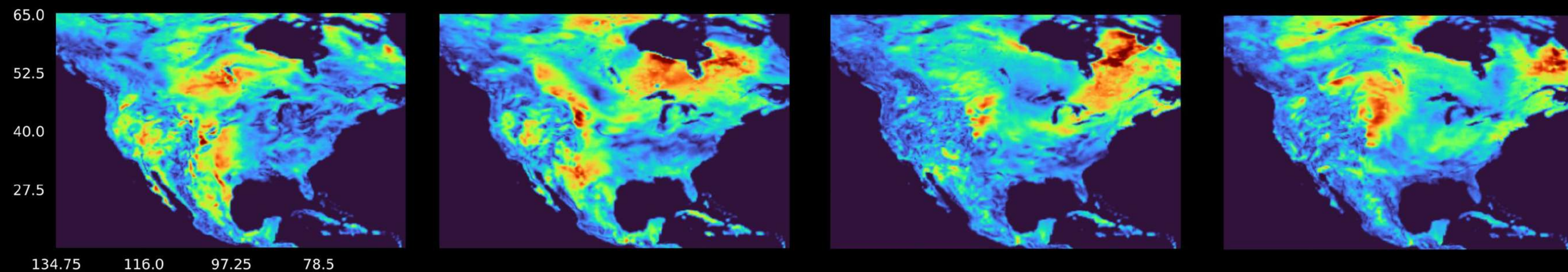


FOURCASTNET PREDICTS NEAR-SURFACE WIND FIELDS OVER LAND ACCURATELY: IMPORTANT IMPLICATIONS FOR WIND ENERGY PLANNING

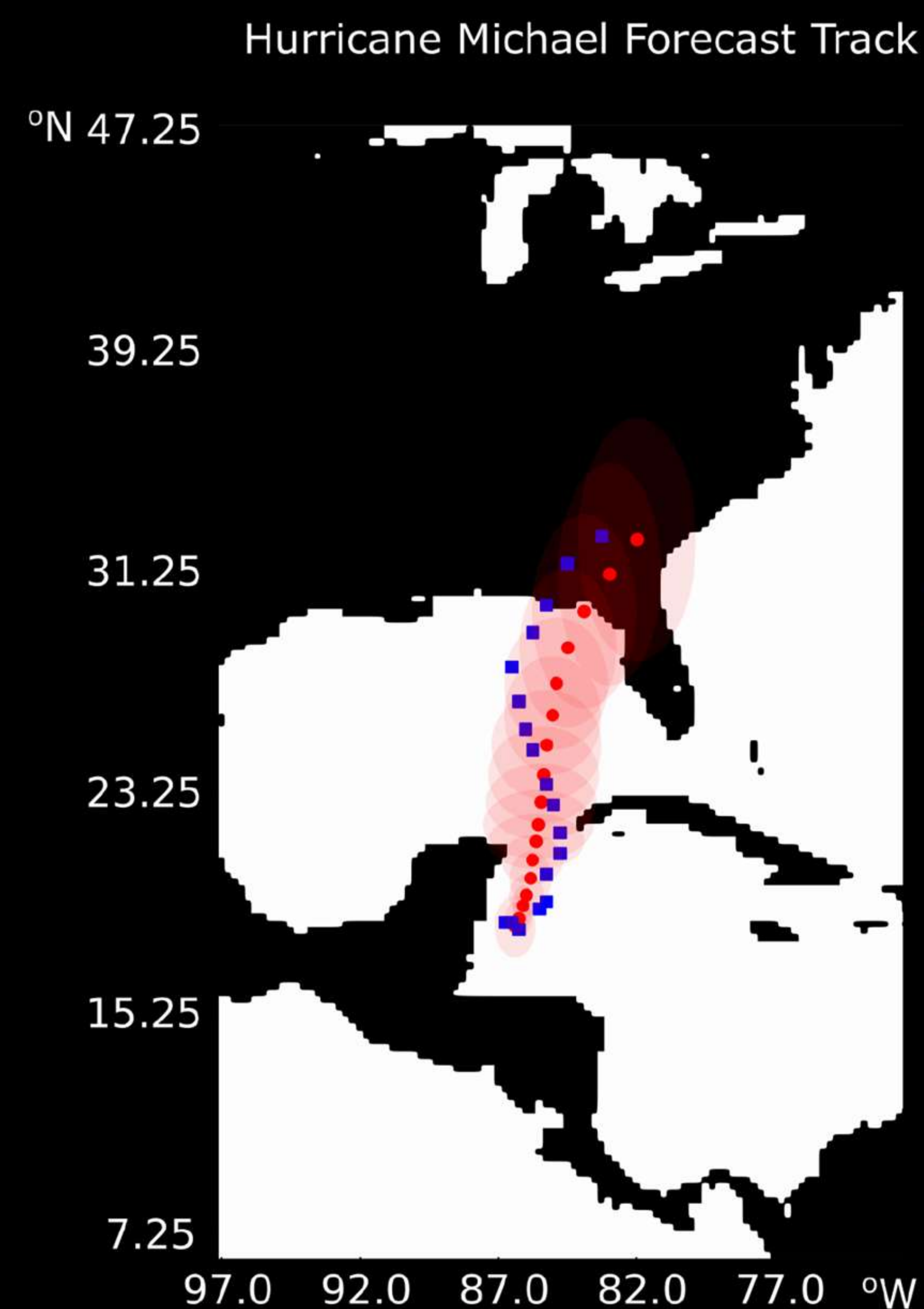
FourCastNet:



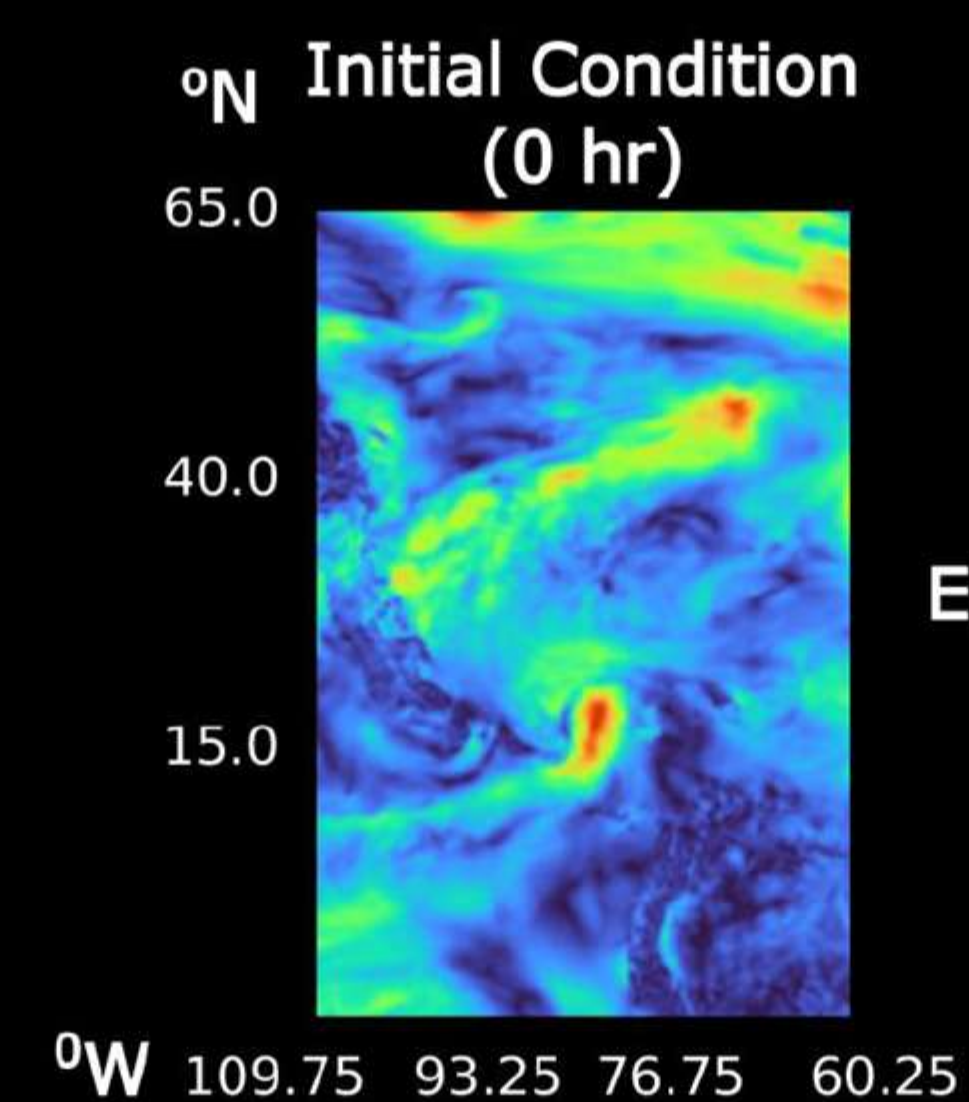
ERA5:



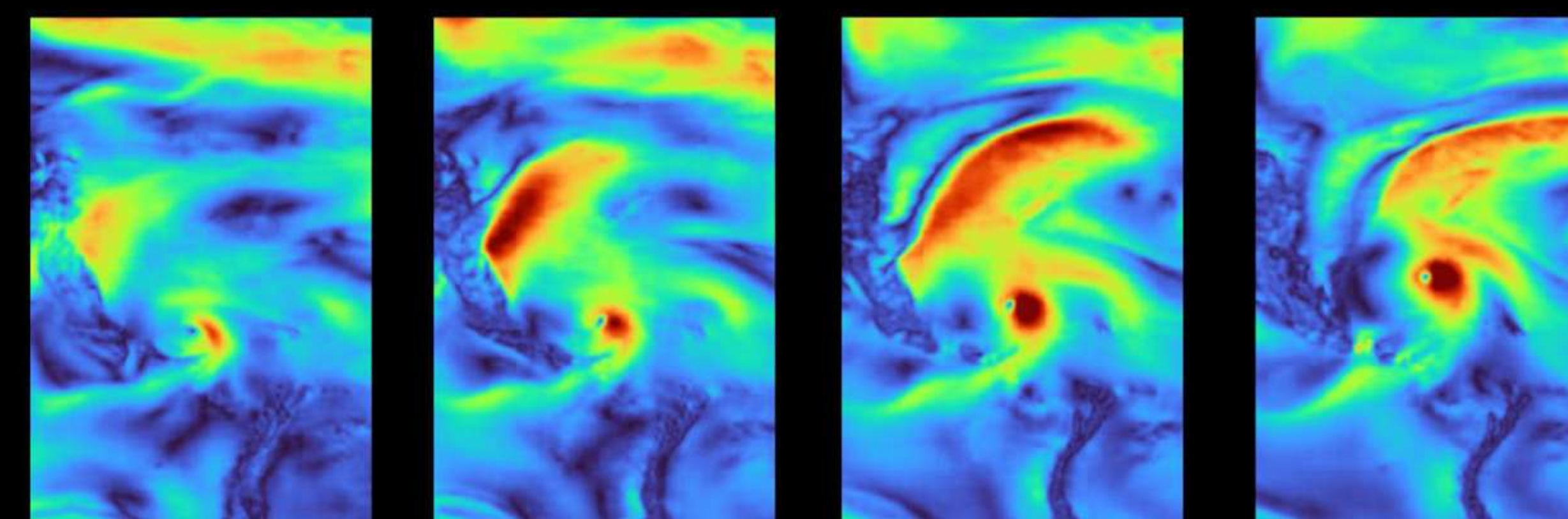
FOURCASTNET PREDICTS HURRICANE PATHS AND INTENSITIES



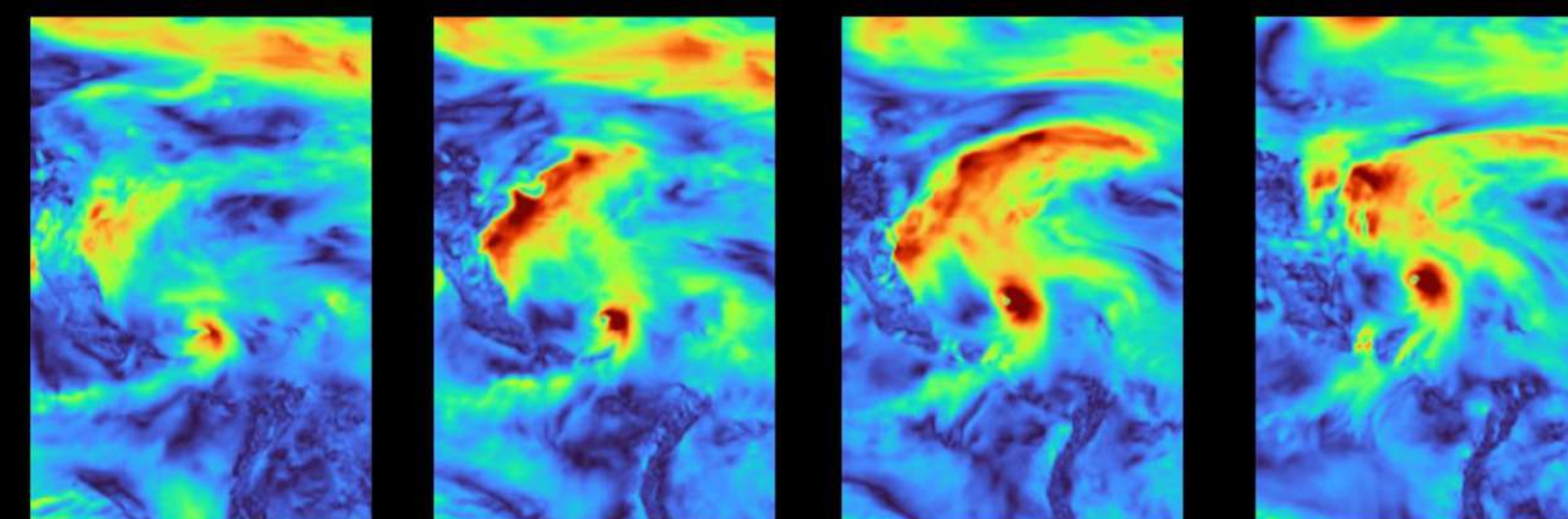
850hPa Wind Speed



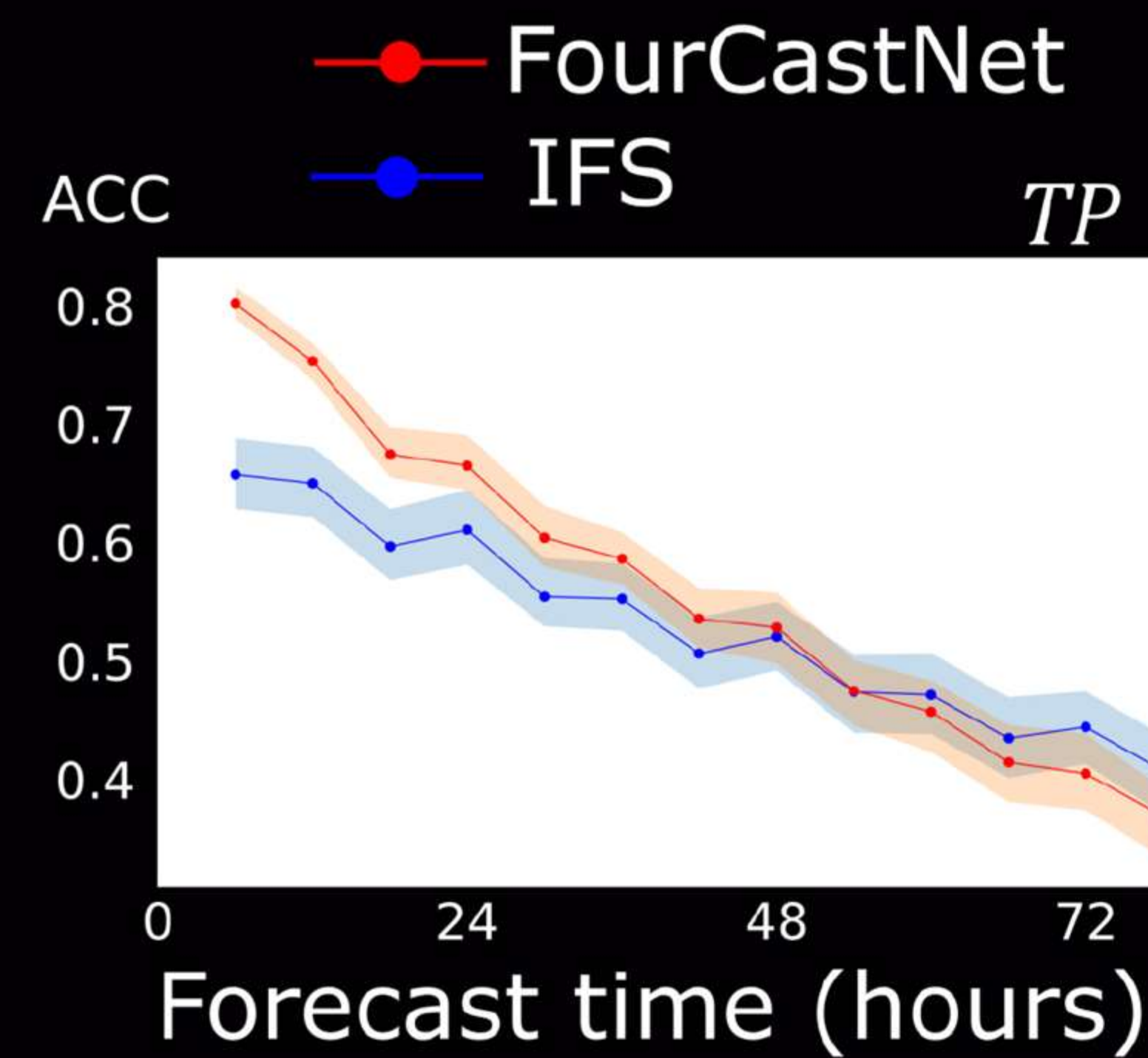
FourCastNet:



ERA5:

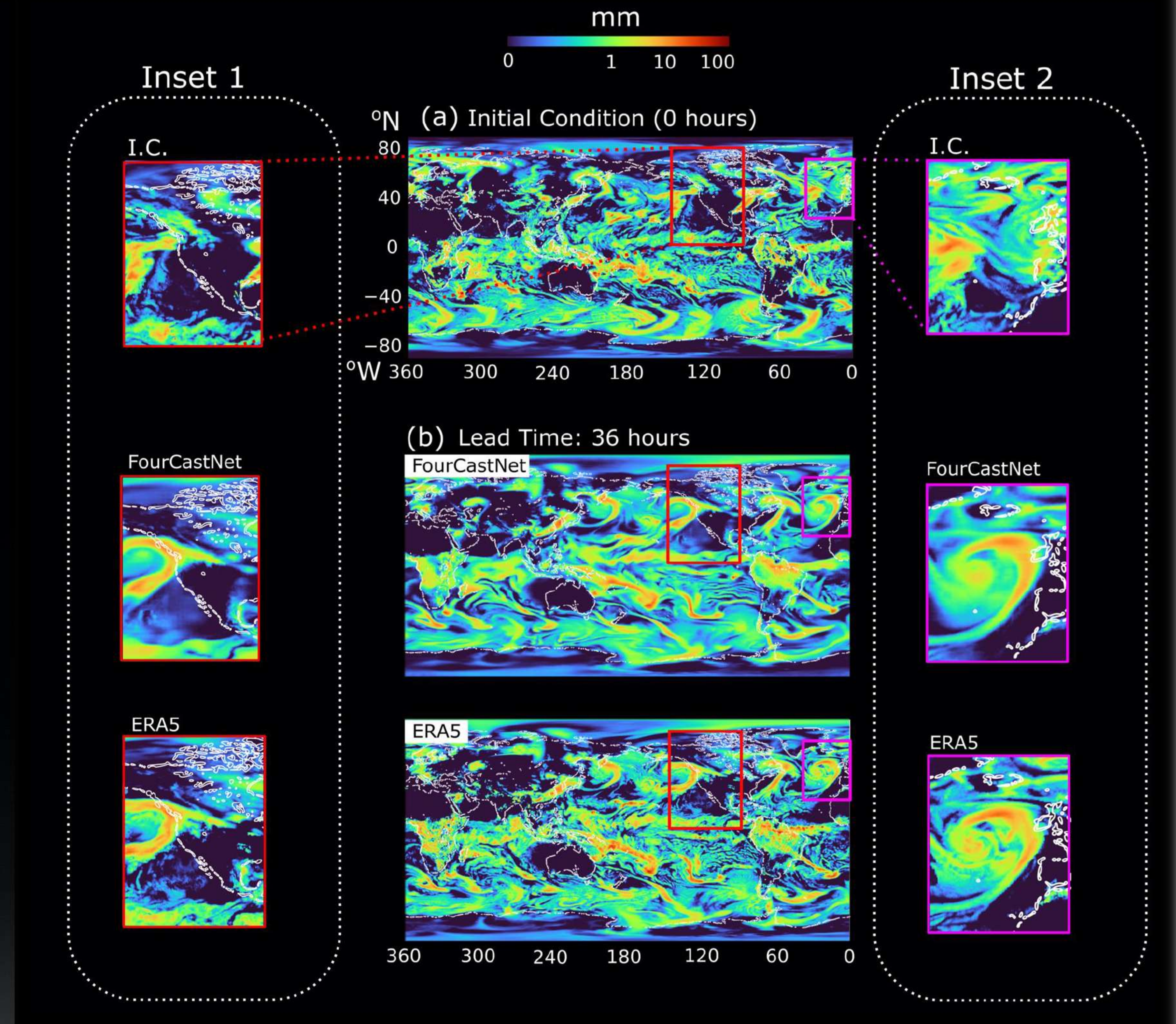


PRECIPITATION



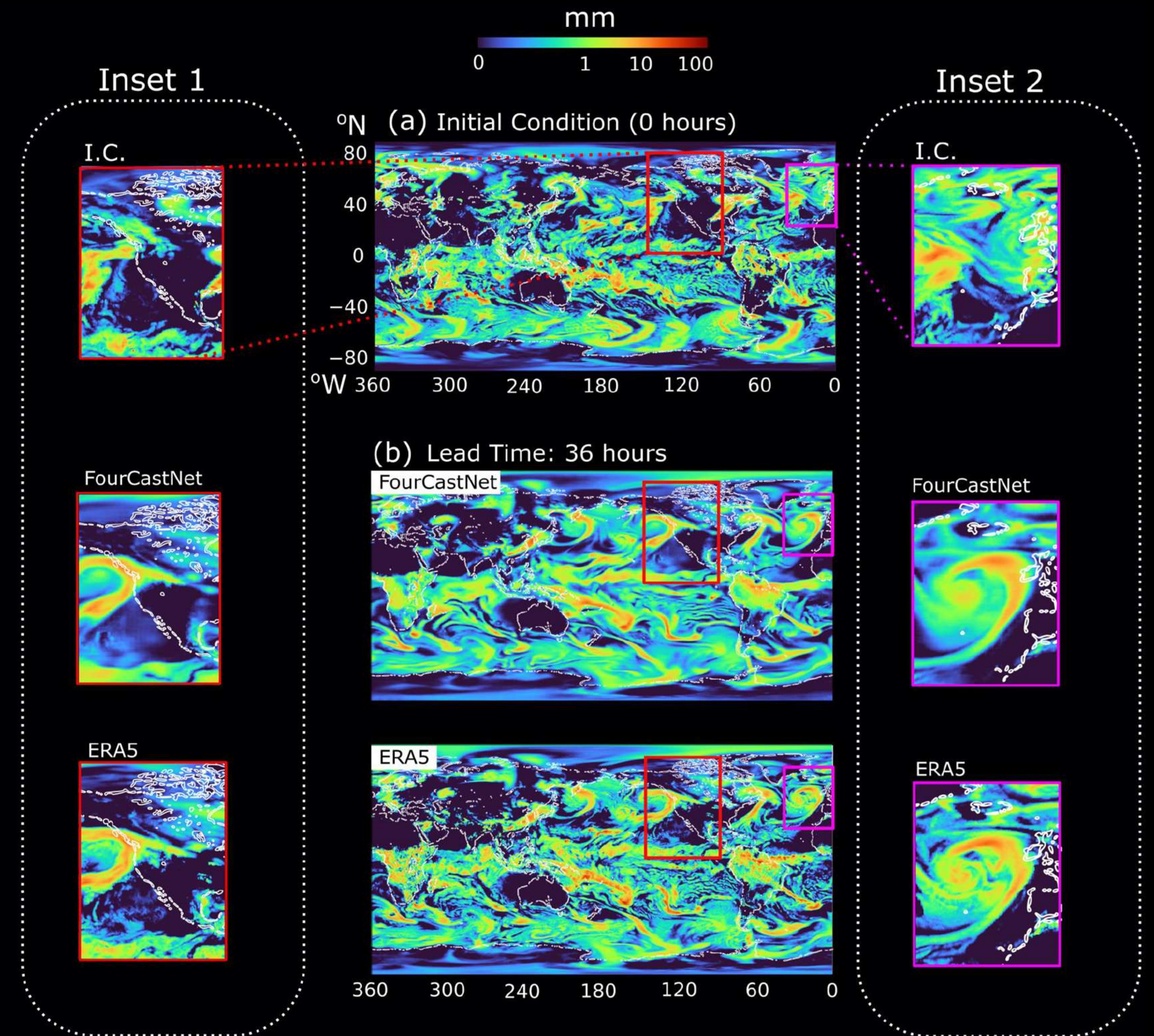
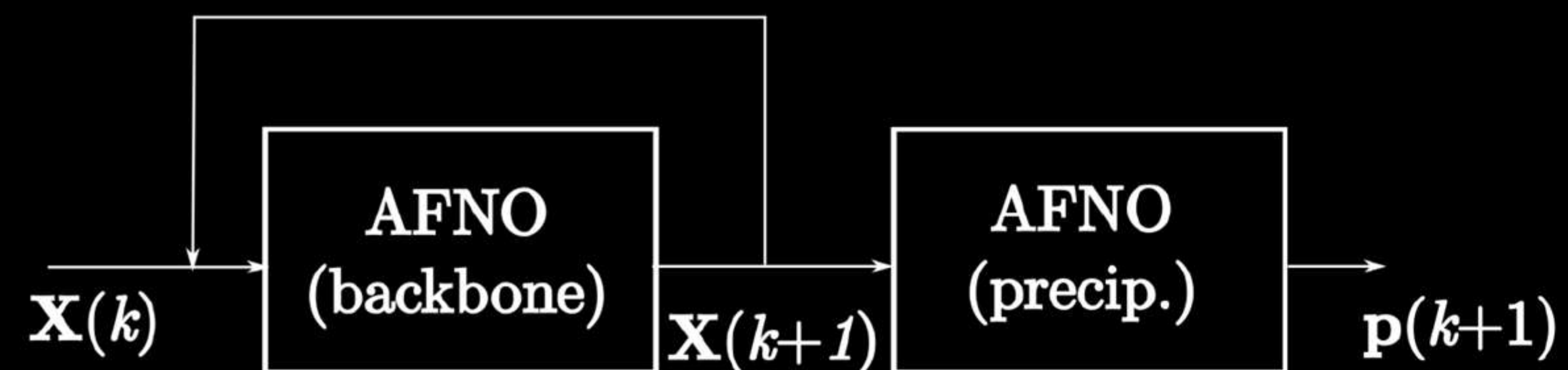
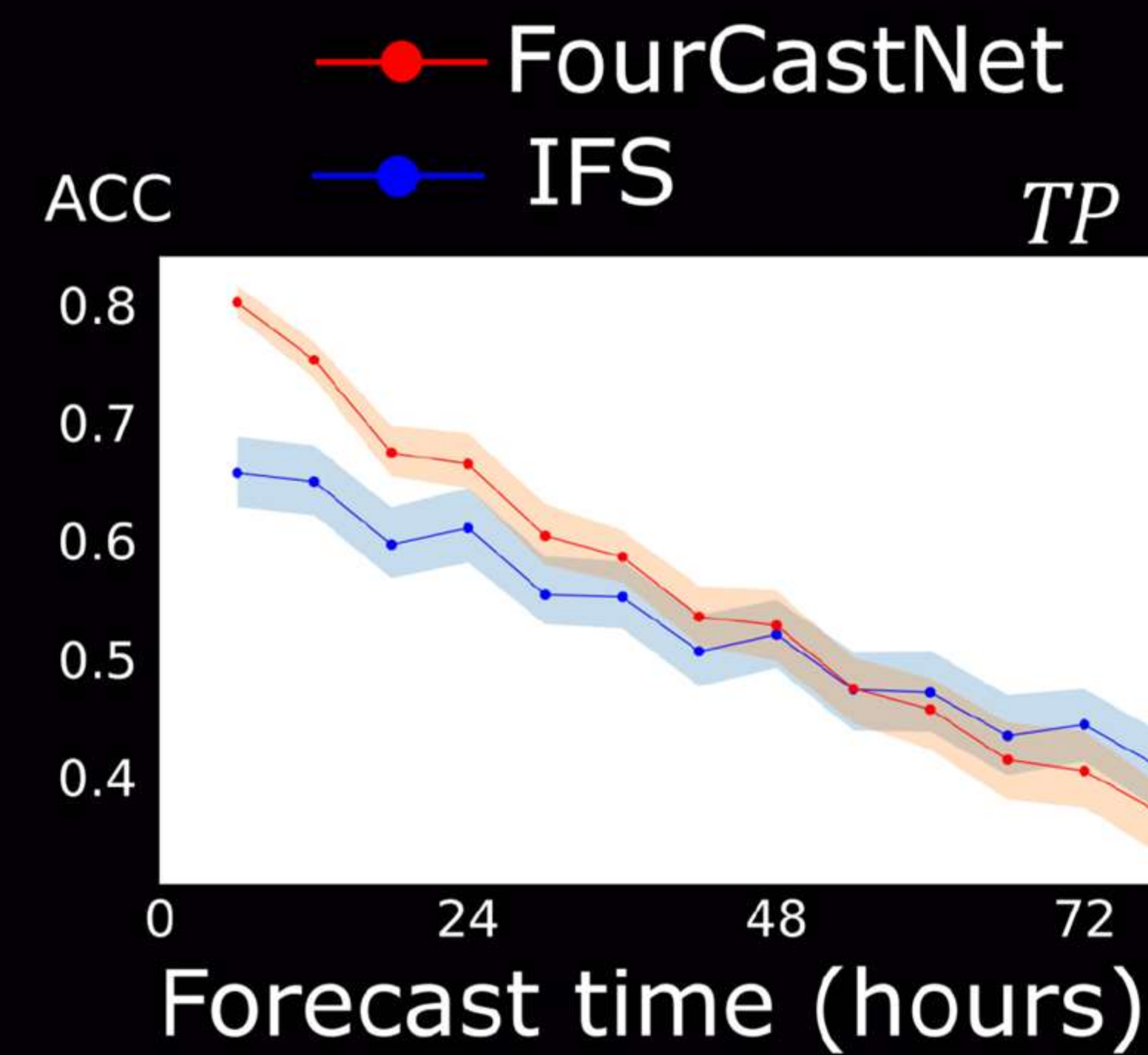
Excellent performance on forecasting precipitation with small scale features captured really well.

Currently we compare our forecasts to ERA5 rather than observations



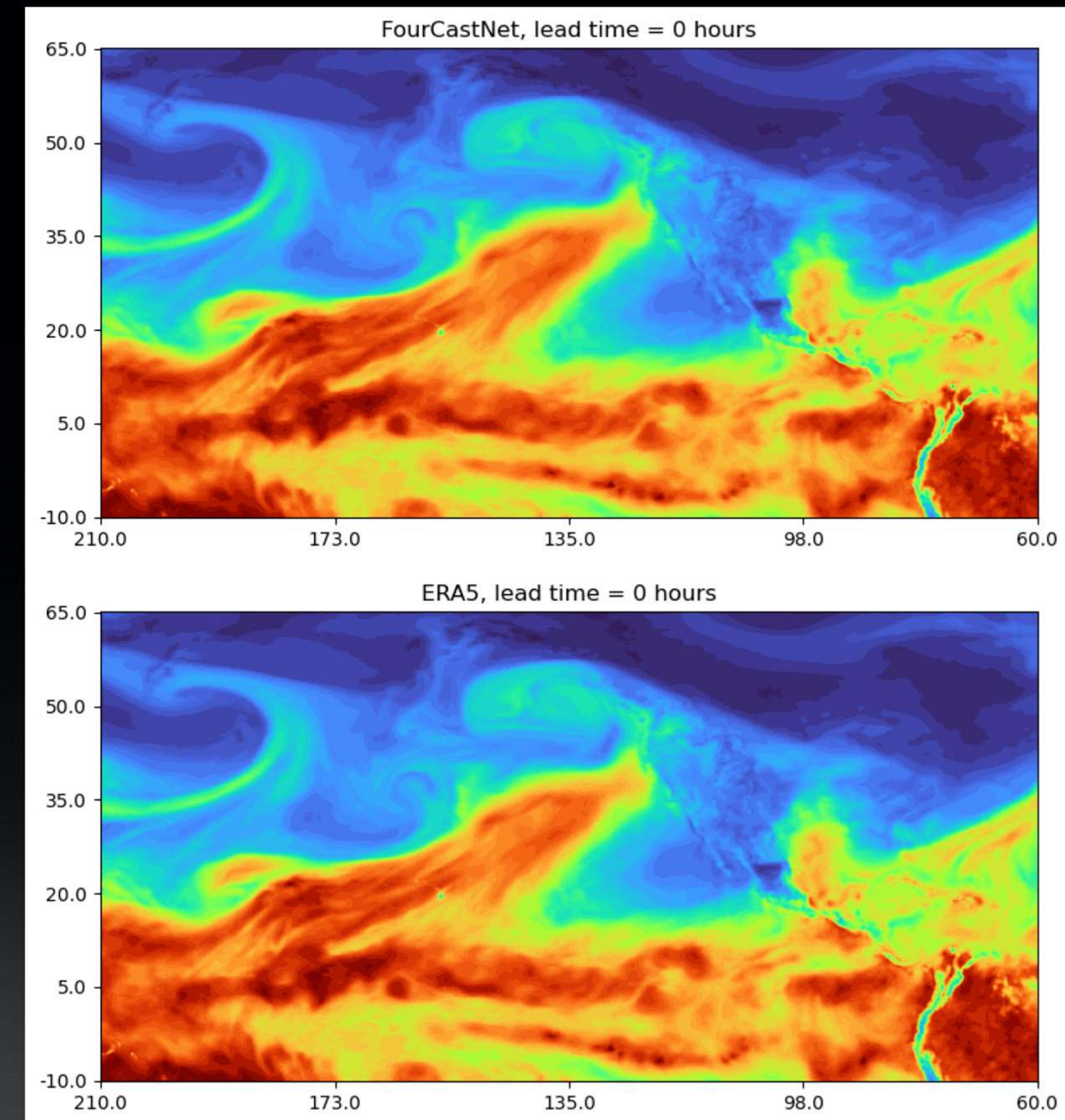
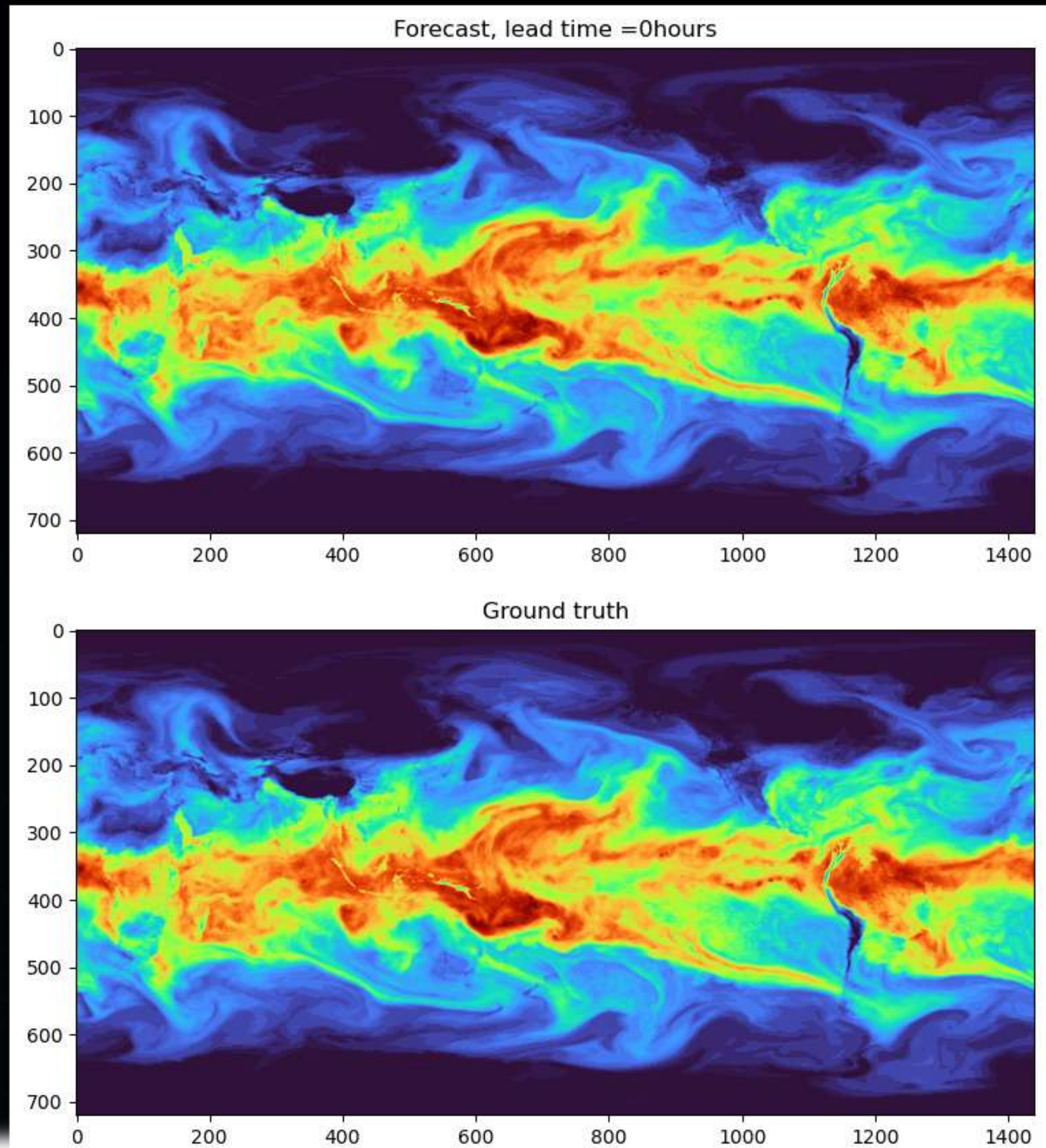
Pathak et al. (2022), FourCastNet: A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators, arXiv:2202.11214

PRECIPITATION



Pathak et al. (2022), FourCastNet: A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators, arXiv:2202.11214

MOISTURE VARIABLES: WATER VAPOR

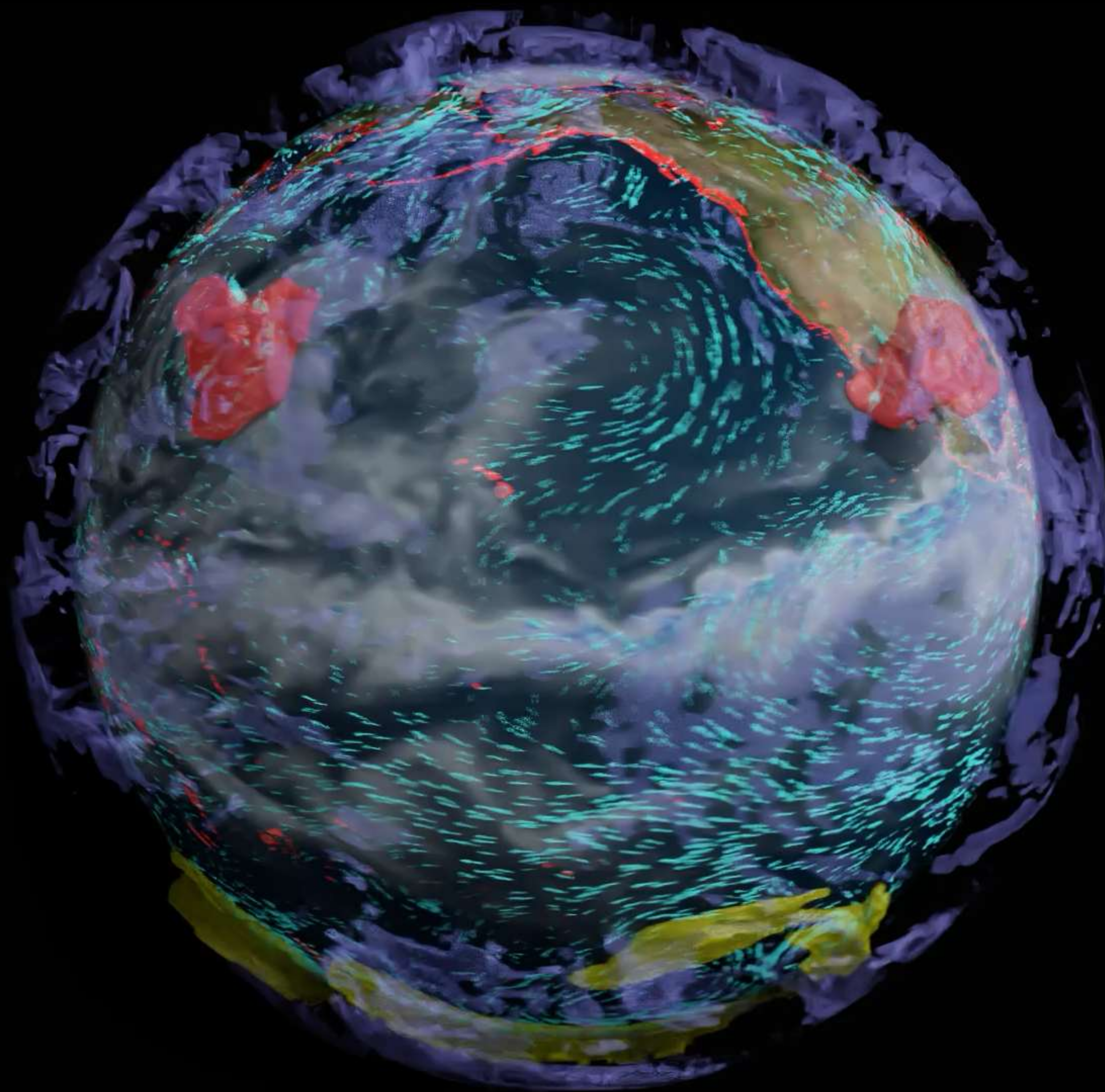


YEAR 2100

+3 C Global Temperature

+60% Extreme Tropical Cyclones

+400% Extreme Atmospheric Rivers



2100, SEP 14

- TROPICAL CYCLONES
- ATMOSPHERIC RIVERS
- CLOUDS
- WINDS



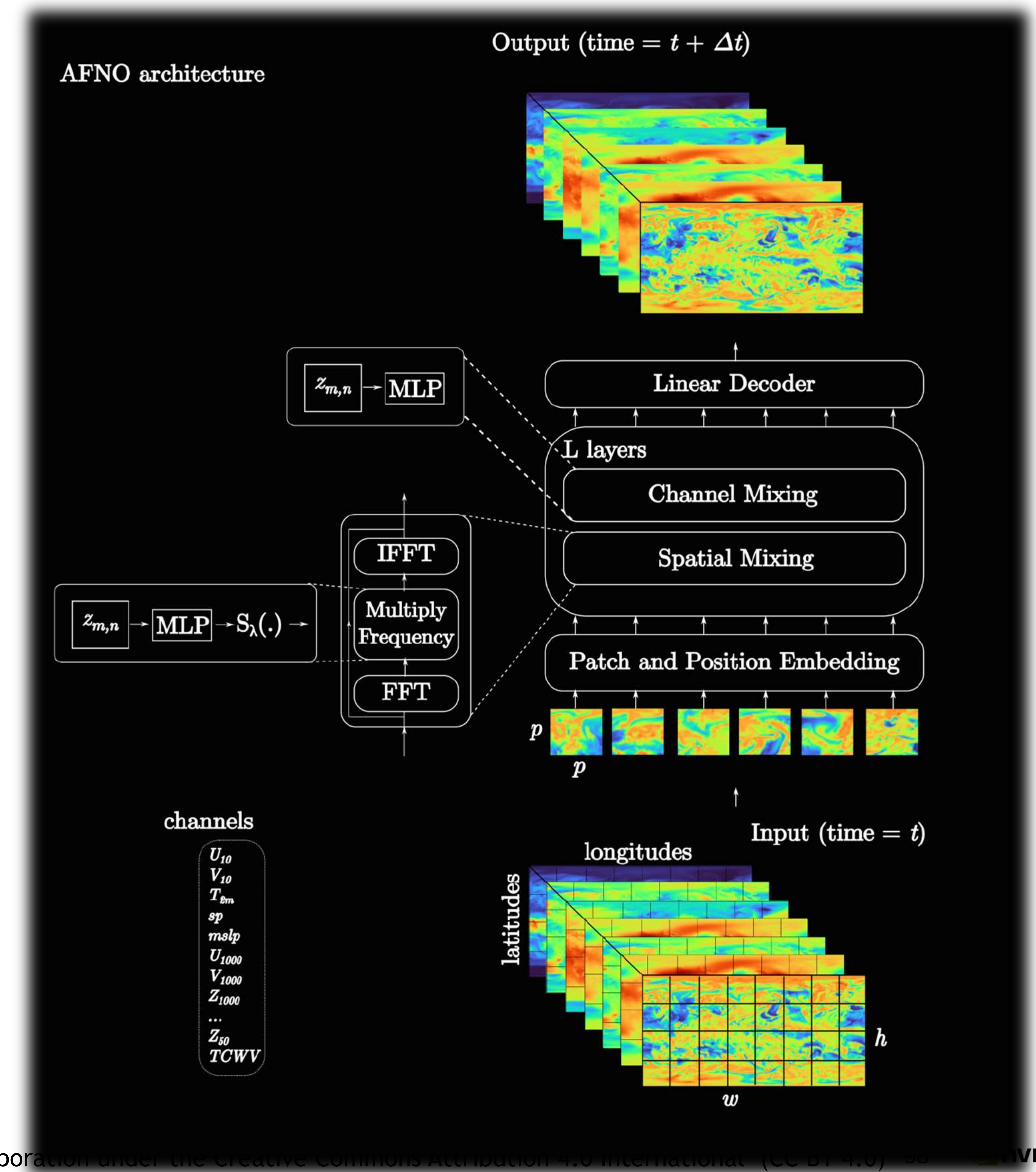
NVIDIA.

Training a Weather prediction model

Training FourCastNet model using PhysicsNeMo

- Goal: Given weather at time-step t , predict weather at time-step $t + \Delta t$
- Purely data-driven AI surrogate model
 - Trained on 0.25-degree ERA5 Reanalysis data
 - Predict 21 atmospheric variables
 - AFNO architecture based on FNOs and ViT

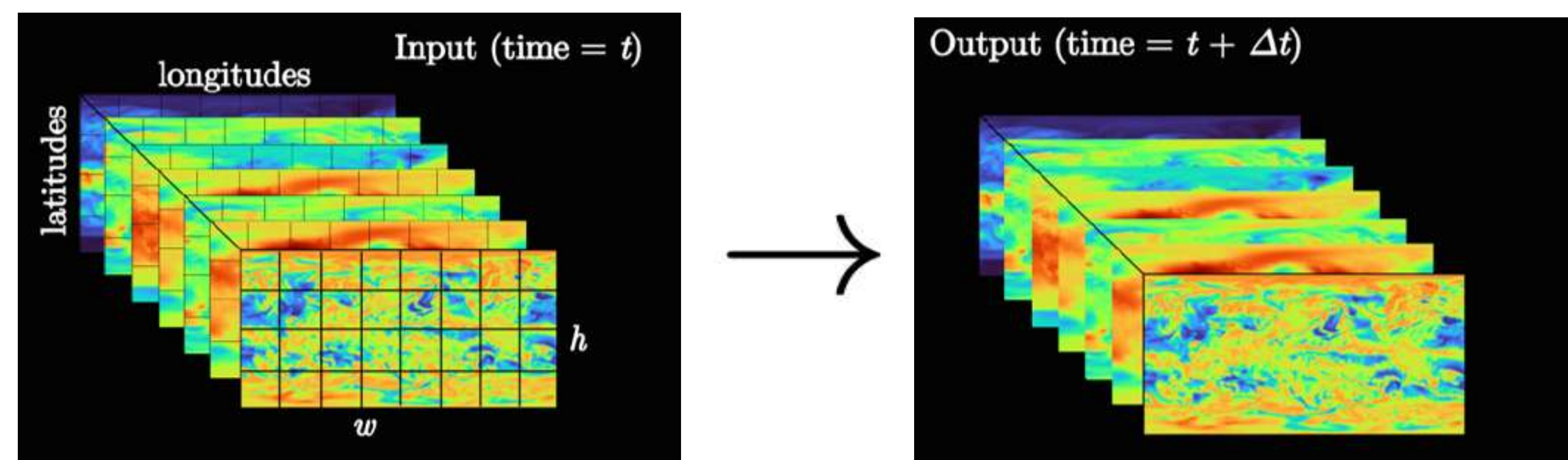
Vertical Levels	Variables
Surface	U_{10} , V_{10} , T_{2m} , SP, MSLP
1000 hPa	U , V , Z
850 hPa	T , U , V , Z , RH
500 hPa	T , U , V , Z , RH
Integrated	TCWV



Training a Weather prediction model

Training FourCastNet model using PhysicsNeMo

- Initialize PhysicsNeMo DistributedManager
 - Unified way to configure and query distributed env parameters
 - Abstractions built on top of `torch.distributed`
 - Facilitates data and model parallelism
- Initialize data loader
 - Optimized data loader using NVIDIA DALI for fast loading of weather datasets
 - Asynchronous and data-parallel
- Initialize AFNO model



```
@hydra.main(version_base="1.2", config_path="conf", config_name="config")
def main(cfg: DictConfig) -> None:
    DistributedManager.initialize() # Initialize the DistributedManager
    dist = DistributedManager() # Get instance of the DistributedManager

    # Setup MLFlow for logging
    initialize_mlflow(
        experiment_name="PhysicsNeMo-Launch-Dev",
        experiment_desc="PhysicsNeMo launch development",
        run_name="FCN-Training",
        run_desc="FCN ERA5 Training",
        user_name="PhysicsNeMo User",
        mode="offline",
    )
    LaunchLogger.initialize(use_mlflow=cfg.use_mlflow) # PhysicsNeMo launch logger
    logger = PythonLogger("main") # General python logger

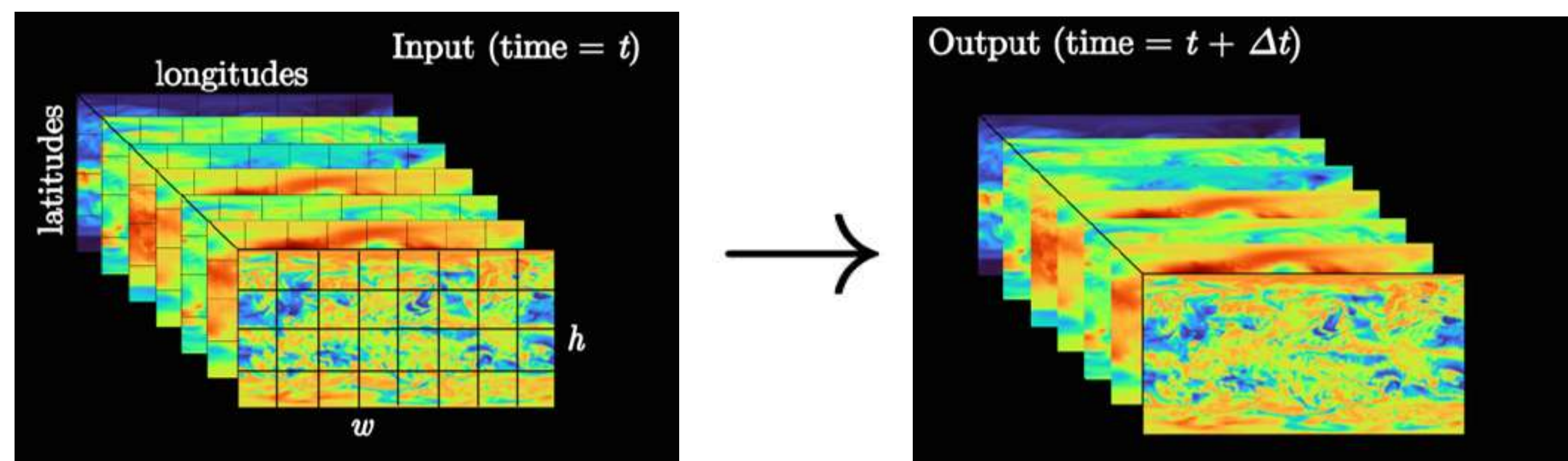
    # Datapipeline using NVIDIA DALI for loading the weather data
    datapipe = ERA5HDF5Datapipe(
        data_dir="/data/train/",
        stats_dir="/data/stats/",
        channels=[i for i in range(20)],
        num_samples_per_year=cfg.num_samples_per_year_train,
        batch_size=2,
        patch_size=(8, 8),
        num_workers=8,
        device=dist.device,
        process_rank=dist.rank,
        world_size=dist.world_size,
    )
    logger.success(f"Loaded datapipe of size {len(datapipe)}")

    fcn_model = AFNO(
        inp_shape=[720, 1440],
        in_channels=20,
        out_channels=20,
        patch_size=[8, 8],
        embed_dim=768,
        depth=12,
        num_blocks=8,
    ).to(dist.device)
```

Training a Weather prediction model

Training FourCastNet model using PhysicsNeMo

- Setup Data-Parallel training using PhysicsNeMo's DistributedManager for passing arguments to the PyTorch's DistributedDataParallel class
- Define train and validation step
- Use StaticCaptureTraining and StaticCaptureEvaluateNoGrad decorators to wrap the forward pass of the model during training and validation
 - Based on the model's MetaData and decorator arguments, applies optimizations like CUDA Graphs, Automatic Mixed Precision (AMP) etc. automatically



```
if dist.rank == 0 and wandb.run is not None:
    wandb.watch(
        fcn_model, log="all", log_freq=1000, log_graph=(True)
    )

# Distributed learning
if dist.world_size > 1:
    ddps = torch.cuda.Stream()
    with torch.cuda.stream(ddps):
        fcn_model = DistributedDataParallel(
            fcn_model,
            device_ids=[dist.local_rank],
            output_device=dist.device,
            broadcast_buffers=dist.broadcast_buffers,
            find_unused_parameters=dist.find_unused_parameters,
        )
    torch.cuda.current_stream().wait_stream(ddps)

# Initialize optimizer and scheduler
optimizer = optimizers.FusedAdam(
    fcn_model.parameters(), betas=(0.9, 0.999), lr=0.0005, weight_decay=0.0
)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=150)

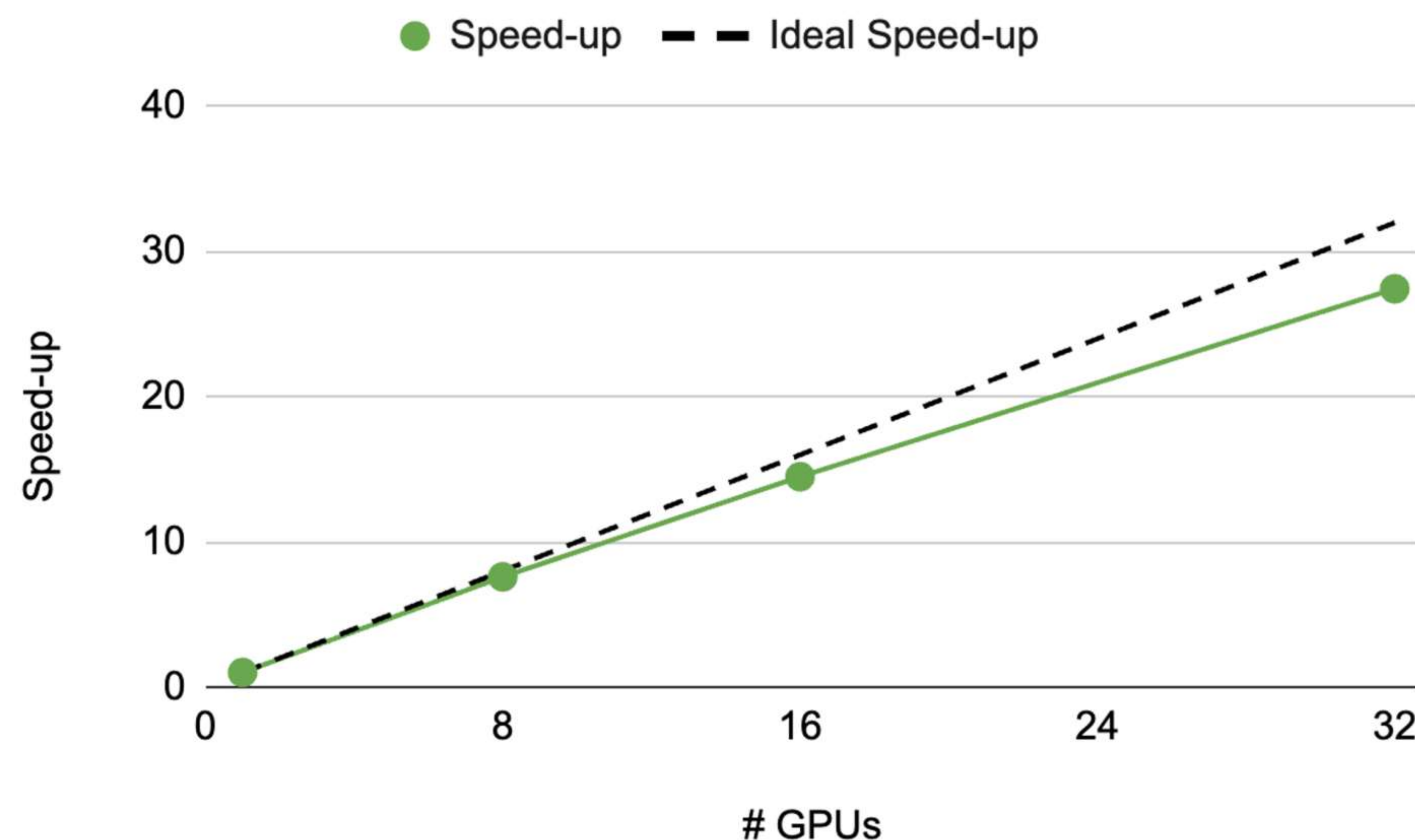
@StaticCaptureEvaluateNoGrad(model=fcn_model, logger=logger, use_graphs=False)
def eval_step_forward(my_model, invar):
    return my_model(invar)

@StaticCaptureTraining(model=fcn_model, optim=optimizer, logger=logger)
def train_step_forward(my_model, invar, outvar):
    # Multi-step prediction
    loss = 0
    # Multi-step not supported
    for t in range(outvar.shape[1]):
        outpred = my_model(invar)
        invar = outpred
        loss += loss_func(outpred, outvar[:, t])
    return loss
```

Training a Weather prediction model

Training FourCastNet model using PhysicsNeMo

- Define training protocol
 - Loop for a set of training epochs and through entire training dataset
- Run the problem using
 - Single GPU: `python <script-name>.py`
 - Multi GPU: `mpirun -np <num-GPUs> python <script-name>.py` (no code changes required)
 - Multi-node execution also supported (refer documentation for details)



```
# Main training loop
max_epoch = cfg.max_epoch
for epoch in range(1, max_epoch + 1):
    # Wrap epoch in launch logger for console / WandB logs
    with LaunchLogger("train", epoch=epoch, num_mini_batch=len(datapipe)) as log:
        # === Training step ===
        for j, data in enumerate(datapipe):
            invar = data[0]["invar"]
            outvar = data[0]["outvar"]
            loss = train_step_forward(fcn_model, invar, outvar)

            log.log_minibatch({"loss": loss.detach()})
            log.log_epoch({"Learning Rate": optimizer.param_groups[0]["lr"]})

        if dist.world_size > 1:
            torch.distributed.barrier()

        scheduler.step()

    if (epoch % 5 == 0 or epoch == 1) and dist.rank == 0:
        # Use PhysicsNeMo checkpoint
        save_checkpoint(
            "./checkpoints",
            models=fcn_model,
            optimizer=optimizer,
            scheduler=scheduler,
            epoch=epoch,
        )

    if dist.rank == 0:
        logger.info("Finished training!")

if __name__ == "__main__":
    main()
```

Demo / Optional Notebook

- Explore Weather prediction using PhysicsNeMo

Training more Weather prediction models

Training FCN, GraphCast and other models using PhysicsNeMo

- [Unified recipe](#) to train and infer global weather forecasting models
- Features to easily change models, data and training procedure
 - Users can either choose existing models or bring your own
- Hooks to score the trained model against atmospheric metrics (ongoing)

README.md

Unified Recipe for Training Global Weather Forecasting Models

This example demonstrates how to train a neural global weather forecast model. The recipe is set up so that modifying the model architecture, data, or the training procedure is straightforward.

Configs

The `conf` directory contains the configuration files for the model, data, training, etc. The configs are given in YAML format and use the `omegaconf` library to manage them. Several example configs are given for generating different datasets, models, and training procedures. For example, AFNO and GraphCast are given with corresponding training procedure and datasets configs. The default configs are set to only download and train a tiny dataset and can be run on an 8GB GPU. To train larger models please adjust `conf/config.yaml` according to the comments.

Getting the ERA5 dataset

In this example we provide scripts to obtain the ERA5 dataset from [ARCO ERA5](#) and perform needed curation and remapping steps. ARCO ERA5 contains a complete lat lon gridded dataset of the ERA5 reanalysis including single and pressure level

```
17 experiment_name: "Unified-Training"
18 experiment_desc: "unified training recipe for global weather models"
19 run_desc: "unified training recipe for global weather models"
20
21 # Get defaults
22 defaults:
23
24     # Dataset
25     - dataset/tiny_6hr_subsample
26     #- dataset/6hr_subsample # uncomment for full dataset
27
28     # Curated Dataset
29     - curated_dataset/tiny_7_var
30     #- curated_dataset/7_var # uncomment for full dataset
31
32     # Transform
33     - transform/trim_lat720
34
35     # Filesystem
36     - filesystem/local
37
38     # Model
39     - model/tiny_afno
40     #- model/afno
41     #- model/graphcastnet # Example of changing model to graphcastnet
42
43     # Training
44     - training/afno
45
46     # Validation
47     - validation/afno
```

Handle data, model and training via Hydra configs

Computing Weather forecasts

And comparing / scoring models

```
import torch
import xarray
import numpy as np
import physicsnemo
from physicsnemo.distributed import DistributedManager
import matplotlib.pyplot as plt

# Load the PhysicsNeMo checkpoint
# All the meta-data is already stored, so no need to instantiate the model
model = physicsnemo.Module.from_checkpoint("./fcn/fcn.mdlus")
device = DistributedManager().device
model = model.to(device)
model.eval()

# Load the initial condition
ic = xarray.open_dataset("./2024-01-01T00_00_00-subsampled.nc")["fields"]

# Load the mean and standard deviation to normalize the inputs
mean = np.load("./fcn/global_means.npy")
std = np.load("./fcn/global_stds.npy")

# Normalize the initial condition
ic_norm = (ic - mean) / std

# Create a tensor input
ic_norm = torch.from_numpy(ic_norm.values).to(device).to(torch.float32)
```

```
# Convert the output from 721x1440 -> 720x1440
ic_norm = ic_norm[:, :, :720, :]

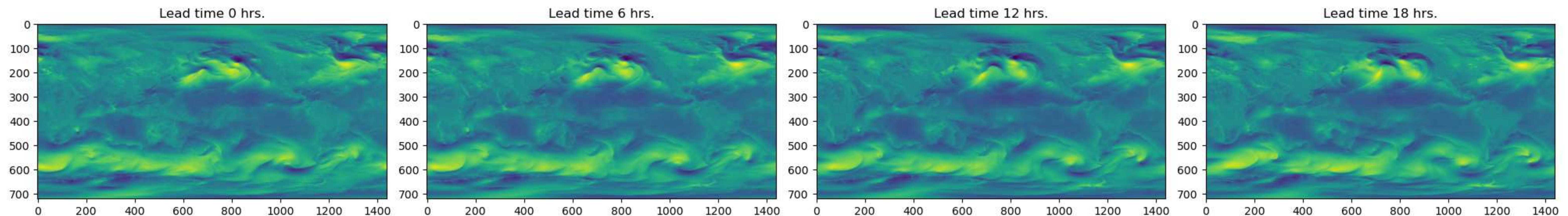
# Generate an auto-regressive inference for 8 rollouts (8*6 = 48hr forecast)
outputs=[]
with torch.no_grad():
    for i in range(8):
        if i == 0:
            out = model(ic_norm)
        else:
            out = model(out)
        outputs.append(out)

# Plot the results
figsize = (5 * 8, 5)
fig, axes = plt.subplots(nrows=1, ncols=8, figsize=figsize,)

for i in range(8):
    out_numpy = outputs[i].cpu().numpy()

    # un-normalize
    out_numpy = out_numpy * std + mean
    axes[i].imshow(out_numpy[0, 0]) # plot the first channel
    axes[i].set_title(f"Lead time {i*6} hrs.")

plt.tight_layout()
plt.show()
```



Explore more scientific capabilities via [Earth-2 MIP](#) and [Earth-2 service](#)

Demo / Optional Notebook

- Explore Weather prediction using PhysicsNeMo-Sym

Core Features

Models

- Built-in & BYO Prognostic Model
- Built-in & BYO Diagnostic Model

Data Sources

- Built-in & BYO Data Sources
- CDS, ARCO, GFS, HRRR, on-prem

IO Support

- Built-in Zarr, NetCDF, and Xarray support
- BYO IO backend
- Support arbitrary writing patterns

Robustness

- CI/CD, QA testing
- Pip wheel and docker support
- Enhanced User Documentation

Statistics

- Built-in & BYO statistics
- Utilities for interfacing with IO

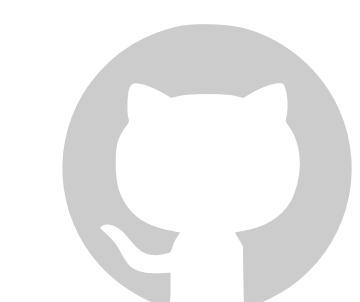
Ensemble Perturbation

- Built-in & BYO initial condition perturbation methods
- Hooks for adding model perturbation

AI Weather Model Inference through Earth-2 Studio

High-level API for Rapid Experimentation and Testing

- Open-source
- Building blocks for bring your own workflows and examples for deterministic and ensemble-based predictions
- Extensive examples for extending built-in APIs
- AI-on-top models for increasing set of variables and spatiotemporal resolution



Produce an AI Weather Forecast with a few lines of code

<https://github.com/NVIDIA/earth2studio>

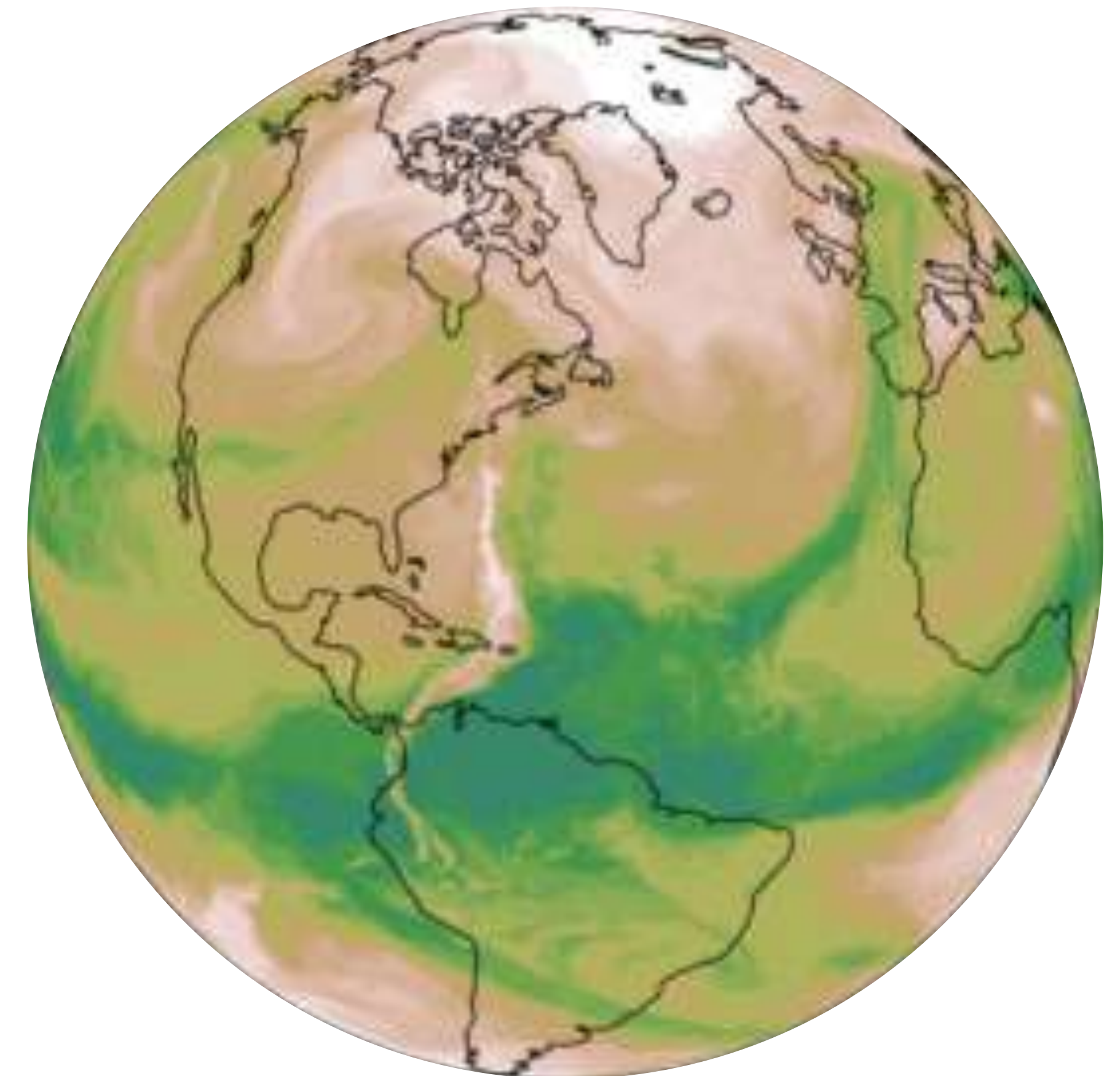
```
from earth2studio.models.px import FCN
from earth2studio.data import GFS
from earth2studio.io import ZarrBackend
import earth2studio.run as run

# Load FourCastNet pretrained model
model = FCN.load_model(FCN.load_default_package())

# Create the data source
data = GFS()

# Create a Zarr IO Backend
io = ZarrBackend()

# Run 20 steps of inference
output_datastore = run.deterministic(["2024-01-01"], 20, model, data, io)
```



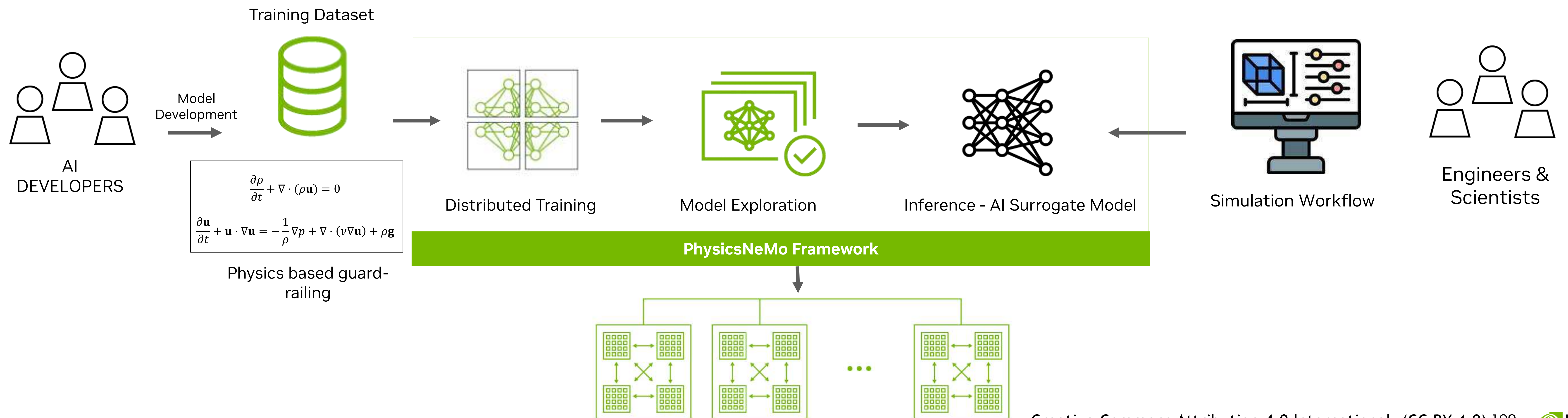
Total column water vapour field
at last time step of the forecast

Demo / Optional Notebook

- Explore Weather prediction using PhysicsNeMo-Sym

Summary: PhysicsNeMo Framework

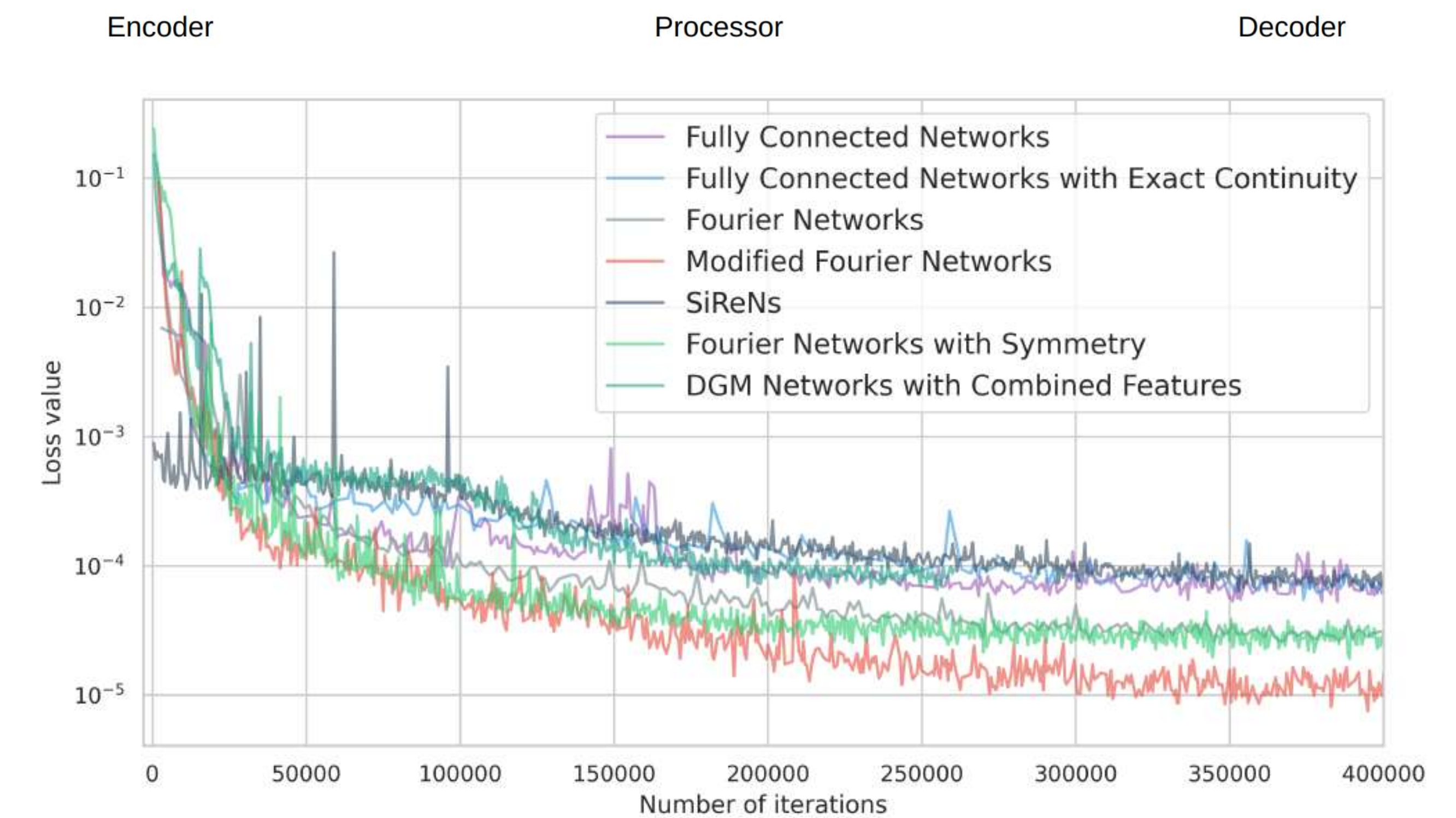
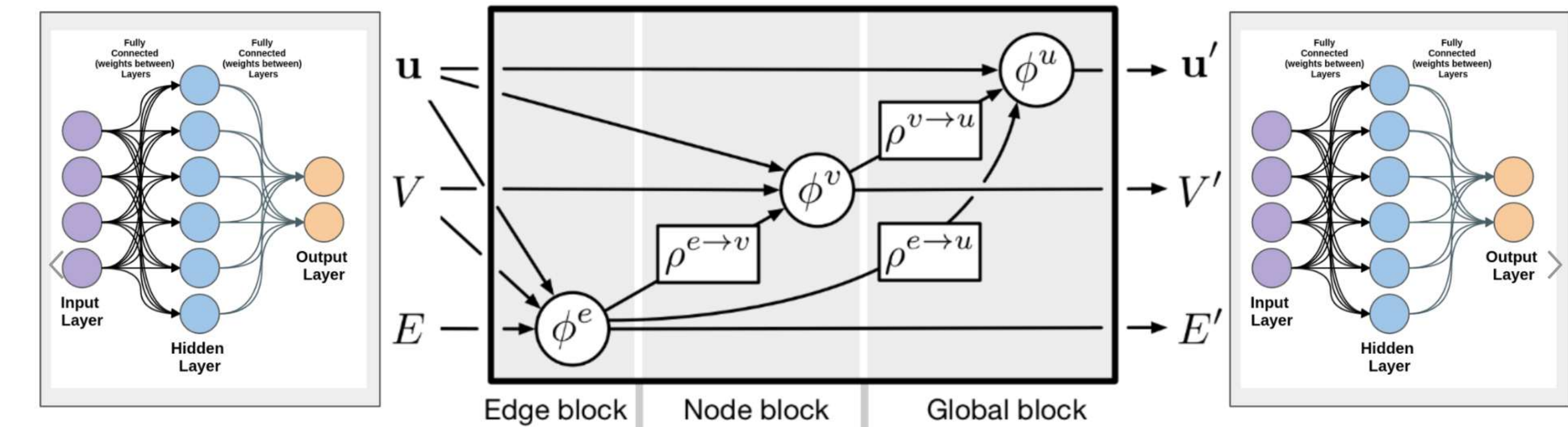
- PhysicsNeMo is a framework to build and customize Physics-ML models and training workflows:
 - Built on top of PyTorch, interoperable with PyTorch
 - Utilities designed to enable DL researchers as well as Domain experts
 - A vast library of optimized models particularly suited for Physics-ML applications
 - Develop performant and scalable training pipelines with ease
 - Utilities to introduce Physics-based guard-railing
 - Several case studies across different domains to enable transition from POC to Industrial problems
 - Open Source and accessible via GitHub, NGC registry, PyPi



Summary: PhysicsNeMo Framework

Variety of model architectures

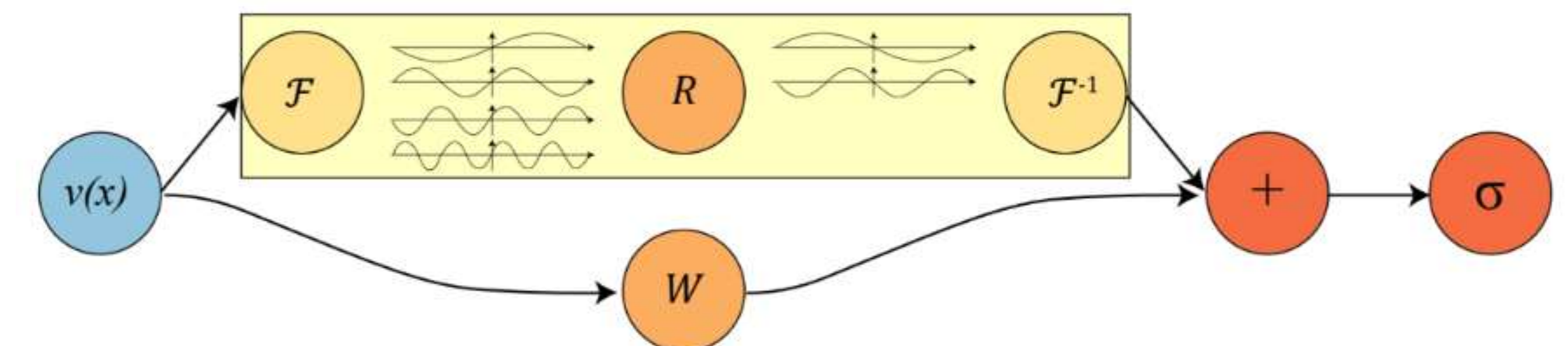
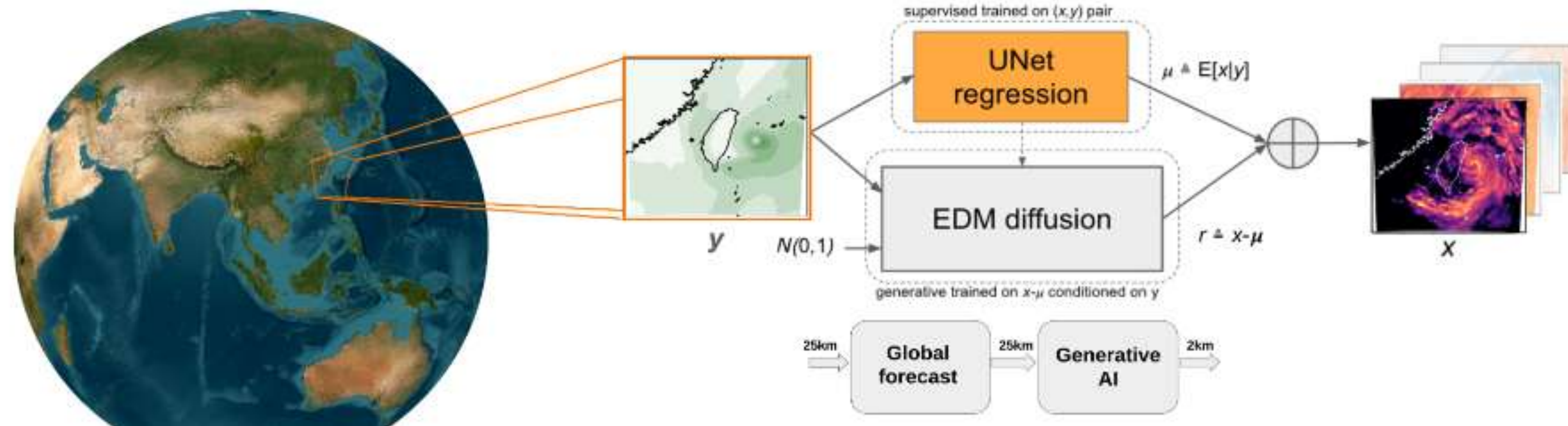
- PhysicsNeMo Model Zoo - Diverse Physics-ML approaches:
 - Fully Physics driven AI models
 - Fully data driven AI models
 - Hybrid (data + Physics) AI models
- Models like GNNs, RNNs, Diffusion Models, Neural Operators (FNO, DeepONet, etc.), MLPs and its variants available to use out-of-the-box or use optimized layers and components in your custom models
- PhysicsNeMo Model enable features like:
 - Argument-less checkpoint loading
 - Model registry



12.5x super-resolution + radar channel synthesis

FEATURES:
ERAS:
36 x 36 (20-ch)

TARGETS:
Radar-assimilating WRF
448x448 (4-ch)



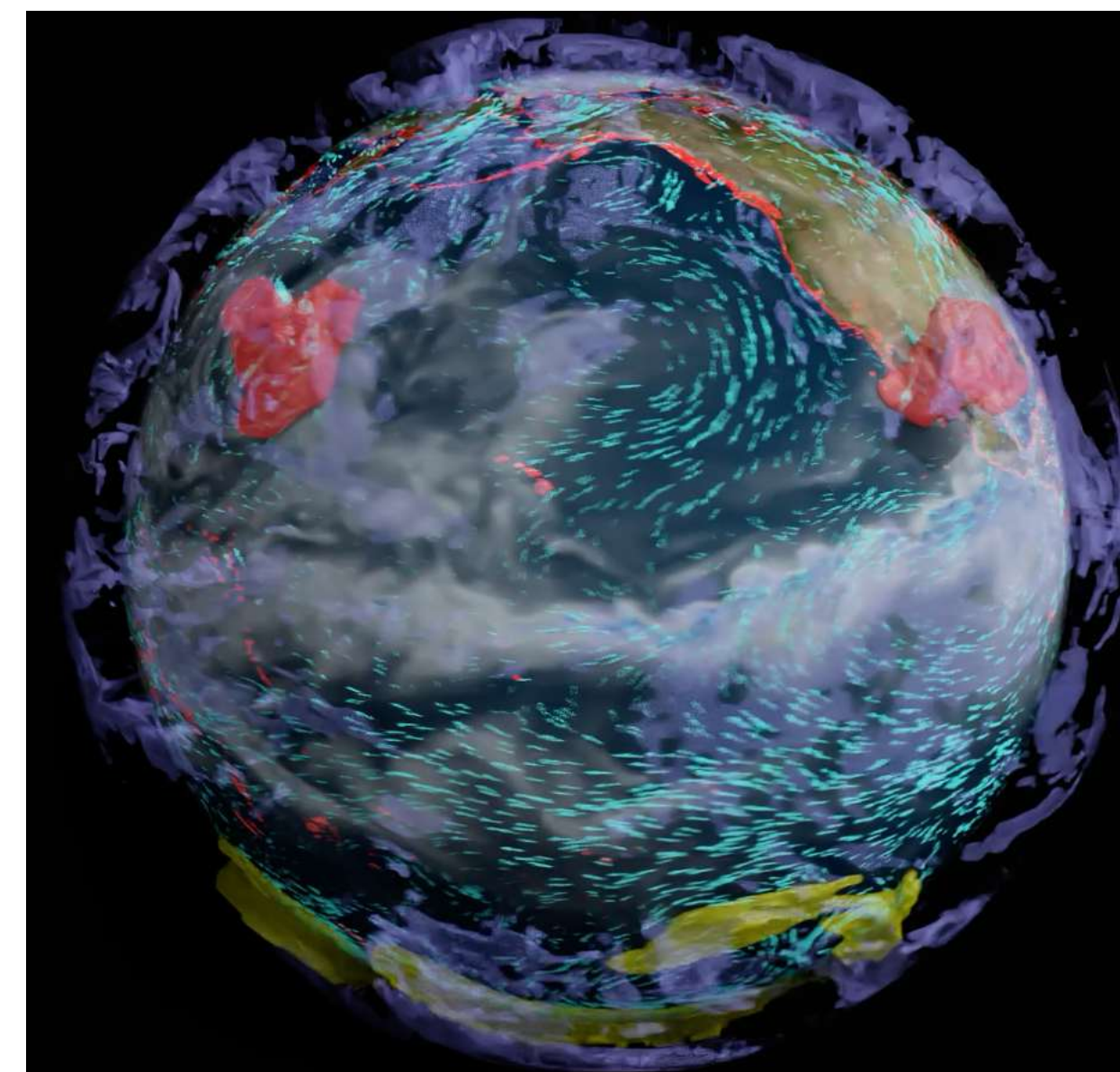
Summary: PhysicsNeMo Framework

Variety of physics applications

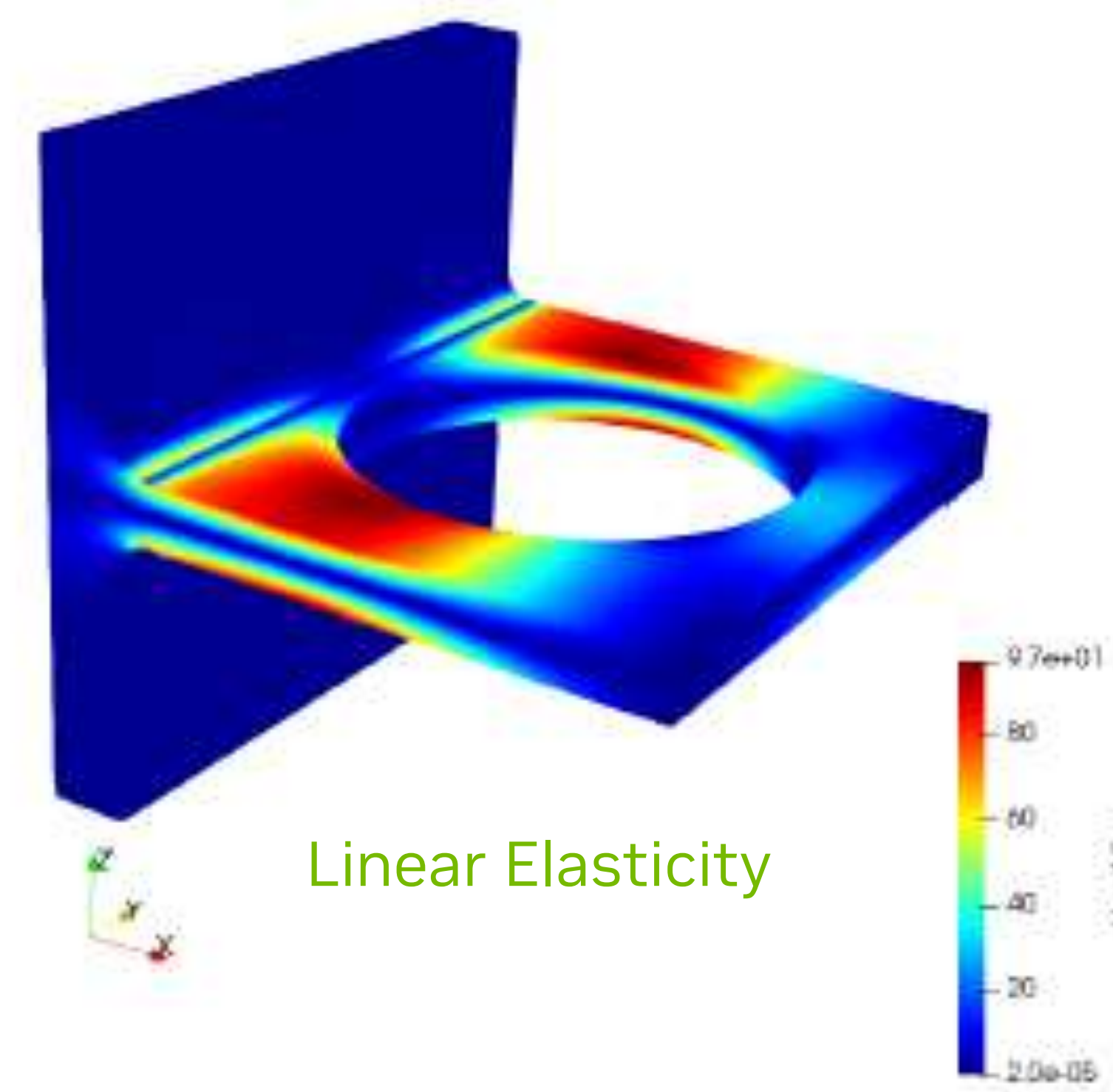
- Sample case studies and use cases covering diverse physical domains like:

- Fluid dynamics
- Heat transfer
- Acoustics and waveform inversion
- Linear Elasticity
- Molecular Dynamics
- Electromagnetics
- Turbulence modeling
- Weather and climate modeling
- And more....

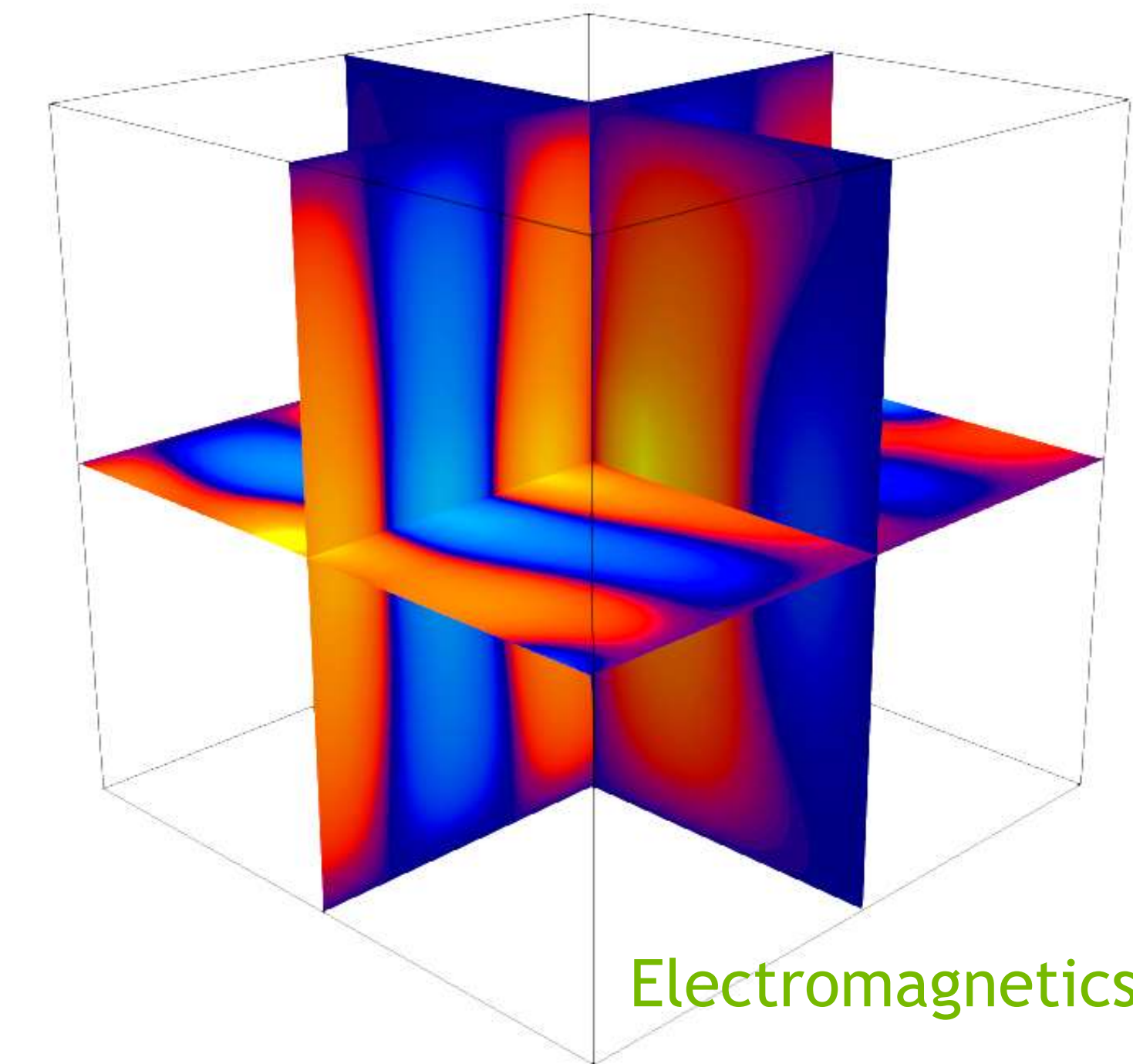
- Reference samples for [PhysicsNeMo](#), and [PhysicsNeMo-Sym](#)



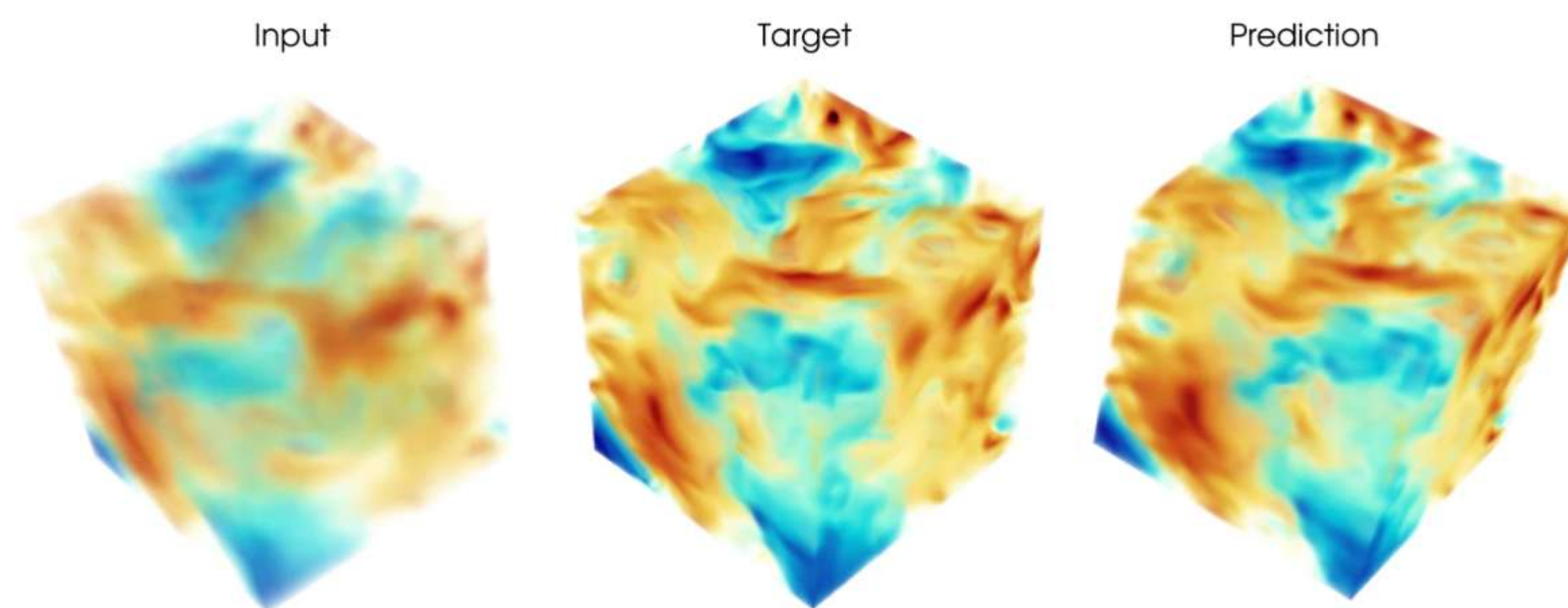
Weather/climate modeling



Linear Elasticity

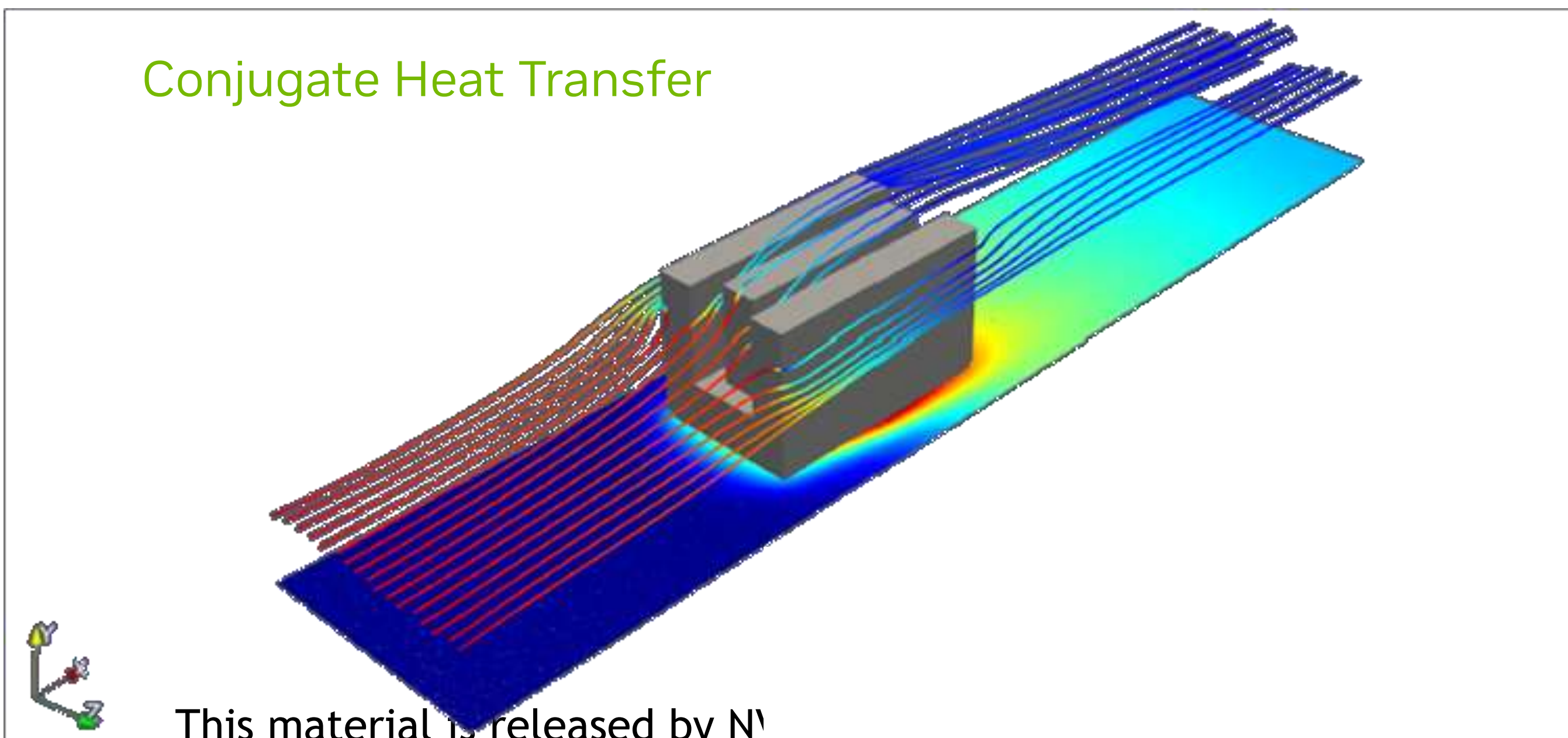


Electromagnetics

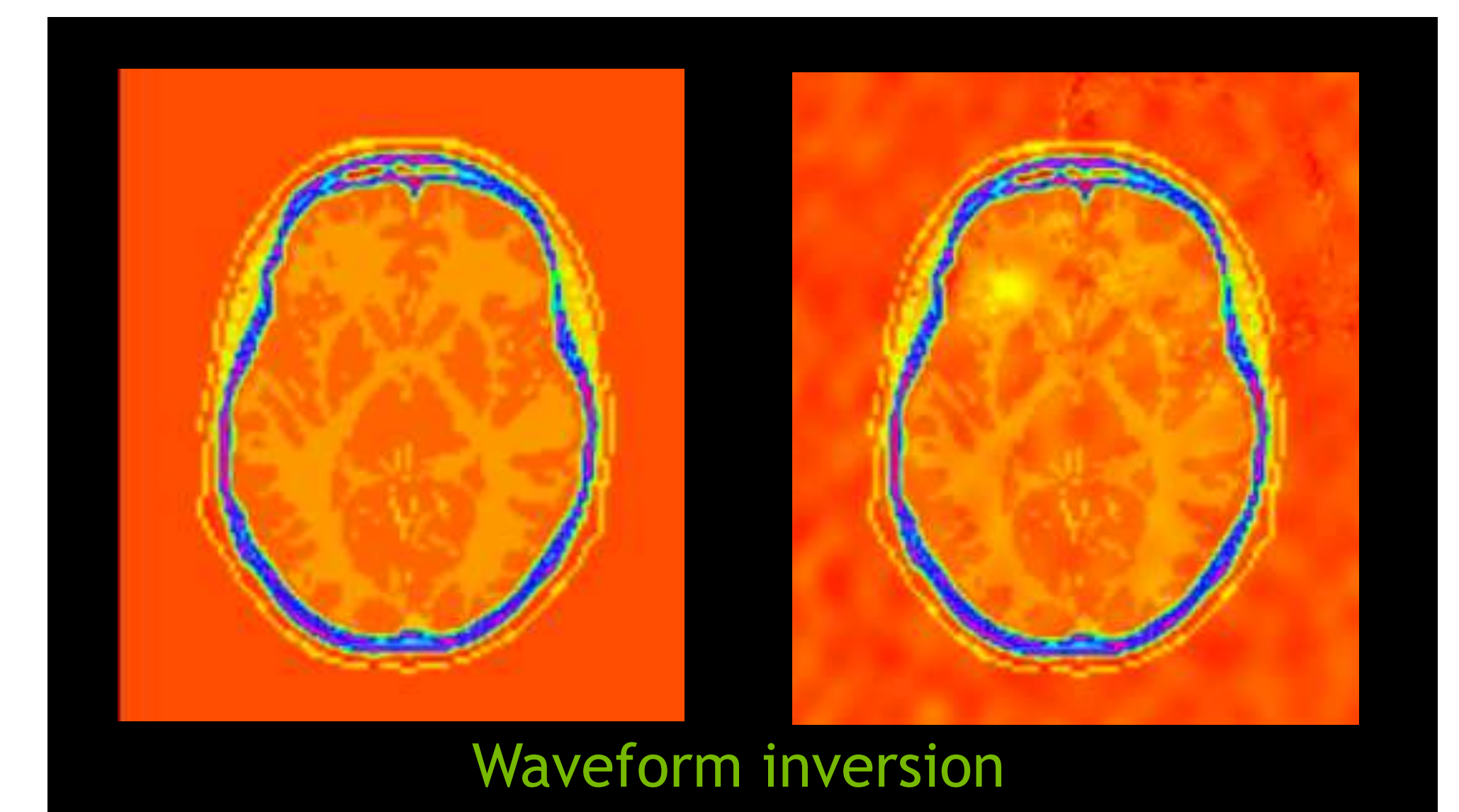


Turbulence Super Resolution

Conjugate Heat Transfer



This material is released by N



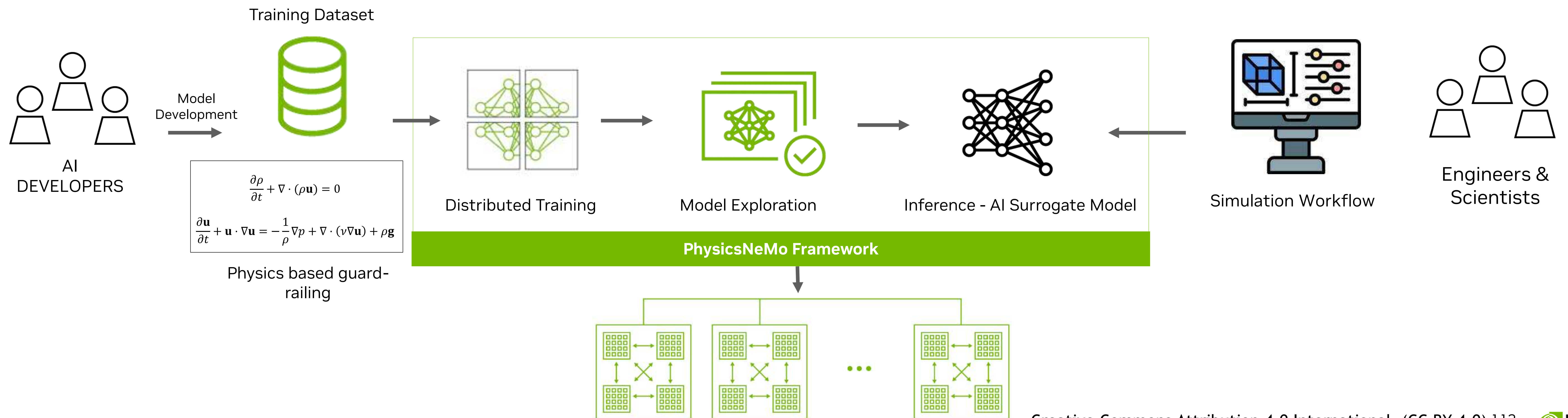
Waveform inversion



Conclusion

Summary: PhysicsNeMo Framework

- PhysicsNeMo is a framework to build and customize Physics-ML models and training workflows:
 - Built on top of PyTorch, interoperable with PyTorch
 - Utilities designed to enable DL researchers as well as Domain experts
 - A vast library of optimized models particularly suited for Physics-ML applications
 - Develop performant and scalable training pipelines with ease
 - Utilities to introduce Physics-based guard-railing
 - Several case studies across different domains to enable transition from POC to Industrial problems
 - Open Source and accessible via GitHub, NGC registry, PyPi



Darcy Flow – Fourier Neural Operators (FNO)

Results and discussion

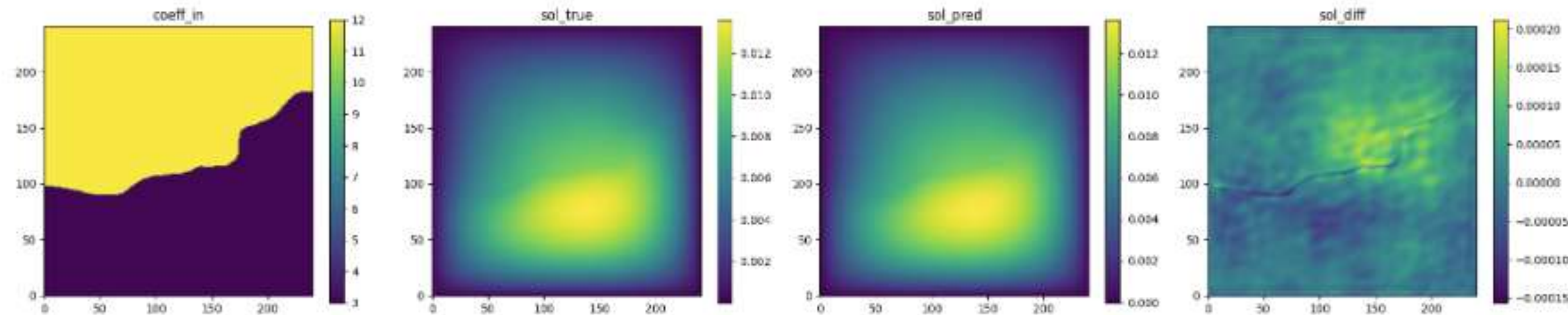


Fig. 61 FNO validation prediction 1. (Left to right) Input permeability, true pressure, predicted pressure, error.

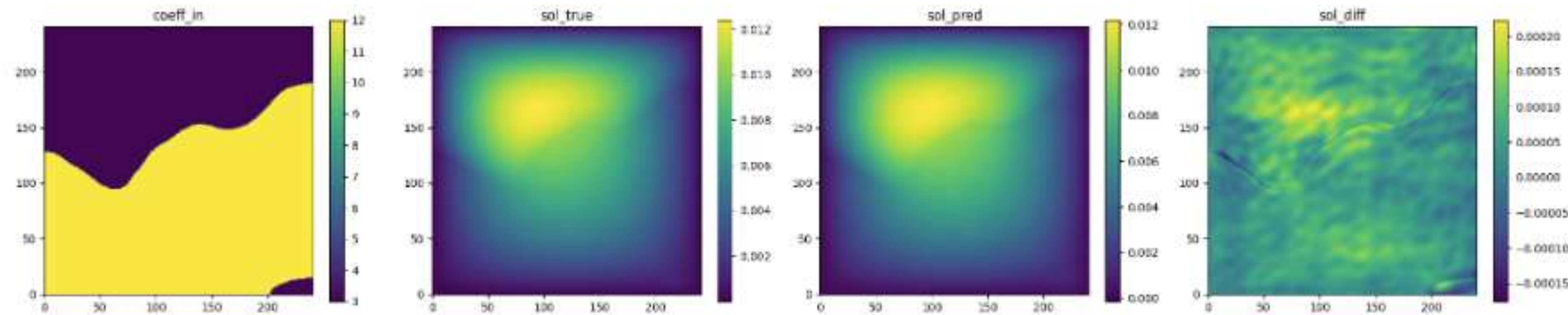


Fig. 62 FNO validation prediction 2. (Left to right) Input permeability, true pressure, predicted pressure, error.

FNO accurately learns the solution of this system.

PhysicsNeMo supports the visualization of results through images (matplotlib), Tensorboard, VTK files and Omniverse for select problems.

For more information, please refer to the official [PhysicsNeMo user guide example](#).

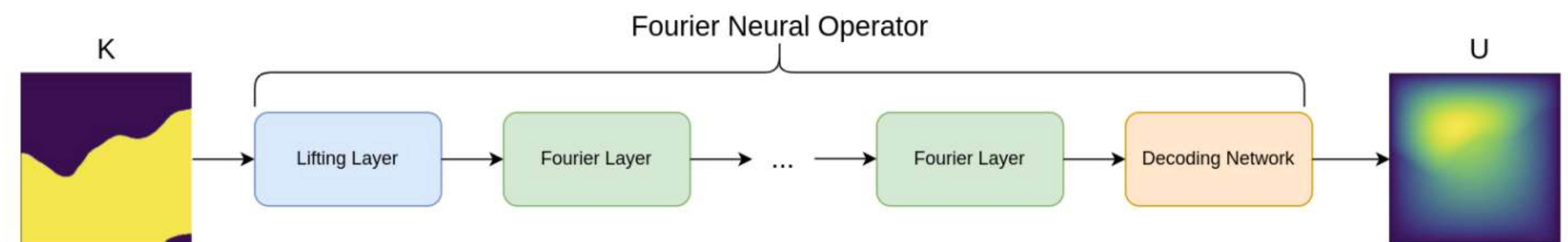
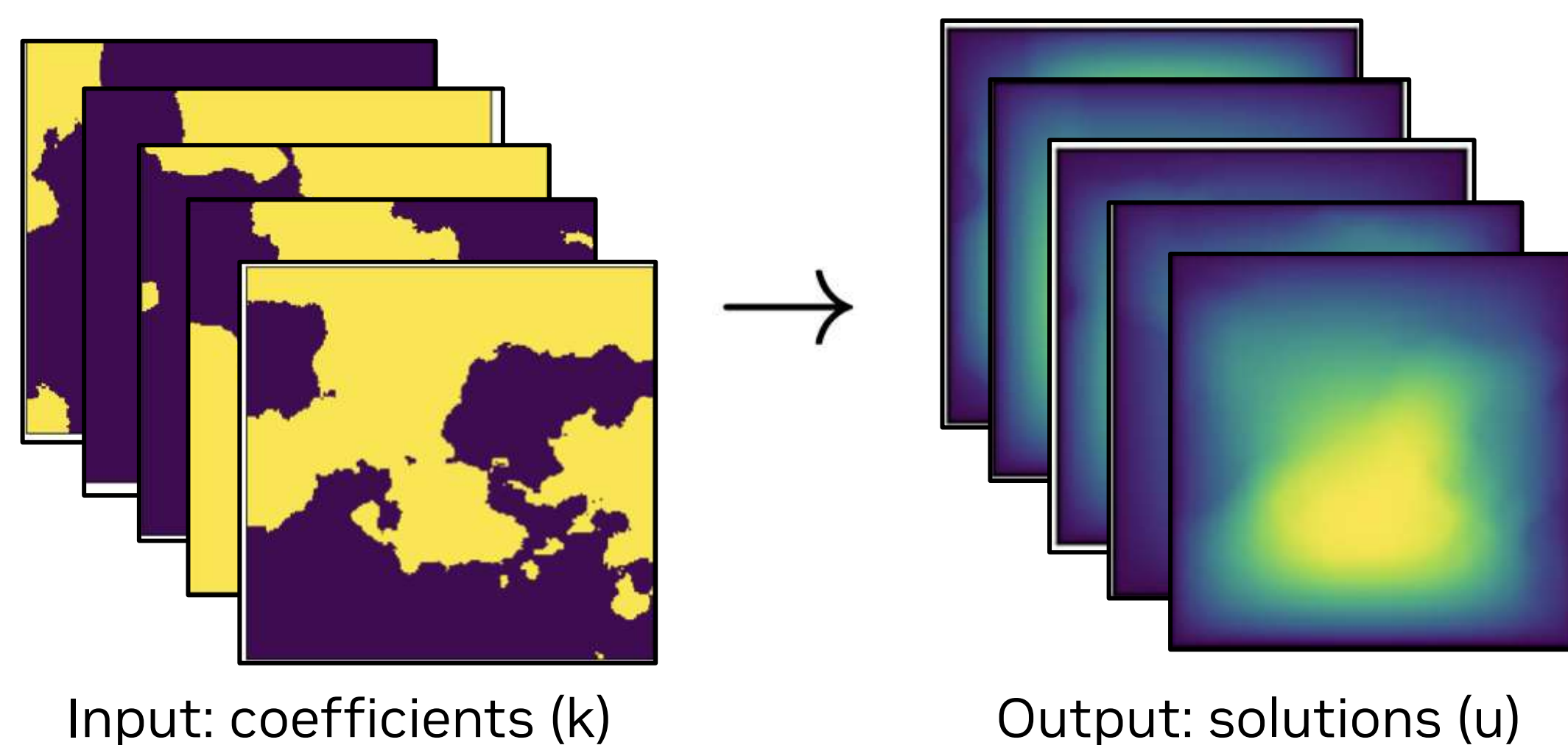
Developing Physics-ML models using data in PhysicsNeMo

Using FNOs to create a surrogate model

- Develop a surrogate model that learns a mapping between a permeability field and pressure field, $\mathbf{K} \rightarrow \mathbf{U}$, for a distribution of permeability fields $\mathbf{K} \sim p(\mathbf{K})$.
- The Darcy PDE (Diffusion equation) is a second order, elliptical PDE with the following form:

$$-\nabla \cdot (k(\mathbf{x}) \nabla u(\mathbf{x})) = f(\mathbf{x}), \quad \mathbf{x} \in D,$$

- $u(\mathbf{x})$: Flow pressure
- $k(\mathbf{x})$: Permeability field
- $f(\cdot)$: Forcing function
- $D = \{x, y \in (0,1)\}$ with the boundary condition $u(\mathbf{x}) = 0, \mathbf{x} \in \partial D$.
- Both the permeability and flow fields are discretized into a 2D matrix $\mathbf{K}, \mathbf{u} \in \mathbb{R}^{N \times N}$.



Developing Physics-ML models using data in PhysicsNeMo

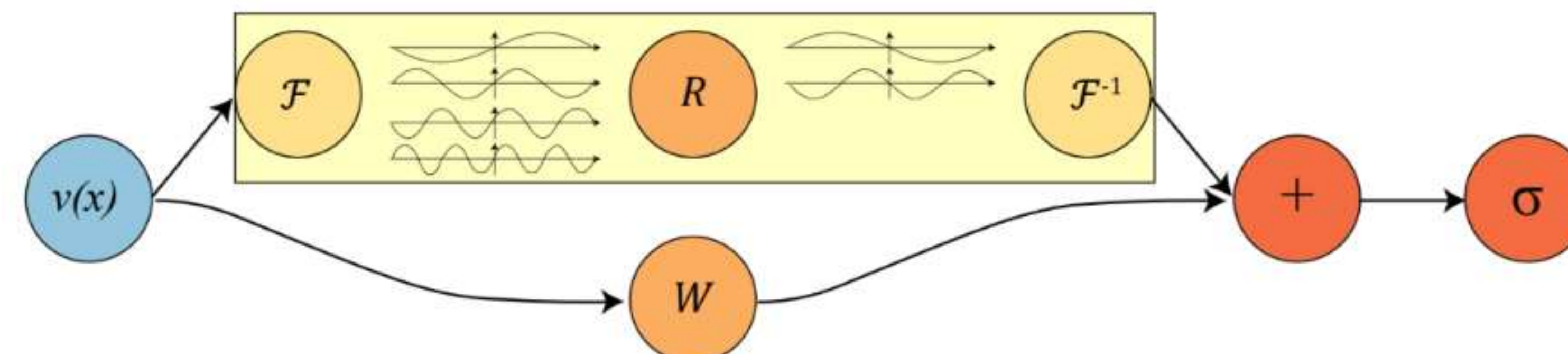
Using FNOs to create a surrogate model

- The key feature of FNO is the spectral convolutions: operations that place the integral kernel in Fourier space:

$$(K(\mathbf{w})\phi)(x) = F^{-1}(R_{\mathbf{w}} \cdot (F)\phi)(x), \quad \forall x \in D$$

- $R_{\mathbf{w}}$: Transformation containing the learnable parameters $\mathbf{w}$, calculated over the entire *structured Euclidean* domain D .
- Fourier coefficients are truncated to only the lower modes - explicit control over the dimensionality of the spectral space and linear operator.
- The FNO model is a composition of a fully-connected “lifting” layer, L spectral convolutions with point-wise linear skip connections and a decoding point-wise fully-connected neural network at the end:

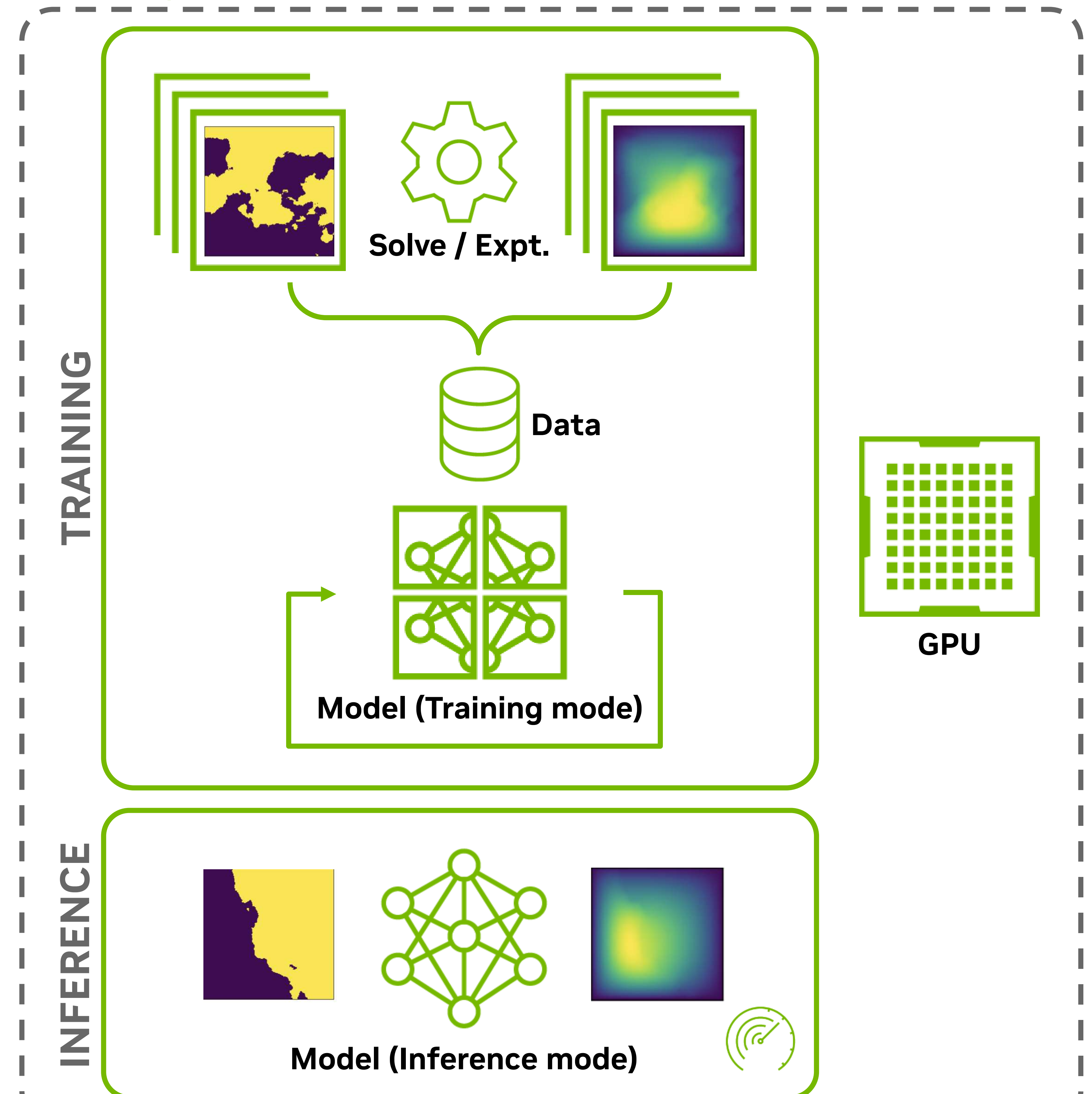
$$u_{net}(\Phi; \theta) = Q \circ \sigma(\mathbf{W}_L + \mathbf{K}_L) \circ \dots \circ \sigma(\mathbf{W}_1 + \mathbf{K}_1) \circ P(\Phi), \quad \Phi = \{\phi(x); \forall x \in D\}$$
 - $\sigma(\mathbf{W}_i + \mathbf{K}_i)$: Spectral convolution layer i with the point-wise linear transform $\mathbf{W}_i$ and activation function $\sigma(\cdot)$.
 - P : Point-wise lifting network that projects the input into a higher dimensional latent space.
 - Q : Point-wise fully-connected decoding network.
- Since all fully-connected components of FNO are point-wise operations, the model is invariant to the dimensionality of the input.



Developing Physics-ML models using data in PhysicsNeMo

Using FNOs to create a surrogate model

- Train a data-driven model:
 - Given input-output pairs, find the map (operator)
- Define the training loop using interoperable PhysicsNeMo-PyTorch utilities
 - Optimized data loaders and data pipes for domain specific use cases
 - Optimized Model Zoo and layers targeted for Physics-ML applications
 - Utilities for Distributed training and scaling
 - Easily apply performance optimizations like CUDA Graphs, AMP
 - Utilities for easier logging
 - Checkpointing utilities to save and load checkpoints
 - Load model checkpoints without model arguments
 - Utilities for deployment and inference



Developing Physics-ML models using data in PhysicsNeMo

Typical training workflow

Python Imports

```
import hydra
from omegaconf import DictConfig
import torch
import numpy as np
import matplotlib.pyplot as plt
from hydra.utils import to_absolute_path
import torch.nn.functional as F
from torch.utils.data import DataLoader
from itertools import chain

from PhysicsNeMo.models.fno import FNO
from PhysicsNeMo.launch.logging import LaunchLogger
from PhysicsNeMo.launch.utils.checkpoint import save_checkpoint

from utils import HDF5MapStyleDataset

@hydra.main(version_base="1.3", config_path="conf", config_name="config_pure_data.yaml")
def main(cfg: DictConfig):

    LaunchLogger.initialize()

    dataset = HDF5MapStyleDataset(
        to_absolute_path("./datasets/Darcy_241/train.hdf5"), device="cuda"
    )
    validation_dataset = HDF5MapStyleDataset(
        to_absolute_path("./datasets/Darcy_241/validation.hdf5"), device="cuda"
    )

    dataloader = DataLoader(dataset, batch_size=2, shuffle=True)
    validation_dataloader = DataLoader(validation_dataset, batch_size=1, shuffle=False)

    model_branch = FNO(
        in_channels=2,
        out_channels=1,
    ).to("cuda")

    optimizer = torch.optim.Adam(
        chain(model_branch.parameters()),
        betas=(0.9, 0.999),
        lr=cfg.start_lr,
        weight_decay=0.0,
    )

    scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=cfg.gamma)
```

Data loading

Model

Optimizer

Training loop with loss computation

Saving model

```
for epoch in range(cfg.max_epochs):
    # wrap epoch in launch logger for console logs
    with LaunchLogger(
        "train",
        epoch=epoch,
        num_mini_batch=len(dataloader),
        epoch_alert_freq=10,
    ) as log:
        for data in dataloader:
            optimizer.zero_grad()
            truevar = data[1]

            # compute forward pass
            outvar = model_branch(data[0][:, 0].unsqueeze(dim=1))

            # Compute data loss
            loss_data = F.mse_loss(outvar, truevar)

            # Compute total loss
            loss = loss_data

            # Backward pass and optimizer and learning rate update
            loss.backward()
            optimizer.step()
            scheduler.step()

            log.log_epoch({"Learning Rate": optimizer.param_groups[0]["lr"]})

        # test model on validation dataset
        with LaunchLogger("valid", epoch=epoch) as log:
            error = validation_step(model_branch, validation_dataloader, epoch)
            log.log_epoch({"Validation error": error})

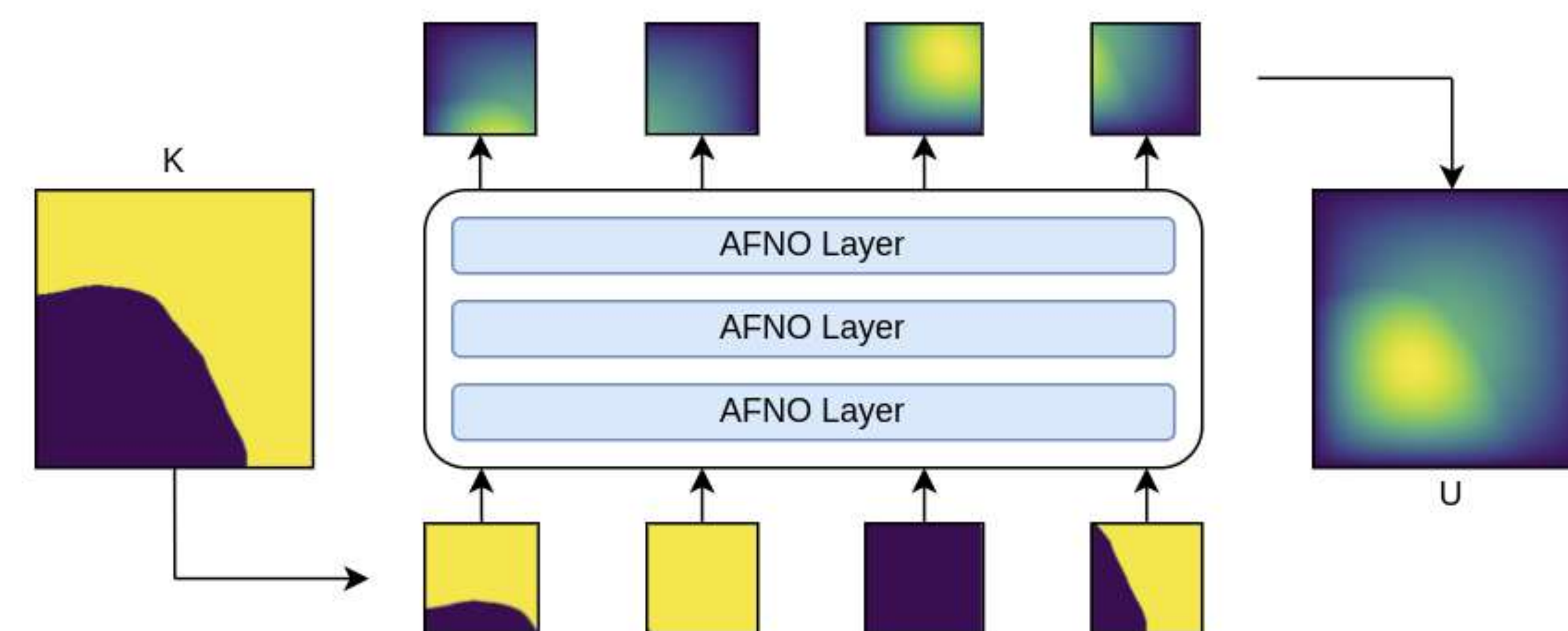
        # save the checkpoint
        save_checkpoint(
            "./checkpoints",
            models=[model_branch],
            optimizer=optimizer,
            scheduler=scheduler,
            epoch=epoch,
        )

if __name__ == "__main__":
    main()
```

Darcy Flow – Adaptive Fourier Neural Operators (AFNO)

Theory

- AFNO combines the Fourier Neural Operator (FNO) with the powerful Vision Transformer (ViT) model for image processing. The ViT and related variants of transformer models have achieved SOTA performance in image processing tasks. The multi-head self-attention (MHSA) mechanism of the ViT is key to its impressive performance.
- The first step in the architecture involves dividing the input image into a regular grid with $h \times w$ equal sized patches of size $p \times p$. The parameter p is referred to as the patch size. For simplicity, we consider a single channel image. Each patch is embedded into a token of size d , the embedding dimension. The patch embedding operation results in a token tensor ($X_{h \times w \times d}$) of size $h \times w \times d$. The patch size and embedding dimension are user selected parameters. A smaller patch size allows the model to capture fine scale details better while increasing the computational cost of training the model. A higher embedding dimension also increases the parameter count of the model. The token tensor is then processed by multiple layers of the transformer architecture performing spatial and channel mixing. The AFNO architecture implements the following operations in each layer.
- The token tensor is first transformed to the Fourier domain with
$$Z_{m,n} = [\text{DFT}(X)]_{m,n}$$
- Here, m, n is the index the patch location and DFT denotes a 2D discrete Fourier transform.



Darcy Flow – Adaptive Fourier Neural Operators (AFNO)

Results and discussion

AFNO accurately learns the solution of this system.

For more information, please refer to the official [PhysicsNeMo user guide example](#).

AFNO is the key backboard for NVIDIA's groundbreaking [FourCastNet](#):

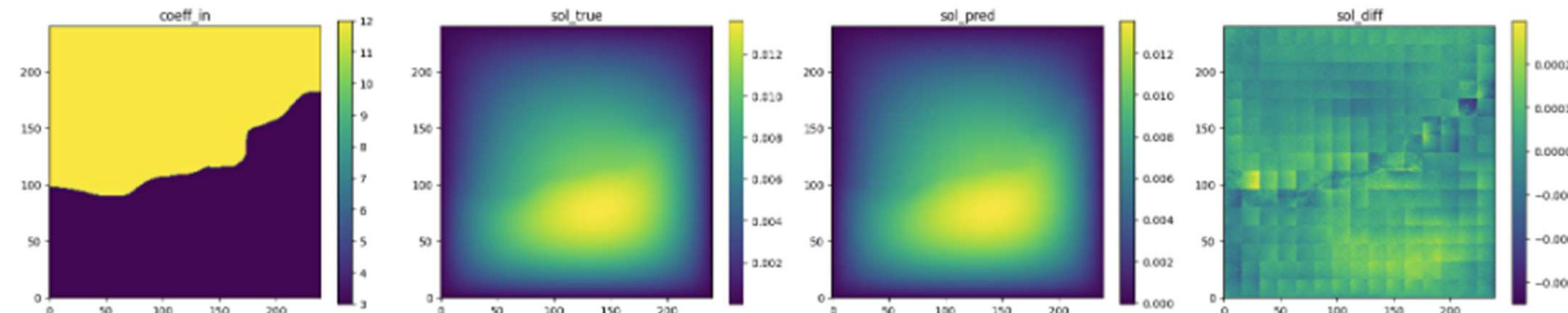
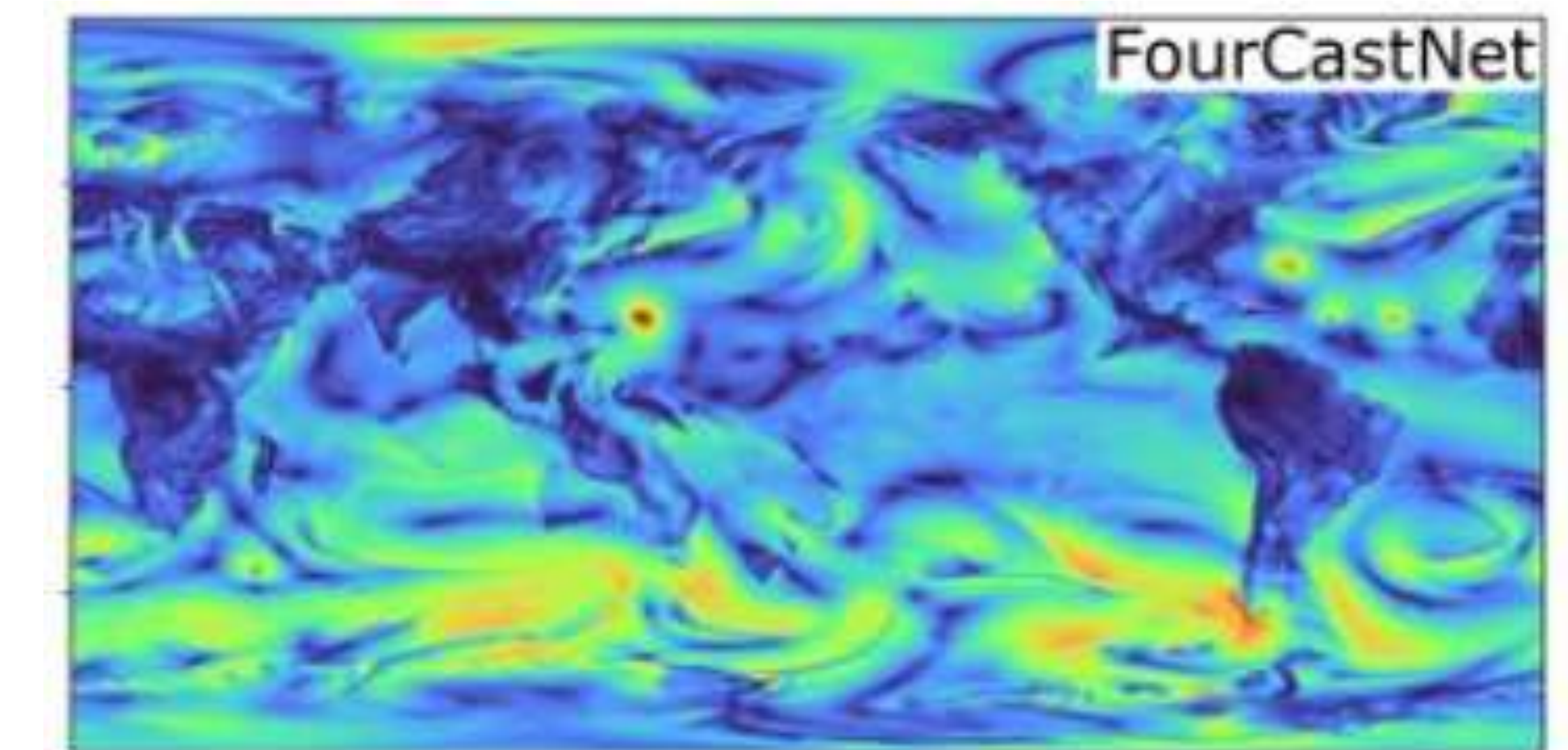


Fig. 65 AFNO validation prediction 1. (Left to right) Input permeability, true pressure, predicted pressure, error.

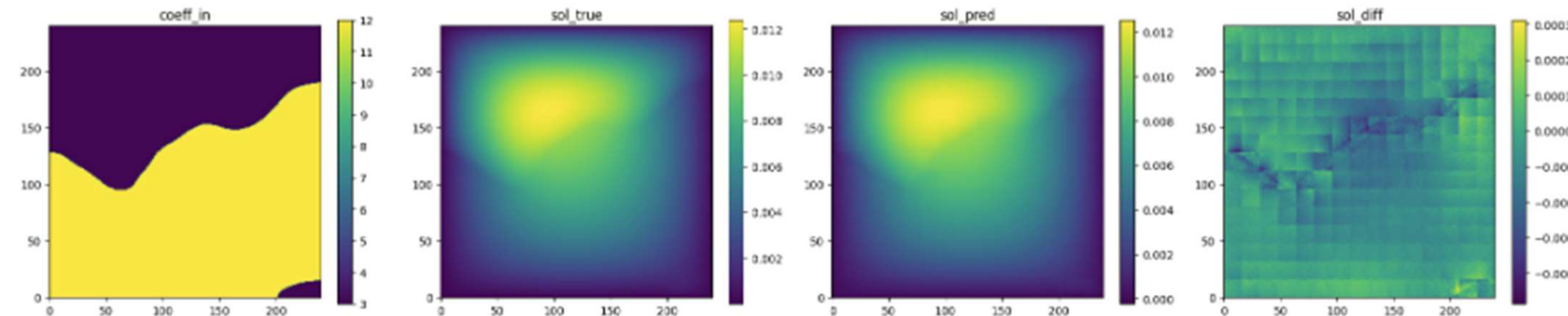
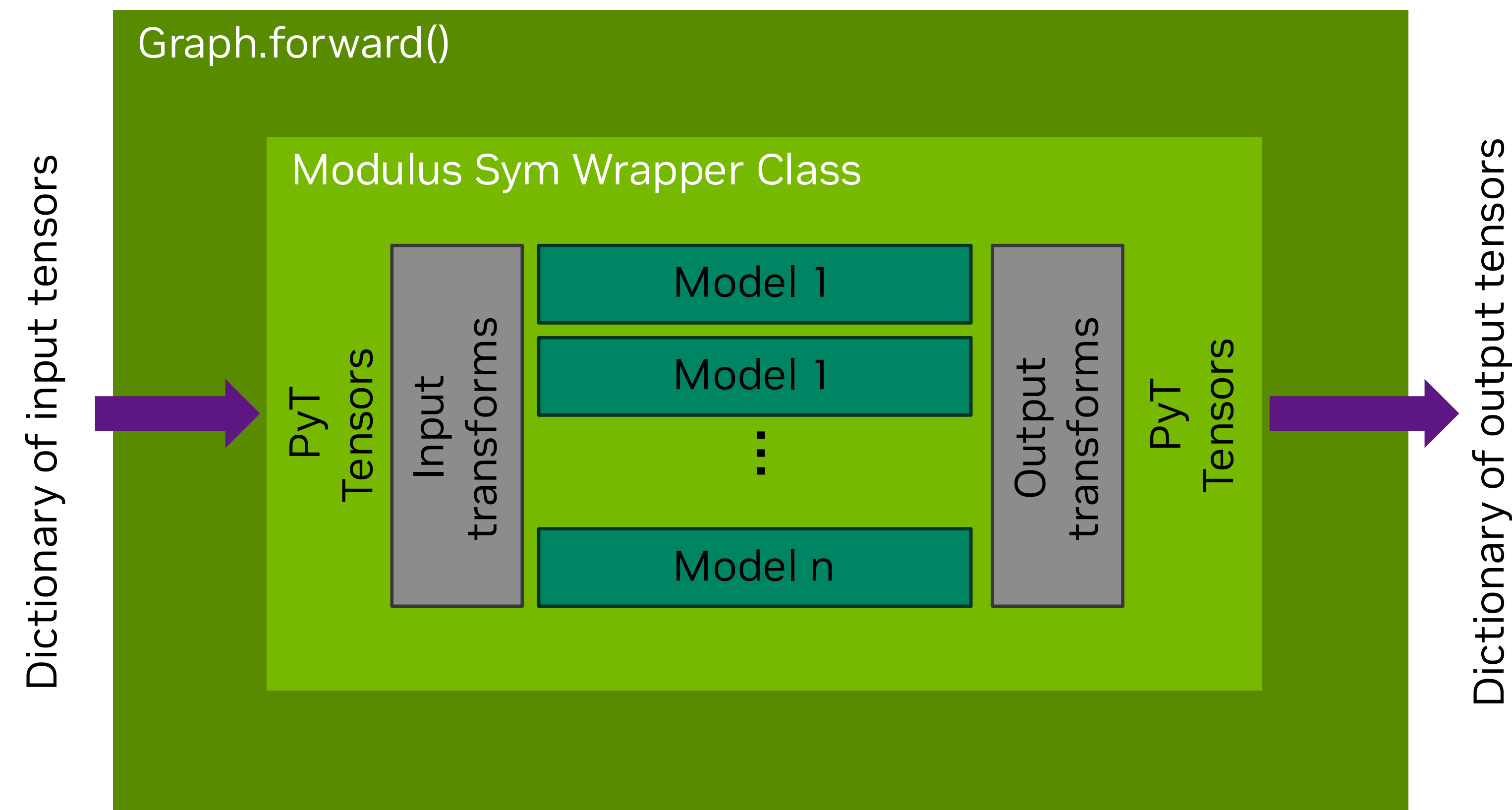


Fig. 66 AFNO validation prediction 2. (Left to right) Input permeability, true pressure, predicted pressure, error.

Computing Physics losses

Using optimized gradient backed from PhysicsNeMo Sym

- PhysicsNeMo or PyTorch models with tensor-in tensor-out are wrapped to work with dictionary of tensors
- This enables gradient computation using PhysicsNeMo Sym's optimized Gradient backend that supports:
 - Gradients using FuncTorch
 - Forward mode AD
 - Meshless Finite Difference
 - etc.



```
import torch
from PhysicsNeMo.sym.models.arch import Arch
from PhysicsNeMo.sym.key import Key
from typing import Dict

class MdlSymWrapper(Arch):
    def __init__(
        self,
        input_keys=[Key("x")],
        output_keys=[Key("u")],
        model=None,
    ):
        super(MdlSymWrapper, self).__init__(
            input_keys=input_keys,
            output_keys=output_keys,
        )

        self.model = model # PhysicsNeMo or PyTorch model

    def forward(self, dict_tensor: Dict[str, torch.Tensor]):

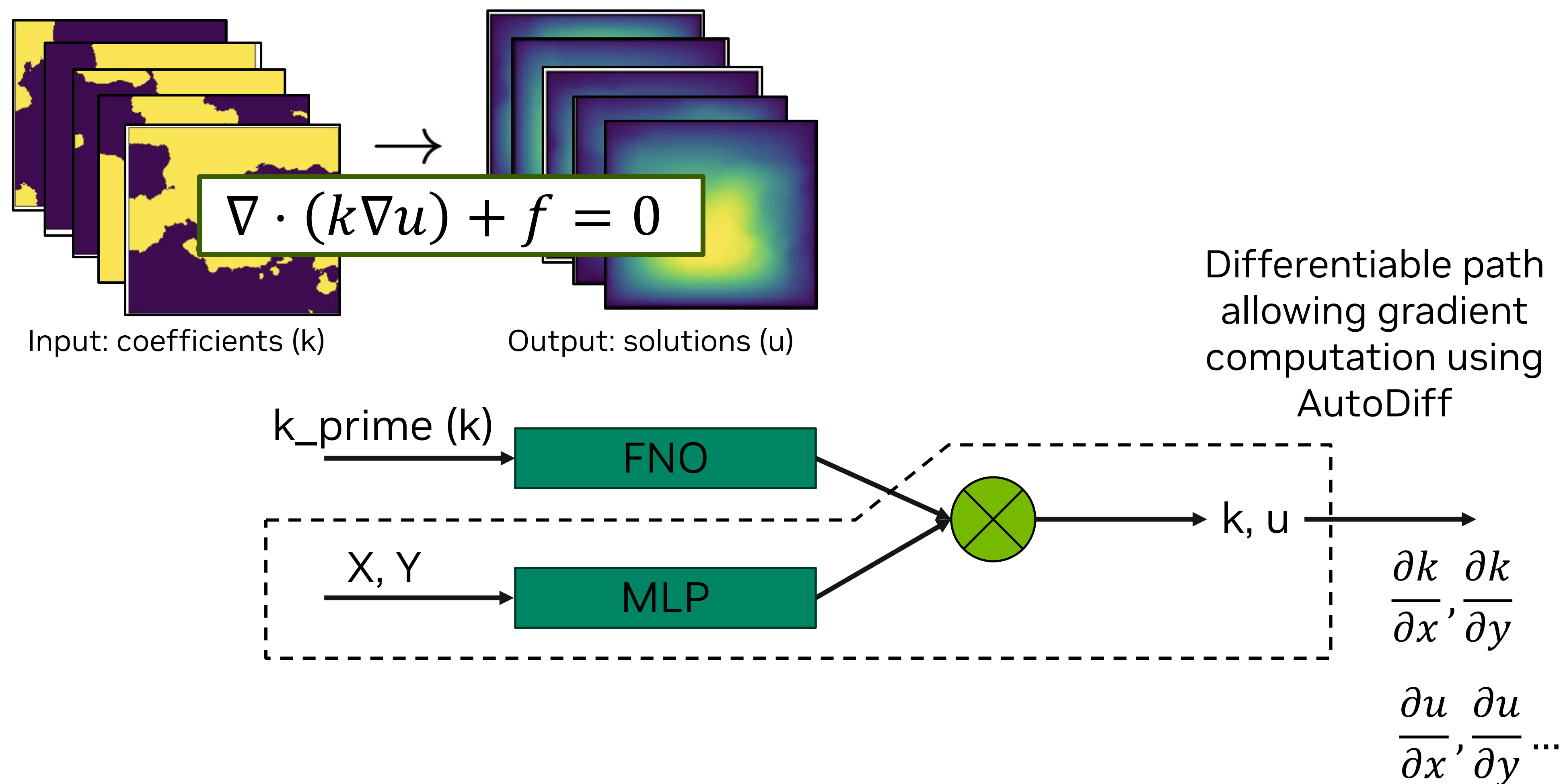
        input = self.concat_input(
            dict_tensor,
            self.input_key_dict,
            detach_dict=self.detach_key_dict,
            dim=-1, # concat along the last dimension to form the feature vector.
        )
        out = self.model(input)
        return self.split_output(
            out, self.output_key_dict, dim=1
        )

PhysicsNeMoSymModel = MdlSymWrapper(
    input_keys=[Key("x")],
    output_keys=[Key("u")],
    model=PhysicsNeMoModel
)
```

Wrapper sample code

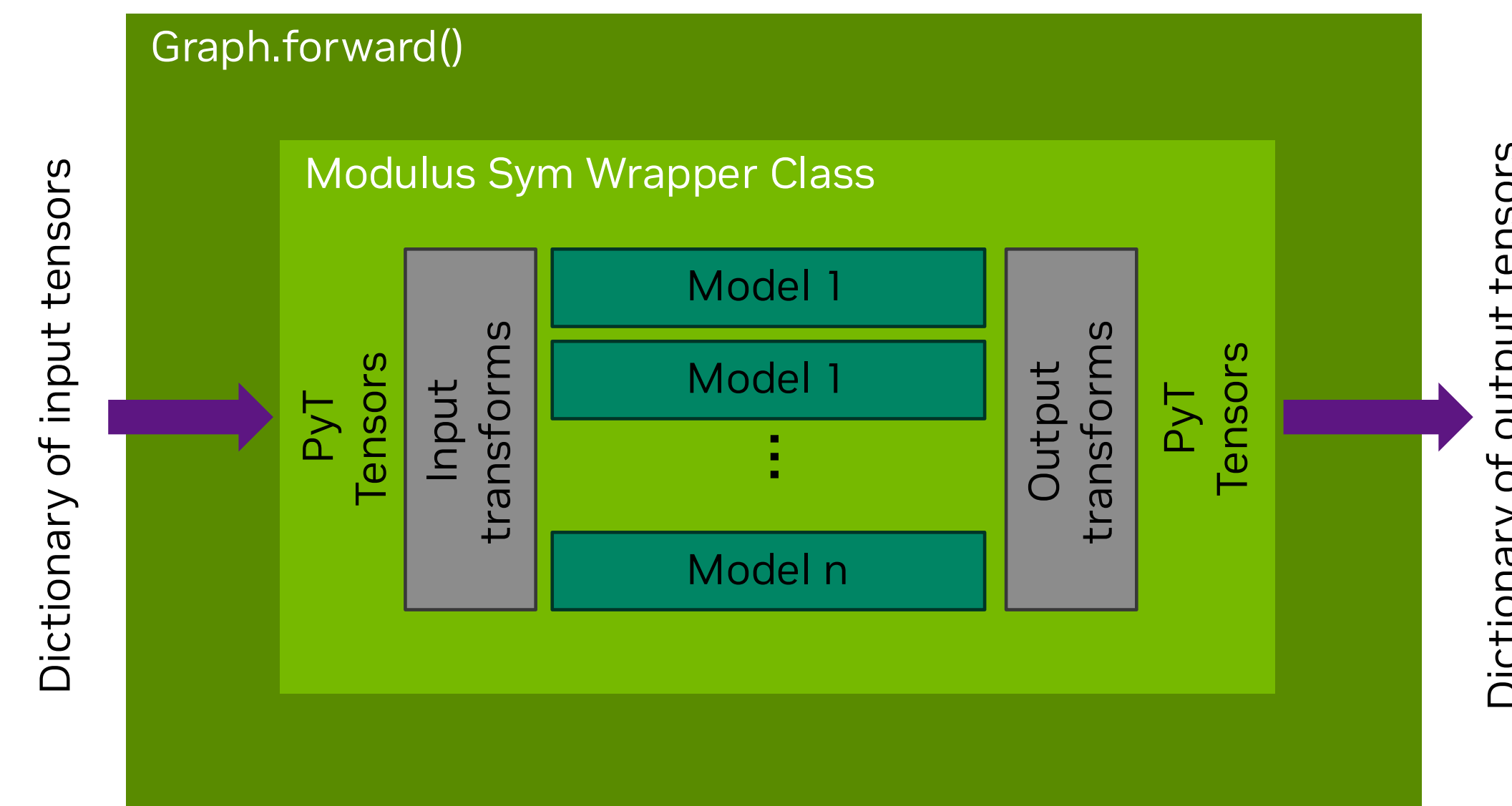
Computing Physics losses: In practice

Hybrid data + physics training workflows



k_prime is a copy of k

k is needed as an input, and as an output of the model to compute gradients of k



```
class MdlSymWrapper(Arch):
    def __init__(
        self,
        input_keys=[Key("k_prime"), Key("x"), Key("y")],
        output_keys=[Key("k"), Key("u")],
        trunk_net=None,
        branch_net=None,
    ):
        super(MdlSymWrapper, self).__init__(
            input_keys=input_keys,
            output_keys=output_keys,
        )

        self.branch_net = branch_net
        self.trunk_net = trunk_net

    def forward(self, dict_tensor: Dict[str, torch.Tensor]):

        xy_input_shape = dict_tensor["x"].shape
        xy = self.concat_input(
            {
                k: dict_tensor[k].view(xy_input_shape[0], -1, 1) for k in ["x", "y"]
            }, # flatten the coordinate dimensions
            ["x", "y"],
            detach_dict=self.detach_key_dict,
            dim=-1, # concat along the last dimension to form the feature vector.
        )
        fc_out = self.trunk_net(xy)
        fno_out = self.branch_net(dict_tensor["k_prime"])
        # reshape the fc_out
        fc_out = fc_out.view(
            xy_input_shape[0], -1, xy_input_shape[-2], xy_input_shape[-1]
        )

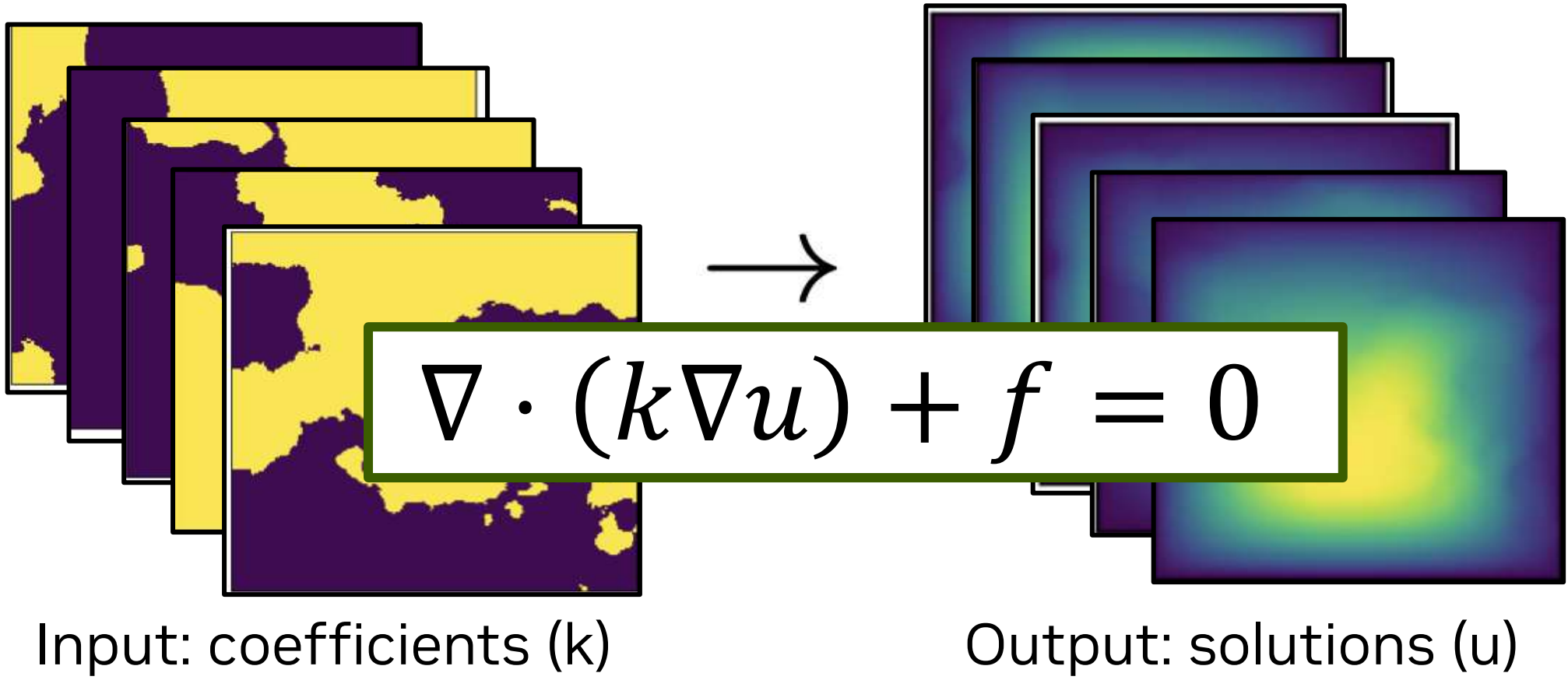
        out = fc_out * fno_out

        return self.split_output(
            out, self.output_key_dict, dim=1
        ) # Split along the channel dimension
```

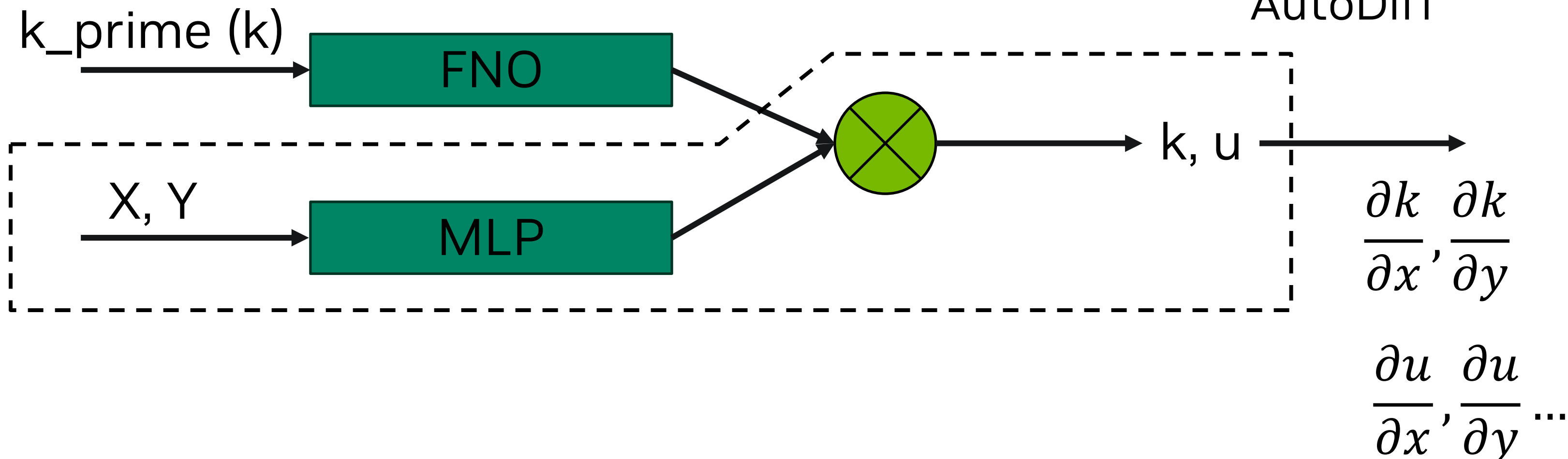
Wrapper definition

Computing Physics losses: In practice

Hybrid data + physics training workflows

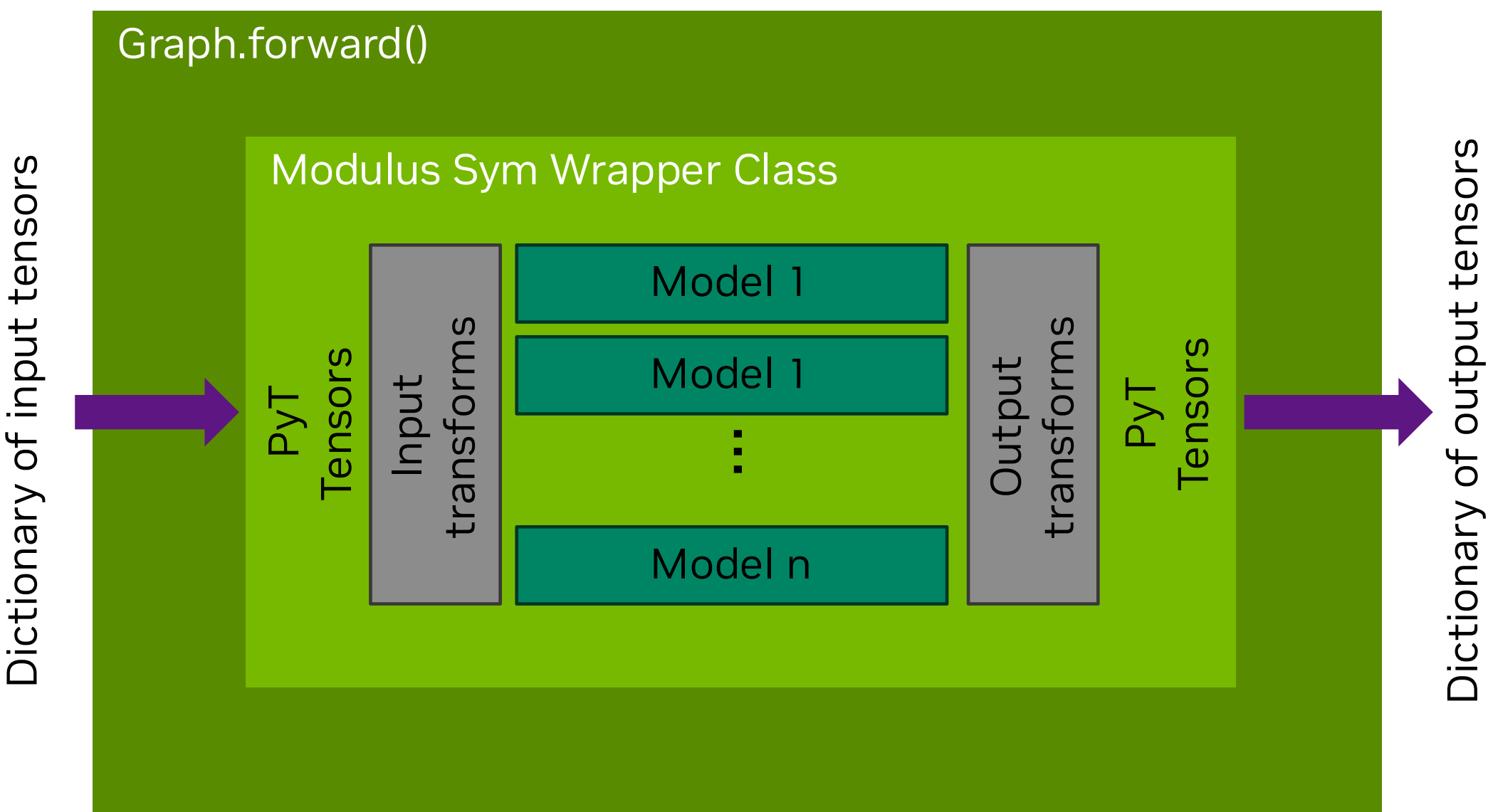


Differentiable path
allowing gradient
computation using
AutoDiff



k_prime is a copy of k

k is needed as an input,
and **as an output of the
model to compute
gradients of k**



```
model_branch = FNO(
    in_channels=1,
    out_channels=2, # one for u and one for k
).to("cuda")

model_trunk = FullyConnected(
    in_features=2, # x and y coordinates
    out_features=2, # one for u and one for k
).to("cuda")

model = MdlsSymWrapper(
    input_keys=[Key("k_prime"), Key("x"), Key("y")],
    output_keys=[Key("k"), Key("u")],
    trunk_net=model_trunk,
    branch_net=model_branch,
)

darcy = Diffusion(T="u", time=False, dim=2, D="k", Q=1.0 * 4.49996e00 * 3.88433e-03)

darcy_node = darcy.make_nodes()

nodes = darcy_node + [model.make_node(name="network", jit=False)]
graph = Graph(
    nodes,
    [Key("k_prime"), Key("x"), Key("y")],
    [Key("k"), Key("u"), Key("diffusion_u")],
    func_arch=False,
).to("cuda")
```

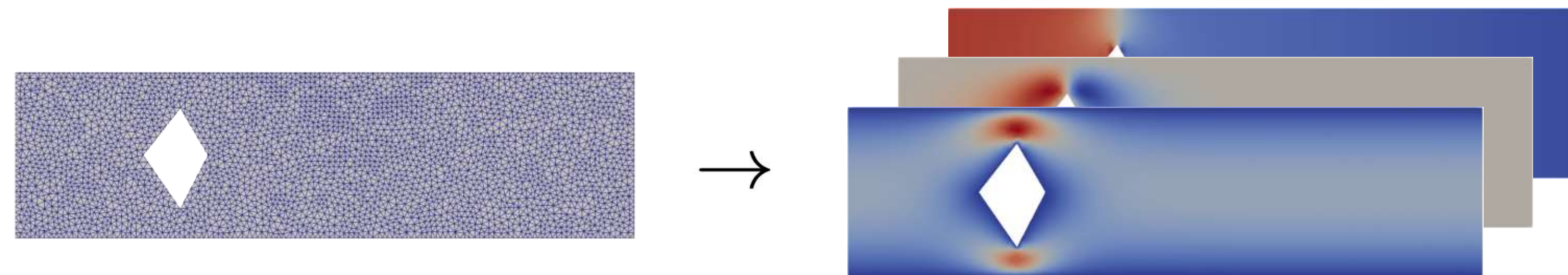
Wrapper and equations usage

Physics guided training workflows

Using physics informed loss in addition to boundary and initial conditions

- Fine-tuning of MeshGraphNets (MGN) predictions
 - Assume a data driven training using MGN that yields approximate predictions of the flow field but lack the finer features due to lack of sufficient training data
 - The predictions of the MGN model can be refined further by training a physics-driven (PINN) model improve the predictions

Data-driven
MGN



Input geometry
(Mesh representation)

Flow Field

```
...
x_no_slip, y_no_slip = coords_noslip[:, 0:1], coords_noslip[:, 1:2]
results_noslip = graph.forward({"x": x_no_slip, "y": y_no_slip})
pred_u_noslip, pred_v_noslip = results_noslip["u"], results_noslip["v"]

x_int, y_int = coords[:, 0:1], coords[:, 1:2]
results_int = graph_full.forward({"x": x_int, "y": y_int})
pred_mom_u, pred_mom_v, pred_cont = (
    results_int["momentum_x"],
    results_int["momentum_y"],
    results_int["continuity"],
)

pred_u, pred_v, pred_p = results_int["u"], results_int["v"], results_int["p"]

# Compute losses
# data loss (training against the MGN predictions)
loss_u = torch.mean((gnn_u - pred_u) ** 2)
loss_v = torch.mean((gnn_v - pred_v) ** 2)
loss_p = torch.mean((gnn_p - pred_p) ** 2)

# noslip boundary condition loss
loss_u_noslip = torch.mean(pred_u_noslip**2)
loss_v_noslip = torch.mean(pred_v_noslip**2)

# pde loss
loss_cont = torch.mean(pred_cont**2)
loss_mom_u = torch.mean(pred_mom_u**2)
loss_mom_v = torch.mean(pred_mom_v**2)
```

Physics-based
fine-tuning
PINN



Ground Truth



MGN results (fully-data driven)



PINN results using MGN predictions
and boundary conditions as inputs

Key challenge: correctly sample the boundary, interior points/regions in the absence of mesh